

VAIJUNTO · SISTEMA DE CARONAS COMPARTILHADAS

Guia Completo

Da regra de negócio a cada linha de código, para quem já programa em Python

Disciplina	Concorrência e Conectividade (PBL) · UEFS
Linguagem	Go 1.27, apenas biblioteca padrão
Código descrito	branch <code>feat/revisao</code> , 14/09/2026
Cobre	protocolo, dados, servidor, cliente, carga, testes e Docker

Como ler este guia

Este guia explica o Vaijunto inteiro, em camadas. Primeiro o problema, sem nenhum código. Depois como as peças se encaixam. Depois o Go que você precisa saber vindo de Python. Só então o código, linha a linha, e por fim os testes, a operação e o que ainda falta.

Parte	O que tem	Para quê
I	Regras de negócio: caronas, trechos, busca, reserva atômica, pagamento, expiração, cancelamentos	Entender o que o sistema faz antes de ver como
II	Arquitetura: pacotes, caminho de uma requisição, protocolo na rede, goroutines, mutex, falhas	Ter o mapa antes de entrar no código
III	Go para quem vem de Python: cada construção usada no projeto, lado a lado com Python	Não travar na sintaxe na Parte IV
IV	O código linha a linha: <code>protocolo</code> , <code>dados</code> , <code>servidor</code> , <code>cliente</code> , <code>carga</code>	O coração do guia
V	Os testes: o que cada um prova e como detectam falhas de concorrência	Saber defender que o sistema funciona
VI	Rodar, demonstrar e empacotar com Docker	Apresentação
VII	O que ainda falta: pendências obrigatórias e limitações conhecidas	Planejar o resto da semana
Apêndice	Todos os arquivos completos, com número de linha	Consulta

Convenções

- **Todo código mostrado é o código real.** Ele foi lido direto dos arquivos no momento em que este PDF foi gerado. Acima de cada bloco aparece o arquivo e a faixa de linhas: no VS Code, `Ctrl+G` e o número levam direto à linha.
- **Blocos "linha a linha"** mostram o bloco inteiro e, abaixo, cada linha (ou pequeno grupo de linhas) com o número à esquerda e a explicação. Linhas que só fecham chaves e os comentários do próprio código não são repetidos.
- **Caixas coloridas:** azul é conceito, verde é comparação com Python ou dica, laranja é armadilha.
- **Blocos lado a lado** mostram a mesma ideia em Python (borda amarela) e em Go (borda azul).
- **Transcrições** mostram conversas reais com o servidor, capturadas rodando o sistema: `-->` é o que o cliente enviou, `<--` é o que o servidor respondeu.
- **Testes** são explicados teste a teste: o que cada um prova e cada construção nova que aparece. Padrões que se repetem entre testes são explicados na primeira vez.

SE O CÓDIGO MUDAR

O guia é gerado por `Documentação/fontes/guia-completo/gerar.py`. Rodando `python gerar.py` nessa pasta, o código de todos os blocos é relido dos arquivos e o PDF é refeito. O gerador também avisa se alguma linha de código ficou sem explicação.

Sumário

Parte I — O problema e as regras de negócio

1. O que é o Vaijunto
2. Quem usa o sistema
3. Carona, rota e trecho
4. Disponibilidade por trecho
5. Buscar uma viagem
6. Reservar: tudo ou nada
7. Pagar, expirar e o ciclo de vida da reserva
8. Cancelamentos
9. Contas, login e sessão
10. Persistência
11. Mapa das regras

Parte II — Arquitetura

12. Visão geral
13. Os pacotes e quem depende de quem
14. O caminho de uma requisição
15. O protocolo de aplicação
16. Goroutines: quem roda ao mesmo tempo
17. Um mutex para todo o estado
18. Resiliência a falhas

Parte III — Go para quem vem de Python

19. Como um programa Go é organizado
20. Variáveis, tipos e valores zero
21. Funções e múltiplos retornos
22. Erros: valores, não exceções
23. Structs, métodos e ponteiros
24. Slices e maps
25. Laços com for
26. if e switch
27. Strings e formatação
28. Visibilidade e documentação
29. defer
30. Goroutines
31. Canais
32. sync.Mutex e sync.WaitGroup
33. Tempo
34. JSON
35. Rede e interfaces
36. Sistema, argumentos e ordenação
37. Testes
38. Cola rápida Python → Go

Parte IV — O código, linha a linha

39. `protocolo/protocolo.go` — o vocabulário da rede
40. `dados/dados.go` (1 de 2) — estado, contas, caronas, busca e reserva
41. `dados/dados.go` (2 de 2) — pagamento, cancelamentos, expiração, listagens e funções internas
42. `servidor/main.go` — conexões, sessões e o roteador
43. `cliente/main.go` — os menus de terminal
44. `carga/main.go` — o teste de carga pela rede

Parte V — Os testes

45. Como rodar e ler o resultado
46. As peças que os testes reaproveitam
47. Como se testa concorrência
48. `dados_test.go`: concorrência na reserva
49. `dados_test.go`: atomicidade e disponibilidade
50. `dados_test.go`: expiração e pagamento
51. `dados_test.go`: contas e cadastro
52. `cancelamento_test.go`

53. `servidor/main_test.go`

Parte VI — Rodar, demonstrar e empacotar

54. Rodar na máquina

55. Roteiro de demonstração

56. Dockerfile

57. `docker-compose.yml`

Parte VII — O que ainda falta

58. Prazos

59. Pendências para a entrega

60. Limitações conhecidas do código

61. Sugestão de ordem

Apêndice — Código completo

62. `go.mod`

63. `protocolo/protocolo.go`

64. `dados/dados.go`

65. `servidor/main.go`

66. `cliente/main.go`

67. `carga/main.go`

68. `dados/dados_test.go`

69. `dados/cancelamento_test.go`

70. `servidor/main_test.go`

71. Dockerfile

72. `docker-compose.yml`

Parte I — O problema e as regras de negócio

O que o sistema precisa fazer, antes de qualquer linha de código. Cada regra desta parte aparece mais adiante implementada numa função e provada por um teste; a última seção desta parte liga as três coisas.

1. O que é o Vaijunto

O Vaijunto é um sistema de caronas compartilhadas entre cidades, feito como PBL da disciplina de Concorrência e Conectividade da UEFS. Motoristas publicam viagens com várias paradas; passageiros procuram um pedaço dessas viagens, reservam um assento e pagam. Um passageiro pode, inclusive, combinar duas caronas de motoristas diferentes para chegar aonde nenhuma das duas chega sozinha.

O tema de negócio é o pretexto. O que a disciplina avalia é o que acontece por baixo: **vários programas conversando pela rede ao mesmo tempo e disputando os mesmos assentos**. Se dois passageiros apertam "reservar" no mesmo instante para o último lugar, só um pode levar, e o sistema precisa garantir isso sozinho, sem um banco de dados fazendo esse trabalho por ele.

Os requisitos que moldaram o projeto

Requisito	O que significa na prática	Como o Vaijunto atende
Comunicação TCP/IP com socket	Nada de HTTP, gRPC ou framework de mensageria	Pacote <code>net</code> da biblioteca padrão e um protocolo próprio: uma linha de JSON por mensagem
Controle de concorrência feito por nós	Não pode delegar a um banco de dados	<code>sync.Mutex</code> protegendo todo o estado
Reserva atômica	"Pegou um trecho, não fica sem o outro"	<code>Reservar</code> em duas fases, dentro de um único bloqueio
Assento não fica preso sem pagamento	Reserva não paga vence em 10 minutos	Goroutine que varre e devolve assentos vencidos
Sem SGBD; JSON permitido	Estado em memória, salvo em arquivo	Arquivo <code>vaijunto.json</code> regravado a cada alteração
Rodar em várias máquinas com Docker	Servidor e clientes em containers	<code>Dockerfile</code> e <code>docker-compose.yml</code> (escritos, ainda não testados)
Teste com vários clientes simultâneos	Provar a concorrência, não só afirmar	Testes automatizados e o programa <code>carga</code> , com até 1000 clientes reais

2. Quem usa o sistema

Existem dois tipos de conta, escolhidos na criação: **motorista** e **passageiro**. O tipo decide qual menu o cliente mostra e quais ações o servidor aceita.

Tipo	Pode fazer	O servidor recusa
Motorista	Cadastrar carona · listar as suas caronas · ver os passageiros de cada trecho · cancelar carona	—
Passageiro	Buscar viagens (diretas e com conexão) · reservar · pagar · listar suas reservas · cancelar reserva	Cadastrar carona · ver passageiros · cancelar carona

DETALHE QUE VALE SABER

O servidor só **restringe ao motorista** as três ações de motorista. Buscar e reservar **não conferem o tipo**: um motorista que mandasse `reservar` direto pelo protocolo conseguiria. Na prática isso não acontece porque o menu do motorista não oferece essas opções. Está listado na Parte VII.

3. Carona, rota e trecho

Uma **carona** é uma viagem oferecida por um motorista numa data. Ela tem uma **rota**: a sequência de cidades por onde passa, na ordem. Entre duas cidades consecutivas existe um **trecho**.

Carona 1 · motorista ana · 2026-09-20 · 2 assentos · R\$30 por trecho

cidade:	Feira	—————	Salvador	—————	Ilheus
índice:	0		1		2
		trecho 0		trecho 1	

- **Uma rota com N cidades tem N – 1 trechos.** Três cidades, dois trechos.
- **A carona anda em um sentido só.** Quem quer ir de Ilhéus para Feira não pode usar essa carona.
- **O número de assentos é o mesmo em todos os trechos:** é o carro que o motorista tem.
- **O preço é por trecho.** Feira → Salvador custa R\$30; Feira → Ilhéus usa dois trechos e custa R\$60.
- **A data é um texto** no formato `2026-09-20`. A busca compara o texto exatamente; não há horário.

Como um pedaço de viagem é descrito

Quando um passageiro quer uma parte da rota, o sistema descreve isso como um **item**: a carona, a posição da cidade de origem (**De**) e a posição da cidade de destino (**Ate**). São **posições na rota**, não nomes de cidade.

Viagem pedida	De	Ate	Trechos usados
Feira → Salvador	0	1	trecho 0
Salvador → Ilhéus	1	2	trecho 1
Feira → Ilhéus	0	2	trechos 0 e 1

A regra geral: **ir da posição `de` até a posição `ate` usa os trechos `de`, `de+1`, ..., `ate-1`**. É exatamente o `range(de, ate)` do Python, que também não inclui o final.

4. Disponibilidade por trecho

Uma carona não fica "cheia" ou "vazia" como um todo. Cada trecho tem sua própria contagem de assentos ocupados. Isso fica guardado num vetor chamado `Ocupados`, com uma posição por trecho.

Acompanhe a carona 1 (2 assentos) recebendo reservas:

Acontecimento	Ocupados	Feira → Salvador	Salvador → Ilhéus	Feira → Ilhéus
Carona criada	[0 0]	2 livres	2 livres	2 livres
bruno reserva Feira → Ilhéus	[1 1]	1 livre	1 livre	1 livre
carla reserva Salvador → Ilhéus	[1 2]	1 livre	0	0
joana reserva Feira → Salvador	[2 2]	0	0	0

Olhe a terceira linha. O trecho o ainda tem um lugar, mas **Feira → Ilhéus já não tem**. O passageiro precisa do mesmo assento do começo ao fim, então quem manda é o **trecho mais cheio do caminho**:

```
livres(Feira → Ilheus) = mínimo( 2 - Ocupados[0], 2 - Ocupados[1] )
                        = mínimo( 2 - 1,          2 - 2 )
                        = 0
```

EM PYTHON

A regra inteira cabe numa linha: `min(assentos - o for o in ocupados[de:ate])`. Em Go ela vira um laço de seis linhas, que você vai ver na função `livres`.

5. Buscar uma viagem

O passageiro informa **origem, destino e data**. O servidor devolve uma lista de **opções**: primeiro as viagens diretas, depois as que exigem trocar de carro uma vez.

Viagem direta

Uma carona serve como viagem direta quando **todas** as condições valem:

1. a data da carona é exatamente a data pedida;
2. a carona não foi cancelada;
3. a origem e o destino estão na rota;
4. a origem vem **antes** do destino na rota (sentido único; origem igual ao destino também não serve);
5. há pelo menos um assento livre em **todo** o caminho entre elas.

O preço da opção é o preço por trecho vezes a quantidade de trechos usados.

Viagem com uma conexão

Quando nenhuma carona sozinha resolve, o sistema tenta combinar duas. A primeira leva o passageiro da origem até uma cidade intermediária, a **baldeação**; a segunda leva da baldeação até o destino.

```
carona 1 (ana)   Feira ——— Salvador ——— Ilheus
                  | troca de carro (baldeação)
carona 2 (davi)                Ilheus ——— Porto Seguro

opção:  Feira → Ilheus na carona 1   +   Ilheus → Porto Seguro na carona 2
preço:  2 trechos × R$30              +   1 trecho × R$50                = R$110
```

As regras da combinação:

- a primeira carona passa pela origem e cada cidade **depois** dela na rota é candidata a baldeação;
- se a candidata já é o destino, a combinação é descartada, porque isso é viagem direta e já foi listado;
- a primeira carona precisa ter vaga da origem até a baldeação;
- a segunda é **outra** carona, da mesma data, não cancelada, que passa pela baldeação e depois pelo destino, com vaga nesse caminho;
- o preço é a soma das duas partes;
- **só uma baldeação**: o sistema não procura viagens com três caronas.

O que a busca devolve

Cada opção tem três coisas: os **itens** (um para viagem direta, dois para conexão), um **resumo** em texto pronto para mostrar, e o **preço**. O cliente não precisa entender os itens: ele mostra o resumo, o passageiro escolhe, e o cliente devolve os itens daquela opção no pedido de reserva.

6. Reservar: tudo ou nada

Reservar é pedir um assento em **todos os itens** de uma opção. Para viagem direta é um item; para conexão, dois. A regra central do enunciado é que isso seja **atômico**: ou todos os itens são garantidos, ou nenhum.

Por que tudo ou nada

Imagine uma implementação ingênua, que vai ocupando item por item:

Passo	Carona 1 · Feira → Ilhéus	Carona 2 · Ilhéus → Porto Seguro
1. ocupa a carona 1	ocupado para o bruno	—
2. tenta a carona 2	ocupado para o bruno	está cheia: falha
Resultado	bruno fica segurando um assento até Ilhéus que não serve para nada, e ninguém mais consegue esse lugar	

Por isso a reserva é feita em **duas fases**.

As duas fases

Fase 1 — verificação. Para cada item, confere, sem alterar nada:

- a carona existe;
- a carona não foi cancelada;
- as posições fazem sentido: `0 ≤ De < Ate ≤ número de trechos`;
- há pelo menos um assento livre em todo o caminho do item.

Qualquer falha devolve um erro e **nada muda**.

Fase 2 — aplicação. Só roda se a fase 1 passou para todos os itens. Ocupa um assento em cada trecho de cada item.

Depois das duas fases, a reserva é registrada com estado **pendente** e um prazo: agora mais 10 minutos.

Por que ninguém se intromete entre as fases

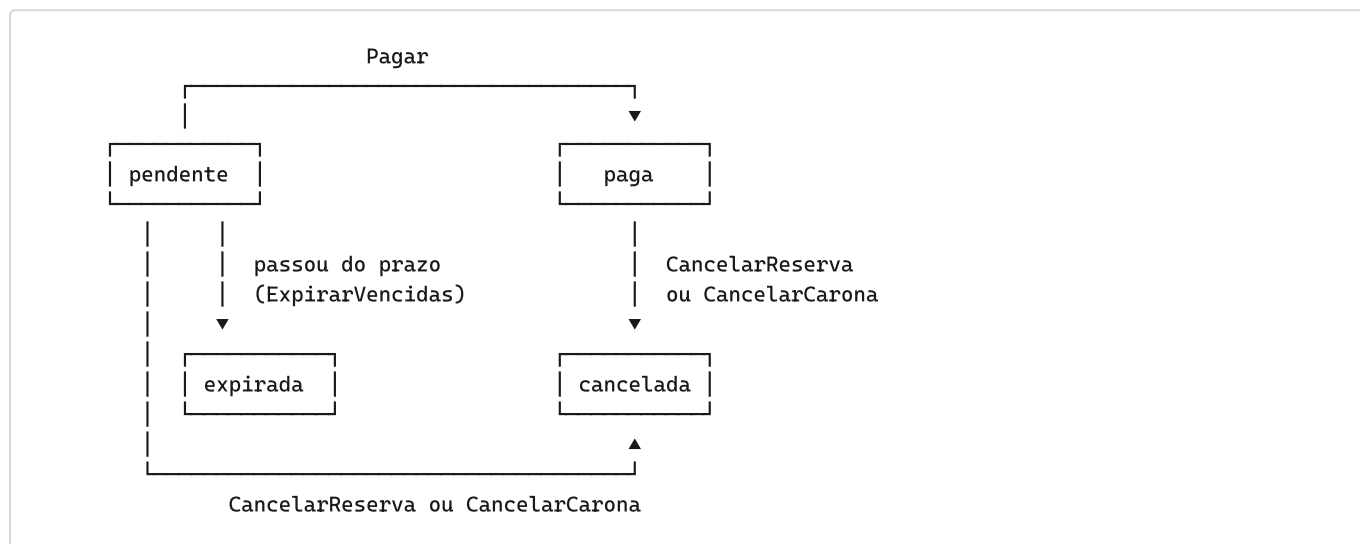
Entre conferir que há vaga e ocupar a vaga, outro passageiro poderia pegar o mesmo assento. O sistema impede isso mantendo o estado inteiro bloqueado do começo ao fim da reserva, com um mutex. A Parte II explica o mecanismo.

Os números de carona e de reserva

Caronas e reservas compartilham **um único contador**. Se a ana cadastra a carona 1 e o davi a carona 2, a próxima reserva será a 3. É o que aparece nas conversas reais da Parte II. O número nunca se repete, mas não dá para deduzir se um número é de carona ou de reserva só olhando para ele.

7. Pagar, expirar e o ciclo de vida da reserva

Uma reserva passa por estados. Ela nasce pendente e termina em um de três destinos:



De	Para	O que causa	Assentos
—	pendente	reserva criada	ocupados
pendente	paga	o dono paga	continuam ocupados
pendente	expirada	prazo vencido, na próxima varredura	devolvidos
pendente ou paga	cancelada	o passageiro cancela, ou o motorista cancela a carona	devolvidos

Expirada e cancelada são estados finais: nada tira a reserva deles.

Regras do pagamento

Situação	Resposta do servidor
A reserva não existe	reserva N nao existe
A reserva é de outra pessoa	esta reserva nao e sua

Já está paga	esta reserva ja foi paga
Expirou	esta reserva expirou; os assentos voltaram para a fila
Foi cancelada	esta reserva foi cancelada (motivo)
Pendente, do próprio passageiro	vira paga

Regras da expiração

- O prazo padrão é de **10 minutos** a partir da criação da reserva.
- O servidor faz uma **varredura a cada 30 segundos**: toda reserva **pendente** cujo prazo já passou vira expirada e devolve seus assentos.
- **Reserva paga nunca expira.**
- Os dois tempos podem ser encurtados para demonstração, pelas variáveis `VAIJUNTO_RESERVA` e `VAIJUNTO_VARREDURA`.

JANELA ENTRE VENCER E EXPIRAR

O pagamento olha apenas o **estado** da reserva, não o horário. Entre o momento em que o prazo vence e a próxima varredura (até 30 segundos depois), a reserva ainda está pendente e **ainda pode ser paga**. O efeito prático é pequeno, mas o prazo não é cumprido ao segundo. Está na Parte VII.

8. Cancelamentos

Cancelar reserva

- Só o **dono** da reserva pode cancelar.
- Só reservas **ativas** (pendente ou paga) podem ser canceladas. Reserva paga também pode.
- Os assentos voltam em **todos os trechos de todos os itens**. Uma conexão devolve o assento nas duas caronas.
- A reserva fica com estado `cancelada` e motivo `cancelada pelo passageiro`.

Cancelar carona

- Só o **motorista dono** pode cancelar.
- A carona **não é apagada**: fica marcada como cancelada, para o histórico continuar coerente.
- Ela **some da busca** e **recusa reservas novas**.
- Toda reserva ativa que usa essa carona é cancelada **inteira**, mesmo se já estava paga, com o motivo `carona N cancelada pelo motorista`.
- A resposta informa quantas reservas foram desfeitas.

O caso que exige cuidado é a conexão:

```
ANTES
reserva 3 (bruno, paga) = carona 1 Feira → Ilheus      + carona 2 Ilheus → Porto Seguro
                        Ocupados [1 1]                  Ocupados [1]

davi cancela a carona 2

DEPOIS
reserva 3 → cancelada ("carona 2 cancelada pelo motorista")
carona 1   Ocupados [0 0]      ← o assento volta TAMBÉM aqui
carona 2   Ocupados [0]        CANCELADA
```

Liberar só a parte da carona 2 deixaria o bruno com uma reserva até Ilhéus que não leva a lugar nenhum, a mesma "meia viagem" que a reserva atômica proíbe. Por isso a reserva cai inteira e o assento volta nas duas caronas.

9. Contas, login e sessão

Criar conta

- O login não pode ser vazio. Espaços nas pontas são removidos, então " ana " vira "ana".
- A senha não pode ser vazia.
- O tipo precisa ser `motorista` ou `passageiro`.
- **O login é único.** Se duas pessoas tentarem criar o mesmo login ao mesmo tempo, só uma consegue.

Entrar e sessão

- Entrar confere login e senha e devolve o tipo da conta, que decide o menu.
- A **sessão pertence à conexão**. Fechou o programa, a sessão acabou. Entrar de novo na mesma conexão troca o usuário; a ação `sair` limpa a sessão.
- Sem login, o servidor só aceita `ping`, `login` e `registrar`. Todo o resto responde `faça login primeiro`.

Usuários de teste

O sistema já nasce com quatro contas, todas com senha `123`: **ana** e **davi** (motoristas), **bruno** e **carla** (passageiros).

SENHA EM TEXTO PURO

As senhas ficam gravadas como texto no arquivo JSON, sem nenhum embaralhamento. É uma simplificação consciente para um trabalho acadêmico e aparece na Parte VII.

10. Persistência

O estado do sistema (contas, caronas, reservas e o contador de números) mora na memória do servidor e é **gravado inteiro num arquivo JSON a cada alteração**: criar conta, cadastrar carona, reservar, pagar, cancelar e expirar. Consultas não gravam nada.

- O arquivo padrão é `vaijunto.json`, na pasta de onde o servidor foi iniciado; a variável `VAIJUNTO_DADOS` muda o caminho.
- Ao subir, o servidor **lê o arquivo**. Se ele não existe, começa do zero com os usuários de teste.
- Reservas pendentes continuam contando o prazo pelo horário gravado: se o servidor ficou 15 minutos desligado, elas expiram na primeira varredura.
- **Não é gravado:** quem está logado (sessões são da conexão) nem o mutex.

ARQUIVO CORROMPIDO

Se o arquivo existir mas não for um JSON válido, o servidor avisa e **começa do zero**. Na primeira alteração, ele regrava o arquivo e os dados antigos se perdem. Isso poderia acontecer se o servidor morresse exatamente durante uma gravação. Está na Parte VII.

Este é o arquivo real gravado ao final das conversas capturadas para este guia:

```
{
  "usuarios": {
    "ana": {
      "login": "ana",
      "senha": "123",
      "tipo": "motorista"
    },
    "bruno": {
      "login": "bruno",
      "senha": "123",
      "tipo": "passageiro"
    },
    "carla": {
      "login": "carla",
      "senha": "123",
      "tipo": "passageiro"
    },
    "davi": {
      "login": "davi",
      "senha": "123",
      "tipo": "motorista"
    }
  },
  "caronas": [
    {
      "id": 1,
      "motorista": "ana",
      "rota": [
        "Feira",
        "Salvador",
        "Ilheus"
      ],
      "data": "2026-09-20",
      "assentos": 2,
      "preco": 30,
      "ocupados": [
        0,
        0
      ]
    },
    {
      "id": 2,
      "motorista": "davi",
      "rota": [
        "Ilheus",
```

```

    "Porto Seguro"
  ],
  "data": "2026-09-20",
  "assentos": 2,
  "preco": 50,
  "ocupados": [
    0
  ],
  "cancelada": true
}
],
"reservas": [
  {
    "id": 3,
    "passageiro": "bruno",
    "itens": [
      {
        "carona_id": 1,
        "de": 0,
        "ate": 2
      },
      {
        "carona_id": 2,
        "de": 0,
        "ate": 1
      }
    ],
    "estado": "cancelada",
    "motivo": "carona 2 cancelada pelo motorista",
    "expira": "2026-09-14T09:57:23.8972464-03:00"
  },
  {
    "id": 4,
    "passageiro": "carla",
    "itens": [
      {
        "carona_id": 1,
        "de": 1,
        "ate": 2
      }
    ],
    "estado": "cancelada",
    "motivo": "cancelada pelo passageiro",
    "expira": "2026-09-14T09:57:23.9747825-03:00"
  }
],
"prox_id": 5
}

```

Repare em três coisas: a carona 2 tem `"cancelada": true` e a carona 1 não tem esse campo (campo falso não é gravado); as duas reservas estão canceladas, cada uma com seu motivo; e `prox_id` é 5, porque os números 1 a 4 já foram usados por duas caronas e duas reservas.

11. Mapa das regras

Cada regra de negócio, a função que a implementa e o teste que a prova. Esta tabela serve de roteiro para o relatório e para responder perguntas na apresentação.

Regra	Onde está	Teste que prova
-------	-----------	-----------------

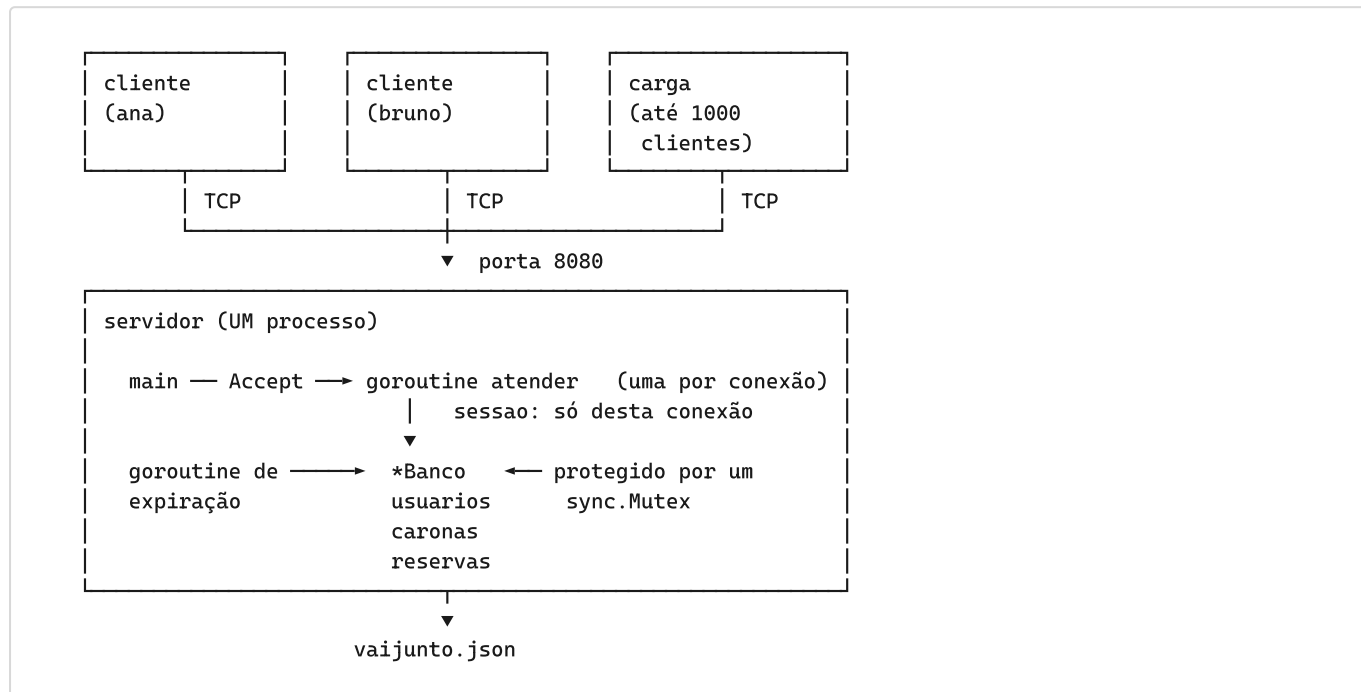
Login único, validação da conta	<code>dados.CadastrarUsuario</code>	<code>TestCadastrarUsuario</code> , <code>TestCadastroConcorrente</code> , <code>TestCadastrosSimultaneos</code>
Conferir login e senha	<code>dados.Autenticar</code>	<code>TestAutenticar</code>
Carona válida: 2+ cidades, 1+ assento, data	<code>dados.CadastrarCarona</code>	<code>TestCadastroInvalido</code> , <code>TestOcupadosNasceZerado</code>
Sentido único, data, cidade fora da rota	<code>buscarDiretas</code>	<code>TestBuscaSentidoUnico</code>
Disponibilidade por trecho	<code>livres</code> , <code>ocupar</code>	<code>TestDisponibilidadePorTrecho</code>
Viagem com uma conexão	<code>buscarComConexao</code>	<code>TestBuscaComConexao</code>
Reserva atômica	<code>dados.Reservar</code>	<code>TestReservaAtomica</code> , <code>TestReservaMultiplaFunciona</code>
Nunca vender o mesmo assento duas vezes	<code>Reservar</code> + mutex	<code>TestReservaConcorrente</code> , <code>TestContagemDeAssentos</code> , programa <code>carga</code>
Reserva não paga expira e devolve o assento	<code>dados.ExpirarVencidas</code>	<code>TestExpiracaoDevolveAssento</code> , <code>TestExpiracaoConcorrente</code>
Reserva paga nunca expira	<code>ExpirarVencidas</code> , <code>Pagar</code>	<code>TestReservaPagaNaoExpira</code>
Regras de pagamento	<code>dados.Pagar</code>	<code>TestRegrasDePagamento</code>
Cancelar reserva devolve todos os trechos	<code>dados.CancelarReserva</code>	<code>TestCancelarReservaDevolveAssento</code> , <code>TestRegrasDeCancelarReserva</code> , <code>TestReservaECancelamentoConcorrentes</code>
Cancelar carona derruba conexões inteiras	<code>dados.CancelarCarona</code>	<code>TestCancelarCaronaCancelaConexaoInteira</code> , <code>TestRegrasDeCancelarCarona</code> , <code>TestCancelarCaronaDuranteReservas</code>
Passageiros por trecho, só para o dono	<code>dados.PassageirosDaCarona</code>	<code>TestPassageirosPorTrecho</code>
Cliente que cai não trava o servidor	<code>servidor.atender</code>	<code>TestQuedaAbruptaEncerraAtendimento</code>
JSON inválido não derruba a conexão	<code>servidor.atender</code>	<code>TestJSONInvalidoNaoDerrubaConexao</code>
Tudo junto, pela rede, sob carga	sistema inteiro	programa <code>carga</code>

Parte II — Arquitetura

Como as peças se encaixam: quais programas existem, como os pacotes dependem uns dos outros, o caminho de uma requisição, o formato das mensagens na rede e como o servidor atende muita gente ao mesmo tempo sem corromper nada.

12. Visão geral

O Vaijunto é formado por **programas separados**, que rodam como processos independentes e podem estar em máquinas diferentes. Eles não compartilham memória. A única ligação entre eles é uma conexão TCP, por onde passam linhas de texto.

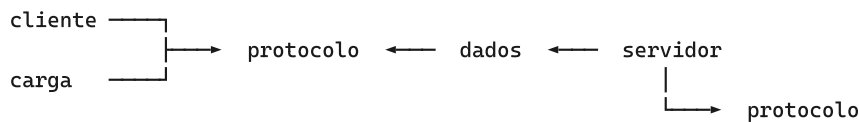


- **Cliente:** menus de terminal. Não tem regra de negócio; só monta pedidos e mostra respostas.
- **Servidor:** um processo só, que aceita conexões e cria uma goroutine para cada uma. Todas elas usam o mesmo `Banco`.
- **Banco:** não é um banco de dados. É uma estrutura em memória com todo o estado, protegida por um mutex e gravada num arquivo JSON.
- **Carga:** um programa de teste que se comporta como centenas de clientes ao mesmo tempo.

13. Os pacotes e quem depende de quem

Em Go, cada pasta é um **pacote**. O projeto tem cinco:

Pacote	Tipo	Responsabilidade	Não sabe nada de
<code>protocolo</code>	biblioteca	O formato das mensagens e as funções de ler e escrever uma linha de JSON	Regras de negócio
<code>dados</code>	biblioteca	O estado, todas as regras de negócio, o mutex e a gravação em arquivo	Rede, sockets, menus
<code>servidor</code>	executável	Aceitar conexões, guardar a sessão, rotear cada ação para o <code>dados</code> , expirar reservas	Como as regras são calculadas
<code>cliente</code>	executável	Menus de terminal para motorista e passageiro	Regras de negócio



A → B significa "A importa B"

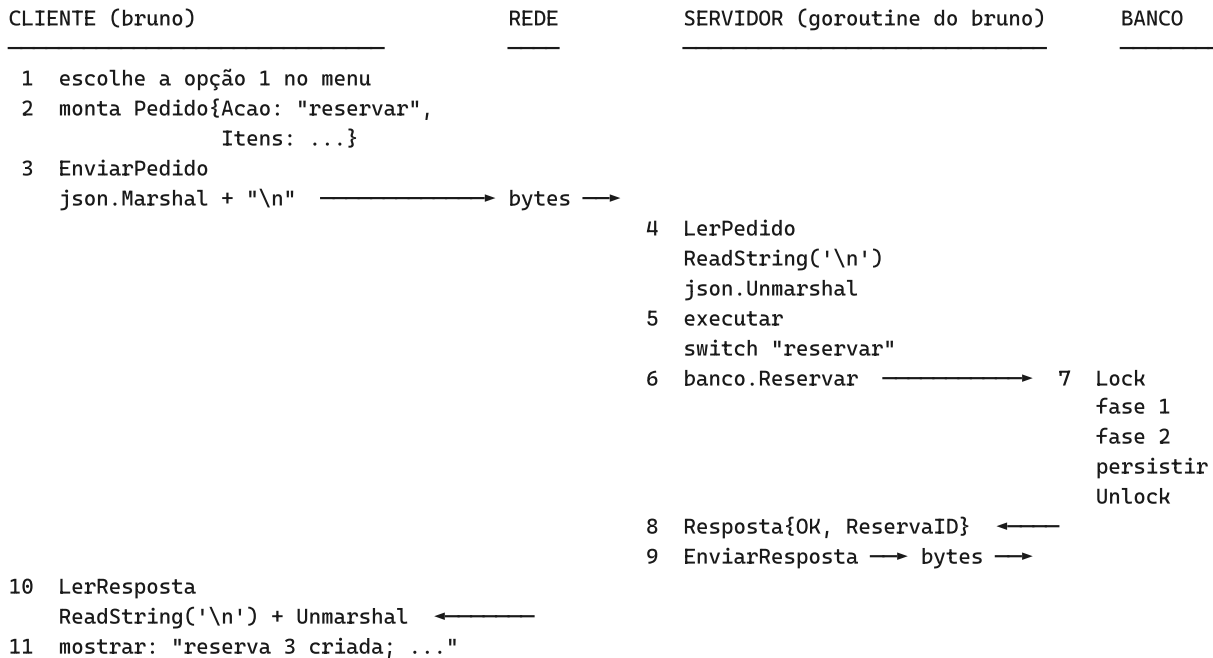
O `dados` importa o `protocolo` porque uma reserva guarda `[]protocolo.Item` e a busca devolve `[]protocolo.Opcao`. O `protocolo`, por sua vez, não importa nenhum pacote do projeto.

EM PYTHON

Em Python, dois módulos podem se importar mutuamente, e o problema só aparece em tempo de execução, como um `ImportError` de importação parcial. Em Go o compilador simplesmente recusa: `import cycle not allowed`. É por isso que `Item` e `Opcao` moram no `protocolo`: se morassem no `dados`, o `protocolo` precisaria importar o `dados`, e haveria um ciclo.

14. O caminho de uma requisição

O que acontece, passo a passo, quando o bruno escolhe uma opção e reserva:



#	Onde	O que acontece
1-2	<code>cliente.buscarEReservar</code>	O cliente já tem as opções da busca; pega os itens da escolhida e monta o pedido.
3	<code>protocolo.EnviarPedido</code>	Converte o pedido em JSON, acrescenta <code>\n</code> e escreve na conexão.
4	<code>protocolo.LerPedido</code>	Na goroutine do bruno, lê bytes até o <code>\n</code> e converte o JSON de volta.

5	<code>servidor.executar</code>	Confere o login e escolhe o <code>case</code> da ação.
6-7	<code>dados.Reservar</code>	Trava o banco, verifica todos os itens, ocupa os assentos, registra a reserva, grava o arquivo e destrava.
8-9	<code>servidor.executar</code> , <code>protocolo.EnviaResposta</code>	Monta a resposta com o número da reserva e a envia como uma linha de JSON.
10-11	<code>cliente.pedir</code> , <code>cliente.mostrar</code>	O cliente, que estava parado esperando, lê a linha e imprime a mensagem.

15. O protocolo de aplicação

Uma linha, uma mensagem

O TCP entrega um **fluxo contínuo de bytes**, sem noção de onde uma mensagem começa ou termina. Duas mensagens podem chegar grudadas numa leitura, e uma mensagem pode chegar partida em duas. Por isso todo protocolo sobre TCP precisa de uma regra de **enquadramento**.

A regra do Vaijunto: **cada mensagem é uma linha de texto terminada em `\n`, e o conteúdo da linha é um JSON**.

Isso funciona porque o JSON gerado pelo Go nunca contém uma quebra de linha de verdade. Se algum texto tiver uma quebra, ela é escrita como os dois caracteres `\` e `n`. Então a única quebra real que passa na rede é a do final de cada mensagem.

POR QUE NÃO PREFIXO DE TAMANHO

A alternativa clássica é mandar, antes de cada mensagem, 4 bytes com o tamanho dela (era a ideia do plano antigo). É mais robusta para dados binários, mas não é legível. Com uma linha de JSON, dá para ver a conversa inteira no terminal e testar o servidor mandando texto à mão, como nas conversas reais mais abaixo. Para um sistema que só troca texto, a linha basta.

Pedido e resposta

- Todo pedido é um `Pedido` e toda resposta é uma `Resposta`. As duas são structs com **todos** os campos possíveis; cada ação usa só os que interessam a ela.
- **Para cada pedido, exatamente uma resposta**, na mesma ordem. O cliente manda e fica esperando.
- A conexão é **com estado**: depois do login, o servidor lembra quem é aquele cliente até a conexão fechar.
- Toda resposta tem `ok`. Se é `false`, o campo `erro` diz por quê.

Todas as ações

Ação	Login	Quem	Campos do pedido	Campos da resposta
<code>ping</code>	não	qualquer	—	<code>mensagem</code> : "pong"
<code>registrar</code>	não	qualquer	<code>usuario</code> , <code>senha</code> , <code>tipo</code>	<code>mensagem</code>
<code>login</code>	não	qualquer	<code>usuario</code> , <code>senha</code>	<code>mensagem</code> , <code>tipo</code>

sair	sim	qualquer	—	mensagem
cadastrar	sim	motorista	rota, data, assentos, preco	mensagem, carona_id
minhas_caronas	sim	qualquer*	—	linhas
passageiros	sim	motorista dono	carona_id	linhas
cancelar_carona	sim	motorista dono	carona_id	mensagem
buscar	sim	qualquer	origem, destino, data	mensagem, opcoes
reservar	sim	qualquer	itens	mensagem, reserva_id
pagar	sim	dono da reserva	reserva_id	mensagem
cancelar_reserva	sim	dono da reserva	reserva_id	mensagem
minhas_reservas	sim	qualquer*	—	linhas

* Lista as do usuário logado; para quem não tem nenhuma, a lista vem vazia. Uma ação que não existe responde `acao desconhecida`. Uma linha que não é JSON responde `mensagem invalida: ...`, e a conexão continua aberta.

Conversas reais

As conversas abaixo foram capturadas rodando o servidor de verdade, com uma ferramenta que manda linhas de JSON cruas pelo socket. Cada sessão é uma conexão nova. Leia na ordem: os números de carona e reserva continuam de uma sessão para a outra.

sessão 1 · ana (motorista): antes e depois do login

```
--> {"acao":"ping"}
<-- {"ok":true,"mensagem":"pong"}
--> {"acao":"minhas_caronas"}
<-- {"ok":false,"erro":"faça login primeiro"}
--> {"acao":"login","usuario":"ana","senha":"errada"}
<-- {"ok":false,"erro":"usuario ou senha invalidos"}
--> {"acao":"login","usuario":"ana","senha":"123"}
<-- {"ok":true,"mensagem":"bem-vindo, ana","tipo":"motorista"}
--> {"acao":"cadastrar","rota":["Feira"],"data":"2026-09-20","assentos":2,"preco":30}
<-- {"ok":false,"erro":"a rota precisa de pelo menos 2 cidades"}
--> {"acao":"cadastrar","rota":["Feira","Salvador","Ilheus"],"data":"2026-09-20","assentos":2,"preco":30}
<-- {"ok":true,"mensagem":"carona 1 cadastrada","carona_id":1}
--> {"acao":"minhas_caronas"}
<-- {"ok":true,"linhas":["carona 1 | 2026-09-20 | Feira -\u003e Salvador -\u003e Ilheus | R$30/trecho | ocupados por trecho: [0 0] de 2"]}
```

Antes do login, só o `ping` passa. O primeiro cadastro é recusado por ter uma cidade só, e esse erro **não gasta número**: a carona válida seguinte é a 1. A resposta do cadastro traz `carona_id`, que o programa de carga usa.

sessão 2 · davi (motorista): uma carona que conecta com a da ana

```
--> {"acao":"login","usuario":"davi","senha":"123"}
<-- {"ok":true,"mensagem":"bem-vindo, davi","tipo":"motorista"}
--> {"acao":"cadastrar","rota":["Ilheus","Porto Seguro"],"data":"2026-09-20","assentos":2,"preco":50}
<-- {"ok":true,"mensagem":"carona 2 cadastrada","carona_id":2}
```

sessão 3 · bruno (passageiro): busca direta, busca com conexão, reserva atômica e pagamento

```
--> {"acao":"login","usuario":"bruno","senha":"123"}
<-- {"ok":true,"mensagem":"bem-vindo, bruno","tipo":"passageiro"}
--> {"acao":"buscar","origem":"Salvador","destino":"Ilheus","data":"2026-09-20"}
<-- {"ok":true,"mensagem":"1 opcao(oes) encontrada(s)","opcoes":[{"itens":
[{"carona_id":1,"de":1,"ate":2}], "resumo":"direto: carona 1 (ana) Salvador -\u003e Ilheus | 2
livre(s)", "preco":30}]}
--> {"acao":"buscar","origem":"Feira","destino":"Porto Seguro","data":"2026-09-20"}
<-- {"ok":true,"mensagem":"1 opcao(oes) encontrada(s)","opcoes":[{"itens":[{"carona_id":1,"de":0,"ate":2},
{"carona_id":2,"de":0,"ate":1}], "resumo":"conexao em Ilheus: carona 1 (ana) + carona 2 (davi)", "preco":110}]}
--> {"acao":"reservar","itens":[{"carona_id":1,"de":0,"ate":2}, {"carona_id":2,"de":0,"ate":1}]}
<-- {"ok":true,"mensagem":"reserva 3 criada; pague em ate 10m0s ou os assentos voltam","reserva_id":3}
--> {"acao":"pagar","reserva_id":3}
<-- {"ok":true,"mensagem":"reserva 3 paga; assentos confirmados"}
--> {"acao":"minhas_reservas"}
<-- {"ok":true,"linhas":["reserva 3 | paga | 2 trecho(s)"]}
```

A primeira busca encontra uma viagem direta: um item, preço 30. A segunda encontra uma conexão: **dois itens**, preço 110. O pedido de reserva devolve exatamente esses dois itens, e o servidor cria a reserva 3 de forma atômica.

sessão 4 · carla (passageira): reserva, cancela e tenta pagar a cancelada

```
--> {"acao":"login","usuario":"carla","senha":"123"}
<-- {"ok":true,"mensagem":"bem-vindo, carla","tipo":"passageiro"}
--> {"acao":"reservar","itens":[{"carona_id":1,"de":1,"ate":2}]}
<-- {"ok":true,"mensagem":"reserva 4 criada; pague em ate 10m0s ou os assentos voltam","reserva_id":4}
--> {"acao":"minhas_reservas"}
<-- {"ok":true,"linhas":["reserva 4 | pendente | 1 trecho(s) | expira em 10m0s"]}
--> {"acao":"cancelar_reserva","reserva_id":4}
<-- {"ok":true,"mensagem":"reserva 4 cancelada; assentos devolvidos"}
--> {"acao":"pagar","reserva_id":4}
<-- {"ok":false,"erro":"esta reserva foi cancelada (cancelada pelo passageiro)"}
--> {"acao":"cadastrar","rota":["Feira","Salvador"],"data":"2026-09-20","assentos":1,"preco":10}
<-- {"ok":false,"erro":"so motorista cadastra carona"}
```

A carla reserva, vê o prazo correndo, cancela e tenta pagar a cancelada. Depois tenta cadastrar carona sendo passageira e é recusada pelo tipo.

sessão 5 · ana: passageiros por trecho e tentativa de cancelar carona alheia

```
--> {"acao":"login","usuario":"ana","senha":"123"}
<-- {"ok":true,"mensagem":"bem-vindo, ana","tipo":"motorista"}
--> {"acao":"passageiros","carona_id":1}
<-- {"ok":true,"linhas":["carona 1 | 2026-09-20 | Feira -\u003e Salvador -\u003e Ilheus"," Feira -\u003e
Salvador [1/2]: bruno (paga)"," Salvador -\u003e Ilheus [1/2]: bruno (paga)"]}
--> {"acao":"cancelar_carona","carona_id":2}
<-- {"ok":false,"erro":"esta carona nao e sua"}
```

Na lista de passageiros, a reserva cancelada da carla não aparece; só reservas ativas contam. A ana não consegue cancelar a carona do davi.

sessão 6 · davi cancela a carona dele (cascata na conexão do bruno)

```
--> {"acao":"login","usuario":"davi","senha":"123"}
<-- {"ok":true,"mensagem":"bem-vindo, davi","tipo":"motorista"}
--> {"acao":"passageiros","carona_id":2}
<-- {"ok":true,"linhas":["carona 2 | 2026-09-20 | Ilheus -\u003e Porto Seguro"," Ilheus -\u003e Porto Seguro
[1/2]: bruno (paga)"]}
--> {"acao":"cancelar_carona","carona_id":2}
<-- {"ok":true,"mensagem":"carona 2 cancelada; 1 reserva(s) desfeita(s) e assentos devolvidos"}
--> {"acao":"minhas_caronas"}
```

```
<-- {"ok":true,"linhas":["carona 2 | 2026-09-20 | Ilheus -\u003e Porto Seguro | R$50/trecho | ocupados por  
trecho: [0] de 2 | CANCELADA"]}]}
```

sessão 7 · bruno depois do cancelamento

```
--> {"acao":"login","usuario":"bruno","senha":"123"}  
<-- {"ok":true,"mensagem":"bem-vindo, bruno","tipo":"passageiro"}  
--> {"acao":"minhas_reservas"}  
<-- {"ok":true,"linhas":["reserva 3 | cancelada | 2 trecho(s) | carona 2 cancelada pelo motorista"]}]  
--> {"acao":"buscar","origem":"Feira","destino":"Porto Seguro","data":"2026-09-20"}  
<-- {"ok":true,"mensagem":"0 opcao(oes) encontrada(s)"}  
--> {"acao":"buscar","origem":"Feira","destino":"Ilheus","data":"2026-09-20"}  
<-- {"ok":true,"mensagem":"1 opcao(oes) encontrada(s)","opcoes":[{"itens":  
[{"carona_id":1,"de":0,"ate":2}], "resumo":"direto: carona 1 (ana) Feira -\u003e Salvador -\u003e Ilheus | 2  
livre(s)","preco":60}]}
```

Aqui está o cancelamento em cascata. O davi cancelou a carona 2; a reserva 3 do bruno, que era uma conexão paga, caiu inteira. Repare na última busca: Feira → Ilhéus mostra de novo **2 livres** na carona 1, porque o assento do bruno também voltou lá.

sessão 8 · erros de protocolo

```
--> isso nao e json  
<-- {"ok":false,"erro":"mensagem invalida: invalid character 'i' looking for beginning of value"}  
--> {"acao":"voar"}  
<-- {"ok":false,"erro":"faca login primeiro"}  
--> {"acao":"login","usuario":"bruno","senha":"123"}  
<-- {"ok":true,"mensagem":"bem-vindo, bruno","tipo":"passageiro"}  
--> {"acao":"voar"}  
<-- {"ok":false,"erro":"acao desconhecida: \"voar\""}  
--> {"acao":"pagar","reserva_id":999}  
<-- {"ok":false,"erro":"reserva 999 nao existe"}
```

Três detalhes que aparecem na rede

1. **->** no lugar de **->**. Por padrão, o `json.Marshal` do Go troca `<`, `>` e `&` por códigos `\u`, para o JSON poder ser colado dentro de HTML sem risco. Não muda o conteúdo: quem lê com `json.Unmarshal` recebe o `->` normal, e é isso que o cliente mostra.
2. **Campos que somem**. Os campos marcados com `omitempty` não aparecem quando estão vazios. Por isso a resposta de um `ping` é só `{"ok":true,"mensagem":"pong"}`. O `ok` não tem essa marca, então `"ok":false` sempre aparece.
3. `"voar"`. A mensagem de ação desconhecida coloca o nome entre aspas; dentro de um texto JSON, aspas precisam ser escritas como `"`.

O que o servidor imprime

Enquanto atendia as conversas acima, o servidor escreveu isto no terminal:

```
sem dados anteriores em C:/Users/Rick/AppData/Local/Temp/claude/c--Users-Rick-Documents-UEFS-5--Semestre-PBL-  
Redes/418c85c4-52ce-4ea1-b3e9-b70854918fd8/scratchpad/guia.json, começando do zero  
reserva expira em 10m0s, varrendo a cada 30s  
servidor ouvindo em :8080  
cliente conectado: [::1]:59058  
recebido de [::1]:59058: acao="ping"  
recebido de [::1]:59058: acao="minhas_caronas"  
recebido de [::1]:59058: acao="login"  
recebido de [::1]:59058: acao="login"  
recebido de [::1]:59058: acao="cadastrar"
```

```

recebido de [::1]:59058: acao="cadastrar"
recebido de [::1]:59058: acao="minhas_caronas"
cliente desconectou: [::1]:59058
cliente conectado: [::1]:59059
recebido de [::1]:59059: acao="login"
recebido de [::1]:59059: acao="cadastrar"
cliente desconectou: [::1]:59059
cliente conectado: [::1]:59060
recebido de [::1]:59060: acao="login"
recebido de [::1]:59060: acao="buscar"
recebido de [::1]:59060: acao="buscar"
recebido de [::1]:59060: acao="reservar"
recebido de [::1]:59060: acao="pagar"
recebido de [::1]:59060: acao="minhas_reservas"
cliente desconectou: [::1]:59060
cliente conectado: [::1]:59061
recebido de [::1]:59061: acao="login"
recebido de [::1]:59061: acao="reservar"
recebido de [::1]:59061: acao="minhas_reservas"
recebido de [::1]:59061: acao="cancelar_reserva"
recebido de [::1]:59061: acao="pagar"
recebido de [::1]:59061: acao="cadastrar"
cliente desconectou: [::1]:59061
cliente conectado: [::1]:59062
recebido de [::1]:59062: acao="login"
recebido de [::1]:59062: acao="passageiros"
recebido de [::1]:59062: acao="cancelar_carona"
cliente desconectou: [::1]:59062
cliente conectado: [::1]:59063
recebido de [::1]:59063: acao="login"
recebido de [::1]:59063: acao="passageiros"
recebido de [::1]:59063: acao="cancelar_carona"
recebido de [::1]:59063: acao="minhas_caronas"
cliente desconectou: [::1]:59063
cliente conectado: [::1]:59064
recebido de [::1]:59064: acao="login"
recebido de [::1]:59064: acao="minhas_reservas"
recebido de [::1]:59064: acao="buscar"
recebido de [::1]:59064: acao="buscar"
cliente desconectou: [::1]:59064
cliente conectado: [::1]:59065
recebido de [::1]:59065: acao="voar"
recebido de [::1]:59065: acao="login"
recebido de [::1]:59065: acao="voar"
recebido de [::1]:59065: acao="pagar"
cliente desconectou: [::1]:59065

```

Cada conexão recebe uma porta diferente do lado do cliente (59058 , 59059 ...), e [::1] é o endereço local em IPv6. `cliente desconectou` é o encerramento normal: o cliente fechou a conexão.

16. Goroutines: quem roda ao mesmo tempo

Programa	Goroutines	O que cada uma faz
<code>servidor</code>	<code>main</code> · uma <code>atender</code> por conexão · uma <code>expirarPeriodicamente</code>	<code>main</code> fica aceitando conexões; cada <code>atender</code> conversa com um cliente; a de expiração acorda a cada 30 s
<code>cliente</code>	só <code>main</code>	Lê o teclado, manda o pedido, espera a resposta
<code>carga</code>	<code>main</code> · uma por cliente simulado	Cada uma conecta, entra e dispara uma reserva

No servidor, o **único dado compartilhado entre goroutines é o `*Banco`**. A sessão de cada cliente é uma variável local da goroutine dele; ninguém mais a enxerga, então ela não precisa de proteção. As funções do `protocolo` não guardam estado nenhum.

O CRITÉRIO

A pergunta que decide se algo precisa de mutex é sempre a mesma: **mais de uma goroutine pode ler e escrever isto ao mesmo tempo?** O `Banco`, sim. A sessão, não. Uma variável local, nunca.

17. Um mutex para todo o estado

O `Banco` tem um único `sync.Mutex`. Um mutex é uma trava: quem chama `Lock()` entra, e qualquer outra goroutine que chamar `Lock()` fica parada esperando até alguém chamar `Unlock()`.

A regra de ouro

Toda função pública do `dados` trava no começo e destrava no fim. Nenhuma função interna trava.

A regra existe porque o `sync.Mutex` **não é reentrante**: se uma goroutine que já está com a trava chama `Lock()` de novo, ela espera por si mesma para sempre. Se `Reservar`, já travado, chamasse `Buscar`, que também trava, o servidor congelaria.

PYTHON

```
trava = threading.Lock()

def buscar():
    with trava:
        ...

def reservar():
    with trava:
        buscar() # espera por si mesmo para
                sempre
```

GO

```
func (b *Banco) Buscar() {
    b.mu.Lock()
    defer b.mu.Unlock()
}

func (b *Banco) Reservar() {
    b.mu.Lock()
    defer b.mu.Unlock()
    b.Buscar() // espera por si mesmo para
              sempre
}
```

O `threading.Lock` do Python tem o mesmo comportamento; só o `threading.RLock` permite travar de novo. Por isso as funções internas, como `buscarDiretas` e `livres`, assumem que quem as chamou já está com a trava.

Por que um mutex só

A alternativa seria um mutex por carona. Parece mais eficiente, porque reservas em caronas diferentes não esperaríamos umas pelas outras. O problema é a reserva com conexão, que precisa de **duas** caronas ao mesmo tempo:

```

goroutine A (reserva carona 1 + carona 2)      goroutine B (reserva carona 2 + carona 1)

trava a carona 1                               trava a carona 2
quer a carona 2 — espera B soltar ————— quer a carona 1 — espera A soltar

                               ninguém solta nunca: DEADLOCK

```

Com um mutex por carona, seria preciso travar sempre na mesma ordem (por exemplo, do menor número para o maior) e tomar cuidado com cancelamentos, que mexem em várias caronas. Com um mutex só, esse problema não existe: **a atomicidade entre caronas vem de graça**, porque tudo acontece dentro de uma única trava.

O preço

Com um mutex só, todas as operações entram **uma de cada vez**. O programa de carga mediu o efeito:

Clientes disputando	Lugares	Latência média	p95	Máxima
200	10	19 ms	21 ms	22 ms
500	50	81 ms	91 ms	94 ms
1000	50	437 ms	474 ms	480 ms

A latência cresce mais rápido que o número de clientes. A explicação mais provável, não medida separadamente: as reservas fazem fila no mutex, e cada reserva aceita regravava o JSON inteiro de dentro da seção crítica. As três rodadas usaram o mesmo servidor, então o arquivo já tinha cerca de 1700 contas na última. É o custo consciente de escolher simplicidade e correção. Mesmo com 1000 clientes, nenhum lugar foi vendido duas vezes.

O que acontece sem o mutex

A linha `c.Ocupados[t]++` parece uma operação só, mas são três: ler o valor, somar um, escrever. Duas goroutines podem se intercalar no meio:

Momento	Goroutine A	Goroutine B
1	lê <code>Ocupados[0]</code> → 0	
2		lê <code>Ocupados[0]</code> → 0
3	escreve <code>0 + 1 = 1</code>	
4		escreve <code>0 + 1 = 1</code>
Duas reservas feitas, contador marca 1. Um incremento foi perdido, e o sistema acha que ainda tem lugar.		

No Python acontece o mesmo com `x += 1` entre threads: o GIL pode trocar de thread entre a leitura e a escrita. Tirando o mutex de propósito, os testes mediram exatamente isso:

- sem trava no `Reservar`: 2000 reservas e `Ocupados` terminou em **1937**;
- sem trava no `CancelarReserva`: tudo cancelado, mas sobraram **11, 6 e 13** assentos ocupados em três rodadas;
- sem trava no `CadastrarUsuario`: o próprio Go derrubou o programa com `fatal error: concurrent map read and map write`.

Validação fora, gravação dentro

Duas escolhas completam o desenho. **Conferir argumentos** (rota com uma cidade, senha vazia) acontece **antes** de travar: não depende do estado e não há motivo para fazer todo mundo esperar. **Gravar o arquivo** acontece **dentro** da trava: assim o JSON nunca é escrito com o estado pela metade de uma operação.

18. Resiliência a falhas

Cliente que fecha normalmente

Quando o cliente encerra a conexão, a leitura do servidor devolve `io.EOF` ("fim do arquivo"). A goroutine daquele cliente imprime `cliente desconectou` e termina.

Cliente que cai

Quando o processo do cliente morre ou a rede cai, a conexão termina com um **RST**, e a leitura devolve outro erro: `connection reset` (no Windows, `wsarecv: An existing connection was forcibly closed by the remote host`).

Uma versão anterior do servidor só encerrava no `EOF` e tratava qualquer outro erro como JSON malformado: respondia e voltava a ler. Numa queda abrupta, a leitura falhava de novo na hora, e a goroutine **girava para sempre**. Medido no servidor real, o tempo de CPU foi de 1,14 s para 6,28 s em cinco segundos: um núcleo inteiro ocupado por um cliente que nem existia mais.

A correção separa os dois casos com um erro-sentinela, `protocolo.ErrMensagemInvalida`: só esse erro faz o servidor continuar lendo; qualquer outro encerra a goroutine. Depois da correção, o tempo de CPU ficou parado em 0,08 s, e o servidor passou a registrar `conexao com ... caiu`. Um teste automatizado reproduz a queda com RST e falha se a goroutine não terminar em 2 segundos.

Mensagem malformada

Uma linha que chega inteira mas não é JSON válido recebe `mensagem invalida: ...`, e a conexão continua atendendo normalmente.

Servidor reiniciado

Ao subir, o servidor relê o arquivo JSON. Contas, caronas e reservas voltam como estavam; reservas pendentes continuam contando o prazo pelo horário gravado.

Porta ocupada

Se já houver outro programa na porta 8080 (inclusive outro servidor esquecido aberto), o servidor imprime `erro ao abrir a porta: ... Only one usage of each socket address ...` e encerra.

O que não é tratado

Arquivo corrompido por queda no meio da gravação, rajadas de conexões novas acima do que o sistema operacional enfileira, e alguns outros casos. Estão todos na Parte VII.

Parte III — Go para quem vem de Python

Cada construção de Go que aparece no Vaijunto, explicada a partir do que você já sabe de Python. Não é um curso de Go: é exatamente o necessário para ler a Parte IV sem travar em nenhuma linha.

19. Como um programa Go é organizado

Python roda o código-fonte com um interpretador. Go é **compilado**: o compilador lê todos os arquivos, confere os tipos e gera um executável que roda sem precisar do Go instalado.

Módulo, pacote e arquivo

Go	O que é	Equivalente aproximado em Python
módulo	O projeto inteiro, definido pelo <code>go.mod</code>	O pacote instalável, com <code>pyproject.toml</code>
pacote	Uma pasta. Todos os <code>.go</code> da pasta formam um pacote só	Um pacote (pasta com <code>__init__.py</code>)
arquivo	Só divide o código; não cria escopo próprio	Um módulo <code>.py</code> (que cria escopo próprio)
<code>package main</code>	Pacote que vira executável; começa pela função <code>main</code>	<code>if __name__ == "__main__":</code>

Uma diferença importante: em Python, cada arquivo é um módulo com seus próprios nomes. Em Go, **todos os arquivos de uma pasta enxergam os nomes uns dos outros**. É por isso que `cancelamento_test.go` usa a função `trecho` definida em `dados_test.go` sem importar nada.

O go.mod

pbl/go.mod · linhas 1–3

```
1 module vaijunto
2
3 go 1.27
```

1 `module vaijunto`

Dá nome ao módulo. É o começo de todo caminho de import interno: a pasta `pbl/dados` é importada como `"vaijunto/dados"`. Não precisa ser um endereço de internet; seria, se o projeto fosse publicado.

3 `go 1.27`

Versão mínima da linguagem. Além de exigir o compilador 1.27 ou mais novo, ela ativa os comportamentos dessa versão, como o laço `for` criar uma variável nova a cada volta (explicado adiante). Não há linhas `require` porque o projeto **não usa nenhuma biblioteca de terceiros**.

PYTHON

```
from vaijunto import dados

banco = dados.NovoBanco()
```

GO

```
import "vaijunto/dados"

banco := dados.NovoBanco()
```

Em Go, o que você usa de outro pacote sempre vem com o nome do pacote na frente: `dados.NovoBanco`, `fmt.Println`, `strings.Split`. Não existe `from x import *`.

Os comandos

Comando	O que faz
<code>go run ./servidor</code>	Compila e executa na hora, sem deixar arquivo. Parecido com <code>python servidor.py</code> .
<code>go build ./servidor</code>	Gera o executável <code>servidor.exe</code> .
<code>go test ./...</code>	Roda os testes de todos os pacotes. <code>./...</code> quer dizer "esta pasta e todas as de baixo".
<code>go vet ./...</code>	Procura erros que compilam mas estão errados, como copiar um mutex.
<code>gofmt -w .</code>	Formata o código no estilo oficial. Não há discussão de estilo em Go.

O COMPILADOR É RIGOROSO

Em Go, **importar um pacote e não usar é erro de compilação**, e declarar uma variável e não usar também. Em Python isso passa despercebido para sempre.

20. Variáveis, tipos e valores zero

PYTHON

```
assentos = 2
assentos = "dois"    # permitido

nome: str = "ana"    # só uma dica
vazio = None
```

GO

```
assentos := 2
assentos = "dois"    // erro de compilação

var nome string = "ana"
var vazio string     // vale ""
                    // automaticamente
```

- `:=` **cria** a variável e deduz o tipo pelo valor. Só pode ser usado dentro de funções.
- `=` **atribui** a uma variável que já existe.
- `var nome tipo` cria a variável dizendo o tipo. Sem valor inicial, ela recebe o **valor zero** do tipo.
- O tipo de uma variável **nunca muda**, e o compilador confere isso antes de rodar.

Valores zero, o "None" do Go

Em Go, **toda variável sempre tem um valor válido**. Não existe variável "não inicializada":

Tipo	Valor zero	Onde aparece no Vaijunto
<code>int</code>	<code>0</code>	Contadores de <code>Ocupados</code> nascem 0
<code>string</code>	<code>""</code>	Sessão sem login: <code>s.usuario.Login == ""</code>
<code>bool</code>	<code>false</code>	Carona nasce com <code>Cancelada == false</code>
ponteiro, slice, map, canal, <code>error</code>	<code>nil</code>	Função que não achou nada devolve ponteiro <code>nil</code>
struct	cada campo no seu zero	<code>var s sessao</code> cria uma sessão vazia

`nil` é o parente mais próximo do `None`, mas só existe para ponteiros, slices, maps, canais, funções e interfaces. Um `int` ou uma `string` nunca são `nil`.

Constantes

`const intervaloPadrao = 30 * time.Second` cria um valor que não pode mudar. Python não tem constante de verdade; a convenção é escrever `INTERVALO_PADRAO` em maiúsculas.

21. Funções e múltiplos retornos

PYTHON

```
def autenticar(login: str, senha: str) ->
    tuple[Usuario, bool]:
    ...
    return usuario, True

u, ok = autenticar("ana", "123")
_, ok = autenticar("ana", "123")
```

GO

```
func Autenticar(login, senha string) (Usuario,
bool) {
    ...
    return u, true
}

u, ok := Autenticar("ana", "123")
_, ok = Autenticar("ana", "123")
```

- O tipo vem depois do nome: `login string`.
- Parâmetros seguidos do mesmo tipo podem dividir a declaração: `login, senha string`.
- O tipo de retorno vem depois dos parênteses. Com mais de um retorno, eles vão entre parênteses: `(Usuario, bool)`.
- Retornar vários valores é nativo: não é uma tupla, são valores separados.
- `_` descarta um valor. Como variável não usada é erro, o `_` é o jeito de dizer "sei que existe e não quero".

Funções como valores

Funções podem ser guardadas em variáveis e criadas sem nome, como o `lambda` do Python, mas com corpo completo:

```
percentil := func(p float64) time.Duration {
    return l[int(p*float64(len(l)-1))]
}
```

Uma função sem nome chamada na hora, `func() { ... }()`, aparece muito com `go` e `defer`.

22. Erros: valores, não exceções

A maior mudança de mentalidade. Em Go **não há** `try/except`. Uma função que pode falhar devolve um `error` como último retorno, e quem chama confere na linha seguinte.

PYTHON

```
class MensagemInvalida(Exception):
    pass

try:
    pedido = ler_pedido(leitor)
except MensagemInvalida as e:
    responder_erro(e)
except ConnectionError:
    return
```

GO

```
var ErrMensagemInvalida = errors.New("mensagem
invalida")

pedido, err := protocolo.LerPedido(leitor)
if errors.Is(err,
protocolo.ErrMensagemInvalida) {
    // responde o erro e continua
}
if err != nil {
    return
}
```

Go	O que faz
<code>error</code>	Um tipo que representa um erro. <code>nil</code> quer dizer "deu certo".
<code>if err != nil { ... }</code>	O padrão mais repetido do Go: se houve erro, trata ali mesmo.
<code>errors.New("texto")</code>	Cria um erro com uma mensagem fixa. Como <code>raise ValueError("texto")</code> , só que devolvido em vez de lançado.
<code>fmt.Errorf("carona %d nao existe", id)</code>	Cria um erro com a mensagem formatada.
<code>fmt.Errorf("%w: %v", ErrX, err)</code>	<code>%w</code> embrulha um erro dentro de outro, sem perder a identidade dele.
<code>errors.Is(err, ErrX)</code>	Confere se <code>err</code> é <code>ErrX</code> ou tem <code>ErrX</code> embrulhado dentro. É o <code>except ErrX</code> .
<code>err.Error()</code>	O texto do erro. Como <code>str(e)</code> .

ERRO-SENTINELA

Uma variável de pacote que guarda um erro fixo, como `ErrMensagemInvalida`, é chamada de **sentinela**. Ela faz o papel de uma classe de exceção: quem recebe um erro pergunta, com `errors.Is`, se é aquele.

23. Structs, métodos e ponteiros

Struct

PYTHON

```
@dataclass
class Carona:
    id: int
    rota: list[str]
    assentos: int

c = Carona(id=1, rota=["Feira", "Salvador"],
assentos=2)
print(c.assentos)
```

GO

```
type Carona struct {
    ID      int
    Rota    []string
    Assentos int
}

c := Carona{ID: 1, Rota: []string{"Feira",
"Salvador"}, Assentos: 2}
fmt.Println(c.Assentos)
```

Uma struct é só dados, como uma dataclass. `Carona{ID: 1, ...}` é um **literal composto**: cria o valor dizendo os campos. Campos não informados ficam no valor zero.

Métodos

PYTHON

```
class Carona:
    def trechos(self) -> int:
        return len(self.rotas) - 1
```

GO

```
func (c Carona) trechos() int {
    return len(c.Rotas) - 1
}
```

Não existe `class`. Um método é uma função com um **receptor** entre parênteses antes do nome: `(c Carona)`. O receptor faz o papel do `self`, só que com o nome que você escolher, e declarado fora da struct.

Ponteiros: a maior diferença

Em Python, **todo objeto é passado por referência**: se uma função altera o objeto recebido, quem chamou vê a alteração. Em Go, **uma struct é copiada** ao ser passada, atribuída ou usada como receptor comum.

PYTHON

```
def zerar(c):
    c.assentos = 0 # altera o original

zerar(carona) # carona.assentos agora é 0
```

GO

```
func zerarCopia(c Carona) { c.Assentos = 0 }
// mexe numa cópia
func zerar(c *Carona)      { c.Assentos = 0 }
// mexe no original

zerarCopia(carona) // carona NÃO mudou
zerar(&carona)     // carona mudou
```

Sintaxe	Significado
<code>*Carona</code>	O tipo "ponteiro para Carona": um endereço onde uma Carona mora.
<code>&carona</code>	O endereço da variável <code>carona</code> .
<code>p.Assentos</code>	Mesmo com <code>p</code> sendo ponteiro, o Go segue o endereço sozinho. Não existe <code>-></code> como em C.
<code>func (c *Carona) ocupar()</code>	Método com receptor ponteiro: pode alterar a carona.
<code>func (c Carona) livres()</code>	Método com receptor valor: recebe uma cópia; serve para ler.

CÓPIA DE STRUCT NÃO COPIA O CONTEÚDO DOS SLICES

Copiar uma struct copia os campos, mas um campo slice é só uma "janela" para um vetor em outro lugar. A cópia recebe uma janela nova para o **mesmo** vetor. Então, numa cópia de carona, `copia.Assentos = 0` não afeta o original, mas `copia.Ocupados[0]++` **afeta**. O Vajunto evita depender disso: tudo que altera usa receptor ponteiro.

Por que `*Banco` e nunca `Banco`

O `Banco` contém um `sync.Mutex`. Uma cópia do `Banco` teria uma cópia da trava, e travar a cópia não protegeria o original. Por isso `NovoBanco` devolve `*Banco`, todos os métodos usam `(b *Banco)`, e o `go vet`

reclama se alguém tentar copiar.

24. Slices e maps

PYTHON

```
rota = ["Feira", "Salvador", "Ilheus"]
rota.append("Porto Seguro")
parte = rota[0:2]
ocupados = [0] * 3
len(rota)

usuarios = {"ana": u}
if "ana" in usuarios: ...
u = usuarios.get("ana")
```

GO

```
rota := []string{"Feira", "Salvador",
"Ilheus"}
rota = append(rota, "Porto Seguro")
parte := rota[0:2]
ocupados := make([]int, 3)
len(rota)

usuarios := map[string]Usuario{"ana": u}
u, existe := usuarios["ana"]
```

Slice: a lista do Go

- `[]string` é um slice de strings. Todos os elementos têm o mesmo tipo.
- `make([]int, 3)` cria um slice com 3 zeros. É como `[0] * 3`.
- `append(s, x)` **devolve** o slice novo, que precisa ser guardado: `s = append(s, x)`. Esquecer a atribuição é um erro comum.
- `s[a:b]` funciona como no Python (inclui `a`, exclui `b`), mas **não copia**: é uma janela para o mesmo vetor.
- Índice fora do tamanho derruba o programa (`panic`), como o `IndexError`. Não existe índice negativo.

PONTEIRO PARA DENTRO DE UM SLICE

`&b.Caronas[i]` é o endereço de um elemento do slice. Se depois alguém fizer `append` e o slice precisar crescer, o Go copia tudo para um vetor novo, e aquele endereço passa a apontar para o vetor **velho**. O Vajunto só pega esses ponteiros dentro de uma operação que não faz `append` no mesmo slice, e nunca os guarda para depois.

Map: o dict do Go

- `map[string]Usuario`: chaves string, valores `Usuario`.
- `u, existe := m["ana"]`: se a chave não existe, `u` vem com o valor zero e `existe` vem `false`. Nunca dá `KeyError`.
- `m["ana"] = u` grava.
- A ordem de percorrer um map é **sorteada** a cada execução. Não dependa dela.
- **Map não aguenta escrita simultânea**: duas goroutines escrevendo derrubam o programa com `fatal error: concurrent map writes`. No Python, o GIL protege operações simples de dict; em Go, quem protege é você.

25. Laços com for

Go tem uma única palavra de laço, `for`, que faz o papel de `for` e de `while`:

PYTHON

```
for i, cidade in enumerate(rota): ...
for t in range(de, ate): ...
while True: ...
for linha in linhas: ...
```

GO

```
for i, cidade := range rota { }
for t := de; t < ate; t++ { }
for { }
for _, linha := range linhas { }
```

- `range` sobre um slice entrega **índice e valor**, como `enumerate`. Use `_` para ignorar um deles, ou `for i := range s` para só o índice.
- `for inicio; condicao; passo { }` é o laço com contador; `t++` soma um.
- `for { }` é o laço infinito.
- `range` sobre um canal lê valores até o canal ser fechado.
- `continue` e `break` funcionam como no Python.

A armadilha do range

PYTHON

```
for c in caronas:
    c.assentos = 0 # altera cada carona
```

GO

```
for _, c := range caronas {
    c.Assentos = 0 // altera uma CÓPIA: nada muda
}

for i := range caronas {
    caronas[i].Assentos = 0 // altera o original
}
```

Em Python, a variável do laço é uma referência para o objeto da lista. Em Go, `c` recebe uma **cópia** de cada struct. Para alterar, é preciso usar o índice. É exatamente por isso que `acharCarona` e `ExpirarVencidas` usam `for i := range` e `&b.Reservas[i]`.

UMA VARIÁVEL NOVA A CADA VOLTA

Desde o Go 1.22 (e o projeto usa 1.27), cada volta do laço cria uma variável nova. Uma goroutine criada dentro do laço que usa `i` vê o `i` daquela volta. Em versões antigas, todas viam a mesma variável, e o programa de carga passa o número como argumento, `go func(n int) { ... }(i)`, que funciona em qualquer versão.

26. if e switch

if com declaração

```
if u, existe := b.Usuarios[login]; !existe {
    return Usuario{}, false
}
```

O `if` não usa parênteses, sempre usa chaves, e pode ter uma declaração antes da condição, separada por `;`. A variável criada ali só existe dentro do `if`. O Python 3.8 tem algo parecido com o `:= (if (n := len(x)) > 0:)`.

Não existe operador ternário. `!` é o `not`, `&&` é o `and`, `||` é o `or`.

switch

PYTHON

```
match acao:
    case "ping":
        ...
    case "0" | "":
        return
    case _:
        print("opcao invalida")
```

GO

```
switch acao {
case "ping":
    ...
case "0", "":
    return
default:
    fmt.Println("opcao invalida")
}
```

- Cada `case` termina sozinho: não precisa de `break`, e não "cai" no próximo.
- Vários valores num `case` vão separados por vírgula.
- `default` é o `case _`.
- `switch { case cond1: ... case cond2: ... }`, sem nada depois do `switch`, é uma cadeia de `if/elif`.

27. Strings e formatação

Formatar texto

PYTHON

```
f"carona {id} cadastrada"
f"{nome!r}"
print(f"recebido: {acao}")
```

GO

```
fmt.Sprintf("carona %d cadastrada", id)
fmt.Sprintf("%q", nome)
fmt.Printf("recebido: %s\n", acao)
```

Verbo	Formata	Exemplo de saída
<code>%d</code>	número inteiro	<code>3</code>
<code>%s</code>	texto (ou algo que sabe virar texto, como uma duração)	<code>ana</code> , <code>10m0s</code>
<code>%q</code>	texto entre aspas, com escapes	<code>"voar"</code>
<code>%v</code>	qualquer valor, no formato padrão	<code>[1 2]</code> para um <code>[]int</code>
<code>%w</code>	só em <code>fmt.Errorf</code> : embrulha um erro	—

Função	Faz
<code>fmt.Println(a, b)</code>	Imprime com espaço entre os valores e quebra de linha no fim. Como <code>print(a, b)</code> .
<code>fmt.Print(x)</code>	Imprime sem quebra de linha. Como <code>print(x, end="")</code> .
<code>fmt.Printf(formato, ...)</code>	Imprime formatado; a quebra de linha precisa estar no formato.
<code>fmt.Sprintf(formato, ...)</code>	Devolve o texto formatado em vez de imprimir.
<code>fmt.Fprint(destino, ...)</code>	Escreve num destino qualquer, como uma conexão.

Funções de texto

Go	Python	Uso no projeto
<code>strings.TrimSpace(s)</code>	<code>s.strip()</code>	Tirar o <code>\r\n</code> do Enter
<code>strings.Split(s, ",")</code>	<code>s.split(",")</code>	Ler a rota digitada
<code>strings.Join(lista, " -> ")</code>	<code>" -> ".join(lista)</code>	Mostrar a rota
<code>strings.Contains(s, "x")</code>	<code>"x" in s</code>	Testes
<code>strings.ToLower(s)</code>	<code>s.lower()</code>	Confirmação s/n
<code>strconv.Atoi(s)</code>	<code>int(s)</code>	Ler números do teclado; devolve erro em vez de lançar
<code>strconv.FormatInt(n, 36)</code>	<code>numpy.base_repr(n, 36)</code>	Sufixo único no teste de carga

Aspas duplas fazem uma `string`. Aspas simples fazem um único caractere (`'\n'`), que é um tipo diferente. Crases fazem um texto cru, sem escapes, usado nas tags de JSON.

28. Visibilidade e documentação

Em Python, `_nome` é uma **convenção** de "privado", que ninguém impede de usar. Em Go, **a primeira letra decide**, e o compilador obriga:

Nome	Visível de onde
<code>Reservar</code> , <code>Carona</code> , <code>ID</code>	Maiúscula: exportado , visível de outros pacotes.
<code>livres</code> , <code>mu</code> , <code>arquivo</code>	Minúscula: só dentro do próprio pacote.

No Vaijunto isso tem dois efeitos práticos. A regra de ouro do mutex coincide com a visibilidade: **maiúscula trava, minúscula não**. E o `encoding/json` só enxerga campos com maiúscula, então `mu` e `arquivo` nunca vão parar no arquivo JSON.

Um comentário logo acima de uma declaração é a **documentação** dela, como uma docstring. Por convenção começa com o nome: `// Reservar segura assentos...`.

29. defer

PYTHON

```
with trava:
    ...                # solta ao sair do
    bloco

conexao = aceitar()
try:
    atender(conexao)
finally:
    conexao.close()
```

GO

```
b.mu.Lock()
defer b.mu.Unlock() // solta ao sair da função

conexao, _ := ouvinte.Accept()
defer conexao.Close()
atender(conexao)
```

- `defer` agenda uma chamada para **quando a função terminar**, seja por `return`, por chegar ao fim ou por `panic`.
- Vários `defer` rodam na ordem inversa: o último agendado roda primeiro.
- A vantagem prática: o `Unlock` fica escrito logo abaixo do `Lock`. Uma função com cinco `return` diferentes nunca esquece de destravar.
- Diferente do `with`, o `defer` vale para a **função inteira**, não para um bloco.

30. Goroutines

PYTHON

```
t = threading.Thread(target=atender, args=
(conexao,))
t.start()
```

GO

```
go atender(conexao, banco)
```

- `go f(x)` começa a executar `f(x)` em paralelo e segue para a linha seguinte **sem esperar**.
- Uma goroutine custa poucos kilobytes; um servidor com dezenas de milhares delas é normal. Uma thread do sistema custa bem mais.
- **Não há GIL.** Goroutines rodam de verdade ao mesmo tempo em vários núcleos. Com 12 núcleos, até 12 goroutines executam no mesmo instante. Isso é ótimo para desempenho e é exatamente o que torna as corridas reais.
- Não existe `join` numa goroutine; para esperar, usa-se `sync.WaitGroup` ou canais.

QUANDO O MAIN TERMINA, TUDO TERMINA

Em Python, o programa espera as threads normais terminarem antes de sair. Em Go, **quando a função `main` retorna, o processo acaba na hora**, e todas as goroutines morrem junto. O servidor nunca sai do laço de `Accept`, e os testes esperam as goroutines com `WaitGroup` antes de conferir o resultado.

31. Canais

Um canal é um tubo por onde goroutines trocam valores com segurança, sem mutex. O parente mais próximo no Python é a `queue.Queue`.

PYTHON

```
fila = queue.Queue(maxsize=10)
fila.put(1)
x = fila.get()
fila.qsize()
```

GO

```
fila := make(chan int, 10)
fila <- 1
x := <-fila
len(fila)
```

Go	O que faz
<code>make(chan int, 10)</code>	Canal com espaço para 10 valores. Enviar só bloqueia quando está cheio.
<code>make(chan struct{})</code>	Canal sem espaço, de um tipo que não ocupa memória. Usado só para sinalizar.
<code>c <- v</code>	Envia <code>v</code> .
<code>v := <-c</code> ou só <code><-c</code>	Recebe; espera se não houver nada.
<code>close(c)</code>	Fecha o canal: não aceita mais envios.
<code>for v := range c</code>	Lê até o canal ser fechado e esvaziado.
<code>len(c)</code>	Quantos valores estão esperando no canal agora.

Dois truques usados no projeto

A largada. Receber de um canal **fechado** não espera: devolve na hora. Então centenas de goroutines podem ficar paradas em `<-largada`, e um único `close(largada)` solta todas ao mesmo tempo. É o `threading.Event` do Python: `evento.wait()` em todas e um `evento.set()`.

O semáforo. Um canal com espaço para 50 limita quantas goroutines fazem algo ao mesmo tempo: cada uma envia antes de começar (se estiver cheio, espera) e recebe ao terminar, liberando a vaga. É o `threading.Semaphore(50)`. O programa de carga usa isso para não abrir conexões demais de uma vez.

select

```
select {
case <-terminou:
    // a goroutine terminou
case <-time.After(2 * time.Second):
    // passaram 2 segundos
}
```

`select` espera o **primeiro** de vários canais ficar pronto. `time.After` devolve um canal que recebe um valor depois do tempo indicado. Juntos, fazem "espere isto, mas no máximo 2 segundos", como um `asyncio.wait_for` com timeout. Aparece no teste de queda abrupta.

32. sync.Mutex e sync.WaitGroup

PYTHON

```
trava = threading.Lock()
with trava:
    contador += 1

threads = [Thread(target=f) for _ in
range(50)]
for t in threads: t.start()
for t in threads: t.join()
```

GO

```
var mu sync.Mutex
mu.Lock()
contador++
mu.Unlock()

var wg sync.WaitGroup
for i := 0; i < 50; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        f()
    }()
}
wg.Wait()
```

- `sync.Mutex` não precisa ser criado: o valor zero já é uma trava destravada, pronta para uso.
- `WaitGroup` é um contador: `Add(1)` antes de criar cada goroutine, `Done()` quando ela termina, `Wait()` espera o contador voltar a zero.
- `Add` vem **antes** do `go`, nunca dentro da goroutine; senão o `Wait` poderia rodar antes de a goroutine ter contado.
- Nenhum dos dois pode ser copiado depois de usado.

33. Tempo

Go	Python	Uso no projeto
<code>time.Duration</code>	<code>timedelta</code>	Prazos
<code>10 * time.Minute</code>	<code>timedelta(minutes=10)</code>	Prazo da reserva
<code>time.Now()</code>	<code>datetime.now()</code>	Criar e conferir prazos
<code>t.Add(d)</code>	<code>t + d</code>	Calcular o vencimento
<code>agora.Before(t)</code>	<code>agora < t</code>	Ainda não venceu?
<code>time.Until(t)</code>	<code>t - datetime.now()</code>	"expira em 9m46s"
<code>time.Since(t)</code>	<code>datetime.now() - t</code>	Medir latência
<code>d.Round(time.Second)</code>	arredondar manualmente	Mostrar tempo sem frações
<code>time.ParseDuration("30s")</code>	sem equivalente direto	Ler variável de ambiente
<code>time.NewTicker(d)</code>	<code>while True: sleep(d)</code>	Varredura de expiração
<code>time.Sleep(d)</code>	<code>time.sleep(s)</code>	Testes

Um `Duration` é um número inteiro de nanossegundos. Escrito com `%s`, vira texto legível: `10m0s`, `30s`.

34. JSON

PYTHON

```
texto = json.dumps({"acao": "ping"})

dados = json.loads(linha)
usuario = dados.get("usuario", "")
```

GO

```
type Pedido struct {
    Acao    string `json:"acao"`
    Usuario string `json:"usuario,omitempty"`
}

bytes, err := json.Marshal(Pedido{Acao:
    "ping"})

var p Pedido
err = json.Unmarshal([]byte(linha), &p)
```

- Em Go o JSON vira uma **struct com tipos fixos**, não um dicionário solto. O texto entre crases depois de cada campo é uma **tag**, que diz o nome do campo no JSON.
- `omitempty`: se o campo estiver com o valor zero, ele não aparece no JSON gerado.
- `json.Marshal` devolve `[]byte` (bytes crus) e um erro. `MarshalIndent` faz o mesmo com indentação, para o arquivo ficar legível.
- `json.Unmarshal` recebe o **endereço** da variável (`&p`), porque precisa preenchê-la.
- Campos que faltam no JSON ficam no valor zero; campos que sobram são ignorados.
- **Só campos exportados** (maiúscula) entram no JSON.
- `[]byte(linha)` converte texto em bytes. Em Go, conversões de tipo são sempre explícitas.

35. Rede e interfaces

PYTHON

```
srv = socket.create_server(("", 8080))
while True:
    conn, addr = srv.accept()
    Thread(target=atender, args=
(conn,)).start()

arquivo = conn.makefile("r")
linha = arquivo.readline()
```

GO

```
ouvinte, err := net.Listen("tcp", ":8080")
for {
    conexao, err := ouvinte.Accept()
    go atender(conexao, banco)
}

leitor := bufio.NewReader(conexao)
linha, err := leitor.ReadString('\n')
```

Go	O que faz
<code>net.Listen("tcp", ":8080")</code>	Abre a porta 8080 em todos os endereços da máquina.
<code>ouvinte.Accept()</code>	Espera o próximo cliente e devolve a conexão com ele.
<code>net.Dial("tcp", "localhost:8080")</code>	Conecta num servidor. Como <code>socket.create_connection</code> .
<code>net.Conn</code>	Uma conexão: dá para ler, escrever e fechar.
<code>bufio.NewReader(conexao)</code>	Põe um buffer na leitura, o que permite ler "até o <code>\n</code> ".
<code>leitor.ReadString('\n')</code>	Lê até achar <code>\n</code> e devolve o texto com o <code>\n</code> no fim.

Interfaces: duck typing conferido pelo compilador

Em Python, se um objeto tem `write`, ele serve onde se espera um arquivo, e isso só é descoberto na hora de rodar. Go tem a mesma ideia, mas conferida na compilação: uma **interface** é uma lista de métodos, e **qualquer tipo que tenha esses métodos** satisfaz a interface automaticamente, sem declarar nada.

`io.Writer` é a interface "tem um método `Write`". Por isso `EnviarResposta(destino io.Writer, ...)` aceita uma conexão de rede, e aceitaria também `os.Stdout` ou um arquivo. `any` é a interface vazia: aceita qualquer valor.

36. Sistema, argumentos e ordenação

Go	Python	Uso no projeto
<code>os.Getenv("VAIJUNTO_DADOS")</code>	<code>os.environ.get("VAIJUNTO_DADOS", "")</code>	Configuração
<code>os.ReadFile(caminho)</code>	<code>Path(caminho).read_bytes()</code>	Carregar o JSON
<code>os.WriteFile(caminho, dados, 0o644)</code>	<code>Path(caminho).write_bytes(dados)</code>	Gravar o JSON
<code>os.Stdin</code>	<code>sys.stdin</code>	Teclado
<code>os.Exit(1)</code>	<code>sys.exit(1)</code>	Carga falhou
<code>flag.Int("clientes", 200, "...")</code>	<code>parser.add_argument("--clientes", type=int, default=200)</code>	Opções da carga
<code>sort.Slice(l, func(i, j int) bool {...})</code>	<code>l.sort(key=...)</code>	Percentis de latência

`0o644` é um número em base 8, e o Python escreve igual. São as permissões do arquivo no Linux: dono lê e escreve, os outros só leem.

`flag.Int` devolve um **ponteiro**: o valor só é preenchido depois de `flag.Parse()`, e é lido com `*clientes`.

37. Testes

PYTHON (PYTEST)

```
# test_dados.py
import pytest

@pytest.mark.parametrize("login,senha,quer", [
    ("ana", "123", True),
    ("ana", "outra", False),
])
def test_autenticar(login, senha, quer):
    banco = novo_banco()
    _, ok = banco.autenticar(login, senha)
    assert ok == quer
```

GO

```
// dados_test.go
func TestAutenticar(t *testing.T) {
    b := NovoBanco()
    casos := []struct {
        login, senha string
        quer          bool
    }{
        {"ana", "123", true},
        {"ana", "outra", false},
    }
    for _, caso := range casos {
        if _, ok := b.Autenticar(caso.login,
            caso.senha); ok != caso.quer {
            t.Errorf("%s: deu %v", caso.login,
                ok)
        }
    }
}
```

Go	O que faz
arquivo <code>*_test.go</code>	Só entra na compilação dos testes, nunca no executável.
<code>func TestNome(t *testing.T)</code>	Todo teste começa com <code>Test</code> e recebe <code>t</code> .
<code>t.Errorf(...)</code>	Marca falha e continua o teste.
<code>t.Fatalf(...)</code> , <code>t.Fatal(...)</code>	Marca falha e para o teste ali.
<code>t.Helper()</code>	Numa função auxiliar: a falha aponta a linha de quem chamou.
<code>t.Cleanup(f)</code>	Roda <code>f</code> no fim do teste, como uma fixture com <code>yield</code> .
<code>[]struct{ ... }{ {...}, {...} }</code>	Tabela de casos com uma struct sem nome: o <code>parametrize</code> do Go.
<code>go test ./... -v -run Nome -count=1</code>	<code>-v</code> mostra cada teste; <code>-run</code> filtra pelo nome; <code>-count=1</code> ignora resultados guardados.
<code>go test -race ./...</code>	Roda com o detector de corrida , que acusa acessos simultâneos sem proteção.

Não há `assert`: a condição é um `if` comum, e a mensagem de falha é escrita à mão.

38. Cola rápida Python → Go

Você quer	Python	Go
Criar variável	<code>x = 10</code>	<code>x := 10</code>
Nada / ausente	<code>None</code>	valor zero, ou <code>nil</code>
Lista	<code>[1, 2]</code>	<code>[]int{1, 2}</code>

Adicionar	<code>l.append(x)</code>	<code>l = append(l, x)</code>
Dicionário	<code>{"a": 1}</code>	<code>map[string]int{"a": 1}</code>
Chave existe?	<code>k in d</code>	<code>_, ok := d[k]</code>
Classe de dados	<code>@dataclass</code>	<code>type X struct { ... }</code>
Método	<code>def f(self)</code>	<code>func (x *X) f()</code>
Privado	<code>_nome</code> (convenção)	<code>nome</code> minúsculo (obrigatório)
Laço com índice	<code>for i, v in enumerate(l)</code>	<code>for i, v := range l</code>
Laço com contador	<code>for i in range(a, b)</code>	<code>for i := a; i < b; i++</code>
Laço infinito	<code>while True:</code>	<code>for { }</code>
Várias condições	<code>if/elif/else</code> , <code>match</code>	<code>if/else if/else</code> , <code>switch</code>
Texto formatado	<code>f"{x}"</code>	<code>fmt.Sprintf("%v", x)</code>
Lançar erro	<code>raise ValueError("x")</code>	<code>return errors.New("x")</code>
Tratar erro	<code>try/except</code>	<code>if err != nil { }</code>
Tipo de erro	<code>except MeuErro</code>	<code>errors.Is(err, ErrMeu)</code>
Limpeza garantida	<code>with</code> , <code>finally</code>	<code>defer</code>
Rodar em paralelo	<code>Thread(target=f).start()</code>	<code>go f()</code>
Esperar terminar	<code>t.join()</code>	<code>wg.Wait()</code>
Trava	<code>threading.Lock()</code>	<code>sync.Mutex</code>
Fila entre threads	<code>queue.Queue()</code>	<code>make(chan T, n)</code>
Sinal para todos	<code>threading.Event</code>	<code>close(canal)</code>
Limitar simultâneos	<code>threading.Semaphore(n)</code>	<code>make(chan struct{}, n)</code>
JSON para texto	<code>json.dumps(d)</code>	<code>json.Marshal(v)</code>
Texto para JSON	<code>json.loads(s)</code>	<code>json.Unmarshal(b, &v)</code>
Servidor TCP	<code>socket.create_server</code>	<code>net.Listen</code>
Cliente TCP	<code>socket.create_connection</code>	<code>net.Dial</code>
Variável de ambiente	<code>os.environ.get("X")</code>	<code>os.Getenv("X")</code>
Texto para número	<code>int(s)</code>	<code>strconv.Atoi(s)</code>

Parte IV — O código, linha a linha

Cada arquivo de produção, bloco por bloco. Acima de cada bloco, o código real com os números de linha; abaixo, cada linha explicada. A ordem segue a dependência: primeiro o protocolo, que não depende de ninguém, depois o dados, o servidor, o cliente e o programa de carga.

39. `protocolo/protocolo.go` — o vocabulário da rede

É o arquivo mais curto e o melhor para começar. Ele define as mensagens que servidor, cliente e carga trocam, e as quatro funções que transformam uma mensagem em uma linha de texto e de volta. Não aplica nenhuma regra de negócio, não abre conexão e não guarda estado. Por isso pode ser usado por várias goroutines ao mesmo tempo sem nenhum cuidado.

Cabeçalho e imports

pbl/protocolo/protocolo.go · linhas 1–14

```
1 // Package protocolo define as mensagens que cliente e servidor trocam.
2 //
3 // A regra do protocolo e simples: UMA LINHA DE TEXTO = UMA MENSAGEM, e o
4 // conteúdo da linha e um JSON. E isso que resolve o problema de o TCP nao ter
5 // fronteira de mensagem: quem le sabe que a mensagem acabou quando acha o \n.
6 package protocolo
7
8 import (
9     "bufio"
10    "encoding/json"
11    "errors"
12    "fmt"
13    "io"
14 )
```

6 `package protocolo`

Declara que este arquivo pertence ao pacote `protocolo`. Todo arquivo `.go` começa com `package`. O nome é o que os outros escrevem antes das coisas daqui: `protocolo.Pedido`. Como não é `main`, este pacote não gera executável; é uma biblioteca. O comentário logo acima, que começa com `Package protocolo`, é a documentação do pacote inteiro e registra a regra do enquadramento.

```
8-14 import (
    "bufio"
    "encoding/json"
    "errors"
    "fmt"
    "io"
)
```

Os pacotes da biblioteca padrão usados no arquivo, um por linha entre parênteses:

- `bufio`: leitura com buffer, que permite ler "até o `\n`";
- `encoding/json`: conversão entre struct e JSON;
- `errors`: criar o erro-sentinela;
- `fmt`: montar a mensagem de erro formatada;
- `io`: a interface `io.Writer`, "qualquer coisa em que se escreve".

O import usa o caminho completo, `"encoding/json"`, mas no código só aparece a última parte: `json.Marshal`. O `gofmt` mantém a lista em ordem alfabética.

O erro-sentinela

pbl/protocolo/protocolo.go · linhas 16–22

```
16 // ErrMensagemInvalida marca o erro de "a linha chegou inteira, mas nao e um
17 // JSON valido". O problema e da MENSAGEM, e a conexao continua boa.
18 //
19 // Existe para quem le poder separar esse caso de um erro da CONEXAO (cliente
20 // fechou, caiu, rede sumiu) com errors.Is. Confundir os dois ja causou um bug:
21 // o servidor tratava queda abrupta como JSON ruim e ficava lendo para sempre.
22 var ErrMensagemInvalida = errors.New("mensagem invalida")
```

```
22 var ErrMensagemInvalida = errors.New("mensagem invalida")
```

Cria uma variável do pacote com um erro fixo, uma única vez, quando o programa começa. Tem maiúscula, então é exportada: o servidor usa `protocolo.ErrMensagemInvalida`. Por convenção, erros-sentinela começam com `Err`.

`errors.New` cria um erro cuja mensagem é o texto dado. O que importa não é o texto, é a **identidade** desta variável: quem recebe um erro pergunta `errors.Is(err, protocolo.ErrMensagemInvalida)`. É o equivalente a declarar `class MensagemInvalida(Exception)` e depois usar `except MensagemInvalida`.

Esta linha é a correção do bug da queda abrupta (Parte II): é ela que permite ao servidor diferenciar "a mensagem veio errada" de "a conexão acabou".

Item e Opcao

pbl/protocolo/protocolo.go · linhas 24–30

```
24 // Item e um pedaco de viagem: "na carona X, da cidade De ate a cidade Ate".
25 // De e Ate sao INDICES dentro da Rota da carona, nao nomes de cidade.
26 type Item struct {
27     CaronaID int `json:"carona_id"`
28     De       int `json:"de"`
29     Ate      int `json:"ate"`
30 }
```

```
26 type Item struct {
```

`type Nome struct { ... }` declara um tipo novo com campos, como uma `@dataclass`. `Item` descreve um pedaço de viagem: uma carona e duas posições na rota dela.

```
27-29 CaronaID int `json:"carona_id"`
De      int `json:"de"`
Ate     int `json:"ate"`
```

Três campos inteiros. Cada linha tem **nome**, **tipo** e **tag**.

Os nomes começam com maiúscula porque o `encoding/json` só enxerga campos exportados. A **tag**, o texto entre crases, é uma anotação que o Go guarda junto do campo; o pacote `json` lê a parte `json:"..."` para saber o nome do campo no JSON. Sem tag, o JSON usaria o próprio nome, `CaronaID`; com ela, sai `carona_id`.

Nenhum dos três tem `omitempty`. `De` vale o com frequência (toda viagem que sai da primeira cidade), e é mais claro que o JSON sempre traga os três números.

```

32 // Opcao e uma viagem possivel que o servidor oferece ao passageiro.
33 //
34 // O servidor ja monta o Resumo pronto para exibir. Assim o cliente nao precisa
35 // conhecer a estrutura interna das caronas: ele so imprime o texto e devolve
36 // os Itens da opcao escolhida.
37 type Opcao struct {
38     Itens []Item `json:"itens"`
39     Resumo string `json:"resumo"`
40     Preco int `json:"preco"`
41 }

```

```
37 type Opcao struct {
```

O que a busca devolve: uma viagem possível.

```
38     Itens []Item `json:"itens"`
```

`[]Item` é um slice de `Item`, declarado logo acima no mesmo arquivo. Uma opção direta tem um item; uma com conexão, dois.

```
39     Resumo string `json:"resumo"`
```

Texto pronto para mostrar, montado pelo servidor. Assim o cliente nunca precisa interpretar números de carona e posições.

```
40     Preco int `json:"preco"`
```

Preço total da opção, em reais inteiros.

Pedido

```

43 // Pedido e o que o cliente manda para o servidor.
44 // Cada acao usa so os campos que interessam a ela; o resto vai vazio.
45 type Pedido struct {
46     Acao string `json:"acao"`
47     Usuario string `json:"usuario,omitempty"`
48     Senha string `json:"senha,omitempty"`
49     Tipo string `json:"tipo,omitempty"` // registrar: motorista ou passageiro
50
51     // cadastrar
52     Rota []string `json:"rota,omitempty"`
53     Data string `json:"data,omitempty"`
54     Assentos int `json:"assentos,omitempty"`
55     Preco int `json:"preco,omitempty"`
56
57     // buscar
58     Origem string `json:"origem,omitempty"`
59     Destino string `json:"destino,omitempty"`
60
61     // reservar
62     Itens []Item `json:"itens,omitempty"`
63
64     // passageiros, cancelar_carona
65     CaronaID int `json:"carona_id,omitempty"`
66
67     // pagar, cancelar_reserva

```

```
68 ReservaID int `json:"reserva_id,omitempty"`
69 }
```

```
45 type Pedido struct {
```

Tudo que um cliente pode mandar, numa struct só. Cada ação usa alguns campos e deixa os outros no valor zero.

A alternativa seria uma struct por ação. Isso obrigaria o servidor a descobrir qual é a ação antes de decodificar o resto. Com uma struct só, um único `json.Unmarshal` preenche o que veio, e o servidor olha o campo `Acao`.

```
46 Acao string `json:"acao"`
```

O único campo sem `omitempty`: toda mensagem precisa dizer a ação.

```
47-48 Usuario string `json:"usuario,omitempty"`
      Senha string `json:"senha,omitempty"`
```

Usados por `registrar` e `login`. `omitempty` faz o campo sumir do JSON quando está vazio, então a linha que vai pela rede só leva o que interessa.

```
49 Tipo string `json:"tipo,omitempty"` // registrar: motorista ou passageiro
```

Só no `registrar`. O `//` depois do campo é um comentário de fim de linha, como o `#` do Python.

```
52-55 Rota []string `json:"rota,omitempty"`
      Data string `json:"data,omitempty"`
      Assentos int `json:"assentos,omitempty"`
      Preco int `json:"preco,omitempty"`
```

Campos do `cadastrar`. `Rota` é um `[]string` e vira uma lista no JSON: `"rota":["Feira", "Salvador"]`. A linha `// cadastrar` acima é só um rótulo para quem lê.

```
58-59 Origem string `json:"origem,omitempty"`
      Destino string `json:"destino,omitempty"`
```

Do `buscar`, que também usa o `Data` declarado acima.

```
62 Itens []Item `json:"itens,omitempty"`
```

Do `reservar`: os itens da opção escolhida, exatamente como vieram na busca.

```
65 CaronaID int `json:"carona_id,omitempty"`
```

Do `passageiros` e do `cancelar_carona`. Um número de carona nunca é 0 (o contador começa em 1), então o `omitempty` não esconde nenhum valor válido.

```
68 ReservaID int `json:"reserva_id,omitempty"`
```

Do `pagar` e do `cancelar_reserva`.

Resposta

pbl/protocolo/protocolo.go · linhas 71–82

```
71 // Resposta e o que o servidor devolve para o cliente.
72 type Resposta struct {
```

```

73 OK      bool    `json:"ok"`
74 Erro    string  `json:"erro,omitempty"`
75 Mensagem string  `json:"mensagem,omitempty"`
76
77 Tipo     string  `json:"tipo,omitempty"`      // login
78 CaronaID int     `json:"carona_id,omitempty"` // cadastrar
79 Opcoes   []Opcao `json:"opcoes,omitempty"`      // buscar
80 Linhas   []string `json:"linhas,omitempty"`      // listagens
81 ReservaID int    `json:"reserva_id,omitempty"` // reservar
82 }

```

72 **type** Resposta **struct** {

Tudo que o servidor pode responder, também numa struct só.

73 OK bool `json:"ok"`

Sem `omitempty` **de propósito**. O valor zero de `bool` é `false`; com `omitempty`, toda resposta de falha sairia sem o campo `ok`. Assim, toda resposta traz `"ok":true` ou `"ok":false` explicitamente.

74-75 Erro string `json:"erro,omitempty"`
Mensagem string `json:"mensagem,omitempty"`

`Erro` é preenchido nas falhas; `Mensagem`, nos sucessos, com o texto que o cliente mostra ao usuário.

77-81 Tipo string `json:"tipo,omitempty"` // login
CaronaID int `json:"carona\_id,omitempty"` // cadastrar
Opcoes []Opcao `json:"opcoes,omitempty"` // buscar
Linhas []string `json:"linhas,omitempty"` // listagens
ReservaID int `json:"reserva\_id,omitempty"` // reservar

Campos que só algumas ações preenchem, cada um anotado com a ação: `Tipo` no login, `CaronaID` no cadastro, `Opcoes` na busca, `Linhas` nas listagens, `ReservaID` na reserva. O `gofmt` alinha em colunas o nome, o tipo, a tag e o comentário; os espaços extras são só visuais.

Ler uma mensagem

pbl/protocolo/protocolo.go · linhas 84–100

```

84 // LerPedido le uma linha e transforma o JSON em Pedido.
85 //
86 // Dois tipos de erro podem voltar, e quem chama PRECISA distinguir:
87 // - erro de leitura (EOF, conexao resetada): a conexao acabou;
88 // - ErrMensagemInvalida: a linha veio, mas o JSON esta errado.
89 func LerPedido(leitor *bufio.Reader) (Pedido, error) {
90     linha, err := leitor.ReadString('\n')
91     if err != nil {
92         return Pedido{}, err
93     }
94
95     var p Pedido
96     if err := json.Unmarshal([]byte(linha), &p); err != nil {
97         return Pedido{}, fmt.Errorf("%w: %v", ErrMensagemInvalida, err)
98     }
99     return p, nil
100 }

```

```
89 func LerPedido(leitor *bufio.Reader) (Pedido, error) {
```

Recebe um `*bufio.Reader` e devolve o pedido e um erro.

O leitor vem como ponteiro porque ele guarda estado entre chamadas: se duas mensagens chegaram grudadas, a segunda fica no buffer dele esperando a próxima leitura. Precisa ser sempre **o mesmo** leitor para a mesma conexão.

O comentário acima da função avisa quem chama que existem dois tipos de erro, e que eles precisam ser tratados de forma diferente.

```
90 linha, err := leitor.ReadString('\n')
```

Lê bytes até achar `\n` e devolve tudo, **incluindo** o `\n`. É aqui que o enquadramento do TCP se resolve: se a mensagem chegou partida, o `ReadString` espera o resto; se chegaram duas juntas, ele devolve só a primeira.

`'\n'` com aspas simples é um caractere, não uma string. Equivale ao `arquivo.readline()` sobre um `socket.makefile()` em Python.

```
91-93 if err != nil {
    return Pedido{}, err
}
```

Erro de leitura: a conexão fechou (`io.EOF`) ou caiu. Devolve `Pedido{}`, um pedido vazio (literal composto sem campos, tudo no valor zero), e o próprio erro, sem alterar. Quem chamou decide o que fazer.

```
95 var p Pedido
```

Cria um `Pedido` vazio para ser preenchido. `p := Pedido{}` faria o mesmo; `var` é o jeito comum quando o valor inicial não importa.

```
96-98 if err := json.Unmarshal([]byte(linha), &p); err != nil {
    return Pedido{}, fmt.Errorf("%w: %v", ErrMensagemInvalida, err)
}
```

Três ideias numa linha:

- `[]byte(linha)` converte o texto em bytes, que é o que o `Unmarshal` aceita. Conversões em Go são sempre explícitas.
- `&p` passa o **endereço** de `p`, para o `Unmarshal` preencher os campos. Passando só `p`, ele receberia uma cópia e o pedido continuaria vazio.
- O `if` com declaração cria um `err` que só existe dentro do `if`. Esse `err` novo esconde o de fora, o que aqui é intencional. Em Python não existe escopo de bloco, então isso não tem equivalente direto.

Se o JSON é inválido, devolve o erro embrulhado: `%w` coloca `ErrMensagemInvalida` dentro do erro novo, e `errors.Is` vai reconhecê-lo; `%v` só acrescenta o texto do erro original. Na rede, isso aparece como `mensagem invalida: invalid character 'i' looking for beginning of value`.

```
99 return p, nil
```

Deu certo: o pedido preenchido e `nil` no lugar do erro.

```

102 // LerResposta lê uma linha e transforma o JSON em Resposta.
103 // Os erros seguem a mesma regra do LerPedido.
104 func LerResposta(leitor *bufio.Reader) (Resposta, error) {
105     linha, err := leitor.ReadString('\n')
106     if err != nil {
107         return Resposta{}, err
108     }
109
110     var r Resposta
111     if err := json.Unmarshal([]byte(linha), &r); err != nil {
112         return Resposta{}, fmt.Errorf("%w: %v", ErrMensagemInvalida, err)
113     }
114     return r, nil
115 }

```

104-114

```

func LerResposta(leitor *bufio.Reader) (Resposta, error) {
    linha, err := leitor.ReadString('\n')
    if err != nil {
        return Resposta{}, err
    }

    var r Resposta
    if err := json.Unmarshal([]byte(linha), &r); err != nil {
        return Resposta{}, fmt.Errorf("%w: %v", ErrMensagemInvalida, err)
    }
    return r, nil
}

```

Idêntica ao `LerPedido`, trocando `Pedido` por `Resposta`. É usada pelo cliente e pela carga. Go tem generics e daria para escrever uma função só para os dois tipos, mas duas funções explícitas e curtas são mais fáceis de ler.

Enviar uma mensagem

pbl/protocolo/protocolo.go · linhas 117-120

```

117 // EnviarPedido escreve o Pedido como uma linha de JSON.
118 func EnviarPedido(destino io.Writer, p Pedido) error {
119     return enviar(destino, p)
120 }

```

118 `func EnviarPedido(destino io.Writer, p Pedido) error {`

O destino é um `io.Writer`, uma **interface**: qualquer coisa que tenha o método `Write`. Uma `net.Conn` tem, então a conexão pode ser passada direto. Também serviria um arquivo ou `os.Stdout`, sem mudar nada aqui.

119 `return enviar(destino, p)`

Só repassa para a função interna. A função pública existe para quem usa o pacote ver a assinatura com o tipo certo, `Pedido`, e não poder mandar qualquer coisa.

pbl/protocolo/protocolo.go · linhas 122-125

```

122 // EnviarResposta escreve a Resposta como uma linha de JSON.
123 func EnviarResposta(destino io.Writer, r Resposta) error {

```

```
124     return enviar(destino, r)
125 }
```

```
123-124 func EnviarResposta(destino io.Writer, r Resposta) error {
        return enviar(destino, r)
    }
```

O mesmo, para `Resposta`. É o que o servidor chama depois de cada ação.

pbl/protocolo/protocolo.go · linhas 127–139

```
127 // enviar converte qualquer valor em JSON e escreve com o \n no fim.
128 // E minúscula, então só existe dentro deste pacote.
129 func enviar(destino io.Writer, valor any) error {
130     bytes, err := json.Marshal(valor)
131     if err != nil {
132         return err
133     }
134
135     bytes = append(bytes, '\n')
136
137     _, err = destino.Write(bytes)
138     return err
139 }
```

```
129 func enviar(destino io.Writer, valor any) error {
```

Minúscula: só existe dentro do `protocolo`. `valor any` aceita um valor de qualquer tipo (`any` é a interface sem métodos, que todo tipo satisfaz), e é por isso que as duas funções públicas podem compartilhá-la.

```
130     bytes, err := json.Marshal(valor)
```

Converte a struct em JSON e devolve `[]byte`. Percorre os campos exportados, usa as tags para os nomes, aplica `omitempty` e troca `<`, `>` e `&` por códigos `\u`. Pode falhar para valores sem representação em JSON (um canal, por exemplo), o que nunca acontece com `Pedido` e `Resposta`.

```
131-133 if err != nil {
        return err
    }
```

Se falhou, devolve o erro para cima.

```
135     bytes = append(bytes, '\n')
```

Acrescenta o `\n` no fim: é a regra do protocolo sendo aplicada. O JSON gerado nunca contém quebra de linha real, então este é o único `\n` da mensagem. O resultado do `append` precisa ser guardado de volta.

```
137     _, err = destino.Write(bytes)
```

`Write` devolve quantos bytes escreveu e um erro. O número vai para `_`, e o erro é guardado com `=`, porque `err` já existe.

A mensagem inteira sai numa chamada só. Se o `\n` fosse escrito separado, duas goroutines escrevendo na mesma conexão poderiam intercalar os pedaços. Neste projeto cada conexão tem uma única goroutine escrevendo, mas é o jeito seguro.

138 `return err`

`nil` se a escrita deu certo; o erro dela (por exemplo, a conexão já fechou) se não deu.

O ARQUIVO EM RESUMO

Dois tipos de mensagem, dois tipos auxiliares, um erro-sentinela e quatro funções públicas. Tudo que cruza a rede passa por aqui, e nada daqui guarda estado entre chamadas. O único cuidado com concorrência é o do chamador: cada conexão deve ter o seu próprio `bufio.Reader`.

40. `dados/dados.go` (1 de 2) — estado, contas, caronas, busca e reserva

O coração do sistema: todo o estado e todas as regras de negócio. O arquivo segue uma ordem fixa: tipos, construtores, **funções públicas** (todas travam o mutex) e, no fim, **funções internas** (nenhuma trava). Esta primeira metade vai até o `Reservar`; a segunda cobre pagamento, cancelamentos, expiração, listagens e as funções internas.

LEIA SEMPRE COM A REGRA DE OURO NA CABEÇA

Maiúscula trava, minúscula não. Toda função pública começa com `b.mu.Lock()` e `defer b.mu.Unlock()`. Toda função interna pode ser chamada por uma pública e confia que a trava já está com quem chamou.

Cabeçalho e imports

pbl/dados/dados.go · linhas 1–19

```
1 // Package dados guarda o estado do sistema e as regras que mexem nele.
2 //
3 // REGRA DE OURO DESTE PACOTE: toda funcao publica (maiúscula) abre com
4 // mu.Lock() e defer mu.Unlock(). Funcoes internas (minúsculas) NUNCA travam,
5 // porque sao chamadas de dentro das publicas -- se elas travassem tambem, o
6 // programa congelaria (deadlock).
7 package dados
8
9 import (
10     "encoding/json"
11     "errors"
12     "fmt"
13     "os"
14     "strings"
15     "sync"
16     "time"
17
18     "vaijunto/protocolo"
19 )
```

7 `package dados`

O pacote de regras. O comentário acima registra a regra de ouro no próprio arquivo, para quem abrir o código sem ter lido este guia.

```

9-19 import (
    "encoding/json"
    "errors"
    "fmt"
    "os"
    "strings"
    "sync"
    "time"

    "vaijunto/protocolo"
)

```

- `encoding/json`: gravar e ler o arquivo de dados;
- `errors` e `fmt`: criar erros com mensagem fixa ou formatada;
- `os`: ler e gravar o arquivo;
- `strings`: `TrimSpace` e `Join`;
- `sync`: o `Mutex`;
- `time`: prazos das reservas;
- `vaijunto/protocolo`: os tipos `Item` e `Opcao`.

A linha em branco separa a biblioteca padrão dos pacotes do próprio projeto, que é a convenção de organização dos imports em Go.

O prazo da reserva

pbl/dados/dados.go · linhas 21-23

```

21 // TempoDeReserva e quanto tempo a reserva fica de pe sem ser paga.
22 // E variavel (nao constante) para os testes poderem encurtar isso.
23 var TempoDeReserva = 10 * time.Minute

```

```

23 var TempoDeReserva = 10 * time.Minute

```

Variável do pacote, exportada, com o prazo padrão. `time.Minute` é uma constante do tipo `time.Duration`; multiplicada por 10, dá dez minutos.

É `var` e não `const` por dois motivos: os testes trocam o valor para 1 milissegundo, e o servidor troca pelo valor de `VAIJUNTO_RESERVA` ao iniciar. É um estado global; os testes que mexem nele restauram o original com `defer`, e testes do mesmo pacote não rodam em paralelo entre si.

Os tipos

pbl/dados/dados.go · linhas 25-30

```

25 // Usuario e quem usa o sistema. Tipo e "motorista" ou "passageiro".
26 type Usuario struct {
27     Login string `json:"login"`
28     Senha string `json:"senha"`
29     Tipo  string `json:"tipo"`
30 }

```

26 `type Usuario struct {`

Uma conta.

27-29 `Login string `json:"login"`
Senha string `json:"senha"`
Tipo string `json:"tipo"``

Três textos com tag para o arquivo JSON. `Senha` fica em texto puro (Parte VII). `Tipo` é um texto livre, conferido no cadastro: Go não tem um `enum.Enum` como o Python, e um texto com dois valores conhecidos resolve.

pbl/dados/dados.go · linhas 32-49

```
32 // Carona e uma viagem oferecida por um motorista.  
33 //  
34 // Rota e a sequencia de cidades: ["Feira", "Salvador", "Ilheus"].  
35 // Os TRECHOS sao os intervalos entre elas: Feira->Salvador e Salvador->Ilheus.  
36 // Uma rota com N cidades tem N-1 trechos.  
37 //  
38 // Ocupados tem um contador por trecho. E o coracao da modelagem: a  
39 // disponibilidade POR TRECHO sai naturalmente, sem estrutura extra.  
40 type Carona struct {  
41     ID          int        `json:"id"`  
42     Motorista   string      `json:"motorista"`  
43     Rota        []string   `json:"rota"`  
44     Data        string      `json:"data"`  
45     Assentos    int         `json:"assentos" // total de assentos, igual em todo trecho  
46     Preco       int         `json:"preco"    // preco POR TRECHO  
47     Ocupados    []int       `json:"ocupados" // len = len(Rota)-1  
48     Cancelada   bool        `json:"cancelada,omitempty"`  
49 }
```

40 `type Carona struct {`

Uma viagem oferecida por um motorista. O comentário acima resume a modelagem de rota e trechos da Parte I.

41-44 `ID int `json:"id"`
Motorista string `json:"motorista"`
Rota []string `json:"rota"`
Data string `json:"data"``

O número da carona, o login do motorista dono, a rota como slice de nomes de cidade, e a data como texto.

45 `Assentos int `json:"assentos" // total de assentos, igual em todo trecho`

Total de lugares, o mesmo em todos os trechos.

46 `Preco int `json:"preco" // preco POR TRECHO`

Preço por trecho, em inteiro. Dinheiro em inteiro evita os erros de arredondamento de ponto flutuante: `0.1 + 0.2` não dá exatamente `0.3`, nem em Go nem em Python.

47 `Ocupados []int `json:"ocupados" // len = len(Rota)-1`

O vetor central do sistema: `Ocupados[t]` é quantos lugares estão ocupados no trecho `t`. Tem `len(Rota)-1` posições, criadas em `CadastrarCarona`.

48 Cancelada `bool` ``json:"cancelada,omitempty"``

Começa `false` (valor zero). Com `omitempty`, só aparece no arquivo quando é `true`, como na carona 2 do arquivo de exemplo da Parte I. Um arquivo gravado antes de este campo existir continua carregando normalmente: o campo ausente vira `false`.

pbl/dados/dados.go · linhas 51–59

```
51 // Reserva e um pedido de assentos, que pode cobrir varias caronas.
52 type Reserva struct {
53     ID          int          `json:"id"`
54     Passageiro  string       `json:"passageiro"`
55     Itens       []protocolo.Item `json:"itens"`
56     Estado      string       `json:"estado" // "pendente" | "paga" | "expirada" | "cancelada"
57     Motivo      string       `json:"motivo,omitempty"`
58     Expira      time.Time    `json:"expira"`
59 }
```

52 `type Reserva struct {`

Um pedido de assentos, que pode cobrir mais de uma carona.

53–54 ID `int` ``json:"id"``
Passageiro `string` ``json:"passageiro"``

O número da reserva e o login de quem reservou, usado para conferir o dono ao pagar e cancelar.

55 Itens `[]protocolo.Item` ``json:"itens"``

`[]protocolo.Item`: um tipo de outro pacote, com o nome do pacote na frente. A reserva guarda exatamente os itens pedidos, e é com eles que os assentos são devolvidos depois.

56 Estado `string` ``json:"estado" // "pendente" | "paga" | "expirada" | "cancelada"`

Um dos quatro estados, como texto. O comentário lista os valores possíveis.

57 Motivo `string` ``json:"motivo,omitempty"``

Só é preenchido quando a reserva é cancelada: pelo passageiro ou porque o motorista cancelou uma carona dela.

58 Expira `time.Time` ``json:"expira"``

`time.Time` é um instante: data, hora e fuso. No JSON vira texto no formato RFC 3339, como `"2026-09-14T09:57:23.89-03:00"`. É um horário absoluto, e é por isso que o prazo continua valendo mesmo depois de o servidor ser reiniciado.

pbl/dados/dados.go · linhas 61–78

```
61 // Banco guarda TODO o estado do sistema em memoria.
62 //
63 // O mutex protege tudo que esta declarado abaixo dele. Como varias goroutines
64 // (uma por cliente conectado) mexem neste mesmo Banco, sem o mutex duas
65 // compras simultaneas poderiam ler o mesmo assento livre e vende-lo duas vezes.
66 //
67 // "mu" e "arquivo" comecam com minuscula, entao o encoding/json os ignora --
```

```

68 // e o que queremos: nem a fechadura nem o caminho do arquivo fazem parte dos
69 // dados salvos.
70 type Banco struct {
71     mu      sync.Mutex
72     arquivo string
73
74     Usuarios map[string]Usuario `json:"usuarios"`
75     Caronas  []Carona    `json:"caronas"`
76     Reservas []Reserva  `json:"reservas"`
77     ProxID   int         `json:"prox_id"`
78 }

```

70 `type Banco struct {`

A struct que guarda **todo** o estado do sistema. Tudo que as goroutines compartilham está aqui dentro, e em nenhum outro lugar.

71 `mu sync.Mutex`

A trava. Minúscula de propósito: nenhum outro pacote consegue travar ou destravar o banco, então ninguém de fora quebra a regra de ouro.

O valor zero de um `sync.Mutex` já é uma trava pronta e destravada; não precisa de construtor. Por convenção, o mutex fica acima dos campos que protege.

72 `arquivo string`

Caminho do arquivo JSON. Vazio quer dizer "não gravar nada", que é o caso dos testes.

74 `Usuarios map[string]Usuario `json:"usuarios"``

`map[string]Usuario`: do login para a conta. O map dá acesso direto pelo login, que é como as contas são procuradas.

75-76 `Caronas []Carona `json:"caronas"``
`Reservas []Reserva `json:"reservas"``

Slices, não maps. A busca precisa varrer todas as caronas de qualquer forma, e o slice mantém a ordem de criação, o que deixa as listagens previsíveis. O custo é achar uma carona pelo número varrendo a lista, aceitável para o tamanho deste projeto.

77 `ProxID int `json:"prox_id"``

O contador de números, compartilhado por caronas e reservas. É exportado para ser gravado no arquivo; sem isso, depois de reiniciar, o servidor voltaria a dar números repetidos.

Os construtores

pbl/dados/dados.go · linhas 80-95

```

80 // NovoBanco cria o banco ja com usuarios de teste, sem persistencia.
81 //
82 // Devolve PONTEIRO (*Banco), nunca valor. Um sync.Mutex nao pode ser copiado:
83 // cada copia teria a sua propria fechadura, e proteger uma copia nao protegeria
84 // as outras. O proprio "go vet" reclama se voce tentar copiar.
85 func NovoBanco() *Banco {
86     return &Banco{
87         Usuarios: map[string]Usuario{
88             "ana": {Login: "ana", Senha: "123", Tipo: "motorista"},

```

```

89         "davi": {Login: "davi", Senha: "123", Tipo: "motorista"},
90         "bruno": {Login: "bruno", Senha: "123", Tipo: "passageiro"},
91         "carla": {Login: "carla", Senha: "123", Tipo: "passageiro"},
92     },
93     ProxID: 1,
94 }
95 }

```

85 `func NovoBanco() *Banco {`

Go não tem `__init__`. A convenção é uma função `NovoAlgo` que devolve o valor pronto. Devolve `*Banco`, um ponteiro, porque o banco contém um mutex e não pode ser copiado.

86 `return &Banco{`

`&Banco{...}` cria a struct e já devolve o endereço dela. O Go percebe que o valor precisa sobreviver ao fim da função e o guarda num lugar adequado sozinho; em C isso seria um erro grave, em Go é o normal.

Os campos não citados ficam no valor zero: trava destravada, caminho vazio e slices `nil`. Um slice `nil` se comporta como uma lista vazia em `len`, `range` e `append`; no JSON, sai como `null` até a primeira carona ser criada.

87 `Usuarios: map[string]Usuario{`

Um map criado já com conteúdo, com a sintaxe de literal: `chave: valor`, separados por vírgula.

88-91 `"ana": {Login: "ana", Senha: "123", Tipo: "motorista"},
"davi": {Login: "davi", Senha: "123", Tipo: "motorista"},
"bruno": {Login: "bruno", Senha: "123", Tipo: "passageiro"},
"carla": {Login: "carla", Senha: "123", Tipo: "passageiro"},`

Os quatro usuários de teste. Dentro de um literal de `map[string]Usuario`, o valor pode ser escrito só como `{Login: ...}`, sem repetir `Usuario`: o Go deduz o tipo. Todo item termina com vírgula, inclusive o último; é por isso que dá para acrescentar uma linha sem mexer nas outras.

93 `ProxID: 1,`

O primeiro número será 1. Começar do 1 garante que o nunca é um número válido, o que importa porque `carona_id` e `reserva_id` têm `omitempty` no protocolo e sumiriam do JSON se valessem o.

pbl/dados/dados.go · linhas 97-119

```

97 // Carregar le o banco de um arquivo JSON. Se o arquivo nao existir (primeira
98 // execucao), comeca do zero. A partir daqui toda alteracao e gravada sozinha.
99 //
100 // O enunciado proibe SGBD mas permite JSON: e exatamente isto.
101 func Carregar(caminho string) *Banco {
102     b := NovoBanco()
103     b.arquivo = caminho
104
105     conteudo, err := os.ReadFile(caminho)
106     if err != nil {
107         fmt.Printf("sem dados anteriores em %s, comecando do zero\n", caminho)
108         return b
109     }
110
111     if err := json.Unmarshal(conteudo, b); err != nil {
112         fmt.Printf("arquivo %s ilegivel (%v), comecando do zero\n", caminho, err)
113         return NovoBancoEm(caminho)

```

```

114 }
115
116 fmt.Printf("dados carregados de %s: %d carona(s), %d reserva(s)\n",
117     caminho, len(b.Caronas), len(b.Reservas))
118 return b
119 }

```

101 `func Carregar(caminho string) *Banco {`

O construtor que o servidor usa: cria o banco ligado a um arquivo e tenta recuperar o estado salvo.

102 `b := NovoBanco()`

Começa pelo banco padrão. Se não houver arquivo, é ele que vale.

103 `b.arquivo = caminho`

A partir daqui, `persistir` grava neste caminho. `Carregar` é do mesmo pacote, então pode mexer num campo minúsculo.

105 `conteudo, err := os.ReadFile(caminho)`

Lê o arquivo inteiro para a memória e devolve `[]byte` e um erro. É o `Path(caminho).read_bytes()`.

106-109 `if err != nil {
 fmt.Printf("sem dados anteriores em %s, começando do zero\n", caminho)
 return b
}`

Arquivo não existe, que é o caso da primeira execução, ou não pôde ser lido: avisa e segue com o banco padrão. A função não diferencia "não existe" de "sem permissão"; os dois viram "começar do zero".

111-114 `if err := json.Unmarshal(conteudo, b); err != nil {
 fmt.Printf("arquivo %s ilegível (%v), começando do zero\n", caminho, err)
 return NovoBancoEm(caminho)
}`

Preenche o banco com o conteúdo do arquivo. `b` já é ponteiro, então não precisa de `&`.

Os campos exportados são substituídos pelo que está no arquivo, com uma exceção: como o map de usuários já existe, o `Unmarshal` acrescenta as contas do arquivo às de teste em vez de trocar o map. Os campos `mu` e `arquivo`, minúsculos, não são tocados.

Se o JSON estiver quebrado, avisa e devolve um banco novo apontando para o mesmo caminho. Na primeira gravação, o arquivo antigo é sobrescrito (limitação na Parte VII).

116-117 `fmt.Printf("dados carregados de %s: %d carona(s), %d reserva(s)\n",
 caminho, len(b.Caronas), len(b.Reservas))`

Informa quanto foi recuperado. A chamada continua na linha de baixo: em Go, uma linha que termina em vírgula continua na próxima.

118 `return b`

Devolve o banco carregado.

```

121 // NovoBancoEm cria um banco vazio que grava no caminho indicado.
122 func NovoBancoEm(caminho string) *Banco {
123     b := NovoBanco()
124     b.arquivo = caminho
125     return b
126 }

```

122 `func NovoBancoEm(caminho string) *Banco {`

Um banco zerado, mas já com caminho para gravar. Usado pelo `Carregar` quando o arquivo existe mas está ilegível.

123-125 `b := NovoBanco()
b.arquivo = caminho
return b`

Mesma ideia do começo do `Carregar`, sem a leitura.

Autenticar

pbl/dados/dados.go · linhas 132-147

```

132 // Autenticar confere login e senha. O segundo retorno diz se deu certo.
133 func (b *Banco) Autenticar(login, senha string) (Usuario, bool) {
134     b.mu.Lock()
135     defer b.mu.Unlock()
136
137     u, existe := b.Usuarios[login]
138     if !existe {
139         return Usuario{}, false
140     }
141
142     if u.Senha != senha {
143         return Usuario{}, false
144     }
145
146     return u, true
147 }

```

133 `func (b *Banco) Autenticar(login, senha string) (Usuario, bool) {`

Primeiro método de `*Banco`: `(b *Banco)` é o receptor, o `self`. Devolve a conta e um `bool` dizendo se deu certo, o padrão "valor, ok" do Go.

134 `b.mu.Lock()`

Trava o banco. Se outra goroutine estiver com a trava, esta fica parada aqui até ela soltar.

Mesmo uma leitura precisa travar: se esta goroutine lesse o map enquanto outra grava uma conta nova, o Go derrubaria o programa. Existe o `sync.RWMutex`, que deixa várias leituras acontecerem juntas; o projeto usa o `Mutex` comum por simplicidade.

135 `defer b.mu.Unlock()`

Agenda a destrava para o fim da função, qualquer que seja o `return`. Este par de linhas abre todas as funções públicas do arquivo.

```
137 u, existe := b.Usuarios[login]
```

Busca no map com dois retornos: a conta (ou uma conta vazia, se não existir) e se a chave existe.

```
138-140 if !existe {  
    return Usuario{}, false  
}
```

Login desconhecido. `!` é o `not`. `Usuario{}` é uma conta vazia.

```
142-144 if u.Senha != senha {  
    return Usuario{}, false  
}
```

Senha errada. As duas falhas devolvem exatamente a mesma coisa: quem tenta entrar não descobre se o login existe ou se só a senha está errada.

```
146 return u, true
```

Deu certo. O `defer` destrava agora.

CadastrarUsuario

pbl/dados/dados.go · linhas 149–180

```
149 // CadastrarUsuario cria uma conta nova.  
150 //  
151 // Repare que a checagem "já existe" e a escrita no mapa acontecem DENTRO do  
152 // mesmo Lock. Se a checagem ficasse fora, duas pessoas registrando o mesmo  
153 // login ao mesmo tempo passariam as duas pela verificacao antes de qualquer  
154 // uma escrever -- e a segunda sobrescreveria a primeira. E o mesmo raciocinio  
155 // das duas fases do Reservar.  
156 func (b *Banco) CadastrarUsuario(login, senha, tipo string) error {  
157     login = strings.TrimSpace(login)  
158  
159     if login == "" {  
160         return errors.New("informe o usuario")  
161     }  
162     if senha == "" {  
163         return errors.New("informe a senha")  
164     }  
165     if tipo != "motorista" && tipo != "passageiro" {  
166         return errors.New(`o tipo precisa ser "motorista" ou "passageiro"`)  
167     }  
168  
169     b.mu.Lock()  
170     defer b.mu.Unlock()  
171  
172     if _, existe := b.Usuarios[login]; existe {  
173         return fmt.Errorf("o usuario %q já existe", login)  
174     }  
175  
176     b.Usuarios[login] = Usuario{Login: login, Senha: senha, Tipo: tipo}  
177     b.persistir()  
178  
179     return nil  
180 }
```

```
156 func (b *Banco) CadastrarUsuario(login, senha, tipo string) error {
```

Devolve só um `error`: `nil` se a conta foi criada.

```
157 login = strings.TrimSpace(login)
```

Tira espaços e quebras de linha das pontas. O resultado é guardado no próprio parâmetro, que é uma cópia local: não afeta quem chamou.

```
159-161 if login == "" {
    return errors.New("informe o usuario")
}
```

`errors.New` cria o erro com a mensagem que vai chegar ao usuário.

```
162-164 if senha == "" {
    return errors.New("informe a senha")
}
```

Senha vazia não é aceita.

```
165-167 if tipo != "motorista" && tipo != "passageiro" {
    return errors.New(`o tipo precisa ser "motorista" ou "passageiro"`)
}
```

`&&` é o `and`. A mensagem está entre crases porque contém aspas duplas; num texto entre crases, nada precisa de escape.

```
169-170 b.mu.Lock()
defer b.mu.Unlock()
```

Só agora trava. As três conferências acima olham apenas os argumentos e não precisam fazer ninguém esperar.

```
172-174 if _, existe := b.Usuarios[login]; existe {
    return fmt.Errorf("o usuario %q ja existe", login)
}
```

`if` com declaração: consulta o map, descarta a conta com `_` e testa só `existe`. `%q` põe o login entre aspas na mensagem.

A checagem e a gravação logo abaixo estão **dentro da mesma trava**. Separadas, duas goroutines poderiam passar pela checagem antes de qualquer uma gravar, e a segunda sobrescreveria a primeira.

`TestCadastroConcorrente` e `TestCadastrosSimultaneos` cobrem exatamente isso.

```
176 b.Usuarios[login] = Usuario{Login: login, Senha: senha, Tipo: tipo}
```

Grava no map. Se o login já existisse, isto sobrescreveria a conta; é justamente o que a checagem acima impede.

```
177 b.persistir()
```

Grava o arquivo. `persistir` é interna e não trava, porque esta goroutine já está com a trava.

```
179 return nil
```

Conta criada.

CadastrarCarona

pbl/dados/dados.go · linhas 182–214

```
182 // CadastrarCarona registra uma carona nova e devolve o ID dela.
183 func (b *Banco) CadastrarCarona(motorista string, rota []string, data string, assentos, preco int)
(int, error) {
184     // A validacao vem ANTES do Lock: nao faz sentido segurar a fechadura
185     // (e travar todos os outros clientes) para conferir argumento.
186     if len(rota) < 2 {
187         return 0, errors.New("a rota precisa de pelo menos 2 cidades")
188     }
189     if assentos < 1 {
190         return 0, errors.New("a carona precisa de pelo menos 1 assento")
191     }
192     if data == "" {
193         return 0, errors.New("informe a data")
194     }
195
196     b.mu.Lock()
197     defer b.mu.Unlock()
198
199     c := Carona{
200         ID:        b.ProxID,
201         Motorista: motorista,
202         Rota:       rota,
203         Data:       data,
204         Assentos:   assentos,
205         Preco:      preco,
206         Ocupados:   make([]int, len(rota)-1), // um contador por trecho, zerado
207     }
208
209     b.ProxID++
210     b.Caronas = append(b.Caronas, c)
211     b.persistir()
212
213     return c.ID, nil
214 }
```

183 `func (b *Banco) CadastrarCarona(motorista string, rota []string, data string, assentos, preco int) (int, error) {`

Vários parâmetros de tipos diferentes; `assentos, preco int` divide o tipo. Devolve o número da carona criada e um erro. O comentário no início do corpo explica por que a validação vem antes da trava.

186–188 `if len(rota) < 2 {
 return 0, errors.New("a rota precisa de pelo menos 2 cidades")
}`

`len` funciona para slice, string, map e canal, como no Python. `return 0, errors.New(...)`: o `0` ocupa o lugar do número, que não existe; quem chama olha o erro antes de usar o número.

189–191 `if assentos < 1 {
 return 0, errors.New("a carona precisa de pelo menos 1 assento")
}`

Uma carona precisa de pelo menos um lugar.

```
192-194  if data == "" {
        return 0, errors.New("informe a data")
    }
```

Só confere que a data não está vazia. O formato não é validado; "amanhã" seria aceito (Parte VII).

```
196-197  b.mu.Lock()
        defer b.mu.Unlock()
```

Com os argumentos conferidos, trava.

```
199      c := Carona{
```

Monta a carona com um literal composto, campo por campo.

```
200-205  ID:         b.ProxID,
        Motorista: motorista,
        Rota:      rota,
        Data:      data,
        Assentos:  assentos,
        Preco:     preco,
```

O número vem do contador; os outros campos copiam os argumentos. `Rota` guarda o mesmo slice que veio no pedido, sem copiar os nomes; como o pedido não é usado depois, não há problema.

```
206      Ocupados: make([]int, len(rota)-1), // um contador por trecho, zerado
```

Cria o vetor de ocupação com um zero para cada trecho. É o `[0] * (len(rota) - 1)` do Python.

```
209      b.ProxID++
```

Avança o contador. Em Go, `++` é uma instrução, não uma expressão: não existe `x = y++`.

```
210      b.Caronas = append(b.Caronas, c)
```

Acrescenta a carona ao slice e guarda o slice de volta no campo. A struct `c` é **copiada** para dentro do slice; alterar `c` depois não mudaria a carona guardada.

```
211      b.persistir()
```

Grava o arquivo, ainda com a trava.

```
213      return c.ID, nil
```

Devolve o número. `c.ID` tem o valor certo porque foi lido do contador antes do `++`.

Buscar

pbl/dados/dados.go · linhas 216–224

```
216 // Buscar devolve as opcoes de viagem de origem ate destino naquela data.
217 // Primeiro as caronas diretas, depois as que exigem UMA baldeacao.
218 func (b *Banco) Buscar(origem, destino, data string) []protocolo.Opcao {
219     b.mu.Lock()
220     defer b.mu.Unlock()
221
222     opcoes := b.buscarDiretas(origem, destino, data)
```

```

223     return append(opcoes, b.buscarComConexao(origem, destino, data)...)
224 }

```

218 `func (b *Banco) Buscar(origem, destino, data string) []protocolo.Opcao {`

Devolve `[]protocolo.Opcao` e nenhum erro: não achar nada é uma lista vazia, não uma falha.

219–220

```

b.mu.Lock()
defer b.mu.Unlock()

```

Mesmo sendo só leitura, trava. Sem a trava, uma reserva poderia mudar `Ocupados` no meio da busca, e uma opção sairia mostrando vagas que já não existem; ou o slice de caronas poderia crescer no meio da varredura.

222 `opcoes := b.buscarDiretas(origem, destino, data)`

As duas buscas de verdade são funções internas, executadas sob esta mesma trava (explicadas na segunda metade).

223 `return append(opcoes, b.buscarComConexao(origem, destino, data)...)`

Junta as duas listas. Os três pontos depois do argumento **espalham** o slice em vários argumentos: `append(a, b...)` é o `a.extend(b)` do Python, ou o `*b` numa chamada.

As diretas entram primeiro, e é por isso que aparecem primeiro no menu. Se as duas vierem vazias, o resultado é `nil`, que o `omitempty` tira da resposta.

Reservar

A função mais importante do projeto. A lógica foi explicada na Parte I; aqui está cada linha.

pbl/dados/dados.go · linhas 226–280

```

226 // Reservar segura assentos em uma ou mais caronas, de forma ATOMICA.
227 //
228 // Esta e a funcao mais importante do projeto. O enunciado exige que, se o
229 // passageiro pegar um trecho, ele nao fique sem o outro. A garantia vem de
230 // duas fases dentro de UM UNICO Lock:
231 //
232 //     FASE 1 confere TODOS os itens sem alterar nada.
233 //     FASE 2 so roda se a fase 1 passou inteira.
234 //
235 // Como nada solta o mutex entre as duas, nenhuma outra goroutine consegue se
236 // intrometer no meio e roubar um assento que acabamos de conferir.
237 func (b *Banco) Reservar(passageiro string, itens []protocolo.Item) (int, error) {
238     if len(itens) == 0 {
239         return 0, errors.New("nenhum trecho escolhido")
240     }
241
242     b.mu.Lock()
243     defer b.mu.Unlock()
244
245     // ---- FASE 1: verificacao. NAO altera nada. ----
246     for _, item := range itens {
247         c := b.acharCarona(item.CaronaID)
248         if c == nil {
249             return 0, fmt.Errorf("carona %d nao existe", item.CaronaID)
250         }
251         if c.Cancelada {
252             return 0, fmt.Errorf("carona %d foi cancelada", item.CaronaID)

```

```

253     }
254     if item.De < 0 || item.Ate > c.trechos() || item.De >= item.Ate {
255         return 0, fmt.Errorf("trecho invalido na carona %d", item.CaronaID)
256     }
257     if c.livres(item.De, item.Ate) < 1 {
258         return 0, fmt.Errorf("sem assento livre na carona %d", item.CaronaID)
259     }
260 }
261
262 // ---- FASE 2: aplicacao. So chega aqui se TUDO passou. ----
263 for _, item := range itens {
264     b.acharCarona(item.CaronaID).ocupar(item.De, item.Ate)
265 }
266
267 r := Reserva{
268     ID:          b.ProxID,
269     Passageiro:  passageiro,
270     Itens:       itens,
271     Estado:      "pendente",
272     Expira:      time.Now().Add(TempoDeReserva),
273 }
274
275 b.ProxID++
276 b.Reservas = append(b.Reservas, r)
277 b.persistir()
278
279 return r.ID, nil
280 }

```

237 `func (b *Banco) Reservar(passageiro string, itens []protocolo.Item) (int, error) {`

Recebe o login de quem reserva e os itens da opção escolhida; devolve o número da reserva e um erro. O comentário acima dela resume as duas fases.

238-240 `if len(itens) == 0 {
 return 0, errors.New("nenhum trecho escolhido")
}`

Pedido sem nenhum item: recusado antes de travar.

242-243 `b.mu.Lock()
defer b.mu.Unlock()`

A trava vale para as duas fases e para o registro da reserva. **Nada solta a trava no meio:** é isso que torna a reserva atômica em relação às outras goroutines.

246 `for _, item := range itens {`

Fase 1: verificação. `item` é uma cópia de cada `Item`, o que aqui está certo, porque esta fase só lê.

247 `c := b.acharCarona(item.CaronaID)`

Um ponteiro para a carona dentro do slice, ou `nil` se o número não existir.

248-250 `if c == nil {
 return 0, fmt.Errorf("carona %d nao existe", item.CaronaID)
}`

Número de carona inexistente. `%d` formata o número na mensagem.

```
251-253  if c.Cancelada {
        return 0, fmt.Errorf("carona %d foi cancelada", item.CaronaID)
    }
```

Carona cancelada não aceita reserva nova.

```
254-256  if item.De < 0 || item.Ate > c.trechos() || item.De >= item.Ate {
        return 0, fmt.Errorf("trecho invalido na carona %d", item.CaronaID)
    }
```

Protege contra itens impossíveis, que um cliente escrito à mão poderia mandar:

- posição negativa;
- destino além do fim da rota: `trechos()` é o número de trechos, e `Ate` pode ir no máximo até ele;
- origem que não vem antes do destino.

Sem esta checagem, `livres` acessaria `Ocupados` fora do tamanho e o servidor cairia com `panic`. `||` é o `or`.

```
257-259  if c.livres(item.De, item.Ate) < 1 {
        return 0, fmt.Errorf("sem assento livre na carona %d", item.CaronaID)
    }
```

Falta lugar em algum trecho do caminho deste item. A reserva é recusada sem que nada tenha sido alterado: nenhuma carona foi tocada até aqui.

```
263      for _, item := range itens {
```

Fase 2: aplicação. Só chega aqui se **todos** os itens passaram pela fase 1.

```
264      b.acharCarona(item.CaronaID).ocupar(item.De, item.Ate)
```

`acharCarona` devolve `*Carona`, e `ocupar` tem receptor ponteiro: a carona dentro do slice é alterada de verdade.

A carona é procurada de novo em vez de reaproveitar o ponteiro da fase 1. É mais simples do que guardar uma lista de ponteiros, e o resultado é o mesmo, porque nada mudou o slice entre as duas fases.

```
267-272  r := Reserva{
        ID:          b.ProxID,
        Passageiro:  passageiro,
        Itens:        itens,
        Estado:      "pendente",
        Expira:      time.Now().Add(TempoDeReserva),
```

Monta a reserva: número do contador, dono, itens, estado `"pendente"` e o vencimento, que é o instante atual mais o prazo: `time.Now().Add(TempoDeReserva)`.

```
275      b.ProxID++
```

Avança o contador.

```
276      b.Reservas = append(b.Reservas, r)
```

Registra a reserva.

```
277      b.persistir()
```

Grava o arquivo já com os assentos ocupados e a reserva registrada, ainda com a trava.

279 `return r.ID, nil`

Devolve o número. O `defer` destrava, e só então outra goroutine pode ver o novo estado.

41. dados/dados.go (2 de 2) — pagamento, cancelamentos, expiração, listagens e funções internas

A segunda metade do arquivo. Primeiro as últimas funções públicas, que seguem o mesmo molde da primeira metade: travam, procuram, conferem, alteram, gravam. Depois as funções internas, que fazem o trabalho pesado e nunca travam.

Pagar

pbl/dados/dados.go · linhas 282–308

```
282 // Pagar confirma uma reserva pendente. Reserva paga nunca mais expira.
283 func (b *Banco) Pagar(passageiro string, reservaID int) error {
284     b.mu.Lock()
285     defer b.mu.Unlock()
286
287     r := b.acharReserva(reservaID)
288     if r == nil {
289         return fmt.Errorf("reserva %d nao existe", reservaID)
290     }
291     if r.Passageiro != passageiro {
292         return errors.New("esta reserva nao e sua")
293     }
294
295     switch r.Estado {
296     case "paga":
297         return errors.New("esta reserva ja foi paga")
298     case "expirada":
299         return errors.New("esta reserva expirou; os assentos voltaram para a fila")
300     case "cancelada":
301         return fmt.Errorf("esta reserva foi cancelada (%s)", r.Motivo)
302     }
303
304     r.Estado = "paga"
305     b.persistir()
306
307     return nil
308 }
```

283 `func (b *Banco) Pagar(passageiro string, reservaID int) error {`

Recebe o login de quem está pagando e o número da reserva; devolve só um erro. Não há dinheiro envolvido: pagar é mudar o estado da reserva.

284–285 `b.mu.Lock()`
`defer b.mu.Unlock()`

Trava: neste mesmo instante, a reserva pode estar sendo expirada pela varredura ou cancelada pelo motorista.

287 `r := b.acharReserva(reservaID)`

Ponteiro para a reserva dentro do slice, ou `nil`. Precisa ser ponteiro, porque o estado vai ser alterado mais abaixo.

288-290

```
if r == nil {  
    return fmt.Errorf("reserva %d nao existe", reservaID)  
}
```

Número inexistente.

291-293

```
if r.Passageiro != passageiro {  
    return errors.New("esta reserva nao e sua")  
}
```

Só o dono paga. O login vem da sessão do servidor, não de um campo do pedido: um cliente não consegue pagar em nome de outro mandando um nome diferente.

295 `switch r.Estado {`

`switch` sobre o estado. Só os três estados que impedem o pagamento têm `case`. `pendente` não tem, então nenhum `case` roda e a execução segue para depois do `switch`. Não há `default` porque não é preciso.

296-297

```
case "paga":  
    return errors.New("esta reserva ja foi paga")
```

Pagar duas vezes é recusado.

298-299

```
case "expirada":  
    return errors.New("esta reserva expirou; os assentos voltaram para a fila")
```

O prazo venceu e a varredura já passou: os assentos já foram devolvidos e podem estar com outra pessoa.

300-301

```
case "cancelada":  
    return fmt.Errorf("esta reserva foi cancelada (%s)", r.Motivo)
```

Recusa mostrando também o motivo do cancelamento.

304 `r.Estado = "paga"`

Altera a reserva dentro do slice, através do ponteiro. A partir daqui, `ExpirarVencidas` ignora esta reserva, porque só olha as pendentes.

Repare que nada aqui olha o horário, só o estado. Uma reserva cujo prazo venceu há 10 segundos, mas que a varredura ainda não alcançou, ainda pode ser paga (Parte VII).

305 `b.persistir()`

Grava a mudança.

307 `return nil`

Pagamento registrado.

CancelarReserva

pbl/dados/dados.go · linhas 310–334

```

310 // CancelarReserva desiste de uma reserva pendente ou paga e devolve os
311 // assentos de TODOS os trechos dela.
312 //
313 // E o espelho do Reservar: se a reserva cobre duas caronas, as duas recebem o
314 // assento de volta juntas, dentro do mesmo Lock.
315 func (b *Banco) CancelarReserva(passageiro string, reservaID int) error {
316     b.mu.Lock()
317     defer b.mu.Unlock()
318
319     r := b.acharReserva(reservaID)
320     if r == nil {
321         return fmt.Errorf("reserva %d nao existe", reservaID)
322     }
323     if r.Passageiro != passageiro {
324         return errors.New("esta reserva nao e sua")
325     }
326     if !r.ativa() {
327         return fmt.Errorf("esta reserva ja esta %s", r.Estado)
328     }
329
330     b.cancelar(r, "cancelada pelo passageiro")
331     b.persistir()
332
333     return nil
334 }

```

```

315 func (b *Banco) CancelarReserva(passageiro string, reservaID int) error {

```

O mesmo molde do `Pagar`: login e número, só erro de retorno.

```

316-317     b.mu.Lock()
317     defer b.mu.Unlock()

```

Trava.

```

319     r := b.acharReserva(reservaID)

```

Ponteiro para a reserva, ou `nil`.

```

320-322     if r == nil {
321         return fmt.Errorf("reserva %d nao existe", reservaID)
322     }

```

Número inexistente.

```

323-325     if r.Passageiro != passageiro {
324         return errors.New("esta reserva nao e sua")
325     }

```

Só o dono cancela.

```

326-328     if !r.ativa() {
327         return fmt.Errorf("esta reserva ja esta %s", r.Estado)
328     }

```

`ativa()` é um método interno de `Reserva`, verdadeiro para pendente e paga. `%s` mostra o estado atual na mensagem.

Esta checagem evita **devolver assentos duas vezes**. Uma reserva expirada já devolveu os dela; se pudesse ser cancelada, devolveria de novo e `Ocupados` ficaria negativo. `TestRegrasDeCancelarReserva` confere exatamente isso.

330 `b.cancelar(r, "cancelada pelo passageiro")`

Função interna que devolve os assentos de todos os itens, marca a reserva como cancelada e guarda o motivo. É a mesma que o `CancelarCarona` usa.

331 `b.persistir()`

Grava.

333 `return nil`

Cancelada.

CancelarCarona

pbl/dados/dados.go · linhas 336–375

```
336 // CancelarCarona tira uma carona do ar e cancela todas as reservas ativas que
337 // passam por ela. Devolve quantas reservas foram afetadas.
338 //
339 // Atencao ao caso da conexao: uma reserva Feira->Ilheus pode usar esta carona
340 // num trecho e OUTRA carona no trecho seguinte. Liberar so a parte desta
341 // carona deixaria o passageiro com meia viagem -- exatamente o que a
342 // atomicidade proibe. Por isso a reserva inteira e cancelada, e os assentos
343 // voltam em TODAS as caronas dela.
344 func (b *Banco) CancelarCarona(motorista string, caronaID int) (int, error) {
345     b.mu.Lock()
346     defer b.mu.Unlock()
347
348     c := b.acharCarona(caronaID)
349     if c == nil {
350         return 0, fmt.Errorf("carona %d nao existe", caronaID)
351     }
352     if c.Motorista != motorista {
353         return 0, errors.New("esta carona nao e sua")
354     }
355     if c.Cancelada {
356         return 0, errors.New("esta carona ja foi cancelada")
357     }
358
359     c.Cancelada = true
360
361     motivo := fmt.Sprintf("carona %d cancelada pelo motorista", caronaID)
362     afetadas := 0
363
364     for i := range b.Reservas {
365         r := &b.Reservas[i]
366         if r.ativa() && r.usa(caronaID) {
367             b.cancelar(r, motivo)
368             afetadas++
369         }
370     }
371
372     b.persistir()
373
374     return afetadas, nil
375 }
```

```
344 func (b *Banco) CancelarCarona(motorista string, caronaID int) (int, error) {
```

Recebe o login do motorista e o número da carona; devolve quantas reservas foram desfeitas.

```
345-346 b.mu.Lock()
defer b.mu.Unlock()
```

A trava garante que nenhuma reserva nova entra nesta carona enquanto as antigas estão sendo desfeitas.

`TestCancelarCaronaDuranteReservas` dispara 500 reservas junto com o cancelamento para provar isso.

```
348 c := b.acharCarona(caronaID)
```

Ponteiro para a carona, ou `nil`.

```
349-351 if c == nil {
    return 0, fmt.Errorf("carona %d nao existe", caronaID)
}
```

Número inexistente.

```
352-354 if c.Motorista != motorista {
    return 0, errors.New("esta carona nao e sua")
}
```

O servidor já confere que quem pede é motorista; aqui se confere que é o motorista dono desta carona.

```
355-357 if c.Cancelada {
    return 0, errors.New("esta carona ja foi cancelada")
}
```

Cancelar duas vezes é recusado.

```
359 c.Cancelada = true
```

Marca a carona através do ponteiro. A partir daqui, a busca a ignora e a reserva a recusa.

A carona **não é removida** do slice. Remover mudaria as posições dos elementos e deixaria reservas antigas guardando números de carona que não existem mais.

```
361 motivo := fmt.Sprintf("carona %d cancelada pelo motorista", caronaID)
```

Monta o texto do motivo uma vez só, antes do laço.

```
362 afetadas := 0
```

Contador de reservas desfeitas.

```
364 for i := range b.Reservas {
```

Percorre pelo **índice**, porque as reservas vão ser alteradas.

```
365 r := &b.Reservas[i]
```

Ponteiro para a reserva de verdade. Com `for _, r := range`, `r` seria uma cópia e o cancelamento não teria efeito nenhum.

```
366-369  if r.ativa() && r.usa(caronaID) {
        b.cancelar(r, motivo)
        afetadas++
    }
```

Reserva ativa com pelo menos um item nesta carona. `&&` só avalia o segundo lado se o primeiro for verdadeiro, como o `and` do Python.

A reserva é cancelada **inteira**, mesmo que tenha outro item em outra carona: `cancelar` devolve os assentos de todos os itens. É o cancelamento em cascata da Parte I. `afetadas++` conta mais uma.

```
372  b.persistir()
```

Uma única gravação no fim, com a carona e todas as reservas já alteradas.

```
374  return afetadas, nil
```

O servidor usa o número na mensagem.

ExpirarVencidas

pbl/dados/dados.go · linhas 377–406

```
377 // ExpirarVencidas devolve os assentos das reservas pendentes que passaram do
378 // prazo. Roda periodicamente numa goroutine do servidor. Devolve quantas
379 // reservas foram expiradas.
380 func (b *Banco) ExpirarVencidas() int {
381     b.mu.Lock()
382     defer b.mu.Unlock()
383
384     agora := time.Now()
385     expiradas := 0
386
387     for i := range b.Reservas {
388         // Ponteiro para o elemento REAL do slice. Com "for _, r := range"
389         // mexeríamos numa copia e o estado nunca mudaria.
390         r := &b.Reservas[i]
391
392         if r.Estado != "pendente" || agora.Before(r.Expira) {
393             continue
394         }
395
396         b.devolverAssentos(r)
397         r.Estado = "expirada"
398         expiradas++
399     }
400
401     if expiradas > 0 {
402         b.persistir()
403     }
404
405     return expiradas
406 }
```

```
380  func (b *Banco) ExpirarVencidas() int {
```

Não recebe nada e devolve quantas reservas expirou. É chamada pela goroutine de varredura do servidor e pelos testes.

```
381-382  b.mu.Lock()
        defer b.mu.Unlock()
```

Disputa a mesma trava das goroutines dos clientes. Por isso uma reserva nunca expira no meio de um pagamento.

```
384  agora := time.Now()
```

Lê o relógio uma vez só. Todas as reservas desta varredura são comparadas com o mesmo instante.

```
385  expiradas := 0
```

Contador.

```
387  for i := range b.Reservas {
```

Pelo índice, para poder alterar.

```
390  r := &b.Reservas[i]
```

Ponteiro para a reserva de verdade; o comentário logo acima dele no código repete o aviso sobre cópias.

```
392-394  if r.Estado != "pendente" || agora.Before(r.Expira) {
        continue
    }
```

Pula o que não interessa: reserva que não está pendente, ou pendente cujo vencimento ainda não chegou. `agora.Before(r.Expira)` é `agora < vencimento`.

O que sobra é exatamente o alvo: pendente com vencimento já passado. `continue` vai para a próxima volta do laço.

```
396  b.devolverAssentos(r)
```

Libera um assento em cada trecho de cada item.

```
397  r.Estado = "expirada"
```

Estado final.

```
398  expiradas++
```

Conta.

```
401-403  if expiradas > 0 {
        b.persistir()
    }
```

Só grava o arquivo se algo mudou. A varredura roda a cada 30 segundos, e regravar o arquivo inteiro sem mudança nenhuma seria desperdício.

```
405  return expiradas
```

O servidor imprime `expirei N reserva(s) nao paga(s)` quando o número é maior que zero.

MinhasCaronas

pbl/dados/dados.go · linhas 408–430

```

408 // MinhasCaronas lista as caronas de um motorista, prontas para exibir.
409 func (b *Banco) MinhasCaronas(motorista string) []string {
410     b.mu.Lock()
411     defer b.mu.Unlock()
412
413     var linhas []string
414
415     for _, c := range b.Caronas {
416         if c.Motorista != motorista {
417             continue
418         }
419
420         situacao := ""
421         if c.Cancelada {
422             situacao = " | CANCELADA"
423         }
424
425         linhas = append(linhas, fmt.Sprintf("carona %d | %s | %s | R$d/trecho | ocupados por trecho: %v
de %d%s",
426             c.ID, c.Data, strings.Join(c.Rota, " -> "), c.Preco, c.Ocupados, c.Assentos, situacao))
427     }
428
429     return linhas
430 }

```

409 `func (b *Banco) MinhasCaronas(motorista string) []string {`

Devolve linhas de texto prontas para imprimir. O servidor coloca isso direto no campo `Linhas` da resposta.

410-411 `b.mu.Lock()`
`defer b.mu.Unlock()`

Trava, mesmo só lendo.

413 `var linhas []string`

Slice `nil`. Se o motorista não tiver caronas, a função devolve `nil`, e o cliente mostra a mensagem de lista vazia.

415 `for _, c := range b.Caronas {`

Aqui `c` pode ser cópia, porque só lê.

416-418 `if c.Motorista != motorista {`
`continue`
`}`

Carona de outro motorista: `continue` pula para a próxima.

420 `situacao := ""`

Texto extra, vazio por padrão.

421-423 `if c.Cancelada {`
`situacao = " | CANCELADA"`
`}`

Marca as canceladas.

425-426

```
linhas = append(linhas, fmt.Sprintf("carona %d | %s | %s | R$%d/trecho | ocupados por trecho: %v
de %d%s",
    c.ID, c.Data, strings.Join(c.Rota, " -> "), c.Preco, c.Ocupados, c.Assentos, situacao))
```

Monta a linha. `%d` para números, `%s` para textos, `%v` para o slice `Ocupados`, que sai como `[1 2]`.
`strings.Join` junta a rota com setas. `R$` é texto comum dentro do formato.

429

```
return linhas
```

Devolve a lista.

PassageirosDaCarona

pbl/dados/dados.go · linhas 432-479

```
432 // PassageirosDaCarona mostra, trecho a trecho, quem esta em cada assento.
433 // So o motorista dono da carona pode ver.
434 func (b *Banco) PassageirosDaCarona(motorista string, caronaID int) ([]string, error) {
435     b.mu.Lock()
436     defer b.mu.Unlock()
437
438     c := b.acharCarona(caronaID)
439     if c == nil {
440         return nil, fmt.Errorf("carona %d nao existe", caronaID)
441     }
442     if c.Motorista != motorista {
443         return nil, errors.New("esta carona nao e sua")
444     }
445
446     situacao := ""
447     if c.Cancelada {
448         situacao = " | CANCELADA"
449     }
450
451     linhas := []string{fmt.Sprintf("carona %d | %s | %s%s",
452         c.ID, c.Data, strings.Join(c.Rota, " -> "), situacao)}
453
454     // Para cada trecho, procura as reservas ativas cujo caminho passa por ele.
455     for t := 0; t < c.trechos(); t++ {
456         var nomes []string
457
458         for _, r := range b.Reservas {
459             if !r.ativa() {
460                 continue
461             }
462             for _, item := range r.Itens {
463                 if item.CaronaID == caronaID && item.De <= t && t < item.Ate {
464                     nomes = append(nomes, fmt.Sprintf("%s (%s)", r.Passageiro, r.Estado))
465                 }
466             }
467         }
468
469         lista := "ninguem"
470         if len(nomes) > 0 {
471             lista = strings.Join(nomes, ", ")
472         }
473
474         linhas = append(linhas, fmt.Sprintf(" %s -> %s [%d/%d]: %s",
475             c.Rota[t], c.Rota[t+1], c.Ocupados[t], c.Assentos, lista))
476     }
477 }
```

```
478     return linhas, nil
479 }
```

```
434 func (b *Banco) PassageirosDaCarona(motorista string, caronaID int) ([]string, error) {
```

Devolve as linhas e um erro, porque pode falhar: carona inexistente ou de outro motorista.

```
435-436     b.mu.Lock()
        defer b.mu.Unlock()
```

Trava.

```
438     c := b.acharCarona(caronaID)
```

Ponteiro para a carona, ou `nil`.

```
439-441     if c == nil {
        return nil, fmt.Errorf("carona %d nao existe", caronaID)
    }
```

`return nil, ...: nil` é o valor zero de um slice.

```
442-444     if c.Motorista != motorista {
        return nil, errors.New("esta carona nao e sua")
    }
```

Só o dono vê quem está na carona.

```
446     situacao := ""
```

Texto extra do cabeçalho.

```
447-449     if c.Cancelada {
        situacao = " | CANCELADA"
    }
```

Marca a carona cancelada.

```
451-452     linhas := []string{fmt.Sprintf("carona %d | %s | %s",
        c.ID, c.Data, strings.Join(c.Rota, " -> "), situacao)}
```

Cria o slice já com um elemento: o cabeçalho. `[]string{x}` é o `[x]` do Python.

```
455     for t := 0; t < c.trechos(); t++ {
```

Um laço por trecho, de 0 até o último.

```
456     var nomes []string
```

Quem está neste trecho. Declarada dentro do laço, começa vazia a cada trecho.

```
458     for _, r := range b.Reservas {
```

Percorre todas as reservas procurando quem passa por este trecho. São três laços aninhados; para o tamanho do projeto, é rápido o suficiente.

```
459-461  if !r.ativa() {  
         continue  
        }
```

Reserva expirada ou cancelada não ocupa lugar e não aparece.

```
462  for _, item := range r.Itens {
```

Cada item da reserva.

```
463-465  if item.CaronaID == caronaID && item.De <= t && t < item.Ate {  
         nomes = append(nomes, fmt.Sprintf("%s (%s)", r.Passageiro, r.Estado))  
        }
```

O item é desta carona e o caminho dele passa pelo trecho `t`: começa em `t` ou antes (`De <= t`) e termina depois (`t < Ate`). É a mesma regra do `range(de, ate)`.

Se passa, acrescenta o nome e o estado, como `bruno (paga)`. Uma reserva com dois itens na mesma carona apareceria duas vezes, e estaria certo: seriam dois assentos.

```
469  lista := "ninguem"
```

Texto padrão para trecho vazio.

```
470-472  if len(nomes) > 0 {  
         lista = strings.Join(nomes, ", ")  
        }
```

Junta os nomes separados por vírgula.

```
474-475  linhas = append(linhas, fmt.Sprintf(" %s -> %s [%d/%d]: %s",  
         c.Rota[t], c.Rota[t+1], c.Ocupados[t], c.Assentos, lista))
```

A linha do trecho: `c.Rota[t]` e `c.Rota[t+1]` são as duas pontas do trecho `t`; depois vêm ocupados e total, e a lista de nomes. A contagem vem de `Ocupados` e os nomes vêm das reservas: são duas fontes da mesma informação, e precisam concordar.

```
478  return linhas, nil
```

Deu certo.

MinhasReservas

pbl/dados/dados.go · linhas 481–506

```
481 // MinhasReservas lista as reservas de um passageiro, prontas para exibir.  
482 func (b *Banco) MinhasReservas(passageiro string) []string {  
483     b.mu.Lock()  
484     defer b.mu.Unlock()  
485  
486     var linhas []string  
487  
488     for _, r := range b.Reservas {  
489         if r.Passageiro != passageiro {  
490             continue  
491         }  
492  
493         detalhe := ""  
494         switch {  
495             case r.Estado == "pendente":
```

```

496         detalhe = fmt.Sprintf(" | expira em %s", time.Until(r.Expira).Round(time.Second))
497     case r.Motivo != "":
498         detalhe = " | " + r.Motivo
499     }
500
501     linhas = append(linhas, fmt.Sprintf("reserva %d | %s | %d trecho(s)%s",
502         r.ID, r.Estado, len(r.Itens), detalhe))
503 }
504
505 return linhas
506 }

```

```

482 func (b *Banco) MinhasReservas(passageiro string) []string {

```

Linhas prontas com as reservas de um passageiro.

```

483-484 b.mu.Lock()
      defer b.mu.Unlock()

```

Trava.

```

486 var linhas []string

```

Começa `nil`.

```

488 for _, r := range b.Reservas {

```

Só lê, então cópia serve.

```

489-491 if r.Passageiro != passageiro {
      continue
      }

```

Reserva de outra pessoa: pula.

```

493 detalhe := ""

```

Complemento da linha, vazio por padrão.

```

494 switch {

```

`switch` sem nada depois: cada `case` é uma condição, testadas de cima para baixo, e só a primeira verdadeira roda. É um `if / elif`.

```

495-496 case r.Estado == "pendente":
      detalhe = fmt.Sprintf(" | expira em %s", time.Until(r.Expira).Round(time.Second))

```

Pendente mostra quanto falta. `time.Until(r.Expira)` é o vencimento menos agora, e `Round(time.Second)` arredonda para segundos inteiros: sai como `9m46s`. Se o prazo já passou mas a varredura ainda não chegou, a duração é negativa e aparece como `-5s`.

```

497-498 case r.Motivo != "":
      detalhe = " | " + r.Motivo

```

Cancelada mostra o motivo. Paga e expirada não têm motivo, então não entram em nenhum `case` e ficam sem detalhe. `+` junta textos, como no Python.

501-502 `linhas = append(linhas, fmt.Sprintf("reserva %d | %s | %d trecho(s)%s",
r.ID, r.Estado, len(r.Itens), detalhe))`

`len(r.Itens)` mostra quantos trechos de carona a reserva tem.

505 `return linhas`

Devolve a lista.

Funções internas: as buscas

Daqui para baixo, **nenhuma função trava**. Todas são chamadas de dentro de uma função pública que já está com a trava.

pbl/dados/dados.go · linhas 513-545

```
513 // buscarDiretas acha caronas que sozinhas levam de origem ate destino.
514 func (b *Banco) buscarDiretas(origem, destino, data string) []protocolo.Opcao {
515     var opcoes []protocolo.Opcao
516
517     for _, c := range b.Caronas {
518         if c.Cancelada || c.Data != data {
519             continue
520         }
521
522         de := c.indice(origem)
523         ate := c.indice(destino)
524
525         // As duas cidades precisam estar na rota, e a origem precisa vir
526         // ANTES do destino: a carona so anda em um sentido.
527         if de == -1 || ate == -1 || de >= ate {
528             continue
529         }
530
531         livres := c.livres(de, ate)
532         if livres < 1 {
533             continue
534         }
535
536         opcoes = append(opcoes, protocolo.Opcao{
537             Itens: []protocolo.Item{{CaronaID: c.ID, De: de, Ate: ate}},
538             Preco: c.Preco * (ate - de),
539             Resumo: fmt.Sprintf("direto: carona %d (%s) %s | %d livre(s)",
540                 c.ID, c.Motorista, strings.Join(c.Rota[de:ate+1], " -> "), livres),
541         })
542     }
543
544     return opcoes
545 }
```

514 `func (b *Banco) buscarDiretas(origem, destino, data string) []protocolo.Opcao {`

Minúscula e sem trava: só o `Buscar` a chama.

515 `var opcoes []protocolo.Opcao`

Começa `nil`.

```
517 for _, c := range b.Caronas {
```

Cópia de cada carona: só leitura.

```
518-520 if c.Cancelada || c.Data != data {  
        continue  
    }
```

Condições 1 e 2 da busca direta (Parte I): mesma data e não cancelada.

```
522-523 de := c.indice(origem)  
        ate := c.indice(destino)
```

Posição da origem e do destino na rota, ou `-1` se a cidade não está nela.

```
527-529 if de == -1 || ate == -1 || de >= ate {  
        continue  
    }
```

Condições 3 e 4: as duas cidades estão na rota, e a origem vem antes. `de >= ate` também descarta origem igual ao destino.

```
531 livres := c.livres(de, ate)
```

Condição 5: vagas em todo o caminho. A variável tem o mesmo nome do método, sem conflito, porque o método é chamado com `c.` na frente.

```
532-534 if livres < 1 {  
        continue  
    }
```

Sem vaga, não é opção.

```
536 opcoes = append(opcoes, protocolo.Opcao{
```

Acrescenta uma opção montada ali mesmo, com um literal composto de outro pacote.

```
537 Itens: []protocolo.Item{{CaronaID: c.ID, De: de, Ate: ate}},
```

Um slice com um único `Item`. As chaves duplas são o slice por fora e o item por dentro; o tipo do item interno não precisa ser repetido.

```
538 Preco: c.Preco * (ate - de),
```

Preço por trecho vezes a quantidade de trechos.

```
539-540 Resumo: fmt.Sprintf("direto: carona %d (%s) %s | %d livre(s)",  
        c.ID, c.Motorista, strings.Join(c.Rota[de:ate+1], " -> "), livres),
```

O texto que o cliente mostra. `c.Rota[de:ate+1]` recorta a rota da origem até o destino, incluindo o destino. O recorte não copia os nomes; como só é lido para montar o texto, não há problema.

```
544 return opcoes
```

As opções diretas.

```

547 // buscarComConexao acha pares de caronas que, juntas, levam de origem ate
548 // destino trocando de veículo numa cidade do meio.
549 //
550 // E aqui que a atomicidade do Reservar ganha sentido: uma opcao destas tem
551 // DOIS itens, e nao adianta conseguir o primeiro e perder o segundo.
552 func (b *Banco) buscarComConexao(origem, destino, data string) []protocolo.Opcao {
553     var opcoes []protocolo.Opcao
554
555     for _, primeira := range b.Caronas {
556         if primeira.Cancelada || primeira.Data != data {
557             continue
558         }
559
560         de := primeira.indice(origem)
561         if de == -1 {
562             continue
563         }
564
565         // Testa cada cidade depois da origem como ponto de baldeacao.
566         for meio := de + 1; meio < len(primeira.Rota); meio++ {
567             baldeacao := primeira.Rota[meio]
568
569             // Se a primeira carona ja chega no destino, isso e viagem direta:
570             // a outra funcao ja cuidou disso.
571             if baldeacao == destino {
572                 continue
573             }
574             if primeira.livres(de, meio) < 1 {
575                 continue
576             }
577
578             for _, segunda := range b.Caronas {
579                 if segunda.ID == primeira.ID || segunda.Cancelada || segunda.Data != data {
580                     continue
581                 }
582
583                 de2 := segunda.indice(baldeacao)
584                 ate2 := segunda.indice(destino)
585                 if de2 == -1 || ate2 == -1 || de2 >= ate2 {
586                     continue
587                 }
588                 if segunda.livres(de2, ate2) < 1 {
589                     continue
590                 }
591
592                 opcoes = append(opcoes, protocolo.Opcao{
593                     Itens: []protocolo.Item{
594                         {CaronaID: primeira.ID, De: de, Ate: meio},
595                         {CaronaID: segunda.ID, De: de2, Ate: ate2},
596                     },
597                     Preco: primeira.Preco*(meio-de) + segunda.Preco*(ate2-de2),
598                     Resumo: fmt.Sprintf("conexao em %s: carona %d (%s) + carona %d (%s)",
599                         baldeacao, primeira.ID, primeira.Motorista, segunda.ID, segunda.Motorista),
600                 })
601             }
602         }
603     }
604
605     return opcoes
606 }

```

552 `func (b *Banco) buscarComConexao(origem, destino, data string) []protocolo.Opcao {`

A busca de viagens com uma baldeação.

```
553 var opcoes []protocolo.Opcao
```

Começa `nil`.

```
555 for _, primeira := range b.Caronas {
```

Cada carona é candidata a primeira perna.

```
556-558 if primeira.Cancelada || primeira.Data != data {
        continue
    }
```

Não cancelada e da data pedida.

```
560 de := primeira.indice(origem)
```

Posição da origem na primeira carona.

```
561-563 if de == -1 {
        continue
    }
```

A primeira carona precisa passar pela origem.

```
566 for meio := de + 1; meio < len(primeira.Rota); meio++ {
```

Cada cidade depois da origem é candidata a baldeação; `meio` é a posição dela na rota da primeira carona.

```
567 baldeacao := primeira.Rota[meio]
```

O nome da cidade candidata.

```
571-573 if baldeacao == destino {
        continue
    }
```

Se a primeira carona já chega ao destino, é viagem direta e já foi listada.

```
574-576 if primeira.livres(de, meio) < 1 {
        continue
    }
```

A primeira perna precisa de vaga da origem até a baldeação.

```
578 for _, segunda := range b.Caronas {
```

Para cada baldeação, procura a segunda perna em todas as caronas.

```
579-581 if segunda.ID == primeira.ID || segunda.Cancelada || segunda.Data != data {
        continue
    }
```

Outra carona, não cancelada, da mesma data.

```
583-584 de2 := segunda.indice(baldeacao)
        ate2 := segunda.indice(destino)
```

Posições da baldeação e do destino na segunda carona.

```
585-587 if de2 == -1 || ate2 == -1 || de2 >= ate2 {
        continue
    }
```

A segunda precisa passar pela baldeação e depois pelo destino.

```
588-590 if segunda.livres(de2, ate2) < 1 {
        continue
    }
```

E ter vaga nesse caminho.

```
592 opcoes = append(opcoes, protocolo.Opcao{
```

Monta a opção com conexão.

```
593-595 Itens: []protocolo.Item{
        {CaronaID: primeira.ID, De: de, Ate: meio},
        {CaronaID: segunda.ID, De: de2, Ate: ate2},
```

Dois itens, um por perna, cada um numa linha terminada em vírgula.

```
597 Preco: primeira.Preco*(meio-de) + segunda.Preco*(ate2-de2),
```

Soma das duas pernas. O `gofmt` tira os espaços em volta do `*` quando a expressão também tem `+`, para destacar que a multiplicação acontece primeiro.

```
598-599 Resumo: fmt.Sprintf("conexao em %s: carona %d (%s) + carona %d (%s)",
        baldeacao, primeira.ID, primeira.Motorista, segunda.ID, segunda.Motorista),
```

Texto com a cidade de baldeação e as duas caronas.

```
605 return opcoes
```

As opções com conexão.

Funções internas: cancelar, devolver e gravar

pbl/dados/dados.go · linhas 608–613

```
608 // cancelar desfaz uma reserva ativa: devolve os assentos e registra o motivo.
609 func (b *Banco) cancelar(r *Reserva, motivo string) {
610     b.devolverAssentos(r)
611     r.Estado = "cancelada"
612     r.Motivo = motivo
613 }
```

```
609 func (b *Banco) cancelar(r *Reserva, motivo string) {
```

Recebe um ponteiro para a reserva, para alterá-la, e o texto do motivo.

```
610 b.devolverAssentos(r)
```

Primeiro devolve os assentos.

611–612 `r.Estado = "cancelada"`
 `r.Motivo = motivo`

Depois marca e registra o motivo.

pbl/dados/dados.go · linhas 615–623

```
615 // devolverAssentos libera um assento em cada trecho de cada item da reserva.
616 // Usada pela expiracao e pelos dois cancelamentos.
617 func (b *Banco) devolverAssentos(r *Reserva) {
618     for _, item := range r.Itens {
619         if c := b.acharCarona(item.CaronaID); c != nil {
620             c.liberar(item.De, item.Ate)
621         }
622     }
623 }
```

617 `func (b *Banco) devolverAssentos(r *Reserva) {`

Usada pela expiração e pelos dois cancelamentos: um único lugar que sabe devolver assentos.

618 `for _, item := range r.Itens {`

Cada item da reserva.

619–621 `if c := b.acharCarona(item.CaronaID); c != nil {`
 `c.liberar(item.De, item.Ate)`
 `}`

`if` com declaração: acha a carona e, se ela existir, libera o caminho do item. A checagem de `nil` é só defesa: toda reserva foi criada com caronas existentes, e caronas nunca são removidas.

pbl/dados/dados.go · linhas 625–644

```
625 // persistir grava o banco no arquivo. Se nao houver arquivo configurado
626 // (como nos testes), nao faz nada.
627 //
628 // Nao trava: quem chama ja esta com o mutex na mao. Gravar aqui dentro
629 // garante que o arquivo nunca pega o estado pela metade.
630 func (b *Banco) persistir() {
631     if b.arquivo == "" {
632         return
633     }
634
635     conteudo, err := json.MarshalIndent(b, "", " ")
636     if err != nil {
637         fmt.Println("erro ao converter os dados:", err)
638         return
639     }
640
641     if err := os.WriteFile(b.arquivo, conteudo, 0o644); err != nil {
642         fmt.Println("erro ao gravar", b.arquivo, ":", err)
643     }
644 }
```

```
630 func (b *Banco) persistir() {
```

Grava o banco inteiro no arquivo. Não trava: quem chama já está com a trava, e gravar sob a trava garante que o arquivo nunca pega o estado no meio de uma operação.

```
631-633 if b.arquivo == "" {
        return
    }
```

Sem caminho, que é o caso dos testes, não grava nada. `return` sozinho encerra uma função que não devolve valor.

```
635 conteudo, err := json.MarshalIndent(b, "", " ")
```

Converte o banco inteiro em JSON indentado: nenhum prefixo e dois espaços por nível. Só campos exportados entram; `mu` e `arquivo` ficam de fora.

```
636-639 if err != nil {
        fmt.Println("erro ao converter os dados:", err)
        return
    }
```

Não devolve erro, só imprime. Uma falha ao gravar não desfaz a operação que já aconteceu na memória (Parte VII).

```
641-643 if err := os.WriteFile(b.arquivo, conteudo, 0o644); err != nil {
        fmt.Println("erro ao gravar", b.arquivo, ":", err)
    }
```

`os.WriteFile` substitui o arquivo inteiro. `0o644` são as permissões, usadas só se o arquivo precisar ser criado: o dono lê e escreve, os outros só leem. Se o processo morrer exatamente durante esta gravação, o arquivo pode ficar pela metade.

Funções internas: achar

pbl/dados/dados.go · linhas 646–658

```
646 // acharCarona devolve um PONTEIRO para a carona dentro do slice.
647 //
648 // Repare no "for i := range" em vez de "for _, c := range": o range por valor
649 // entrega uma COPIA de cada carona, e alterar a copia nao mudaria nada. Para
650 // modificar o elemento de verdade e preciso pegar o endereco dele.
651 func (b *Banco) acharCarona(id int) *Carona {
652     for i := range b.Caronas {
653         if b.Caronas[i].ID == id {
654             return &b.Caronas[i]
655         }
656     }
657     return nil
658 }
```

```
651 func (b *Banco) acharCarona(id int) *Carona {
```

Devolve `*Carona`: um ponteiro para dentro do slice, ou `nil`.

```
652 for i := range b.Caronas {
```

Só o índice.

```
653-655 if b.Caronas[i].ID == id {  
        return &b.Caronas[i]  
    }
```

`&b.Caronas[i]` é o endereço do elemento guardado no slice. Com `for _, c := range`, `&c` seria o endereço da cópia, e qualquer alteração feita através dele se perderia. O comentário acima da função registra esse cuidado.

```
657 return nil
```

Não achou.

pbl/dados/dados.go · linhas 660–667

```
660 func (b *Banco) acharReserva(id int) *Reserva {  
661     for i := range b.Reservas {  
662         if b.Reservas[i].ID == id {  
663             return &b.Reservas[i]  
664         }  
665     }  
666     return nil  
667 }
```

```
660-666 func (b *Banco) acharReserva(id int) *Reserva {  
        for i := range b.Reservas {  
            if b.Reservas[i].ID == id {  
                return &b.Reservas[i]  
            }  
        }  
        return nil  
    }
```

Idêntica, para reservas.

Métodos de Reserva

pbl/dados/dados.go · linhas 669–672

```
669 // ativa diz se a reserva ainda segura assentos.  
670 func (r Reserva) ativa() bool {  
671     return r.Estado == "pendente" || r.Estado == "paga"  
672 }
```

```
670 func (r Reserva) ativa() bool {
```

Receptor valor, `(r Reserva)`: só lê, então recebe uma cópia.

```
671 return r.Estado == "pendente" || r.Estado == "paga"
```

Devolve direto o resultado de uma expressão booleana.

pbl/dados/dados.go · linhas 674–682

```

674 // usa diz se algum item da reserva e da carona indicada.
675 func (r Reserva) usa(caronaID int) bool {
676     for _, item := range r.Itens {
677         if item.CaronaID == caronaID {
678             return true
679         }
680     }
681     return false
682 }

```

```
675 func (r Reserva) usa(caronaID int) bool {
```

Diz se a reserva tem algum item na carona indicada.

```
676 for _, item := range r.Itens {
```

Cada item.

```

677-679 if item.CaronaID == caronaID {
        return true
    }

```

Devolve `true` no primeiro que bate, sem olhar o resto.

```
681 return false
```

Nenhum bateu.

Métodos de Carona

pbl/dados/dados.go · linhas 684–687

```

684 // trechos diz quantos trechos a carona tem: uma cidade a menos que a rota.
685 func (c Carona) trechos() int {
686     return len(c.Rota) - 1
687 }

```

```
685 func (c Carona) trechos() int {
```

Receptor valor: só lê.

```
686 return len(c.Rota) - 1
```

Uma cidade a menos que a rota.

pbl/dados/dados.go · linhas 689–697

```

689 // indice devolve a posicao da cidade na rota, ou -1 se ela nao estiver la.
690 func (c Carona) indice(cidade string) int {
691     for i, nome := range c.Rota {
692         if nome == cidade {
693             return i
694         }
695     }
696     return -1
697 }

```

```
690 func (c Carona) indice(cidade string) int {
```

Posição de uma cidade na rota.

```
691 for i, nome := range c.Rota {
```

Índice e nome, como `enumerate`.

```
692-694 if nome == cidade {  
        return i  
    }
```

Textos são comparados pelo conteúdo com `==`, como no Python. Devolve a primeira posição encontrada.

```
696 return -1
```

Não está na rota. Usar `-1` para "não achei" é o mesmo costume do `str.find` do Python.

pbl/dados/dados.go · linhas 699–715

```
699 // livres diz quantos assentos estao livres em TODO o caminho de "de" ate "ate".  
700 //  
701 // Viajar da cidade "de" ate a cidade "ate" usa os trechos de, de+1, ..., ate-1.  
702 // O passageiro precisa do mesmo assento no caminho inteiro, entao o que vale e  
703 // o trecho MAIS CHEIO: e ele que limita.  
704 func (c Carona) livres(de, ate int) int {  
705     menor := c.Assentos  
706  
707     for t := de; t < ate; t++ {  
708         livre := c.Assentos - c.Ocupados[t]  
709         if livre < menor {  
710             menor = livre  
711         }  
712     }  
713  
714     return menor  
715 }
```

```
704 func (c Carona) livres(de, ate int) int {
```

`de, ate int`: dois parâmetros do mesmo tipo. É a regra "o trecho mais cheio manda", da Parte I.

```
705 menor := c.Assentos
```

Começa pelo máximo possível.

```
707 for t := de; t < ate; t++ {
```

Os trechos do caminho: o `range(de, ate)`.

```
708 livre := c.Assentos - c.Ocupados[t]
```

Vagas neste trecho.

```
709-711 if livre < menor {  
        menor = livre  
    }
```

Guarda o menor.

714 `return` menor

O trecho mais cheio define a disponibilidade. Em Python: `min(assentos - o for o in ocupados[de:ate])`. Se `de == ate`, o laço não roda e o total volta; quem chama nunca passa esse caso.

pbl/dados/dados.go · linhas 717–722

```
717 // ocupar marca um assento em cada trecho do caminho.
718 func (c *Carona) ocupar(de, ate int) {
719     for t := de; t < ate; t++ {
720         c.Ocupados[t]++
721     }
722 }
```

718 `func` (c *Carona) ocupar(de, ate int) {

Receptor **ponteiro**, (c *Carona): altera a carona de verdade.

719 `for` t := de; t < ate; t++ {

Os trechos do caminho.

720 `c.Ocupados[t]++`

Um assento a mais em cada trecho. Sem o mutex, é esta linha que perde incrementos: lê, soma e escreve são três passos, e duas goroutines podem se intercalar entre eles.

pbl/dados/dados.go · linhas 724–729

```
724 // liberar devolve um assento em cada trecho do caminho.
725 func (c *Carona) liberar(de, ate int) {
726     for t := de; t < ate; t++ {
727         c.Ocupados[t]--
728     }
729 }
```

725 `func` (c *Carona) liberar(de, ate int) {

O inverso de `ocupar`.

726 `for` t := de; t < ate; t++ {

Os mesmos trechos.

727 `c.Ocupados[t]--`

Um assento a menos em cada trecho.

42. `servidor/main.go` — conexões, sessões e o roteador

O servidor é pequeno porque não tem regra de negócio: ele abre a porta, cria uma goroutine para cada cliente, guarda quem está logado em cada conexão e traduz cada pedido numa chamada ao `dados`. Toda a concorrência

perigosa está do outro lado, protegida pelo mutex do `Banco`.

Cabeçalho e imports

pbl/servidor/main.go · linhas 1–14

```
1 package main
2
3 import (
4     "bufio"
5     "errors"
6     "fmt"
7     "io"
8     "net"
9     "os"
10    "time"
11
12    "vaijundo/dados"
13    "vaijundo/protocolo"
14 )
```

1 package main

Pacote `main`: esta pasta gera um executável, e a execução começa pela função `main`.

3–14 import (

```
    "bufio"
    "errors"
    "fmt"
    "io"
    "net"
    "os"
    "time"

    "vaijundo/dados"
    "vaijundo/protocolo"
)
```

- `bufio`: o leitor com buffer de cada conexão;
- `errors`: `errors.Is` para classificar erros;
- `fmt`: mensagens no terminal e textos das respostas;
- `io`: a constante `io.EOF`;
- `net`: abrir a porta e aceitar conexões;
- `os`: variáveis de ambiente;
- `time`: intervalos;
- `vaijundo/dados` e `vaijundo/protocolo`: os dois pacotes do projeto.

Constante e sessão

pbl/servidor/main.go · linhas 16–18

```
16 // intervaloPadrao e de quanto em quanto tempo o servidor procura reservas
17 // vencidas. Pode ser encurtado por variavel de ambiente na hora de demonstrar.
18 const intervaloPadrao = 30 * time.Second
```

18 `const intervaloPadrao = 30 * time.Second`

Constante do pacote, minúscula (só este arquivo usa). `30 * time.Second` é uma `Duration` de 30 segundos: o intervalo da varredura quando nenhuma variável de ambiente muda isso.

pbl/servidor/main.go · linhas 20–27

```
20 // sessao guarda quem esta logado NAQUELA conexao.
21 //
22 // Repare que ela NAO precisa de mutex: cada goroutine de atender() cria a sua
23 // propria sessao como variavel local, entao ninguem compartilha nada. So o
24 // Banco, que e o mesmo para todos, precisa de protecao.
25 type sessao struct {
26     usuario dados.Usuario
27 }
```

25 `type sessao struct {`

Struct minúscula, usada só neste arquivo. O comentário acima dela explica por que não precisa de mutex.

26 `usuario dados.Usuario`

A conta logada nesta conexão. Enquanto ninguém entrou, é o valor zero, com `Login` vazio. Guarda a conta inteira, inclusive o tipo, para as checagens de motorista não precisarem consultar o banco de novo.

main

pbl/servidor/main.go · linhas 29–58

```
29 func main() {
30     varredura := configurarTempos()
31     banco := dados.Carregar(arquivoDeDados())
32
33     fmt.Printf("reserva expira em %s, varrendo a cada %s\n",
34         dados.TempoDeReserva, varredura)
35
36     // Goroutine de fundo: fica devolvendo assentos de reservas nao pagas.
37     go expirarPeriodicamente(banco, varredura)
38
39     ouvinte, err := net.Listen("tcp", ":8080")
40     if err != nil {
41         fmt.Println("erro ao abrir a porta:", err)
42         return
43     }
44     defer ouvinte.Close()
45
46     fmt.Println("servidor ouvindo em :8080")
47
48     for {
49         conexao, err := ouvinte.Accept()
50         if err != nil {
51             fmt.Println("erro ao aceitar conexao:", err)
52             continue
53         }
54
55         fmt.Println("cliente conectado:", conexao.RemoteAddr())
56         go atender(conexao, banco)
57     }
58 }
```

29 `func main() {`

Ponto de entrada. Não recebe argumentos e não devolve nada; em Go, argumentos de linha de comando e código de saída usam os pacotes `os` e `flag`.

30 `varredura := configurarTempos()`

Lê as variáveis de ambiente de tempo antes de tudo: o prazo da reserva precisa estar ajustado antes de a primeira reserva ser criada.

31 `banco := dados.Carregar(arquivoDeDados())`

Cria o **único** `Banco` do processo, carregando o arquivo. Esta variável é o `*Banco` que todas as goroutines vão compartilhar.

33-34 `fmt.Printf("reserva expira em %s, varrendo a cada %s\n",
dados.TempoDeReserva, varredura)`

Mostra os tempos em uso. `%s` formata as durações como `10m0s` e `30s`. A chamada continua na linha de baixo porque a primeira termina em vírgula.

37 `go expirarPeriodicamente(banco, varredura)`

A primeira goroutine: a varredura de expiração. Ela começa a rodar em paralelo, e `main` segue imediatamente para a próxima linha.

39 `ouvinte, err := net.Listen("tcp", ":8080")`

Abre a porta 8080. `":8080"`, sem endereço antes dos dois-pontos, quer dizer "em todos os endereços da máquina". Com `"localhost:8080"`, só conexões da própria máquina seriam aceitas, e o servidor não funcionaria dentro de um container.

40-43 `if err != nil {
 fmt.Println("erro ao abrir a porta:", err)
 return
}`

Não conseguiu abrir a porta, normalmente porque outro programa já está nela: imprime e sai. `return` em `main` encerra o programa.

44 `defer ouvinte.Close()`

Fecharia a porta ao sair de `main`. Na prática, `main` nunca sai do laço abaixo, e o sistema operacional libera a porta quando o processo termina; o `defer` fica por boa prática.

46 `fmt.Println("servidor ouvindo em :8080")`

Aviso de pronto. Os scripts de teste esperam esta frase aparecer antes de conectar.

48 `for {`

Laço infinito: o servidor aceita clientes para sempre.

49 `conexao, err := ouvinte.Accept()`

Bloqueia até um cliente conectar e devolve a conexão com ele. Cada chamada entrega uma conexão nova e independente.

```
50-53  if err != nil {
        fmt.Println("erro ao aceitar conexao:", err)
        continue
    }
```

Um erro ao aceitar um cliente **não** derruba o servidor: imprime e volta a esperar o próximo. Compare com o erro do `Listen` acima, que encerra o programa.

```
55  fmt.Println("cliente conectado:", conexao.RemoteAddr())
```

`RemoteAddr()` é o endereço do cliente, como `:::1:59058`.

```
56  go atender(conexao, banco)
```

A linha que define o servidor. Cada cliente ganha uma goroutine própria, e o laço volta na hora para o `Accept`, pronto para o próximo.

Sem o `go`, o servidor atenderia um cliente de cada vez: o segundo só seria aceito quando o primeiro desconectasse. Todas as goroutines recebem o mesmo `banco`; cada uma recebe a sua `conexao`.

A varredura de expiração

pbl/servidor/main.go · linhas 60–73

```
60  // expirarPeriodicamente roda para sempre, numa goroutine propria.
61  //
62  // Ela tambem mexe no Banco, entao disputa o mesmo mutex que as goroutines dos
63  // clientes -- e e justamente por isso que nao da problema.
64  func expirarPeriodicamente(banco *dados.Banco, intervalo time.Duration) {
65      relógio := time.NewTicker(intervalo)
66      defer relógio.Stop()
67
68      for range relógio.C {
69          if n := banco.ExpirarVencidas(); n > 0 {
70              fmt.Printf("expirei %d reserva(s) nao paga(s)\n", n)
71          }
72      }
73  }
```

```
64  func expirarPeriodicamente(banco *dados.Banco, intervalo time.Duration) {
```

Recebe o banco e o intervalo. Nunca retorna.

```
65  relógio := time.NewTicker(intervalo)
```

Um `Ticker` entrega um valor no canal `relógio.C` a cada intervalo.

```
66  defer relógio.Stop()
```

Pararia o ticker ao sair. Como a função nunca sai, é só boa prática.

```
68  for range relógio.C {
```

`range` sobre o canal: uma volta a cada tique, esperando entre eles sem gastar CPU. O valor recebido (a hora do tique) não interessa, e por isso não há variável antes do `range`. É o `while True: sleep(intervalo)` do Python, com a diferença de que o tempo gasto na varredura não empurra os tiques seguintes.

```
69-71  if n := banco.ExpirarVencidas(); n > 0 {
        fmt.Printf("expirei %d reserva(s) nao paga(s)\n", n)
    }
```

Chama a varredura, que trava o banco. Só imprime quando algo expirou, para não encher o terminal a cada 30 segundos.

Configuração por variáveis de ambiente

pbl/servidor/main.go · linhas 75–82

```
75 // arquivoDeDados diz onde o banco JSON fica gravado.
76 // No Docker apontamos para um volume, para os dados sobreviverem ao container.
77 func arquivoDeDados() string {
78     if v := os.Getenv("VAIJUNTO_DADOS"); v != "" {
79         return v
80     }
81     return "vaijunto.json"
82 }
```

```
77 func arquivoDeDados() string {
```

Decide onde fica o arquivo de dados.

```
78-80  if v := os.Getenv("VAIJUNTO_DADOS"); v != "" {
        return v
    }
```

`os.Getenv` devolve "" quando a variável não existe, então "vazia" e "não definida" são tratadas igual.

```
81     return "vaijunto.json"
```

Caminho relativo: o arquivo fica na pasta de onde o servidor foi iniciado.

pbl/servidor/main.go · linhas 84–103

```
84 // configurarTempos le as variaveis de ambiente que encurtam os prazos.
85 // Para demonstrar a expiracao sem esperar 10 minutos:
86 //
87 //     VAIJUNTO_RESERVA=20s VAIJUNTO_VARREDURA=5s go run ./servidor
88 func configurarTempos() time.Duration {
89     if v := os.Getenv("VAIJUNTO_RESERVA"); v != "" {
90         if d, err := time.ParseDuration(v); err == nil {
91             dados.TempoDeReserva = d
92         }
93     }
94
95     intervalo := intervaloPadrao
96     if v := os.Getenv("VAIJUNTO_VARREDURA"); v != "" {
97         if d, err := time.ParseDuration(v); err == nil {
98             intervalo = d
99         }
100    }
101
102    return intervalo
103 }
```

```
88 func configurarTempos() time.Duration {
```

Ajusta o prazo da reserva direto na variável do pacote `dados` e devolve o intervalo da varredura.

```
89-91 if v := os.Getenv("VAIJUNTO_RESERVA"); v != "" {
    if d, err := time.ParseDuration(v); err == nil {
        dados.TempoDeReserva = d
    }
}
```

Dois `if` com declaração, um dentro do outro. `time.ParseDuration` entende textos como `30s`, `10m` e `1h30m`. Se o texto for inválido, `err` não é `nil` e o valor padrão continua valendo, sem aviso.

```
95 intervalo := intervaloPadrao
```

Começa pelo padrão de 30 segundos.

```
96-98 if v := os.Getenv("VAIJUNTO_VARREDURA"); v != "" {
    if d, err := time.ParseDuration(v); err == nil {
        intervalo = d
    }
}
```

A mesma lógica, para o intervalo.

```
102 return intervalo
```

Devolve o intervalo decidido.

atender: a vida de uma conexão

pbl/servidor/main.go · linhas 105–148

```
105 func atender(conexao net.Conn, banco *dados.Banco) {
106     defer conexao.Close()
107
108     leitor := bufio.NewReader(conexao)
109
110     // A sessao nasce e morre junto com esta conexao.
111     var s sessao
112
113     for {
114         pedido, err := protocolo.LerPedido(leitor)
115
116         // A linha chegou inteira, mas nao e JSON valido: o problema e da
117         // mensagem, nao da conexao. Avisa o cliente e continua atendendo.
118         if errors.Is(err, protocolo.ErrMensagemInvalida) {
119             protocolo.EnviaResposta(conexao, protocolo.Resposta{OK: false, Erro: err.Error()})
120             continue
121         }
122
123         // Qualquer OUTRO erro e da conexao: o cliente fechou (EOF), caiu de
124         // forma abrupta (connection reset) ou a rede sumiu. Em todos os casos
125         // nao ha mais ninguem do outro lado, entao a goroutine precisa acabar.
126         //
127         // Uma versao anterior so encerrava no EOF e tratava o resto como JSON
128         // ruim: numa queda abrupta, voltava a ler, recebia o mesmo erro na hora
129         // e girava para sempre a 100% de um nucleo.
130         if err != nil {
131             if errors.Is(err, io.EOF) {
132                 fmt.Println("cliente desconectou:", conexao.RemoteAddr())
133             } else {
134                 fmt.Printf("conexao com %v caiu: %v\n", conexao.RemoteAddr(), err)
135             }
136             return
137         }
138     }
139 }
```

```

137     }
138
139     fmt.Printf("recebido de %v: acao=%q\n", conexao.RemoteAddr(), pedido.Acao)
140
141     resposta := executar(pedido, &s, banco)
142
143     if err := protocolo.EnviarResposta(conexao, resposta); err != nil {
144         fmt.Println("erro ao responder:", err)
145         return
146     }
147 }
148 }

```

105 `func` atender(conexao net.Conn, banco *dados.Banco) {

O corpo de cada goroutine de cliente. Recebe a conexão dela e o banco compartilhado.

106 `defer` conexao.Close()

Garante que a conexão é fechada quando esta goroutine terminar, qualquer que seja o motivo.

108 `leitor := bufio.NewReader(conexao)`

O leitor com buffer desta conexão, criado uma vez e usado em todas as leituras. Se fosse recriado a cada volta, o que sobrou no buffer anterior se perderia.

111 `var` s sessao

A sessão desta conexão, vazia. É uma variável local desta goroutine: nenhuma outra a enxerga, e é por isso que não precisa de mutex.

113 `for` {

Atende pedidos até a conexão acabar.

114 `pedido, err := protocolo.LerPedido(leitor)`

Espera a próxima linha do cliente.

118-121 `if` errors.Is(err, protocolo.ErrMensagemInvalida) {
 protocolo.EnviarResposta(conexao, protocolo.Resposta{OK: false, Erro: err.Error()})
 continue
}

A linha chegou inteira, mas não é JSON válido: responde o erro e continua atendendo.

`errors.Is(nil, ...)` devolve `false`, então num pedido válido este `if` é simplesmente pulado. O erro do `EnviarResposta` é ignorado aqui: se a conexão tiver caído, a próxima leitura descobre.

130 `if` err != nil {

Qualquer outro erro é da conexão. Esta linha é a correção do bug da queda abrupta: antes, só `io.EOF` encerrava a goroutine, e o resto era tratado como JSON ruim.

```
131-134  if errors.Is(err, io.EOF) {
        fmt.Println("cliente desconectou:", conexao.RemoteAddr())
    } else {
        fmt.Printf("conexao com %v caiu: %v\n", conexao.RemoteAddr(), err)
    }
```

Só muda a mensagem: `io.EOF` é o encerramento normal; qualquer outro erro, como `connection reset`, é uma queda. `} else {` precisa ficar na mesma linha da chave que fecha o `if`; em Go isso é obrigatório.

```
136  return
```

Encerra a goroutine. O `defer` fecha a conexão, e a sessão desaparece junto.

```
139  fmt.Printf("recebido de %v: acao=%q\n", conexao.RemoteAddr(), pedido.Acao)
```

Registra cada ação recebida. `%v` formata o endereço; `%q` põe o nome da ação entre aspas.

```
141  resposta := executar(pedido, &s, banco)
```

Decide e executa. `&s` passa o **endereço** da sessão, porque `login` e `sair` precisam alterá-la.

```
143-146  if err := protocolo.EnviaResposta(conexao, resposta); err != nil {
        fmt.Println("erro ao responder:", err)
        return
    }
```

Manda a resposta. Se não conseguir escrever, a conexão acabou: encerra.

executar: o roteador

O índice do sistema inteiro: cada ação do protocolo é um `case`, e cada `case` chama uma função do `dados` e traduz o resultado numa `Resposta`. A função não sabe nada de rede.

pbl/servidor/main.go · linhas 150–284

```
150 // executar decide o que fazer com cada acao. Este switch e o roteador do
151 // servidor: e o indice do sistema inteiro.
152 //
153 // A sessao vem como PONTEIRO (*sessao) porque o case "login" precisa
154 // MODIFICAR quem esta logado. Se viesse por valor, alteraria uma copia.
155 func executar(p protocolo.Pedido, s *sessao, banco *dados.Banco) protocolo.Resposta {
156     // Barreira de autenticacao: vale para toda acao nova que criarmos daqui
157     // em diante, sem precisar repetir a checagem em cada case.
158     if exigeLogin(p.Acao) && s.usuario.Login == "" {
159         return erro("faca login primeiro")
160     }
161
162     switch p.Acao {
163     case "ping":
164         return protocolo.Resposta{OK: true, Mensagem: "pong"}
165
166     case "registrar":
167         if err := banco.CadastrarUsuario(p.Usuario, p.Senha, p.Tipo); err != nil {
168             return erro(err.Error())
169         }
170         return protocolo.Resposta{
171             OK: true,
172             Mensagem: fmt.Sprintf("conta criada para %s (%s)", p.Usuario, p.Tipo),
173         }
174
175     case "login":
176         u, ok := banco.Autenticar(p.Usuario, p.Senha)
```

```

177     if !ok {
178         return erro("usuario ou senha invalidos")
179     }
180
181     s.usuario = u
182     return protocolo.Resposta{
183         OK:      true,
184         Tipo:    u.Tipo,
185         Mensagem: fmt.Sprintf("bem-vindo, %s", u.Login),
186     }
187
188     case "sair":
189         s.usuario = dados.Usuario{}
190         return protocolo.Resposta{OK: true, Mensagem: "sessao encerrada"}
191
192     // ---- acoes do motorista ----
193
194     case "cadastrar":
195         if s.usuario.Tipo != "motorista" {
196             return erro("so motorista cadastra carona")
197         }
198
199         id, err := banco.CadastrarCarona(s.usuario.Login, p.Rota, p.Data, p.Assentos, p.Preco)
200         if err != nil {
201             return erro(err.Error())
202         }
203         return protocolo.Resposta{
204             OK:      true,
205             CaronaID: id,
206             Mensagem: fmt.Sprintf("carona %d cadastrada", id),
207         }
208
209     case "minhas_caronas":
210         return protocolo.Resposta{OK: true, Linhas: banco.MinhasCaronas(s.usuario.Login)}
211
212     case "passageiros":
213         if s.usuario.Tipo != "motorista" {
214             return erro("so motorista ve os passageiros de uma carona")
215         }
216
217         linhas, err := banco.PassageirosDaCarona(s.usuario.Login, p.CaronaID)
218         if err != nil {
219             return erro(err.Error())
220         }
221         return protocolo.Resposta{OK: true, Linhas: linhas}
222
223     case "cancelar_carona":
224         if s.usuario.Tipo != "motorista" {
225             return erro("so motorista cancela carona")
226         }
227
228         afetadas, err := banco.CancelarCarona(s.usuario.Login, p.CaronaID)
229         if err != nil {
230             return erro(err.Error())
231         }
232         return protocolo.Resposta{
233             OK: true,
234             Mensagem: fmt.Sprintf("carona %d cancelada; %d reserva(s) desfeita(s) e assentos
devolvidos",
235                 p.CaronaID, afetadas),
236         }
237
238     // ---- acoes do passageiro ----
239
240     case "buscar":
241         opcoes := banco.Buscar(p.Origem, p.Destino, p.Data)
242         return protocolo.Resposta{

```

```

243         OK:         true,
244         Opcoes:     opcoes,
245         Mensagem:   fmt.Sprintf("%d opcao(oes) encontrada(s)", len(opcoes)),
246     }
247
248     case "reservar":
249         id, err := banco.Reservar(s.usuario.Login, p.Itens)
250         if err != nil {
251             return erro(err.Error())
252         }
253         return protocolo.Resposta{
254             OK:         true,
255             ReservaID: id,
256             Mensagem:   fmt.Sprintf("reserva %d criada; pague em ate %s ou os assentos voltam",
257                             id, dados.TempoDeReserva),
258         }
259
260     case "pagar":
261         if err := banco.Pagar(s.usuario.Login, p.ReservaID); err != nil {
262             return erro(err.Error())
263         }
264         return protocolo.Resposta{
265             OK:         true,
266             Mensagem:   fmt.Sprintf("reserva %d paga; assentos confirmados", p.ReservaID),
267         }
268
269     case "cancelar_reserva":
270         if err := banco.CancelarReserva(s.usuario.Login, p.ReservaID); err != nil {
271             return erro(err.Error())
272         }
273         return protocolo.Resposta{
274             OK:         true,
275             Mensagem:   fmt.Sprintf("reserva %d cancelada; assentos devolvidos", p.ReservaID),
276         }
277
278     case "minhas_reservas":
279         return protocolo.Resposta{OK: true, Linhas: banco.MinhasReservas(s.usuario.Login)}
280
281     default:
282         return erro(fmt.Sprintf("acao desconhecida: %q", p.Acao))
283     }
284 }

```

```

155 func executar(p protocolo.Pedido, s *sessao, banco *dados.Banco) protocolo.Resposta {

```

Recebe o pedido, um ponteiro para a sessão e o banco; devolve a resposta. O comentário acima explica por que a sessão é ponteiro.

```

158-160 if exigeLogin(p.Acao) && s.usuario.Login == "" {
        return erro("faca login primeiro")
    }

```

A barreira de autenticação, antes do `switch`. Vale para toda ação, inclusive as que ainda não existem: uma ação nova já nasce exigindo login.

```

162 switch p.Acao {

```

Um `case` por ação.

```

163-164 case "ping":
        return protocolo.Resposta{OK: true, Mensagem: "pong"}

```

Não toca no banco. Serve para saber se o servidor está vivo.

```
166-169 case "registrar":
    if err := banco.CadastrarUsuario(p.Usuario, p.Senha, p.Tipo); err != nil {
        return erro(err.Error())
    }
```

Cria a conta. Qualquer erro do `dados` volta como resposta de erro com a mesma mensagem, através de `err.Error()`.

```
170-172 return protocolo.Resposta{
    OK:      true,
    Mensagem: fmt.Sprintf("conta criada para %s (%s)", p.Usuario, p.Tipo),
}
```

Literal composto em várias linhas; cada campo termina com vírgula.

```
175-176 case "login":
    u, ok := banco.Autenticar(p.Usuario, p.Senha)
```

Confere login e senha.

```
177-179 if !ok {
    return erro("usuario ou senha invalidos")
}
```

Recusado.

```
181 s.usuario = u
```

Grava a conta na sessão. `s` é ponteiro, então a alteração vale para a sessão do `atender`, e as próximas ações desta conexão já estão logadas.

```
182-185 return protocolo.Resposta{
    OK:      true,
    Tipo:    u.Tipo,
    Mensagem: fmt.Sprintf("bem-vindo, %s", u.Login),
}
```

Devolve o tipo da conta, que o cliente usa para escolher o menu.

```
188-190 case "sair":
    s.usuario = dados.Usuario{}
    return protocolo.Resposta{OK: true, Mensagem: "sessao encerrada"}
```

`dados.Usuario{}` é uma conta vazia: a sessão volta a ser de quem não entrou. O cliente atual não oferece essa opção, mas ela existe no protocolo.

```
194-197 case "cadastrar":
    if s.usuario.Tipo != "motorista" {
        return erro("so motorista cadastra carona")
    }
```

Restrição de tipo: só motorista cadastra.

```
199-202 id, err := banco.CadastrarCarona(s.usuario.Login, p.Rota, p.Data, p.Assentos, p.Preco)
if err != nil {
    return erro(err.Error())
}
```

O motorista dono é sempre o login da sessão, nunca um campo do pedido.

```
203-206 return protocolo.Resposta{
    OK: true,
    CaronaID: id,
    Mensagem: fmt.Sprintf("carona %d cadastrada", id),
```

Devolve o número também como campo, para programas, como a carga, não precisarem ler o texto da mensagem.

```
209-210 case "minhas_caronas":
    return protocolo.Resposta{OK: true, Linhas: banco.MinhasCaronas(s.usuario.Login)}
```

Lista as caronas do login da sessão. Não confere o tipo: para um passageiro, a lista vem vazia.

```
212-215 case "passageiros":
    if s.usuario.Tipo != "motorista" {
        return erro("so motorista ve os passageiros de uma carona")
    }
```

Só motorista.

```
217-220 linhas, err := banco.PassageirosDaCarona(s.usuario.Login, p.CaronaID)
if err != nil {
    return erro(err.Error())
}
```

O `dados` confere se é o dono.

```
221 return protocolo.Resposta{OK: true, Linhas: linhas}
```

Quando cabe, o literal fica numa linha só.

```
223-226 case "cancelar_carona":
    if s.usuario.Tipo != "motorista" {
        return erro("so motorista cancela carona")
    }
```

Só motorista.

```
228-231 afetadas, err := banco.CancelarCarona(s.usuario.Login, p.CaronaID)
if err != nil {
    return erro(err.Error())
}
```

Cancela e recebe quantas reservas foram desfeitas.

```
232-235 return protocolo.Resposta{
    OK: true,
    Mensagem: fmt.Sprintf("carona %d cancelada; %d reserva(s) desfeita(s) e assentos devolvidos",
        p.CaronaID, afetadas),
```

A mensagem informa esse número.

```
240-245 case "buscar":
    opcoes := banco.Buscar(p.Origem, p.Destino, p.Data)
    return protocolo.Resposta{
        OK: true,
        Opcoes: opcoes,
        Mensagem: fmt.Sprintf("%d opcao(oes) encontrada(s)", len(opcoes)),
```

A busca não falha; a mensagem traz a contagem. Não há checagem de tipo (Parte VII).

```
248-252 case "reservar":
    id, err := banco.Reservar(s.usuario.Login, p.Itens)
    if err != nil {
        return erro(err.Error())
    }
```

O login da sessão é o dono da reserva.

```
253-257 return protocolo.Resposta{
    OK: true,
    ReservaID: id,
    Mensagem: fmt.Sprintf("reserva %d criada; pague em ate %s ou os assentos voltam",
        id, dados.TempoDeReserva),
}
```

A mensagem informa o prazo em uso, lido da variável do pacote `dados`.

```
260-263 case "pagar":
    if err := banco.Pagar(s.usuario.Login, p.ReservaID); err != nil {
        return erro(err.Error())
    }
```

`if` com declaração: chama e testa o erro na mesma linha.

```
264-266 return protocolo.Resposta{
    OK: true,
    Mensagem: fmt.Sprintf("reserva %d paga; assentos confirmados", p.ReservaID),
}
```

Confirmação.

```
269-272 case "cancelar_reserva":
    if err := banco.CancelarReserva(s.usuario.Login, p.ReservaID); err != nil {
        return erro(err.Error())
    }
```

O `dados` confere o dono e o estado.

```
273-275 return protocolo.Resposta{
    OK: true,
    Mensagem: fmt.Sprintf("reserva %d cancelada; assentos devolvidos", p.ReservaID),
}
```

Confirmação.

```
278-279 case "minhas_reservas":
    return protocolo.Resposta{OK: true, Linhas: banco.MinhasReservas(s.usuario.Login)}
```

Lista as reservas do login da sessão.

```
281-282 default:
    return erro(fmt.Sprintf("acao desconhecida: %q", p.Acao))
```

Qualquer outra ação.

Funções auxiliares

pbl/servidor/main.go · linhas 286–289

```
286 // erro monta uma Resposta de falha. Existe so para encurtar os cases.
287 func erro(mensagem string) protocolo.Resposta {
```

```
288     return protocolo.Resposta{OK: false, Erro: mensagem}
289 }
```

```
287 func erro(mensagem string) protocolo.Resposta {
```

Existe só para encurtar os `case` s.

```
288     return protocolo.Resposta{OK: false, Erro: mensagem}
```

Escrever `erro("x")` em vez de `protocolo.Resposta{OK: false, Erro: "x"}` em cada `case`.

pbl/servidor/main.go · linhas 291–296

```
291 // exigeLogin diz quais acoes precisam de usuario autenticado.
292 // So "ping", "login" e "registrar" ficam de fora -- quem esta criando conta
293 // ainda nao tem como estar logado. Todo o resto exige.
294 func exigeLogin(acao string) bool {
295     return acao != "ping" && acao != "login" && acao != "registrar"
296 }
```

```
294 func exigeLogin(acao string) bool {
```

Diz se uma ação precisa de login.

```
295     return acao != "ping" && acao != "login" && acao != "registrar"
```

Lista as **exceções**, não as ações protegidas. Uma ação nova que ninguém lembrar de listar aqui passa a exigir login, que é o lado seguro do erro.

O SERVIDOR EM RESUMO

Três tipos de goroutine (`main`, uma por conexão e a varredura), uma única variável compartilhada (o `banco`) e uma variável por conexão que nunca é compartilhada (a `sessao`). Com isso, a única pergunta de concorrência do servidor é respondida pelo mutex do `dados`.

43. cliente/main.go — os menus de terminal

O cliente não tem nenhuma regra de negócio. Ele roda numa única goroutine, guarda só a conexão e os dois leitores, e toda ação segue o mesmo roteiro: ler o teclado, montar um `Pedido`, mandar, esperar a `Resposta` e mostrar. Por isso o arquivo é longo mas repetitivo: entendendo uma ação, entende-se todas.

Cabeçalho e imports

pbl/cliente/main.go · linhas 1–12

```
1 package main
2
3 import (
4     "bufio"
5     "fmt"
6     "net"
```

```

7  "os"
8  "strconv"
9  "strings"
10
11 "vaijunto/protocolo"
12 )

```

1 `package main`

Outro executável: esta pasta gera o programa do cliente.

3-12 `import (`

```

    "bufio"
    "fmt"
    "net"
    "os"
    "strconv"
    "strings"

    "vaijunto/protocolo"
)

```

- `bufio`: leitores com buffer para o teclado e para a conexão;
- `fmt`: tudo que aparece na tela;
- `net`: `net.Dial` para conectar;
- `os`: `os.Stdin` (o teclado) e variáveis de ambiente;
- `strconv`: converter o que foi digitado em número;
- `strings`: `Split`, `TrimSpace`, `ToLower`;
- `vaijunto/protocolo`: as mensagens.

O cliente **não importa o dados**: ele não conhece nenhuma regra, só o protocolo.

A struct cliente

pbl/cliente/main.go · linhas 14–21

```

14 // cliente junta as tres coisas que toda tela precisa: por onde falar com o
15 // servidor, por onde ouvir a resposta, e por onde ler o teclado.
16 // Sem isso, cada funcao de menu teria que receber os tres como argumento.
17 type cliente struct {
18     conexao net.Conn
19     servidor *bufio.Reader
20     teclado  *bufio.Reader
21 }

```

17 `type cliente struct {`

Junta as três coisas que toda tela precisa. É o equivalente a guardar atributos em `self` numa classe Python, para não passar os três como argumento em todas as funções.

18 `conexao net.Conn`

A conexão com o servidor: é por aqui que se escreve.

19 `servidor *bufio.Reader`

O leitor com buffer sobre a conexão: é por aqui que as respostas são lidas.

20 `teclado *bufio.Reader`

O leitor com buffer sobre a entrada padrão.

main e endereco

pbl/cliente/main.go · linhas 23–48

```
23 func main() {
24     conexao, err := net.Dial("tcp", endereco())
25     if err != nil {
26         fmt.Println("nao consegui conectar:", err)
27         return
28     }
29     defer conexao.Close()
30
31     c := &cliente{
32         conexao: conexao,
33         servidor: bufio.NewReader(conexao),
34         teclado:  bufio.NewReader(os.Stdin),
35     }
36
37     tipo, ok := c.entrada()
38     if !ok {
39         return
40     }
41
42     // O tipo do usuario decide qual menu aparece.
43     if tipo == "motorista" {
44         c.menuMotorista()
45     } else {
46         c.menuPassageiro()
47     }
48 }
```

23 `func main() {`

Ponto de entrada do cliente.

24 `conexao, err := net.Dial("tcp", endereco())`

Conecta no servidor. `endereco()` decide o destino. É o `socket.create_connection` do Python.

25–28 `if err != nil {`
 `fmt.Println("nao consegui conectar:", err)`
 `return`
`}`

Servidor desligado ou endereço errado: avisa e sai.

29 `defer conexao.Close()`

Fecha a conexão quando o programa terminar. Nesse momento, a leitura do servidor recebe `io.EOF` e a goroutine deste cliente termina do lado de lá.

```
31-34 c := &cliente{
      conexao: conexao,
      servidor: bufio.NewReader(conexao),
      teclado: bufio.NewReader(os.Stdin),
```

Cria o cliente e guarda o endereço dele, porque os métodos têm receptor ponteiro. `bufio.NewReader` aparece duas vezes: uma sobre a conexão e outra sobre `os.Stdin`, o teclado.

```
37 tipo, ok := c.entrada()
```

A tela de entrar ou criar conta. Devolve o tipo da conta.

```
38-40 if !ok {
      return
    }
```

O usuário desistiu (opção 0) ou a entrada acabou: sai.

```
43-46 if tipo == "motorista" {
      c.menuMotorista()
    } else {
      c.menuPassageiro()
```

O tipo escolhe o menu. Quando o menu retorna, `main` termina e o `defer` fecha a conexão. A palavra `else` precisa ficar na mesma linha da chave que fecha o `if`.

pbl/cliente/main.go · linhas 50–58

```
50 // endereco permite apontar para outro host, o que sera necessario no Docker:
51 //
52 //     VAIJUNTO_SERVIDOR=servidor:8080 go run ./cliente
53 func endereco() string {
54     if v := os.Getenv("VAIJUNTO_SERVIDOR"); v != "" {
55         return v
56     }
57     return "localhost:8080"
58 }
```

```
53 func endereco() string {
```

Decide a que servidor conectar.

```
54-56 if v := os.Getenv("VAIJUNTO_SERVIDOR"); v != "" {
      return v
    }
```

Permite apontar para outro servidor. No Docker, o compose define `VAIJUNTO_SERVIDOR=servidor:8080`.

```
57 return "localhost:8080"
```

Padrão: o servidor na própria máquina.

Entrar e criar conta

pbl/cliente/main.go · linhas 64–87

```

64 // entrada e a primeira tela: entrar com uma conta existente ou criar uma nova.
65 // Devolve o tipo do usuario, que decide qual menu aparece depois.
66 func (c *cliente) entrada() (string, bool) {
67     fmt.Println("=== Vaijunto ===")
68
69     for {
70         fmt.Println("\n1) entrar    2) criar conta    0) sair")
71
72         switch c.opcao() {
73             case "1":
74                 if tipo, ok := c.entrar(); ok {
75                     return tipo, true
76                 }
77             case "2":
78                 if tipo, ok := c.criarConta(); ok {
79                     return tipo, true
80                 }
81             case "0", "":
82                 return "", false
83             default:
84                 fmt.Println(" opcao invalida")
85             }
86         }
87     }

```

66 `func (c *cliente) entrada() (string, bool) {`

Método com receptor ponteiro. Devolve o tipo da conta e se deu certo.

67 `fmt.Println("=== Vaijunto ===")`

O título, mostrado uma vez.

69 `for {`

Repete o menu até alguém entrar ou desistir.

70 `fmt.Println("\n1) entrar 2) criar conta 0) sair")`

O `\n` no começo do texto pula uma linha antes do menu.

72 `switch c.opcao() {`

`switch` direto sobre o retorno de uma função: `c.opcao()` lê a escolha.

73-76 `case "1":`
`if tipo, ok := c.entrar(); ok {`
`return tipo, true`
`}`

`if` com declaração e dois valores: chama `entrar()` e, se deu certo, sai da função devolvendo o tipo. Se não deu (senha errada, por exemplo), o `for` mostra o menu de novo.

77-80 `case "2":`
`if tipo, ok := c.criarConta(); ok {`
`return tipo, true`
`}`

O mesmo, para criar conta.

```
81-82 case "0", "":
    return "", false
```

Dois valores num `case`: `0` ou linha vazia saem. `opcao()` também devolve `"0"` quando a entrada acaba, o que evita um laço infinito lendo um teclado que não existe mais.

```
83-84 default:
    fmt.Println(" opcao invalida")
```

Qualquer outra coisa.

pbl/cliente/main.go · linhas 89–102

```
89 // entrar pede usuario e senha de uma conta que ja existe.
90 func (c *cliente) entrar() (string, bool) {
91     usuario, ok := c.ler("usuario: ")
92     if !ok {
93         return "", false
94     }
95
96     senha, ok := c.ler("senha: ")
97     if !ok {
98         return "", false
99     }
100
101     return c.autenticar(usuario, senha)
102 }
```

```
90 func (c *cliente) entrar() (string, bool) {
```

Pede usuário e senha de uma conta existente.

```
91-94 usuario, ok := c.ler("usuario: ")
    if !ok {
        return "", false
    }
```

`ler` mostra o rótulo e lê uma linha. `ok` falso quer dizer que a entrada acabou.

```
96-99 senha, ok := c.ler("senha: ")
    if !ok {
        return "", false
    }
```

`senha, ok :=` com `ok` já existente: o `:=` é permitido porque `senha` é nova; `ok` só recebe um valor novo.

```
101 return c.autenticar(usuario, senha)
```

Devolve direto o que `autenticar` devolver: os dois valores passam adiante.

pbl/cliente/main.go · linhas 104–155

```
104 // criarConta registra um usuario novo e ja entra com ele.
105 func (c *cliente) criarConta() (string, bool) {
106     usuario, ok := c.ler("novo usuario: ")
107     if !ok {
108         return "", false
109     }
```

```

110
111     senha, ok := c.ler("senha: ")
112     if !ok {
113         return "", false
114     }
115
116     fmt.Println(" 1) motorista (oferece caronas)")
117     fmt.Println(" 2) passageiro (procura caronas)")
118
119     escolha, ok := c.ler("tipo: ")
120     if !ok {
121         return "", false
122     }
123
124     var tipo string
125     switch escolha {
126     case "1":
127         tipo = "motorista"
128     case "2":
129         tipo = "passageiro"
130     default:
131         fmt.Println(" escolha 1 ou 2")
132         return "", false
133     }
134
135     r, err := c.pedir(protocolo.Pedido{
136         Acao:    "registrar",
137         Usuario: usuario,
138         Senha:   senha,
139         Tipo:    tipo,
140     })
141     if err != nil {
142         fmt.Println("conexao perdida:", err)
143         return "", false
144     }
145
146     if !r.OK {
147         fmt.Println(" erro:", r.Erro)
148         return "", false
149     }
150
151     fmt.Println(" ", r.Mensagem)
152
153     // Conta criada: entra direto, sem obrigar a digitar tudo de novo.
154     return c.autenticar(usuario, senha)
155 }

```

105 `func (c *cliente) criarConta() (string, bool) {`

Cria uma conta e já entra com ela.

106-109 `usuario, ok := c.ler("novo usuario: ")`
`if !ok {`
 `return "", false`
`}`

O login desejado.

111-114 `senha, ok := c.ler("senha: ")`
`if !ok {`
 `return "", false`
`}`

A senha.

```
116-117  fmt.Println("  1) motorista (oferece caronas)")
        fmt.Println("  2) passageiro (procura caronas)")
```

Mostra as duas opções de tipo.

```
119-122  escolha, ok := c.ler("tipo: ")
        if !ok {
            return "", false
        }
```

Lê a escolha.

```
124      var tipo string
```

Declara sem valor; o `switch` abaixo preenche.

```
125-132  switch escolha {
        case "1":
            tipo = "motorista"
        case "2":
            tipo = "passageiro"
        default:
            fmt.Println("  escolha 1 ou 2")
            return "", false
    }
```

Traduz a escolha para o texto do protocolo. Qualquer outra resposta abandona a criação e volta ao menu.

```
135-139  r, err := c.pedir(protocolo.Pedido{
        Acao:    "registrar",
        Usuario:  usuario,
        Senha:    senha,
        Tipo:     tipo,
```

Monta o pedido de registro com um literal em várias linhas e envia.

```
141-144  if err != nil {
        fmt.Println("conexao perdida:", err)
        return "", false
    }
```

Não conseguiu falar com o servidor.

```
146-149  if !r.OK {
        fmt.Println("  erro:", r.Erro)
        return "", false
    }
```

O servidor recusou, por exemplo por login repetido. `r.Erro` traz o motivo.

```
151      fmt.Println(" ", r.Mensagem)
```

`Println` separa os argumentos com espaço, então `" "` mais o espaço automático dão dois espaços de recuo.

```
154      return c.autenticar(usuario, senha)
```

Conta criada: entra com ela na hora, sem pedir os dados de novo.

```

157 // autenticar manda o login e devolve o tipo do usuario.
158 func (c *cliente) autenticar(usuario, senha string) (string, bool) {
159     r, err := c.pedir(protocolo.Pedido{Acao: "login", Usuario: usuario, Senha: senha})
160     if err != nil {
161         fmt.Println("conexao perdida:", err)
162         return "", false
163     }
164
165     if !r.OK {
166         fmt.Println(" erro:", r.Erro)
167         return "", false
168     }
169
170     fmt.Println(" ", r.Mensagem)
171     return r.Tipo, true
172 }

```

158 `func (c *cliente) autenticar(usuario, senha string) (string, bool) {`

Faz o login e devolve o tipo da conta.

159 `r, err := c.pedir(protocolo.Pedido{Acao: "login", Usuario: usuario, Senha: senha})`

O pedido de login numa linha só.

160-163 `if err != nil {
 fmt.Println("conexao perdida:", err)
 return "", false
}`

Conexão perdida.

165-168 `if !r.OK {
 fmt.Println(" erro:", r.Erro)
 return "", false
}`

Usuário ou senha inválidos.

170 `fmt.Println(" ", r.Mensagem)`

Mostra `bem-vindo, ana.`

171 `return r.Tipo, true`

O tipo que decide o menu.

Menu do motorista

pbl/cliente/main.go · linhas 178-197

```

178 func (c *cliente) menuMotorista() {
179     for {
180         fmt.Println("\n1) cadastrar carona    2) minhas caronas    3) passageiros    4) cancelar carona
0) sair")
181
182         switch c.opcao() {
183             case "1":
184                 c.cadastrarCarona()
185             case "2":

```

```

186         c.listar(protocolo.Pedido{Acao: "minhas_caronas"}, "voce ainda nao tem caronas")
187     case "3":
188         c.passageiros()
189     case "4":
190         c.cancelarCarona()
191     case "0", "":
192         return
193     default:
194         fmt.Println(" opcao invalida")
195 }
196 }
197 }

```

```
178 func (c *cliente) menuMotorista() {
```

Não devolve nada; termina quando o usuário escolhe sair.

```
179 for {
```

O menu se repete.

```
180     fmt.Println("\n1) cadastrar carona    2) minhas caronas    3) passageiros    4) cancelar carona    0)
sair")
```

O menu inteiro numa linha.

```
182     switch c.opcao() {
```

Uma escolha por volta.

```
183-184     case "1":
        c.cadastrarCarona()
```

Cadastrar.

```
185-186     case "2":
        c.listar(protocolo.Pedido{Acao: "minhas_caronas"}, "voce ainda nao tem caronas")
```

`listar` recebe um pedido pronto e o texto para lista vazia.

```
187-188     case "3":
        c.passageiros()
```

Passageiros por trecho.

```
189-190     case "4":
        c.cancelarCarona()
```

Cancelar carona.

```
191-192     case "0", "":
        return
```

`return` num método sem retorno encerra o método. `main` continua e termina.

```
193-194     default:
        fmt.Println(" opcao invalida")
```

Qualquer outra coisa.

```

199 func (c *cliente) cadastrarCarona() {
200     texto, ok := c.ler("rota (cidades separadas por virgula): ")
201     if !ok {
202         return
203     }
204
205     // "Feira, Salvador, Ilheus" vira ["Feira" "Salvador" "Ilheus"].
206     var rota []string
207     for _, cidade := range strings.Split(texto, ",") {
208         cidade = strings.TrimSpace(cidade)
209         if cidade != "" {
210             rota = append(rota, cidade)
211         }
212     }
213
214     data, ok := c.ler("data (2026-09-14): ")
215     if !ok {
216         return
217     }
218
219     assentos, ok := c.lerNumero("assentos: ")
220     if !ok {
221         return
222     }
223
224     preco, ok := c.lerNumero("preco por trecho: ")
225     if !ok {
226         return
227     }
228
229     c.mostrar(c.pedir(protocolo.Pedido{
230         Acao:      "cadastrar",
231         Rota:       rota,
232         Data:       data,
233         Assentos:   assentos,
234         Preco:      preco,
235     })))
236 }

```

199 func (c *cliente) cadastrarCarona() {

Pergunta os dados da carona e envia.

200–203 texto, ok := c.ler("rota (cidades separadas por virgula): ")
 if !ok {
 return
 }

A rota inteira numa linha só.

206 var rota []string

Começa vazia.

207 for _, cidade := range strings.Split(texto, ",") {

strings.Split corta o texto nas vírgulas: "Feira, Salvador" vira ["Feira", " Salvador"].

```
208 cidade = strings.TrimSpace(cidade)
```

Tira o espaço que sobrou depois de cada vírgula. Alterar a variável do laço é seguro aqui, porque só esta cópia é usada logo abaixo.

```
209-211 if cidade != "" {
        rota = append(rota, cidade)
    }
```

Ignora pedaços vazios, como o que sobra de uma vírgula no final, e acrescenta as cidades de verdade.

```
214-217 data, ok := c.ler("data (2026-09-14): ")
if !ok {
    return
}
```

A data, como texto.

```
219-222 assentos, ok := c.lerNumero("assentos: ")
if !ok {
    return
}
```

`lerNumero` converte para `int`; se não for número, avisa e devolve `ok` falso, e o cadastro é abandonado.

```
224-227 preco, ok := c.lerNumero("preco por trecho: ")
if !ok {
    return
}
```

O preço por trecho.

```
229-234 c.mostrar(c.pedir(protocolo.Pedido{
    Acao:      "cadastrar",
    Rota:      rota,
    Data:      data,
    Assentos:  assentos,
    Preco:     preco,
```

`c.mostrar(c.pedir(...))`: `pedir` devolve dois valores, e Go permite passar os dois direto como argumentos de uma função que recebe exatamente esses dois tipos. Assim, cada ação termina numa única chamada.

pbl/cliente/main.go · linhas 238–246

```
238 // passageiros mostra quem esta em cada trecho de uma carona.
239 func (c *cliente) passageiros() {
240     id, ok := c.lerNumero("numero da carona: ")
241     if !ok {
242         return
243     }
244
245     c.listar(protocolo.Pedido{Acao: "passageiros", CaronaID: id}, "carona sem dados")
246 }
```

```
239 func (c *cliente) passageiros() {
```

Mostra os passageiros de uma carona.

```
240-243 id, ok := c.lerNumero("numero da carona: ")
        if !ok {
            return
        }
```

O número da carona.

```
245 c.listar(protocolo.Pedido{Acao: "passageiros", CaronaID: id}, "carona sem dados")
```

O servidor monta as linhas por trecho; o cliente só imprime.

pbl/cliente/main.go · linhas 248–263

```
248 // cancelarCarona pede confirmacao antes, porque desfaz as reservas de todos
249 // os passageiros daquela carona.
250 func (c *cliente) cancelarCarona() {
251     id, ok := c.lerNumero("numero da carona: ")
252     if !ok {
253         return
254     }
255
256     resposta, ok := c.ler(fmt.Sprintf("cancelar a carona %d? as reservas dos passageiros serao desfeitas
(s/n): ", id))
257     if !ok || strings.ToLower(resposta) != "s" {
258         fmt.Println(" nada foi cancelado")
259         return
260     }
261
262     c.mostrar(c.pedir(protocolo.Pedido{Acao: "cancelar_carona", CaronaID: id}))
263 }
```

```
250 func (c *cliente) cancelarCarona() {
```

Cancela uma carona com confirmação.

```
251-254 id, ok := c.lerNumero("numero da carona: ")
        if !ok {
            return
        }
```

O número da carona.

```
256 resposta, ok := c.ler(fmt.Sprintf("cancelar a carona %d? as reservas dos passageiros serao
desfeitas (s/n): ", id))
```

O rótulo é montado com o número da carona e avisa que as reservas serão desfeitas: a ação afeta outras pessoas.

```
257-260 if !ok || strings.ToLower(resposta) != "s" {
        fmt.Println(" nada foi cancelado")
        return
    }
```

Qualquer resposta diferente de `s` ou `S` desiste.

```
262 c.mostrar(c.pedir(protocolo.Pedido{Acao: "cancelar_carona", CaronaID: id}))
```

Confirmado: envia.

Menu do passageiro

pbl/cliente/main.go · linhas 269–288

```
269 func (c *cliente) menuPassageiro() {
270     for {
271         fmt.Println("\n1) buscar e reservar    2) minhas reservas    3) pagar    4) cancelar reserva    0)
sair")
272
273         switch c.opcao() {
274             case "1":
275                 c.buscarEReservar()
276             case "2":
277                 c.listar(protocolo.Pedido{Acao: "minhas_reservas"}, "voce ainda nao tem reservas")
278             case "3":
279                 c.pagar()
280             case "4":
281                 c.cancelarReserva()
282             case "0", "":
283                 return
284             default:
285                 fmt.Println(" opcao invalida")
286         }
287     }
288 }
```

269–285

```
func (c *cliente) menuPassageiro() {
    for {
        fmt.Println("\n1) buscar e reservar    2) minhas reservas    3) pagar    4) cancelar reserva
0) sair")

        switch c.opcao() {
            case "1":
                c.buscarEReservar()
            case "2":
                c.listar(protocolo.Pedido{Acao: "minhas_reservas"}, "voce ainda nao tem reservas")
            case "3":
                c.pagar()
            case "4":
                c.cancelarReserva()
            case "0", "":
                return
            default:
                fmt.Println(" opcao invalida")
        }
    }
}
```

O mesmo desenho do menu do motorista, com as ações de passageiro: buscar e reservar, listar, pagar e cancelar reserva.

pbl/cliente/main.go · linhas 290–346

```
290 func (c *cliente) buscarEReservar() {
291     origem, ok := c.ler("origem: ")
292     if !ok {
293         return
294     }
295
296     destino, ok := c.ler("destino: ")
297     if !ok {
298         return
299     }
300
301     data, ok := c.ler("data (2026-09-14): ")
```

```

302     if !ok {
303         return
304     }
305
306     r, err := c.pedir(protocolo.Pedido{
307         Acao:    "buscar",
308         Origem:   origem,
309         Destino:  destino,
310         Data:    data,
311     })
312     if err != nil {
313         fmt.Println("conexao perdida:", err)
314         return
315     }
316
317     if !r.OK {
318         fmt.Println(" erro:", r.Erro)
319         return
320     }
321
322     if len(r.Opcoes) == 0 {
323         fmt.Println(" nenhuma carona encontrada")
324         return
325     }
326
327     // O servidor ja mandou o Resumo pronto: aqui so numeramos e imprimimos.
328     for i, o := range r.Opcoes {
329         fmt.Printf(" %d) %s | R$d\n", i+1, o.Resumo, o.Preco)
330     }
331
332     escolha, ok := c.lerNumero("reservar qual? (0 cancela): ")
333     if !ok || escolha == 0 {
334         return
335     }
336
337     if escolha < 1 || escolha > len(r.Opcoes) {
338         fmt.Println(" opcao invalida")
339         return
340     }
341
342     c.mostrar(c.pedir(protocolo.Pedido{
343         Acao:    "reservar",
344         Itens:   r.Opcoes[escolha-1].Itens,
345     }))
346 }

```

```
290 func (c *cliente) buscarEReservar() {
```

O fluxo mais longo: pergunta, busca, mostra as opções e reserva a escolhida.

```

291-294 origem, ok := c.ler("origem: ")
      if !ok {
          return
      }

```

Origem.

```

296-299 destino, ok := c.ler("destino: ")
      if !ok {
          return
      }

```

Destino.

```
301-304 data, ok := c.ler("data (2026-09-14): ")
      if !ok {
          return
      }
```

Data.

```
306-310 r, err := c.pedir(protocolo.Pedido{
      Acao:    "buscar",
      Origem:  origem,
      Destino: destino,
      Data:    data,
```

O pedido de busca.

```
312-315 if err != nil {
      fmt.Println("conexao perdida:", err)
      return
  }
```

Conexão perdida.

```
317-320 if !r.OK {
      fmt.Println(" erro:", r.Erro)
      return
  }
```

O servidor recusou, por exemplo por falta de login.

```
322-325 if len(r.Opcoes) == 0 {
      fmt.Println(" nenhuma carona encontrada")
      return
  }
```

Nada encontrado.

```
328-330 for i, o := range r.Opcoes {
      fmt.Printf(" %d) %s | R$%d\n", i+1, o.Resumo, o.Preco)
  }
```

Numera as opções a partir de 1 (`i+1`), com o resumo e o preço. O cliente não interpreta os itens: só mostra o texto que o servidor montou.

```
332-335 escolha, ok := c.lerNumero("reservar qual? (0 cancela): ")
      if !ok || escolha == 0 {
          return
      }
```

`0` desiste.

```
337-340 if escolha < 1 || escolha > len(r.Opcoes) {
      fmt.Println(" opcao invalida")
      return
  }
```

Número fora da lista.

```
342-344 c.mostrar(c.pedir(protocolo.Pedido{
        Acao: "reservar",
        Itens: r.Opcoes[escolha-1].Itens,
```

Manda de volta exatamente os itens da opção escolhida. `escolha-1` converte a numeração do usuário, que começa em 1, no índice do slice, que começa em 0.

pbl/cliente/main.go · linhas 348–355

```
348 func (c *cliente) pagar() {
349     id, ok := c.lerNumero("numero da reserva: ")
350     if !ok {
351         return
352     }
353
354     c.mostrar(c.pedir(protocolo.Pedido{Acao: "pagar", ReservaID: id}))
355 }
```

```
348 func (c *cliente) pagar() {
```

Paga uma reserva.

```
349-352 id, ok := c.lerNumero("numero da reserva: ")
        if !ok {
            return
        }
```

O número da reserva.

```
354 c.mostrar(c.pedir(protocolo.Pedido{Acao: "pagar", ReservaID: id}))
```

Envia e mostra.

pbl/cliente/main.go · linhas 357–364

```
357 func (c *cliente) cancelarReserva() {
358     id, ok := c.lerNumero("numero da reserva: ")
359     if !ok {
360         return
361     }
362
363     c.mostrar(c.pedir(protocolo.Pedido{Acao: "cancelar_reserva", ReservaID: id}))
364 }
```

```
357-363 func (c *cliente) cancelarReserva() {
        id, ok := c.lerNumero("numero da reserva: ")
        if !ok {
            return
        }

        c.mostrar(c.pedir(protocolo.Pedido{Acao: "cancelar_reserva", ReservaID: id}))
```

O mesmo desenho do `pagar`, com a ação `cancelar_reserva`.

Peças reaproveitadas

pbl/cliente/main.go · linhas 370–377

```
370 // pedir envia um Pedido e espera a Resposta. Toda conversa com o servidor
371 // passa por aqui.
372 func (c *cliente) pedir(p protocolo.Pedido) (protocolo.Resposta, error) {
373     if err := protocolo.EnviarPedido(c.conexao, p); err != nil {
374         return protocolo.Resposta{}, err
375     }
376     return protocolo.LerResposta(c.servidor)
377 }
```

372 `func (c *cliente) pedir(p protocolo.Pedido) (protocolo.Resposta, error) {`

Toda conversa com o servidor passa por aqui: um pedido, uma resposta.

373–375 `if err := protocolo.EnviarPedido(c.conexao, p); err != nil {`
 `return protocolo.Resposta{}, err`
`}`

`c.conexao` é uma `net.Conn`, que satisfaz `io.Writer`.

376 `return protocolo.LerResposta(c.servidor)`

Espera a resposta. O programa fica parado nesta linha até o servidor responder: é o comportamento síncrono do protocolo.

pbl/cliente/main.go · linhas 379–400

```
379 // listar faz um pedido que devolve Linhas e imprime uma por uma.
380 func (c *cliente) listar(p protocolo.Pedido, seVazio string) {
381     r, err := c.pedir(p)
382     if err != nil {
383         fmt.Println("conexao perdida:", err)
384         return
385     }
386
387     if !r.OK {
388         fmt.Println(" erro:", r.Erro)
389         return
390     }
391
392     if len(r.Linhas) == 0 {
393         fmt.Println(" ", seVazio)
394         return
395     }
396
397     for _, linha := range r.Linhas {
398         fmt.Println(" " + linha)
399     }
400 }
```

380 `func (c *cliente) listar(p protocolo.Pedido, seVazio string) {`

Para as ações que devolvem `Linhas`.

381 r, err := c.pedir(p)

Envia o pedido recebido.

```
382-385 if err != nil {  
    fmt.Println("conexao perdida:", err)  
    return  
}
```

Conexão perdida.

```
387-390 if !r.OK {  
    fmt.Println(" erro:", r.Erro)  
    return  
}
```

Recusado.

```
392-395 if len(r.Linhas) == 0 {  
    fmt.Println(" ", seVazio)  
    return  
}
```

Lista vazia mostra o texto recebido em `seVazio`.

```
397-399 for _, linha := range r.Linhas {  
    fmt.Println(" " + linha)  
}
```

Imprime cada linha com recuo. `+` junta textos.

pbl/cliente/main.go · linhas 402–415

```
402 // mostrar imprime o resultado de um pedir(). Recebe os DOIS retornos de uma  
403 // vez, o que deixa as chamadas curtas: c.mostrar(c.pedir(...)).  
404 func (c *cliente) mostrar(r protocolo.Resposta, err error) {  
405     if err != nil {  
406         fmt.Println("conexao perdida:", err)  
407         return  
408     }  
409  
410     if r.OK {  
411         fmt.Println(" ", r.Mensagem)  
412     } else {  
413         fmt.Println(" erro:", r.Erro)  
414     }  
415 }
```

404 func (c *cliente) mostrar(r protocolo.Resposta, err error) {

Recebe como parâmetros os dois retornos do `pedir`, o que permite a forma `c.mostrar(c.pedir(...))`.

```
405-408 if err != nil {  
    fmt.Println("conexao perdida:", err)  
    return  
}
```

Conexão perdida.

```
410-413  if r.OK {
        fmt.Println(" ", r.Mensagem)
    } else {
        fmt.Println(" erro:", r.Erro)
```

Mensagem de sucesso ou de erro.

pbl/cliente/main.go · linhas 417-423

```
417 func (c *cliente) opcao() string {
418     escolha, ok := c.ler("> ")
419     if !ok {
420         return "0"
421     }
422     return escolha
423 }
```

```
417 func (c *cliente) opcao() string {
```

Lê a escolha de um menu.

```
418     escolha, ok := c.ler("> ")
```

O prompt dos menus.

```
419-421  if !ok {
        return "0"
    }
```

Entrada encerrada vira "0", que todos os menus tratam como sair.

```
422     return escolha
```

A escolha digitada.

pbl/cliente/main.go · linhas 425-438

```
425 // ler mostra o rotulo e le uma linha do teclado. O segundo retorno e false
426 // quando a entrada acabou (ctrl+Z no Windows, ctrl+D no Linux).
427 func (c *cliente) ler(rotulo string) (string, bool) {
428     fmt.Print(rotulo)
429
430     texto, err := c.teclado.ReadString('\n')
431     if err != nil {
432         return "", false
433     }
434
435     // TrimSpace tira o \r\n do Enter. Sem isso "ping" viraria "ping\r\n" e
436     // nunca casaria com o case do servidor.
437     return strings.TrimSpace(texto), true
438 }
```

```
427 func (c *cliente) ler(rotulo string) (string, bool) {
```

Todas as leituras do teclado passam por aqui.

```
428     fmt.Print(rotulo)
```

`Print`, sem quebra de linha: o cursor fica logo depois do rótulo.

```
430     texto, err := c.teclado.ReadString('\n')
```

Espera o usuário apertar Enter. É o `input()` do Python, que também espera a linha inteira.

```
431-433     if err != nil {  
         return "", false  
     }
```

Quase sempre é o fim da entrada: `Ctrl+Z` no Windows, `Ctrl+D` no Linux, ou o fim de um texto redirecionado para o programa, como nos testes deste guia.

```
437     return strings.TrimSpace(texto), true
```

No Windows, o Enter gera `\r\n`, e o `ReadString` devolve os dois. Sem o `TrimSpace`, `"ana\r\n"` nunca seria igual a `"ana"`. O `input()` do Python já tira a quebra; aqui é manual.

pbl/cliente/main.go · linhas 440–453

```
440     func (c *cliente) lerNumero(rotulo string) (int, bool) {  
441         texto, ok := c.ler(rotulo)  
442         if !ok {  
443             return 0, false  
444         }  
445  
446         n, err := strconv.Atoi(texto)  
447         if err != nil {  
448             fmt.Println(" precisa ser um numero")  
449             return 0, false  
450         }  
451  
452         return n, true  
453     }
```

```
440     func (c *cliente) lerNumero(rotulo string) (int, bool) {
```

Lê uma linha e converte em número.

```
441         texto, ok := c.ler(rotulo)
```

Lê.

```
442-444         if !ok {  
             return 0, false  
         }
```

Entrada encerrada.

```
446         n, err := strconv.Atoi(texto)
```

Converte texto em `int`. Diferente do `int()` do Python, não lança exceção: devolve um erro.

```
447-450  if err != nil {
        fmt.Println("  precisa ser um numero")
        return 0, false
    }
```

Não é número: avisa e desiste.

```
452      return n, true
```

O número.

44. `carga/main.go` — o teste de carga pela rede

Os testes do pacote `dados` chamam as funções direto. Este programa faz o contrário: se comporta como centenas de clientes reais, cada um com a sua conexão TCP, e confere o resultado **só pelo protocolo**, sem olhar dentro do servidor. Ele exercita tudo junto: socket, JSON, goroutine por conexão e o mutex do `Banco`.

1. um motorista cria a carona A (2N lugares) e a carona B (N lugares)
2. C clientes conectam e entram, no máximo 50 ao mesmo tempo → todos param na largada
3. `close(largada)`: os C clientes pedem o itinerário A + B no mesmo instante
4. conta aceita e recusadas, e mede a latência de cada reserva
5. uma "sonda" conta as vagas que sobraram em A e em B, reservando uma a uma
6. confere as 4 verificações e cancela A e B (limpeza)

Os resultados medidos com 200, 500 e 1000 clientes estão na Parte II.

Cabeçalho e imports

pbl/carga/main.go · linhas 1–27

```
1  // Programa de teste de carga: N clientes REAIS, cada um com a sua conexao TCP,
2  // disputando os mesmos assentos ao mesmo tempo contra o servidor de verdade.
3  //
4  // Os testes do pacote dados chamam as funcoes direto. Aqui tudo passa pela
5  // rede: socket, protocolo JSON, goroutine por conexao no servidor e o mutex
6  // do Banco. E o teste que mostra o sistema inteiro aguentando concorrência.
7  //
8  // Com o servidor ja rodando:
9  //
10 //    go run ./carga
11 //    go run ./carga -clientes 500 -assentos 25
12 //    go run ./carga -servidor 192.168.0.10:8080
13 package main
14
15 import (
16     "bufio"
17     "flag"
18     "fmt"
19     "net"
20     "os"
21     "sort"
22     "strconv"
23     "sync"
24     "time"
25
```

```
26 "vaijunto/protocolo"
27 )
```

13 `package` main

Mais um executável. O comentário do pacote traz os comandos de uso.

```
15-27 import (
    "bufio"
    "flag"
    "fmt"
    "net"
    "os"
    "sort"
    "strconv"
    "sync"
    "time"

    "vaijunto/protocolo"
)
```

- `bufio`, `net`: a conexão de cada cliente simulado;
- `flag`: as opções `-servidor`, `-clientes` e `-assentos`;
- `fmt`, `os`: relatório e código de saída;
- `sort`: ordenar as latências para calcular percentis;
- `strconv`: montar o sufixo único da rodada;
- `sync`: os `WaitGroup`;
- `time`: cronometrar;
- `vaijunto/protocolo`: as mensagens.

main

pbl/carga/main.go · linhas 29–46

```
29 func main() {
30     padrao := "localhost:8080"
31     if v := os.Getenv("VAIJUNTO_SERVIDOR"); v != "" {
32         padrao = v
33     }
34
35     endereco := flag.String("servidor", padrao, "endereço do servidor")
36     clientes := flag.Int("clientes", 200, "quantos clientes disputam ao mesmo tempo")
37     assentos := flag.Int("assentos", 10, "lugares da carona mais apertada do itinerário")
38     flag.Parse()
39
40     if err := rodar(*endereco, *clientes, *assentos); err != nil {
41         fmt.Println("\nFALHOU:", err)
42         os.Exit(1)
43     }
44
45     fmt.Println("\nPASSOU")
46 }
```

```
29 func main() {
```

Lê as opções e roda o cenário.

```
30 padrao := "localhost:8080"
```

Endereço padrão.

```
31-33 if v := os.Getenv("VAIJUNTO_SERVIDOR"); v != "" {
    padrao = v
}
```

A mesma variável de ambiente do cliente vira o padrão.

```
35 endereco := flag.String("servidor", padrao, "endereco do servidor")
```

Declara a opção `-servidor`. `flag.String` recebe nome, valor padrão e texto de ajuda (`go run ./carga -h` mostra), como o `add_argument` do `argparse`. Devolve um `*string`, que só recebe o valor depois do `Parse`.

```
36 clientes := flag.Int("clientes", 200, "quantos clientes disputam ao mesmo tempo")
```

Opção `-clientes`, padrão 200.

```
37 assentos := flag.Int("assentos", 10, "lugares da carona mais apertada do itinerario")
```

Opção `-assentos`, padrão 10: os lugares da carona mais apertada.

```
38 flag.Parse()
```

Lê a linha de comando e preenche as três variáveis.

```
40-43 if err := rodar(*endereco, *clientes, *assentos); err != nil {
    fmt.Println("\nFALHOU:", err)
    os.Exit(1)
}
```

`*endereco` lê o valor apontado pelo ponteiro. Se o teste falhar, imprime e sai com `os.Exit(1)`: um script que rode a carga percebe a falha pelo código de saída.

```
45 fmt.Println("\nPASSOU")
```

Tudo conferiu. Código de saída 0.

rodar: o cenário completo

pbl/carga/main.go · linhas 52–245

```
52 func rodar(endereco string, clientes, assentos int) error {
53     // Sufixo unico: a data e os logins desta rodada nao colidem com nada que
54     // ja exista no servidor, entao o teste pode rodar varias vezes seguidas.
55     tag := strconv.FormatInt(time.Now().UnixNano(), 36)
56     data := "carga-" + tag
57
58     fmt.Printf("servidor %s | %d clientes | %d lugares disputados\n\n", endereco, clientes, assentos)
59
60     // ---- 1. O motorista monta o cenario ----
61
62     motorista, err := conectar(endereco)
```

```

63     if err != nil {
64         return fmt.Errorf("o motorista nao conectou: %w", err)
65     }
66     defer motorista.Close()
67
68     if err := entrar(motorista, "mot_"+tag, "motorista"); err != nil {
69         return err
70     }
71
72     // A primeira carona tem o DOBRO de lugares da segunda. Se a atomicidade
73     // falhar -- uma reserva ocupar a primeira sem ocupar a segunda -- sobra
74     // lugar ocupado na primeira sem par, e a verificacao do passo 4 enxerga.
75     a, err := motorista.exigir(protoccolo.Pedido{
76         Acao: "cadastrar", Rota: []string{"Feira", "Salvador"},
77         Data: data, Assentos: 2 * assentos, Preco: 30,
78     })
79     if err != nil {
80         return err
81     }
82
83     b, err := motorista.exigir(protoccolo.Pedido{
84         Acao: "cadastrar", Rota: []string{"Salvador", "Ilheus"},
85         Data: data, Assentos: assentos, Preco: 40,
86     })
87     if err != nil {
88         return err
89     }
90
91     // O itinerario disputado usa as DUAS caronas: tudo ou nada.
92     itinerario := []protoccolo.Item{
93         {CaronaID: a.CaronaID, De: 0, Ate: 1},
94         {CaronaID: b.CaronaID, De: 0, Ate: 1},
95     }
96
97     fmt.Printf("itinerario: carona %d Feira->Salvador (%d lugares) + carona %d Salvador->Ilheus (%d
lugares)\n",
98         a.CaronaID, 2*assentos, b.CaronaID, assentos)
99
100    // ---- 2. Os clientes conectam, fazem login e disparam juntos ----
101
102    resultados := make(chan resultado, clientes)
103    largada := make(chan struct{})
104
105    // A preparacao (conectar + login) roda com no maximo 50 clientes por vez.
106    //
107    // O que este teste mede e a DISPUTA pela reserva, e ela continua com todos
108    // juntos, depois da largada. Ja abrir centenas de conexoes no mesmo
109    // milissegundo so estoura a fila de conexoes pendentes do sistema
110    // operacional, que tem tamanho limitado e recusa o excedente antes de o
111    // servidor sequer ver a conexao.
112    preparando := make(chan struct{}, 50)
113
114    var prontos, terminados sync.WaitGroup
115
116    for i := 0; i < clientes; i++ {
117        prontos.Add(1)
118        terminados.Add(1)
119
120        go func(n int) {
121            defer terminados.Done()
122
123            // Preparacao: cada cliente tem a SUA conexao e o SEU usuario.
124            preparando <- struct{}{} // pega uma das 50 vagas
125            c, err := conectar(endereco)
126            if err == nil {
127                defer c.Close()
128                err = entrar(c, fmt.Sprintf("pas_%s_%d", tag, n), "passageiro")

```

```

129     }
130     <-preparando // devolve a vaga
131
132     prontos.Done()
133     <-largada // espera todos estarem prontos
134
135     if err != nil {
136         resultados <- resultado{erro: err}
137         return
138     }
139
140     inicio := time.Now()
141     r, err := c.pedir(protocolo.Pedido{Acao: "reservar", Itens: itinerario})
142     latencia := time.Since(inicio)
143
144     switch {
145     case err != nil:
146         resultados <- resultado{erro: err}
147     case !r.OK:
148         resultados <- resultado{recusado: true, latencia: latencia}
149     default:
150         resultados <- resultado{reservaID: r.ReservaID, latencia: latencia}
151     }
152     }(i)
153 }
154
155 prontos.Wait()
156 fmt.Printf("%d clientes conectados e logados; disparando todos juntos...\n\n", clientes)
157
158 inicio := time.Now()
159 close(largada)
160 terminados.Wait()
161 duracao := time.Since(inicio)
162 close(resultados)
163
164 // ---- 3. Contagem ----
165
166 var latencias []time.Duration
167 ids := map[int]bool{}
168 aceitas, recusadas, falhas := 0, 0, 0
169 var primeiraFalha error
170
171 for r := range resultados {
172     switch {
173     case r.erro != nil:
174         falhas++
175         if primeiraFalha == nil {
176             primeiraFalha = r.erro
177         }
178         continue
179     case r.recusado:
180         recusadas++
181     default:
182         aceitas++
183         ids[r.reservaID] = true
184     }
185     latencias = append(latencias, r.latencia)
186 }
187
188 fmt.Printf("reservas aceitas: %d\n", aceitas)
189 fmt.Printf("reservas recusadas: %d (sem lugar: e o esperado)\n", recusadas)
190 fmt.Printf("falhas de conexao: %d\n", falhas)
191 if primeiraFalha != nil {
192     fmt.Println(" primeira falha:", primeiraFalha)
193 }
194 fmt.Printf("tempo da disputa: %s\n", duracao.Round(time.Millisecond))
195 imprimirLatencias(latencias)

```

```

196
197 // ---- 4. Verificacao ----
198
199 fmt.Println("\nverificacao:")
200 v := &verificacao{}
201
202 v.conferir("nenhum lugar vendido duas vezes",
203     aceitas <= assentos,
204     fmt.Sprintf("%d reservas aceitas para %d lugares", aceitas, assentos))
205
206 esperadas := min(assentos, clientes-falhas)
207 v.conferir("todos os lugares disponiveis foram vendidos",
208     aceitas == esperadas,
209     fmt.Sprintf("esperava %d, vieram %d", esperadas, aceitas))
210
211 v.conferir("cada reserva aceita tem numero unico",
212     len(ids) == aceitas,
213     fmt.Sprintf("%d reservas mas so %d numeros diferentes", aceitas, len(ids)))
214
215 // Atomicidade: uma sonda conta, pelo proprio protocolo, quantos lugares
216 // sobraram em cada carona. Se toda reserva aceita ocupou as DUAS, sobra
217 // exatamente (lugares - aceitas) em cada uma.
218 sonda, err := conectar(endereco)
219 if err != nil {
220     return fmt.Errorf("a sonda nao conectou: %w", err)
221 }
222 defer sonda.Close()
223
224 if err := entrar(sonda, "sonda_"+tag, "passageiro"); err != nil {
225     return err
226 }
227
228 sobraB, errB := contarVagas(sonda, b.CaronaID, assentos)
229 sobraA, errA := contarVagas(sonda, a.CaronaID, 2*assentos)
230
231 v.conferir("nenhum itinerario pela metade",
232     errA == nil && errB == nil && sobraA == 2*assentos-aceitas && sobraB == assentos-aceitas,
233     fmt.Sprintf("sobraram %d na primeira (esperado %d) e %d na segunda (esperado %d)",
234         sobraA, 2*assentos-aceitas, sobraB, assentos-aceitas))
235
236 // Limpeza: cancelar as caronas desfaz tudo que esta rodada criou.
237 for _, id := range []int{a.CaronaID, b.CaronaID} {
238     motorista.pedir(protocolo.Pedido{Acao: "cancelar_carona", CaronaID: id})
239 }
240
241 if v.falhou {
242     return fmt.Errorf("alguma verificacao nao passou")
243 }
244 return nil
245 }

```

```
52 func rodar(endereco string, clientes, assentos int) error {
```

O cenário inteiro. Devolve erro se algo impediu o teste ou se alguma verificação falhou.

```
55 tag := strconv.FormatInt(time.Now().UnixNano(), 36)
```

Um sufixo único para esta rodada: o instante atual em nanossegundos, escrito em base 36 (dígitos e letras), o que dá um texto curto. Logins e a data da rodada levam esse sufixo.

```
56 data := "carga-" + tag
```

A "data" das caronas é um texto que ninguém usaria, então a busca e as reservas desta rodada não se misturam com caronas reais nem com rodadas anteriores.

```
58 fmt.Printf("servidor %s | %d clientes | %d lugares disputados\n\n", endereco, clientes, assentos)
```

Cabeçalho com os parâmetros.

```
62 motorista, err := conectar(endereco)
```

A conexão que monta o cenário.

```
63-65 if err != nil {  
    return fmt.Errorf("o motorista nao conectou: %w", err)  
}
```

`%w` embrulha o erro original na mensagem nova.

```
66 defer motorista.Close()
```

Fecha no fim.

```
68-70 if err := entrar(motorista, "mot_"+tag, "motorista"); err != nil {  
    return err  
}
```

Cria um motorista novo e entra com ele.

```
75-77 a, err := motorista.exigir(protocolo.Pedido{  
    Acao: "cadastrar", Rota: []string{"Feira", "Salvador"},  
    Data: data, Assentos: 2 * assentos, Preco: 30,
```

A carona A, com o dobro de lugares. Um literal pode ter vários campos na mesma linha. `exigir` transforma uma recusa do servidor em erro.

```
79-81 if err != nil {  
    return err  
}
```

Sem a carona, não há teste.

```
83-85 b, err := motorista.exigir(protocolo.Pedido{  
    Acao: "cadastrar", Rota: []string{"Salvador", "Ilheus"},  
    Data: data, Assentos: assentos, Preco: 40,
```

A carona B, com `assentos` lugares: é ela que limita o itinerário.

```
87-89 if err != nil {  
    return err  
}
```

Idem.

```
92-94  itinerario := []protocolo.Item{
        {CaronaID: a.CaronaID, De: 0, Ate: 1},
        {CaronaID: b.CaronaID, De: 0, Ate: 1},
```

O itinerário disputado: Feira → Salvador na carona A mais Salvador → Ilhéus na carona B. Os números vêm do campo `CaronaID` das respostas de cadastro.

```
97-98  fmt.Printf("itinerario: carona %d Feira->Salvador (%d lugares) + carona %d Salvador->Ilheus (%d
        lugares)\n",
        a.CaronaID, 2*assentos, b.CaronaID, assentos)
```

Mostra o cenário montado.

```
102    resultados := make(chan resultado, clientes)
```

Canal com espaço para todos os resultados. Nenhuma goroutine fica presa ao enviar, e juntar os resultados não precisa de mutex.

```
103    largada := make(chan struct{})
```

O sinal de largada: todos esperam neste canal, e um `close` solta todos.

```
112    preparando := make(chan struct{}, 50)
```

O semáforo da preparação, com 50 vagas. O comentário acima dele explica por que existe: na primeira versão, com 500 clientes abrindo conexão no mesmo milissegundo, o sistema operacional recusou 222 conexões antes de o servidor vê-las.

```
114    var prontos, terminados sync.WaitGroup
```

Dois contadores: "todos conectaram e entraram" e "todos terminaram".

```
116    for i := 0; i < clientes; i++ {
```

Um cliente simulado por volta.

```
117-118  prontos.Add(1)
        terminados.Add(1)
```

`Add` antes de criar a goroutine, nunca dentro dela.

```
120    go func(n int) {
```

Uma goroutine por cliente. A função recebe o número do cliente como parâmetro `n`, passado no fim, em `i`.

```
121    defer terminados.Done()
```

Qualquer que seja o caminho de saída, este cliente conta como terminado.

```
124    preparando <- struct{}{} // pega uma das 50 vagas
```

Pega uma vaga do semáforo. `struct{}{}` é o tipo `struct{}{}` seguido do literal vazio `{}`: um valor que não ocupa memória. Se as 50 vagas estiverem ocupadas, espera aqui.

```
125    c, err := conectar(endereco)
```

Cada cliente tem a sua própria conexão TCP, como um cliente real.

```
126-129  if err == nil {
         defer c.Close()
         err = entrar(c, fmt.Sprintf("pas_%s_%d", tag, n), "passageiro")
       }
```

Se conectou, agenda o fechamento e cria o usuário deste cliente, com login único formado pelo sufixo e pelo número. `err` passa a guardar o resultado do `entrar`.

```
130  <-preparando // devolve a vaga
```

Devolve a vaga: recebe um valor do canal, abrindo espaço para o próximo.

```
132  prontos.Done()
```

Este cliente está pronto, com ou sem erro.

```
133  <-largada // espera todos estarem prontos
```

Espera o sinal. Todas as goroutines ficam paradas nesta linha até o `close(largada)`.

```
135-138  if err != nil {
         resultados <- resultado{erro: err}
         return
       }
```

Se a preparação falhou, registra a falha e termina sem disputar.

```
140  inicio := time.Now()
```

Começa a cronometrar este cliente.

```
141  r, err := c.pedir(protocolo.Pedido{Acao: "reservar", Itens: itinerario})
```

O momento da disputa: todos os clientes mandam o mesmo itinerário quase no mesmo instante.

```
142  latencia := time.Since(inicio)
```

Tempo entre mandar o pedido e receber a resposta, incluindo a espera na fila do mutex do servidor.

```
144  switch {
```

Classifica o resultado.

```
145-146  case err != nil:
         resultados <- resultado{erro: err}
```

Erro de rede durante a reserva.

```
147-148  case !r.OK:
         resultados <- resultado{recusado: true, latencia: latencia}
```

Recusada: não havia mais lugar. É o esperado para a maioria.

```
149-150  default:
         resultados <- resultado{reservaID: r.ReservaID, latencia: latencia}
```

Aceita: guarda o número da reserva.

152 }(*i*)

Fecha a função e a chama na hora, passando `i`. Cada goroutine recebe o seu próprio número.

155 `prontos.Wait()`

Espera todos os clientes estarem conectados e logados.

156 `fmt.Printf("%d clientes conectados e logados; disparando todos juntos...\n\n", clientes)`

Avisa que vai disparar.

158 `inicio := time.Now()`

O cronômetro da disputa inteira. Este `inicio` é da função `rodar`; os de dentro das goroutines são outras variáveis, uma por goroutine.

159 `close(largada)`

O disparo: todas as goroutines paradas em `<-largada` seguem ao mesmo tempo.

160 `terminados.Wait()`

Espera todas terminarem.

161 `duracao := time.Since(inicio)`

Quanto durou a disputa.

162 `close(resultados)`

Ninguém mais vai enviar. Fechar o canal permite ler com `for range` até o fim.

166 `var latencias []time.Duration`

As latências, para o relatório.

167 `ids := map[int]bool{}`

Um map usado como **conjunto**, o `set()` do Python. Go não tem um tipo conjunto; map de chave para `bool` é o costume.

168 `aceitas, recusadas, falhas := 0, 0, 0`

Três variáveis numa linha.

169 `var primeiraFalha error`

`nil` até aparecer uma falha.

171 `for r := range resultados {`

Lê todos os resultados. O laço termina porque o canal foi fechado.

172 `switch {`

Classifica.

```
173-178 case r.erro != nil:
    falhas++
    if primeiraFalha == nil {
        primeiraFalha = r.erro
    }
    continue
```

Conta a falha, guarda só a primeira para mostrar, e `continue` pula o registro de latência logo abaixo: uma falha de rede não tem latência de reserva.

```
179-180 case r.recusado:
    recusadas++
```

Conta as recusadas.

```
181-183 default:
    aceitas++
    ids[r.reservaID] = true
```

Conta as aceitas e marca o número no conjunto.

```
185 latencias = append(latencias, r.latencia)
```

Aceitas e recusadas têm latência.

```
188-190 fmt.Printf("reservas aceitas:  %d\n", aceitas)
fmt.Printf("reservas recusadas: %d (sem lugar: e o esperado)\n", recusadas)
fmt.Printf("falhas de conexao: %d\n", falhas)
```

O relatório. Os espaços dentro dos textos alinham os números.

```
191-193 if primeiraFalha != nil {
    fmt.Println(" primeira falha:", primeiraFalha)
}
```

Mostra a primeira falha, se houve.

```
194 fmt.Printf("tempo da disputa:  %s\n", duracao.Round(time.Millisecond))
```

`Round(time.Millisecond)` arredonda para milissegundos.

```
195 imprimirlatencias(latencias)
```

Mínima, média, percentis e máxima.

```
199 fmt.Println("\nverificacao:")
```

O `\n` no começo pula uma linha.

```
200 v := &verificacao{}
```

Guarda se alguma verificação falhou.

```
202-204 v.conferir("nenhum lugar vendido duas vezes",
    aceitas <= assentos,
    fmt.Sprintf("%d reservas aceitas para %d lugares", aceitas, assentos))
```

Verificação 1: nunca mais reservas aceitas do que lugares na carona B. Se o mutex falhasse, este é o primeiro número a estourar. Cada `conferir` recebe o nome, a condição e o texto de detalhe.

```
206     esperadas := min(assentos, clientes-falhas)
```

`min` é uma função embutida desde o Go 1.21. Com mais clientes do que lugares, todos os lugares devem ser vendidos; com menos, todo cliente deve conseguir.

```
207-209     v.conferir("todos os lugares disponiveis foram vendidos",
              aceitas == esperadas,
              fmt.Sprintf("esperava %d, vieram %d", esperadas, aceitas))
```

Verificação 2: não sobrou lugar por engano. Pega o erro oposto ao da primeira: recusar quando ainda havia vaga.

```
211-213     v.conferir("cada reserva aceita tem numero unico",
              len(ids) == aceitas,
              fmt.Sprintf("%d reservas mas so %d numeros diferentes", aceitas, len(ids)))
```

Verificação 3: nenhum número de reserva repetido. `len(ids)` é o tamanho do conjunto.

```
218     sonda, err := conectar(endereco)
```

Uma conexão extra, que examina o resultado de fora, só pelo protocolo.

```
219-221     if err != nil {
              return fmt.Errorf("a sonda nao conectou: %w", err)
            }
```

Sem sonda, não há como verificar.

```
222     defer sonda.Close()
```

Fecha no fim.

```
224-226     if err := entrar(sonda, "sonda_"+tag, "passageiro"); err != nil {
              return err
            }
```

Cria o usuário da sonda.

```
228-229     sobraB, errB := contarVagas(sonda, b.CaronaID, assentos)
     sobraA, errA := contarVagas(sonda, a.CaronaID, 2*assentos)
```

Conta quantos lugares sobraram em cada carona, reservando um por um até ser recusado. Como cada reserva da sonda usa uma carona só, a contagem de uma não interfere na outra.

```
231-234     v.conferir("nenhum itinerario pela metade",
              errA == nil && errB == nil && sobraA == 2*assentos-aceitas && sobraB == assentos-aceitas,
              fmt.Sprintf("sobraram %d na primeira (esperado %d) e %d na segunda (esperado %d)",
                           sobraA, 2*assentos-aceitas, sobraB, assentos-aceitas))
```

Verificação 4, a atomicidade. Toda reserva aceita ocupou um lugar em A e um em B. Então deve sobrar `2N - aceitas` em A e `N - aceitas` em B. Se alguma reserva tivesse ocupado só A, sobraria menos em A do que a conta prevê.

```
237-239     for _, id := range []int{a.CaronaID, b.CaronaID} {
              motorista.pedir(protocolo.Pedido{Acao: "cancelar_carona", CaronaID: id})
            }
```

Limpeza: o motorista cancela as duas caronas, e o cancelamento em cascata desfaz todas as reservas da rodada, inclusive as da sonda. Os erros são ignorados: a limpeza não muda o resultado do teste.

```
241-243  if v.falhou {
        return fmt.Errorf("alguma verificacao nao passou")
    }
```

Alguma verificação falhou: devolve erro, e `main` sai com código 1.

```
244  return nil
```

Tudo conferiu.

contarVagas

pbl/carga/main.go · linhas 247–264

```
247 // contarVagas descobre quantos lugares sobraram numa carona reservando um por
248 // um ate o servidor recusar. Usa so o protocolo, sem olhar dentro do servidor.
249 func contarVagas(c *conexao, caronaID, limite int) (int, error) {
250     item := []protocolo.Item{{CaronaID: caronaID, De: 0, Ate: 1}}
251
252     for vagas := 0; vagas <= limite; vagas++ {
253         r, err := c.pedir(protocolo.Pedido{Acao: "reservar", Itens: item})
254         if err != nil {
255             return vagas, err
256         }
257         if !r.OK {
258             return vagas, nil
259         }
260     }
261
262     // Mais vagas do que a carona tem: o estado do servidor esta corrompido.
263     return limite + 1, nil
264 }
```

```
249  func contarVagas(c *conexao, caronaID, limite int) (int, error) {
```

Descobre quantos lugares restam numa carona sem olhar dentro do servidor.

```
250  item := []protocolo.Item{{CaronaID: caronaID, De: 0, Ate: 1}}
```

Um item de um trecho, da posição 0 à 1.

```
252  for vagas := 0; vagas <= limite; vagas++ {
```

Tenta até uma vez a mais do que o limite.

```
253  r, err := c.pedir(protocolo.Pedido{Acao: "reservar", Itens: item})
```

Reserva um lugar.

```
254-256  if err != nil {
        return vagas, err
    }
```

Erro de rede.

```
257-259  if !r.OK {
        return vagas, nil
    }
```

Primeira recusa: `vagas` é quantas reservas deram certo antes dela.

```
263  return limite + 1, nil
```

Conseguiu reservar mais do que a carona tem: o estado do servidor está corrompido, e o número impossível faz a verificação falhar.

O cliente mínimo

pbl/carga/main.go · linhas 270–274

```
270  // conexao e um cliente minimo: o socket e o leitor com buffer dele.
271  type conexao struct {
272      net.Conn
273      leitor *bufio.Reader
274  }
```

```
271  type conexao struct {
```

Um cliente mínimo: o socket e o leitor dele.

```
272  net.Conn
```

Campo embutido: só o tipo, sem nome. A struct ganha os métodos da `net.Conn`, então `c.Close()` e `c.Write(...)` funcionam direto numa `*conexao`. É composição, não herança; o nome do campo passa a ser o nome do tipo, `Conn`.

```
273  leitor *bufio.Reader
```

O leitor com buffer desta conexão.

pbl/carga/main.go · linhas 276–282

```
276  func conectar(endereco string) (*conexao, error) {
277      c, err := net.Dial("tcp", endereco)
278      if err != nil {
279          return nil, err
280      }
281      return &conexao{Conn: c, leitor: bufio.NewReader(c)}, nil
282  }
```

```
276  func conectar(endereco string) (*conexao, error) {
```

Conecta e monta a `conexao`.

```
277  c, err := net.Dial("tcp", endereco)
```

Abre o socket.

```
278-280  if err != nil {  
        return nil, err  
    }
```

`return nil, err`: sem conexão, o ponteiro volta `nil`.

```
281  return &conexao{Conn: c, leitor: bufio.NewReader(c)}, nil
```

O campo embutido é preenchido pelo nome do tipo: `Conn`.

pbl/carga/main.go · linhas 284–289

```
284 func (c *conexao) pedir(p protocolo.Pedido) (protocolo.Resposta, error) {  
285     if err := protocolo.EnviarPedido(c, p); err != nil {  
286         return protocolo.Resposta{}, err  
287     }  
288     return protocolo.LerResposta(c.leitor)  
289 }
```

```
284  func (c *conexao) pedir(p protocolo.Pedido) (protocolo.Resposta, error) {
```

Um pedido, uma resposta.

```
285-287  if err := protocolo.EnviarPedido(c, p); err != nil {  
        return protocolo.Resposta{}, err  
    }
```

Passa `c`, a `*conexao` inteira, como `io.Writer`: ela tem `Write` por causa da `net.Conn` embutida.

```
288  return protocolo.LerResposta(c.leitor)
```

Espera a resposta.

pbl/carga/main.go · linhas 291–302

```
291 // exigir e o pedir das etapas de preparacao: ali uma recusa do servidor  
292 // tambem e erro, porque o teste nao consegue seguir sem aquilo.  
293 func (c *conexao) exigir(p protocolo.Pedido) (protocolo.Resposta, error) {  
294     r, err := c.pedir(p)  
295     if err != nil {  
296         return r, err  
297     }  
298     if !r.OK {  
299         return r, fmt.Errorf("%s recusado: %s", p.Acao, r.Erro)  
300     }  
301     return r, nil  
302 }
```

```
293  func (c *conexao) exigir(p protocolo.Pedido) (protocolo.Resposta, error) {
```

Para a preparação, onde uma recusa impede o teste de seguir.

```
294  r, err := c.pedir(p)
```

Envia.

```
295-297  if err != nil {
        return r, err
    }
```

Erro de rede.

```
298-300  if !r.OK {
        return r, fmt.Errorf("%s recusado: %s", p.Acao, r.Erro)
    }
```

Transforma a recusa num erro com a ação e o motivo.

```
301      return r, nil
```

Deu certo.

pbl/carga/main.go · linhas 304–311

```
304  // entrar cria um usuario novo e faz login com ele na mesma conexao.
305  func entrar(c *conexao, login, tipo string) error {
306      if _, err := c.exigir(protocolo.Pedido{Acao: "registrar", Usuario: login, Senha: "carga", Tipo:
tipo}); err != nil {
307          return err
308      }
309      _, err := c.exigir(protocolo.Pedido{Acao: "login", Usuario: login, Senha: "carga"})
310      return err
311  }
```

```
305  func entrar(c *conexao, login, tipo string) error {
```

Cria um usuário e entra com ele, na mesma conexão.

```
306-308  if _, err := c.exigir(protocolo.Pedido{Acao: "registrar", Usuario: login, Senha: "carga", Tipo:
tipo}); err != nil {
        return err
    }
```

Todos os usuários da carga usam a senha `carga`.

```
309      _, err := c.exigir(protocolo.Pedido{Acao: "login", Usuario: login, Senha: "carga"})
```

Descarta a resposta e fica só com o erro.

```
310      return err
```

`nil` se entrou.

Resultados, verificações e latências

pbl/carga/main.go · linhas 313–318

```
313  type resultado struct {
314      reservaID int
315      recusado  bool
316      erro      error
317      latencia  time.Duration
318  }
```

313 `type resultado struct {`

O que cada goroutine manda pelo canal.

314-317 `reservaID int
recusado bool
erro error
latencia time.Duration`

O número da reserva (se aceita), se foi recusada, o erro (se houve) e a latência. Campos minúsculos: é um tipo interno deste programa.

pbl/carga/main.go · linhas 320-322

```
320 type verificacao struct {  
321     falhou bool  
322 }
```

320-321 `type verificacao struct {
 falhou bool`

Guarda se alguma verificação falhou.

pbl/carga/main.go · linhas 324-331

```
324 func (v *verificacao) conferir(nome string, ok bool, detalhe string) {  
325     if ok {  
326         fmt.Printf(" [ok]      %s\n", nome)  
327         return  
328     }  
329     fmt.Printf(" [FALHOU] %s: %s\n", nome, detalhe)  
330     v.falhou = true  
331 }
```

324 `func (v *verificacao) conferir(nome string, ok bool, detalhe string) {`

Receptor ponteiro, porque altera `falhou`.

325-328 `if ok {
 fmt.Printf(" [ok] %s\n", nome)
 return
}`

Passou: imprime `[ok]` e sai da função.

329 `fmt.Printf(" [FALHOU] %s: %s\n", nome, detalhe)`

Não passou: imprime com o detalhe.

330 `v.falhou = true`

Marca a falha.

pbl/carga/main.go · linhas 333-354

```

333 func imprimirLatencias(l []time.Duration) {
334     if len(l) == 0 {
335         return
336     }
337
338     sort.Slice(l, func(i, j int) bool { return l[i] < l[j] })
339
340     var soma time.Duration
341     for _, d := range l {
342         soma += d
343     }
344
345     percentil := func(p float64) time.Duration {
346         return l[int(p*float64(len(l)-1))]
347     }
348
349     arred := func(d time.Duration) time.Duration { return d.Round(100 * time.Microsecond) }
350
351     fmt.Printf("latencia da reserva: min %s | media %s | p50 %s | p95 %s | max %s\n",
352         arred(l[0]), arred(soma/time.Duration(len(l))), arred(percentil(0.50)),
353         arred(percentil(0.95)), arred(l[len(l)-1]))
354 }

```

333 `func imprimirLatencias(l []time.Duration) {`

Resume as latências.

334-336 `if len(l) == 0 {
 return
}`

Nada para medir. Também evita dividir por zero logo abaixo.

338 `sort.Slice(l, func(i, j int) bool { return l[i] < l[j] })`

Ordena da menor para a maior. `sort.Slice` recebe o slice e uma função que diz se o elemento `i` vem antes do `j`. Ordena no lugar, alterando o próprio slice, como o `list.sort()`.

340 `var soma time.Duration`

Começa em zero.

341-343 `for _, d := range l {
 soma += d
}`

Soma todas. Durations são inteiros e podem ser somadas.

345-347 `percentil := func(p float64) time.Duration {
 return l[int(p*float64(len(l)-1))]
}`

Uma função guardada numa variável. Para o percentil `p`, pega a posição `p × (n-1)` do slice ordenado. `float64(...)` e `int(...)` são conversões explícitas: Go não mistura inteiro e ponto flutuante numa mesma conta.

349 `arred := func(d time.Duration) time.Duration { return d.Round(100 * time.Microsecond) }`

Uma função de uma linha que arredonda para 0,1 milissegundo.

351-353

```
fmt.Printf("latencia da reserva: min %s | media %s | p50 %s | p95 %s | max %s\n",
    arred(l[0]), arred(soma/time.Duration(len(l))), arred(percentil(0.50)),
    arred(percentil(0.95)), arred(l[len(l)-1]))
```

Mínima (a primeira), média (soma dividida pela quantidade, convertida para `Duration`), p50, p95 e máxima (a última). p95 quer dizer que 95% das reservas responderam em até esse tempo.

Parte V — Os testes

Vinte e seis testes automatizados, em três arquivos. Esta parte explica como rodá-los, o que cada um prova, e a lição mais importante que eles ensinaram: nem todo teste de concorrência detecta a falta de um mutex.

45. Como rodar e ler o resultado

```
cd pbl
go test ./...           # todos os pacotes
go test ./dados/ -v      # um pacote, mostrando cada teste
go test ./... -run Cancelar # só testes com "Cancelar" no nome
go test ./... -count=1    # sem usar resultados guardados
go test -race ./...       # com o detector de corrida (exige Go 64 bits)
```

Tudo passando aparece assim:

```
?      vaijunto/carga      [no test files]
?      vaijunto/cliente   [no test files]
ok     vaijunto/dados     0.8s
?      vaijunto/protocolo [no test files]
ok     vaijunto/servidor  1.0s
```

Uma falha mostra o nome do teste, o arquivo, a linha e a mensagem escrita no `t.Errorf`:

```
--- FAIL: TestContagemDeAssentos (0.02s)
    dados_test.go:128: perdi incrementos: Ocupados = 1937, esperado 2000
```

Arquivo	Testes	Cobre
<code>dados/dados_test.go</code>	17	Concorrência na reserva, atomicidade, disponibilidade por trecho, busca, expiração, pagamento, contas e cadastro de carona
<code>dados/cancelamento_test.go</code>	7	Os dois cancelamentos, passageiros por trecho e concorrência nos cancelamentos
<code>servidor/main_test.go</code>	2	Queda abrupta de cliente e JSON inválido, com conexões TCP de verdade

O DETECTOR DE CORRIDA AINDA NÃO RODOU

O `go test -race` acusa qualquer acesso simultâneo sem proteção, mesmo quando o resultado por acaso sai certo. Ele exige Go de 64 bits, e o Go instalado na máquina de desenvolvimento é de 32 bits (`windows/386`). A Parte VII mostra como rodar de outro jeito.

46. As peças que os testes reaproveitam

Os dois arquivos de teste do `dados` declaram `package dados`, o mesmo pacote do código. Isso tem duas consequências: os testes enxergam nomes minúsculos, como `acharCarona` e `Ocupados`; e as funções auxiliares de um arquivo de teste servem no outro sem importar nada.

pbl/dados/dados_test.go · linhas 12–15

```
12 // trecho monta o item "carona X, da cidade De ate a cidade Ate".
13 func trecho(caronaID, de, ate int) []protocolo.Item {
```

```
14 return []protocolo.Item{{CaronaID: caronaID, De: de, Ate: ate}}
15 }
```

```
13 func trecho(caronaID, de, ate int) []protocolo.Item {
```

Monta a lista de um item só, que quase todo teste usa. Evita repetir `[]protocolo.Item{{CaronaID: ..., De: ..., Ate: ...}}` dezenas de vezes.

```
14 return []protocolo.Item{{CaronaID: caronaID, De: de, Ate: ate}}
```

Slice por fora, item por dentro.

pbl/dados/dados_test.go · linhas 17–28

```
17 // bancoComCarona devolve um banco novo com uma carona ja cadastrada.
18 func bancoComCarona(t *testing.T, rota []string, assentos int) (*Banco, int) {
19     t.Helper()
20
21     b := NovoBanco()
22     id, err := b.CadastrarCarona("ana", rota, "2026-09-14", assentos, 30)
23     if err != nil {
24         t.Fatalf("nao consegui cadastrar a carona: %v", err)
25     }
26
27     return b, id
28 }
```

```
18 func bancoComCarona(t *testing.T, rota []string, assentos int) (*Banco, int) {
```

Cria um banco novo com uma carona da ana, e devolve os dois. Recebe o `t` para poder falhar o teste se o cadastro não funcionar.

```
19 t.Helper()
```

Marca a função como auxiliar: se ela chamar `t.Fatalf`, a mensagem aponta a linha do **teste que a chamou**, e não esta.

```
21 b := NovoBanco()
```

Cada teste usa um banco próprio, sem arquivo. Um teste nunca interfere no outro.

```
22 id, err := b.CadastrarCarona("ana", rota, "2026-09-14", assentos, 30)
```

Data fixa, preço 30; rota e assentos vêm do teste.

```
23-25 if err != nil {
    t.Fatalf("nao consegui cadastrar a carona: %v", err)
}
```

`t.Fatalf` marca a falha e **para** o teste na hora: sem a carona, nada mais faria sentido.

```
27 return b, id
```

O banco e o número da carona.

47. Como se testa concorrência

Todo teste de concorrência do projeto segue o mesmo desenho:

```
var wg sync.WaitGroup
largada := make(chan struct{})

for i := 0; i < N; i++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        <-largada // todas param aqui
        // ... a operação disputada ...
    }()
}

close(largada) // solta todas ao mesmo tempo
wg.Wait()     // espera todas terminarem
// ... confere o estado final ...
```

- A **largada** junta as goroutines num mesmo ponto antes de começar, para aumentar a chance de elas se sobreporem.
- O **WaitGroup** garante que o teste só confere o estado depois de todas terminarem.
- Quando é preciso contar sucessos, cada goroutine manda um valor num **canal com espaço**, e o teste conta com `len` no fim. Assim a contagem não precisa de outro mutex.
- `t.Errorf` pode ser chamado de dentro das goroutines. `t.Fatal` não: ele só pode ser chamado pela goroutine do próprio teste.

A lição: nem todo teste detecta a falta do mutex

Para saber se um teste de concorrência vale alguma coisa, o único jeito é **tirar o mutex de propósito** e ver se ele falha. Foi feito com vários deles:

Teste	Desenho	Sem o mutex
<code>TestReservaConcorrente</code>	muitas goroutines, 1 lugar	Passou (com 50 e com 2000 goroutines): não detecta
<code>TestContagemDeAssentos</code>	2000 goroutines, 2000 lugares	Falhou: <code>Ocupados = 1937</code> , 1839 reservas
<code>TestCadastroConcorrente</code>	500 goroutines, mesmo login	Passou: não detecta
<code>TestCadastrosSimultaneos</code>	300 goroutines, logins diferentes	Derrubou o programa: <code>concurrent map read and map write</code>
<code>TestReservaECancelamentoConcorrentes</code>	1000 goroutines, todas reservam e cancelam	Falhou: sobraram 11, 6 e 13 lugares ocupados

Por que os testes de "um vencedor" não detectam: o `close(largada)` acorda as goroutines uma de cada vez, bem rápido, mas não no mesmo instante. A primeira acordada já termina a operação e ocupa o único lugar antes de a maioria das outras começar; todas as demais encontram o lugar ocupado e falham **corretamente**, por um motivo legítimo. A disputa real quase não acontece.

Por que os testes de "todos escrevem" detectam: quando todas as goroutines têm lugar e todas escrevem, elas ficam o tempo inteiro dentro da seção crítica ao mesmo tempo, e as escritas perdidas aparecem na contagem final.

POR QUE MANTER OS DOIS TIPOS

O teste de "um vencedor" continua útil: ele descreve a **regra de negócio** ("dois passageiros, um lugar, só um leva") e pega erros de lógica. O teste de "todos escrevem" é o **detector** de falta de proteção. Um documenta o que o sistema promete; o outro prova que a proteção existe.

48. dados_test.go: concorrência na reserva

pbl/dados/dados_test.go · linhas 34–93

```
34 // TestReservaConcorrente e o teste que prova o tratamento de concorrência.
35 //
36 // Uma carona com UM assento e 50 goroutines tentando reservar ao mesmo tempo.
37 // So uma pode vencer. Sem o mutex em Reservar, varias leriam "1 livre" antes
38 // de qualquer uma escrever, e o sistema venderia o mesmo assento varias vezes.
39 //
40 // Rode com o detector de corrida:
41 //
42 // go test -race ./...
43 //
44 // Para ver o teste falhar de proposito, comente o b.mu.Lock() de Reservar.
45 func TestReservaConcorrente(t *testing.T) {
46     const tentativas = 200
47
48     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
49
50     // Canal com espaco para todos: cada goroutine que conseguir reservar
51     // joga um aviso aqui. Usar canal evita precisar de outro mutex so para
52     // contar os sucessos.
53     sucessos := make(chan int, tentativas)
54
55     // WaitGroup e o contador que espera todas as goroutines terminarem.
56     var wg sync.WaitGroup
57
58     // largada trava todas as goroutines no mesmo ponto. Sem isso, a primeira
59     // a nascer ja teria reservado antes de a ultima existir, e nao haveria
60     // disputa nenhuma para testar.
61     largada := make(chan struct{})
62
63     for i := 0; i < tentativas; i++ {
64         wg.Add(1)
65
66         go func() {
67             defer wg.Done()
68
69             <-largada // espera aqui ate o canal ser fechado
70
71             if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err == nil {
72                 sucessos <- 1
73             }
74         }()
75     }
76
77     close(largada) // solta as 50 de uma vez
78     wg.Wait()     // espera todas terminarem
79     close(sucessos)
```

```

80
81 if n := len(sucessos); n != 1 {
82     t.Errorf("esperava exatamente 1 reserva bem sucedida, deu %d", n)
83 }
84
85 c := b.acharCarona(id)
86 if c.Ocupados[0] != 1 {
87     t.Errorf("esperava o trecho com 1 assento ocupado, deu %d", c.Ocupados[0])
88 }
89
90 if n := len(b.Reservas); n != 1 {
91     t.Errorf("esperava 1 reserva registrada, deu %d", n)
92 }
93 }

```

```
45 func TestReservaConcorrente(t *testing.T) {
```

Todo teste começa com `Test`, recebe `t *testing.T` e não devolve nada. O `go test` encontra e roda todas as funções com esse formato.

```
46 const tentativas = 200
```

Uma constante local. O comentário acima do teste e o da linha do `close` ainda falam em 50 goroutines, de uma versão anterior (Parte VII).

```
48 b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
```

Uma carona com **um** lugar.

```
53 sucessos := make(chan int, tentativas)
```

Canal com espaço para todos: quem conseguir reservar manda um valor.

```
56 var wg sync.WaitGroup
```

O contador de goroutines.

```
61 largada := make(chan struct{})
```

O sinal de largada.

```
63 for i := 0; i < tentativas; i++ {
```

Cria as goroutines.

```
64 wg.Add(1)
```

Conta uma a mais, antes de criá-la.

```
66 go func() {
```

Uma função sem nome rodando como goroutine. Ela usa `b`, `id`, `sucessos` e `largada` do escopo de fora: é uma **closure**, como uma função interna do Python que enxerga as variáveis da função que a contém.

```
67 defer wg.Done()
```

Desconta ao terminar.

```
69 <-largada // espera aqui ate o canal ser fechado
```

Espera o sinal.

```
71-73 if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err == nil {
      sucessos <- 1
    }
```

Tenta reservar; se deu certo, avisa pelo canal.

```
74 }()
```

Fecha a função e a chama na hora.

```
77 close(largada) // solta as 50 de uma vez
```

Solta todas.

```
78 wg.Wait() // espera todas terminarem
```

Espera todas.

```
79 close(sucessos)
```

Fecha o canal; `len` continua informando quantos valores ficaram dentro.

```
81-83 if n := len(sucessos); n != 1 {
      t.Errorf("esperava exatamente 1 reserva bem sucedida, deu %d", n)
    }
```

Exatamente uma reserva aceita.

```
85 c := b.acharCarona(id)
```

O teste está no mesmo pacote, então pode usar `acharCarona`, que é minúscula.

```
86-88 if c.Ocupados[0] != 1 {
      t.Errorf("esperava o trecho com 1 assento ocupado, deu %d", c.Ocupados[0])
    }
```

O contador marca exatamente um lugar ocupado.

```
90-92 if n := len(b.Reservas); n != 1 {
      t.Errorf("esperava 1 reserva registrada, deu %d", n)
    }
```

E existe exatamente uma reserva registrada.

pbl/dados/dados_test.go · linhas 95–135

```
95 // TestContagemDeAssentos e o teste que pega a falta do mutex de forma
96 // CONFIÁVEL.
97 //
98 // O teste acima (1 assento, muitas goroutines) e a garantia de negocio, mas
99 // sozinho ele nao detecta bem o bug: quem largar primeiro ja ocupou o assento
100 // antes de as outras nascerem, e todas as demais falham "corretamente".
101 //
102 // Aqui e diferente: sao N assentos para N goroutines, entao TODAS entram na
103 // secao critica e TODAS escrevem. Sem o mutex, dois "Ocupados[t]++"
104 // simultaneos viram um so (o classico "lost update") e a contagem final fica
```

```

105 // MENOR que N.
106 func TestContagemDeAssentos(t *testing.T) {
107     const pedidos = 2000
108
109     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, pedidos)
110
111     var wg sync.WaitGroup
112     largada := make(chan struct{})
113
114     for i := 0; i < pedidos; i++ {
115         wg.Add(1)
116         go func() {
117             defer wg.Done()
118             <-largada
119             if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
120                 t.Errorf("reserva deveria funcionar, sobra assento: %v", err)
121             }
122         }()
123     }
124
125     close(largada)
126     wg.Wait()
127
128     if n := b.acharCarona(id).Ocupados[0]; n != pedidos {
129         t.Errorf("perdi incrementos: Ocupados = %d, esperado %d", n, pedidos)
130     }
131
132     if n := len(b.Reservas); n != pedidos {
133         t.Errorf("perdi reservas: %d registradas, esperado %d", n, pedidos)
134     }
135 }

```

106 `func TestContagemDeAssentos(t *testing.T) {`

O detector confiável de falta de mutex no `Reservar`. O comentário acima dele explica a diferença para o teste anterior.

107 `const pedidos = 2000`

2000 pedidos...

109 `b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, pedidos)`

...para 2000 lugares: todos devem conseguir.

119-121 `if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
 t.Errorf("reserva deveria funcionar, sobra assento: %v", err)
}`

Aqui toda reserva deveria funcionar; se alguma falhar, é erro. `t.Errorf` de dentro de uma goroutine é permitido.

128-130 `if n := b.acharCarona(id).Ocupados[0]; n != pedidos {
 t.Errorf("perdi incrementos: Ocupados = %d, esperado %d", n, pedidos)
}`

Sem o mutex: 1937. Incrementos perdidos.

```
132-134  if n := len(b.Reservas); n != pedidos {
        t.Errorf("perdi reservas: %d registradas, esperado %d", n, pedidos)
    }
}
```

Sem o mutex: 1839. O `append` no slice também perde elementos quando duas goroutines crescem o slice ao mesmo tempo.

pbl/dados/dados_test.go · linhas 137–173

```
137 // TestExpiracaoConcorrente roda a expiracao ao mesmo tempo que as reservas.
138 // Sao duas goroutines mexendo no MESMO contador de assentos por caminhos
139 // diferentes -- exatamente o cenario do servidor de verdade.
140 func TestExpiracaoConcorrente(t *testing.T) {
141     original := TempoDeReserva
142     TempoDeReserva = time.Millisecond
143     defer func() { TempoDeReserva = original }()
144
145     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 5)
146
147     var wg sync.WaitGroup
148
149     for i := 0; i < 30; i++ {
150         wg.Add(1)
151         go func() {
152             defer wg.Done()
153             b.Reservar("bruno", trecho(id, 0, 1))
154         }()
155     }
156
157     for i := 0; i < 30; i++ {
158         wg.Add(1)
159         go func() {
160             defer wg.Done()
161             b.ExpirarVencidas()
162         }()
163     }
164
165     wg.Wait()
166
167     // O numero final depende do sorteio das goroutines, mas o estado tem que
168     // ser COERENTE: nunca negativo, nunca acima do total de assentos.
169     c := b.acharCarona(id)
170     if c.Ocupados[0] < 0 || c.Ocupados[0] > c.Assentos {
171         t.Errorf("contador de assentos incoerente: %d (total %d)", c.Ocupados[0], c.Assentos)
172     }
173 }
```

```
140 func TestExpiracaoConcorrente(t *testing.T) {
```

Reservas e expirações ao mesmo tempo, mexendo no mesmo contador por caminhos diferentes.

```
141-143 original := TempoDeReserva
        TempoDeReserva = time.Millisecond
        defer func() { TempoDeReserva = original }()
```

Guarda o prazo global, troca por 1 milissegundo e agenda a restauração. `defer func() { ... }()` agenda uma função sem nome, que roda quando o teste terminar.

```
145 b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 5)
```

Cinco lugares.

```
149 for i := 0; i < 30; i++ {
```

30 goroutines reservando.

```
153 b.Reservar("bruno", trecho(id, 0, 1))
```

O erro é ignorado: com cinco lugares, várias vão falhar.

```
157 for i := 0; i < 30; i++ {
```

E 30 expirando.

```
161 b.ExpirarVencidas()
```

Cada uma varre as reservas.

```
165 wg.Wait()
```

Não há largada neste teste: as goroutines começam assim que são criadas.

```
170-172 if c.Ocupados[0] < 0 || c.Ocupados[0] > c.Assentos {
    t.Errorf("contador de assentos incoerente: %d (total %d)", c.Ocupados[0], c.Assentos)
}
```

O número final depende da ordem, então o teste confere só a **coerência**: nunca negativo, nunca acima do total. É um teste mais fraco, e não foi sabotado para confirmar se detecta a falta do mutex.

49. dados_test.go: atomicidade e disponibilidade

pbl/dados/dados_test.go · linhas 179–208

```
179 // TestReservaAtomica confere o "pegou um trecho nao fica sem o outro".
180 // Se o segundo item falhar, o primeiro NAO pode ter sido aplicado.
181 func TestReservaAtomica(t *testing.T) {
182     b, livre := bancoComCarona(t, []string{"Ilheus", "Porto Seguro"}, 2)
183
184     cheia, err := b.CadastrarCarona("davi", []string{"Feira", "Salvador"}, "2026-09-14", 1, 30)
185     if err != nil {
186         t.Fatalf("nao consegui cadastrar: %v", err)
187     }
188
189     // Enche a segunda carona.
190     if _, err := b.Reservar("carla", trecho(cheia, 0, 1)); err != nil {
191         t.Fatalf("a primeira reserva deveria funcionar: %v", err)
192     }
193
194     // Agora pede as duas juntas. A carona livre vem PRIMEIRO de proposito:
195     // se a implementacao aplicasse item a item, ela ja teria sido ocupada
196     // quando a segunda falhasse.
197     _, err = b.Reservar("bruno", []protocolo.Item{
198         {CaronaID: livre, De: 0, Ate: 1},
199         {CaronaID: cheia, De: 0, Ate: 1},
200     })
201     if err == nil {
```

```

202     t.Fatal("a reserva deveria falhar: a segunda carona esta cheia")
203 }
204
205 if ocupados := b.acharCarona(livre).Ocupados[0]; ocupados != 0 {
206     t.Errorf("a carona livre foi alterada apesar da falha: ocupados = %d", ocupados)
207 }
208 }

```

181 `func TestReservaAtomica(t *testing.T) {`

Prova o "pegou um trecho, não fica sem o outro".

182 `b, livre := bancoComCarona(t, []string{"Ilheus", "Porto Seguro"}, 2)`

O nome da variável diz o papel: esta carona terá lugar.

184-187 `cheia, err := b.CadastrarCarona("davi", []string{"Feira", "Salvador"}, "2026-09-14", 1, 30)`
`if err != nil {`
 `t.Fatalf("nao consegui cadastrar: %v", err)`
`}`

Uma segunda carona, com um lugar só.

190-192 `if _, err := b.Reservar("carla", trecho(cheia, 0, 1)); err != nil {`
 `t.Fatalf("a primeira reserva deveria funcionar: %v", err)`
`}`

A carla ocupa o único lugar da segunda.

197-199 `_, err = b.Reservar("bruno", []protocolo.Item{`
 `{CaronaID: livre, De: 0, Ate: 1},`
 `{CaronaID: cheia, De: 0, Ate: 1},`

O bruno pede as duas juntas. `=` e não `:=`, porque `err` já existe. A carona livre vem **primeiro** de propósito: uma implementação que aplicasse item por item já a teria ocupado quando a segunda falhasse.

201-203 `if err == nil {`
 `t.Fatal("a reserva deveria falhar: a segunda carona esta cheia")`
`}`

A reserva tem que falhar.

205-207 `if ocupados := b.acharCarona(livre).Ocupados[0]; ocupados != 0 {`
 `t.Errorf("a carona livre foi alterada apesar da falha: ocupados = %d", ocupados)`
`}`

A prova: a carona livre, que veio primeiro no pedido, continua intocada.

pbl/dados/dados_test.go · linhas 210-234

```

210 // TestReservaMultiplaFunciona garante que o caso feliz de duas caronas
211 // tambem funciona -- senao o teste acima passaria com um Reservar que
212 // simplesmente recusa tudo.
213 func TestReservaMultiplaFunciona(t *testing.T) {
214     b, primeira := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
215
216     segunda, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 1, 40)
217     if err != nil {
218         t.Fatalf("nao consegui cadastrar: %v", err)
219     }

```

```

220
221 if _, err := b.Reservar("bruno", []protocolo.Item{
222     {CaronaID: primeira, De: 0, Ate: 1},
223     {CaronaID: segunda, De: 0, Ate: 1},
224 }); err != nil {
225     t.Fatalf("a reserva das duas deveria funcionar: %v", err)
226 }
227
228 if n := b.acharCarona(primeira).Ocupados[0]; n != 1 {
229     t.Errorf("primeira carona: esperava 1 ocupado, deu %d", n)
230 }
231 if n := b.acharCarona(segunda).Ocupados[0]; n != 1 {
232     t.Errorf("segunda carona: esperava 1 ocupado, deu %d", n)
233 }
234 }

```

213 `func TestReservaMultiplaFunciona(t *testing.T) {`

O par do teste anterior. Sem ele, um `Reservar` que recusasse **tudo** passaria no teste de atomicidade.

216-219 `segunda, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 1, 40)`
`if err != nil {`
 `t.Fatalf("nao consegui cadastrar: %v", err)`
`}`

Duas caronas com um lugar cada.

221-225 `if _, err := b.Reservar("bruno", []protocolo.Item{`
 `{CaronaID: primeira, De: 0, Ate: 1},`
 `{CaronaID: segunda, De: 0, Ate: 1},`
`}); err != nil {`
 `t.Fatalf("a reserva das duas deveria funcionar: %v", err)`

Reserva as duas juntas, e tem que funcionar.

228-232 `if n := b.acharCarona(primeira).Ocupados[0]; n != 1 {`
 `t.Errorf("primeira carona: esperava 1 ocupado, deu %d", n)`
`}`
`if n := b.acharCarona(segunda).Ocupados[0]; n != 1 {`
 `t.Errorf("segunda carona: esperava 1 ocupado, deu %d", n)`

As duas ficaram ocupadas.

pbl/dados/dados_test.go · linhas 240–267

```

240 // TestDisponibilidadePorTrecho e o comportamento que diferencia o sistema:
241 // ocupar um trecho NAO ocupa a carona inteira.
242 func TestDisponibilidadePorTrecho(t *testing.T) {
243     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 1)
244
245     // Reserva so o primeiro trecho.
246     if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
247         t.Fatalf("deveria funcionar: %v", err)
248     }
249
250     casos := []struct {
251         nome          string
252         origem, destino string
253         querOpcoes    int
254     }{
255         {"o trecho seguinte continua livre", "Salvador", "Ilheus", 1},
256         {"o trecho reservado ficou cheio", "Feira", "Salvador", 0},

```

```

257     {"a viagem inteira precisa dos dois", "Feira", "Ilheus", 0},
258 }
259
260 for _, caso := range casos {
261     opcoes := b.Buscar(caso.origem, caso.destino, "2026-09-14")
262     if len(opcoes) != caso.querOpcoes {
263         t.Errorf("%s: %s->%s deu %d opcao(oes), queria %d",
264             caso.nome, caso.origem, caso.destino, len(opcoes), caso.querOpcoes)
265     }
266 }
267 }

```

242 `func TestDisponibilidadePorTrecho(t *testing.T) {`

O comportamento que diferencia o sistema.

246-248 `if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {`
`t.Fatalf("deveria funcionar: %v", err)`
`}`

Numa carona de 3 cidades e 1 lugar, o bruno reserva só o primeiro trecho.

250-253 `casos := []struct {`
 `nome string`
 `origem, destino string`
 `querOpcoes int`
`}`

Uma **tabela de casos**: um slice de uma **struct anônima**, um tipo sem nome declarado ali mesmo.

`origem, destino string` divide o tipo, como nos parâmetros de função.

É o `@pytest.mark.parametrize` do Go.

255-257 `{"o trecho seguinte continua livre", "Salvador", "Ilheus", 1},`
`{"o trecho reservado ficou cheio", "Feira", "Salvador", 0},`
`{"a viagem inteira precisa dos dois", "Feira", "Ilheus", 0},`

Os três casos da tabela da Parte I, cada um com a quantidade de opções esperada.

260 `for _, caso := range casos {`

Um laço roda todos.

261 `opcoes := b.Buscar(caso.origem, caso.destino, "2026-09-14")`

Busca o par do caso.

262-264 `if len(opcoes) != caso.querOpcoes {`
`t.Errorf("%s: %s->%s deu %d opcao(oes), queria %d",`
 `caso.nome, caso.origem, caso.destino, len(opcoes), caso.querOpcoes)`

A mensagem inclui o nome do caso: quando falha, diz exatamente qual situação quebrou.

pbl/dados/dados_test.go · linhas 269-285

```

269 // TestBuscaSentidoUnico: a carona so anda para frente.
270 func TestBuscaSentidoUnico(t *testing.T) {
271     b, _ := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 2)
272
273     if n := len(b.Buscar("Ilheus", "Feira", "2026-09-14")); n != 0 {
274         t.Errorf("sentido contrario deveria dar 0 opcoes, deu %d", n)

```

```

275 }
276 if n := len(b.Buscar("Salvador", "Salvador", "2026-09-14")); n != 0 {
277     t.Errorf("origem igual ao destino deveria dar 0 opcoes, deu %d", n)
278 }
279 if n := len(b.Buscar("Feira", "Recife", "2026-09-14")); n != 0 {
280     t.Errorf("cidade fora da rota deveria dar 0 opcoes, deu %d", n)
281 }
282 if n := len(b.Buscar("Feira", "Ilheus", "2026-12-25")); n != 0 {
283     t.Errorf("data diferente deveria dar 0 opcoes, deu %d", n)
284 }
285 }

```

271 b, _ := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 2)

_ descarta o número da carona, que este teste não usa.

273-283

```

if n := len(b.Buscar("Ilheus", "Feira", "2026-09-14")); n != 0 {
    t.Errorf("sentido contrario deveria dar 0 opcoes, deu %d", n)
}
if n := len(b.Buscar("Salvador", "Salvador", "2026-09-14")); n != 0 {
    t.Errorf("origem igual ao destino deveria dar 0 opcoes, deu %d", n)
}
if n := len(b.Buscar("Feira", "Recife", "2026-09-14")); n != 0 {
    t.Errorf("cidade fora da rota deveria dar 0 opcoes, deu %d", n)
}
if n := len(b.Buscar("Feira", "Ilheus", "2026-12-25")); n != 0 {
    t.Errorf("data diferente deveria dar 0 opcoes, deu %d", n)
}

```

Quatro buscas que não podem dar resultado: sentido contrário, origem igual ao destino, cidade fora da rota e data diferente. Cada `if` declara o seu próprio `n`.

pbl/dados/dados_test.go · linhas 287-308

```

287 // TestBuscaComConexao: duas caronas que juntas levam o passageiro ao destino.
288 func TestBuscaComConexao(t *testing.T) {
289     b, _ := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
290
291     if _, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 2, 40); err !=
nil {
292         t.Fatalf("nao consegui cadastrar: %v", err)
293     }
294
295     opcoes := b.Buscar("Feira", "Ilheus", "2026-09-14")
296     if len(opcoes) != 1 {
297         t.Fatalf("esperava 1 opcao com conexao, deu %d", len(opcoes))
298     }
299
300     if n := len(opcoes[0].Itens); n != 2 {
301         t.Errorf("a opcao deveria ter 2 itens, tem %d", n)
302     }
303
304     // 30 do primeiro trecho + 40 do segundo.
305     if opcoes[0].Preco != 70 {
306         t.Errorf("preco esperado 70, deu %d", opcoes[0].Preco)
307     }
308 }

```

289 b, _ := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)

Feira → Salvador...

```
291-293 if _, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 2, 40); err
      != nil {
      t.Fatalf("nao consegui cadastrar: %v", err)
      }
```

...e Salvador → Ilhéus, de outro motorista.

```
295 opcoes := b.Buscar("Feira", "Ilheus", "2026-09-14")
```

Nenhuma carona sozinha faz Feira → Ilhéus.

```
296-298 if len(opcoes) != 1 {
      t.Fatalf("esperava 1 opcao com conexao, deu %d", len(opcoes))
      }
```

`t.Fatalf`: se não houver exatamente uma opção, `opcoes[0]` logo abaixo poderia nem existir e o teste cairia com `panic`.

```
300-302 if n := len(opcoes[0].Itens); n != 2 {
      t.Errorf("a opcao deveria ter 2 itens, tem %d", n)
      }
```

A opção tem dois itens.

```
305-307 if opcoes[0].Preco != 70 {
      t.Errorf("preco esperado 70, deu %d", opcoes[0].Preco)
      }
```

E o preço é a soma: 30 + 40.

50. dados_test.go: expiração e pagamento

pbl/dados/dados_test.go · linhas 314–341

```
314 // TestExpiracaoDevolveAssento: quem nao paga perde o assento.
315 func TestExpiracaoDevolveAssento(t *testing.T) {
316     original := TempoDeReserva
317     TempoDeReserva = time.Millisecond
318     defer func() { TempoDeReserva = original }()
319
320     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
321
322     if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
323         t.Fatalf("deveria funcionar: %v", err)
324     }
325
326     // Antes de expirar, ninguem mais consegue.
327     if _, err := b.Reservar("carla", trecho(id, 0, 1)); err == nil {
328         t.Fatal("carla nao deveria conseguir: o assento esta ocupado")
329     }
330
331     time.Sleep(5 * time.Millisecond)
332
333     if n := b.ExpirarVencidas(); n != 1 {
334         t.Fatalf("esperava 1 reserva expirada, deu %d", n)
335     }
336
337     // Agora sim.
338     if _, err := b.Reservar("carla", trecho(id, 0, 1)); err != nil {
```

```
339     t.Errorf("carla deveria conseguir depois da expiracao: %v", err)
340 }
341 }
```

317 TempoDeReserva = time.Millisecond

Prazo de 1 ms, restaurado pelo `defer` da linha seguinte.

```
322-324  if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
        t.Fatalf("deveria funcionar: %v", err)
    }
```

O bruno reserva o único lugar.

```
327-329  if _, err := b.Reservar("carla", trecho(id, 0, 1)); err == nil {
        t.Fatal("carla nao deveria conseguir: o assento esta ocupado")
    }
```

Antes de expirar, a carla não consegue.

331 time.Sleep(5 * time.Millisecond)

Espera o prazo passar.

```
333-335  if n := b.ExpirarVencidas(); n != 1 {
        t.Fatalf("esperava 1 reserva expirada, deu %d", n)
    }
```

Chama a varredura à mão, sem precisar da goroutine do servidor.

```
338-340  if _, err := b.Reservar("carla", trecho(id, 0, 1)); err != nil {
        t.Errorf("carla deveria conseguir depois da expiracao: %v", err)
    }
```

Agora a carla consegue.

pbl/dados/dados_test.go · linhas 343–369

```
343 // TestReservaPagaNaoExpira: pagou, o assento e seu para sempre.
344 func TestReservaPagaNaoExpira(t *testing.T) {
345     original := TempoDeReserva
346     TempoDeReserva = time.Millisecond
347     defer func() { TempoDeReserva = original }()
348
349     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
350
351     reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
352     if err != nil {
353         t.Fatalf("deveria funcionar: %v", err)
354     }
355
356     if err := b.Pagar("bruno", reserva); err != nil {
357         t.Fatalf("o pagamento deveria funcionar: %v", err)
358     }
359
360     time.Sleep(5 * time.Millisecond)
361
362     if n := b.ExpirarVencidas(); n != 0 {
363         t.Errorf("reserva paga nao pode expirar, mas expirou %d", n)
364     }
365 }
```

```

366     if _, err := b.Reservar("carla", trecho(id, 0, 1)); err == nil {
367         t.Error("o assento deveria continuar ocupado")
368     }
369 }

```

```

351-354     reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
        if err != nil {
            t.Fatalf("deveria funcionar: %v", err)
        }

```

Reserva com prazo de 1 ms.

```

356-358     if err := b.Pagar("bruno", reserva); err != nil {
        t.Fatalf("o pagamento deveria funcionar: %v", err)
    }

```

Paga na hora.

```

360     time.Sleep(5 * time.Millisecond)

```

Passa do prazo.

```

362-364     if n := b.ExpirarVencidas(); n != 0 {
        t.Errorf("reserva paga nao pode expirar, mas expirou %d", n)
    }

```

Nada pode expirar.

```

366-368     if _, err := b.Reservar("carla", trecho(id, 0, 1)); err == nil {
        t.Error("o assento deveria continuar ocupado")
    }

```

E o lugar continua ocupado.

pbl/dados/dados_test.go · linhas 371–392

```

371 // TestRegrasDePagamento cobre os erros possiveis ao pagar.
372 func TestRegrasDePagamento(t *testing.T) {
373     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
374
375     reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
376     if err != nil {
377         t.Fatalf("deveria funcionar: %v", err)
378     }
379
380     if err := b.Pagar("carla", reserva); err == nil {
381         t.Error("carla nao deveria pagar reserva do bruno")
382     }
383     if err := b.Pagar("bruno", 999); err == nil {
384         t.Error("reserva inexistente deveria dar erro")
385     }
386     if err := b.Pagar("bruno", reserva); err != nil {
387         t.Errorf("o dono deveria conseguir pagar: %v", err)
388     }
389     if err := b.Pagar("bruno", reserva); err == nil {
390         t.Error("pagar duas vezes deveria dar erro")
391     }
392 }

```

```

380-390  if err := b.Pagar("carla", reserva); err == nil {
        t.Error("carla nao deveria pagar reserva do bruno")
    }
    if err := b.Pagar("bruno", 999); err == nil {
        t.Error("reserva inexistente deveria dar erro")
    }
    if err := b.Pagar("bruno", reserva); err != nil {
        t.Errorf("o dono deveria conseguir pagar: %v", err)
    }
    if err := b.Pagar("bruno", reserva); err == nil {
        t.Error("pagar duas vezes deveria dar erro")
    }

```

Quatro chamadas em sequência, cada uma com o resultado esperado: outra pessoa, número inexistente, o dono (funciona) e o dono de novo (recusado). A ordem importa: o pagamento válido precisa vir antes do repetido.

51. dados_test.go: contas e cadastro

pbl/dados/dados_test.go · linhas 398–417

```

398 func TestAutenticar(t *testing.T) {
399     b := NovoBanco()
400
401     casos := []struct {
402         nome, login, senha string
403         quer                bool
404     }{
405         {"usuario e senha certos", "ana", "123", true},
406         {"senha errada", "ana", "outra", false},
407         {"usuario inexistente", "ninguem", "123", false},
408         {"senha vazia", "ana", "", false},
409     }
410
411     for _, caso := range casos {
412         if _, ok := b.Autenticar(caso.login, caso.senha); ok != caso.quer {
413             t.Errorf("%s: Autenticar(%q, %q) deu %v, queria %v",
414                 caso.nome, caso.login, caso.senha, ok, caso.quer)
415         }
416     }
417 }

```

```

401-403  casos := []struct {
        nome, login, senha string
        quer                bool
    }

```

Tabela de casos com três textos e o resultado esperado.

```

405-408  {"usuario e senha certos", "ana", "123", true},
        {"senha errada", "ana", "outra", false},
        {"usuario inexistente", "ninguem", "123", false},
        {"senha vazia", "ana", "", false},

```

Um caso certo e três errados.

```

411-414   for _, caso := range casos {
           if _, ok := b.Autenticar(caso.login, caso.senha); ok != caso.quer {
               t.Errorf("%s: Autenticar(%q, %q) deu %v, queria %v",
                   caso.nome, caso.login, caso.senha, ok, caso.quer)
           }
       }

```

if _, ok := ...; ok != caso.quer : chama, descarta a conta e compara só o `ok` com o esperado.

pbl/dados/dados_test.go · linhas 419-456

```

419 func TestCadastrarUsuario(t *testing.T) {
420     b := NovoBanco()
421
422     if err := b.CadastrarUsuario("henrick", "senha1", "motorista"); err != nil {
423         t.Fatalf("deveria funcionar: %v", err)
424     }
425
426     u, ok := b.Autenticar("henrick", "senha1")
427     if !ok {
428         t.Fatal("deveria conseguir entrar com a conta recém-criada")
429     }
430     if u.Tipo != "motorista" {
431         t.Errorf("tipo esperado motorista, deu %q", u.Tipo)
432     }
433
434     casos := []struct {
435         nome, login, senha, tipo string
436     }{
437         {"login repetido", "henrick", "outra", "passageiro"},
438         {"login que ja vem no banco", "ana", "x", "passageiro"},
439         {"login vazio", "", "senha", "passageiro"},
440         {"login so com espacos", " ", "senha", "passageiro"},
441         {"senha vazia", "novo", "", "passageiro"},
442         {"tipo invalido", "novo", "senha", "piloto"},
443         {"tipo vazio", "novo", "senha", ""},
444     }
445
446     for _, caso := range casos {
447         if err := b.CadastrarUsuario(caso.login, caso.senha, caso.tipo); err == nil {
448             t.Errorf("%s: deveria dar erro, mas passou", caso.nome)
449         }
450     }
451
452     // O cadastro repetido nao pode ter trocado a senha do dono original.
453     if _, ok := b.Autenticar("henrick", "senha1"); !ok {
454         t.Error("a tentativa repetida sobrescreveu o usuario original")
455     }
456 }

```

```

422-424   if err := b.CadastrarUsuario("henrick", "senha1", "motorista"); err != nil {
           t.Fatalf("deveria funcionar: %v", err)
       }

```

Cria uma conta.

```

426-431 u, ok := b.Autenticar("henrick", "senha1")
        if !ok {
            t.Fatal("deveria conseguir entrar com a conta recém-criada")
        }
        if u.Tipo != "motorista" {
            t.Errorf("tipo esperado motorista, deu %q", u.Tipo)
        }

```

Consegue entrar com ela, e o tipo foi gravado.

```

434-443 casos := []struct {
    nome, login, senha, tipo string
}{
    {"login repetido", "henrick", "outra", "passageiro"},
    {"login que ja vem no banco", "ana", "x", "passageiro"},
    {"login vazio", "", "senha", "passageiro"},
    {"login so com espacos", " ", "senha", "passageiro"},
    {"senha vazia", "novo", "", "passageiro"},
    {"tipo invalido", "novo", "senha", "piloto"},
    {"tipo vazio", "novo", "senha", ""},
}

```

Sete cadastros que precisam ser recusados, incluindo um login só com espaços, que o `TrimSpace` transforma em vazio.

```

446-448 for _, caso := range casos {
    if err := b.CadastrarUsuario(caso.login, caso.senha, caso.tipo); err == nil {
        t.Errorf("%s: deveria dar erro, mas passou", caso.nome)
    }
}

```

Cada um tem que devolver erro.

```

453-455 if _, ok := b.Autenticar("henrick", "senha1"); !ok {
    t.Error("a tentativa repetida sobrescreveu o usuario original")
}

```

A tentativa de recriar `henrick` com outra senha não sobrescreveu a conta original.

pbl/dados/dados_test.go · linhas 458-488

```

458 // TestCadastroConcorrente: 500 goroutines tentando criar O MESMO login.
459 // So uma pode vencer. Se a checagem "ja existe" estivesse fora do Lock,
460 // varias passariam pela verificacao antes de qualquer uma escrever.
461 func TestCadastroConcorrente(t *testing.T) {
462     const tentativas = 500
463
464     b := NovoBanco()
465
466     sucessos := make(chan int, tentativas)
467     largada := make(chan struct{})
468     var wg sync.WaitGroup
469
470     for i := 0; i < tentativas; i++ {
471         wg.Add(1)
472         go func() {
473             defer wg.Done()
474             <-largada
475             if err := b.CadastrarUsuario("disputado", "senha", "passageiro"); err == nil {
476                 sucessos <- 1
477             }
478         }()
479     }
480
481     close(largada)
482     wg.Wait()

```

```

483     close(sucessos)
484
485     if n := len(sucessos); n != 1 {
486         t.Errorf("esperava exatamente 1 cadastro bem sucedido, deu %d", n)
487     }
488 }

```

462 const tentativas = 500

500 goroutines tentando criar o mesmo login.

475-477 if err := b.CadastrarUsuario("disputado", "senha", "passageiro"); err == nil {
 sucessos <- 1
 }

Quem conseguir, avisa pelo canal.

485-487 if n := len(sucessos); n != 1 {
 t.Errorf("esperava exatamente 1 cadastro bem sucedido, deu %d", n)
 }

Só uma pode conseguir. Como visto acima, este teste passa mesmo sem o mutex: ele descreve a regra, mas não detecta a falta de proteção.

pbl/dados/dados_test.go · linhas 490-523

```

490 // TestCadastrosSimultaneos e o par do TestContagemDeAssentos, agora para o
491 // mapa de usuarios: 300 logins DIFERENTES criados ao mesmo tempo.
492 //
493 // Como cada goroutine escreve mesmo (nenhuma e recusada), todas mexem no mapa
494 // em paralelo. Sem o Lock, o proprio runtime do Go derruba o programa com
495 // "fatal error: concurrent map writes" -- mapa nao suporta escrita simultanea.
496 func TestCadastrosSimultaneos(t *testing.T) {
497     const quantos = 300
498
499     b := NovoBanco()
500     iniciais := len(b.Usuarios)
501
502     var wg sync.WaitGroup
503     largada := make(chan struct{})
504
505     for i := 0; i < quantos; i++ {
506         wg.Add(1)
507         go func(n int) {
508             defer wg.Done()
509             <-largada
510             login := fmt.Sprintf("usuario%d", n)
511             if err := b.CadastrarUsuario(login, "senha", "passageiro"); err != nil {
512                 t.Errorf("cadastro de %s falhou: %v", login, err)
513             }
514         }(i)
515     }
516
517     close(largada)
518     wg.Wait()
519
520     if n := len(b.Usuarios); n != iniciais+quantos {
521         t.Errorf("esperava %d usuarios, deu %d", iniciais+quantos, n)
522     }
523 }

```

```
500    iniciais := len(b.Usuarios)
```

Os quatro usuários de teste já existem.

```
507    go func(n int) {
```

A goroutine recebe o número como parâmetro.

```
510    login := fmt.Sprintf("usuario%d", n)
```

Um login diferente por goroutine: todas vão **escrever** no map.

```
514    }(i)
```

Passa o número da volta.

```
520-522    if n := len(b.Usuarios); n != iniciais+quantos {  
        t.Errorf("esperava %d usuarios, deu %d", iniciais+quantos, n)  
    }
```

Todos os cadastros entraram. Sem o mutex, este teste nem chega aqui: o Go derruba o programa ao detectar escrita simultânea no map.

pbl/dados/dados_test.go · linhas 525–545

```
525 func TestCadastroInvalido(t *testing.T) {  
526     b := NovoBanco()  
527  
528     casos := []struct {  
529         nome      string  
530         rota      []string  
531         data      string  
532         assentos   int  
533     }{  
534         {"rota com uma cidade so", []string{"Feira"}, "2026-09-14", 2},  
535         {"rota vazia", nil, "2026-09-14", 2},  
536         {"sem assentos", []string{"Feira", "Salvador"}, "2026-09-14", 0},  
537         {"sem data", []string{"Feira", "Salvador"}, "", 2},  
538     }  
539  
540     for _, caso := range casos {  
541         if _, err := b.CadastrarCarona("ana", caso.rota, caso.data, caso.assentos, 30); err == nil {  
542             t.Errorf("%s: deveria dar erro, mas passou", caso.nome)  
543         }  
544     }  
545 }
```

```
528-537    casos := []struct {  
        nome      string  
        rota      []string  
        data      string  
        assentos   int  
    }{  
        {"rota com uma cidade so", []string{"Feira"}, "2026-09-14", 2},  
        {"rota vazia", nil, "2026-09-14", 2},  
        {"sem assentos", []string{"Feira", "Salvador"}, "2026-09-14", 0},  
        {"sem data", []string{"Feira", "Salvador"}, "", 2},  
    }
```

Quatro cadastros de carona inválidos. `nil` no lugar da rota é um slice vazio válido, com tamanho zero.

```
540-542 for _, caso := range casos {
    if _, err := b.CadastrarCarona("ana", caso.rota, caso.data, caso.assentos, 30); err == nil {
        t.Errorf("%s: deveria dar erro, mas passou", caso.nome)
    }
}
```

Todos precisam ser recusados.

pbl/dados/dados_test.go · linhas 547–561

```
547 // TestOcupadosNasceZero: uma rota de N cidades tem N-1 contadores.
548 func TestOcupadosNasceZero(t *testing.T) {
549     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus", "Porto Seguro"}, 3)
550
551     c := b.acharCarona(id)
552     if len(c.Ocupados) != 3 {
553         t.Fatalf("rota de 4 cidades deveria ter 3 trechos, tem %d", len(c.Ocupados))
554     }
555
556     for i, n := range c.Ocupados {
557         if n != 0 {
558             t.Errorf("trecho %d deveria nascer zero, esta em %d", i, n)
559         }
560     }
561 }
```

```
551 c := b.acharCarona(id)
```

Uma carona de 4 cidades.

```
552-554 if len(c.Ocupados) != 3 {
    t.Fatalf("rota de 4 cidades deveria ter 3 trechos, tem %d", len(c.Ocupados))
}
```

Tem 3 contadores, um por trecho.

```
556-558 for i, n := range c.Ocupados {
    if n != 0 {
        t.Errorf("trecho %d deveria nascer zero, esta em %d", i, n)
    }
}
```

Todos em zero.

52. cancelamento_test.go

Um arquivo à parte para as funções novas. Ele usa as auxiliares `trecho` e `bancoComCarona` do outro arquivo, e importa `strings` para procurar nomes dentro das linhas de passageiros.

pbl/dados/cancelamento_test.go · linhas 16–37

```
16 func TestCancelarReservaDevolveAssento(t *testing.T) {
17     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 1)
18
19     reserva, err := b.Reservar("bruno", trecho(id, 0, 2))
20     if err != nil {
21         t.Fatalf("deveria funcionar: %v", err)
22     }
23
24     if err := b.CancelarReserva("bruno", reserva); err != nil {
25         t.Fatalf("o cancelamento deveria funcionar: %v", err)
26     }
27 }
```

```

26 }
27
28 for i, n := range b.acharCarona(id).Ocupados {
29     if n != 0 {
30         t.Errorf("trecho %d deveria voltar a 0, esta em %d", i, n)
31     }
32 }
33
34 if _, err := b.Reservar("carla", trecho(id, 0, 2)); err != nil {
35     t.Errorf("carla deveria conseguir o assento devolvido: %v", err)
36 }
37 }

```

```

19-22 reserva, err := b.Reservar("bruno", trecho(id, 0, 2))
    if err != nil {
        t.Fatalf("deveria funcionar: %v", err)
    }

```

O bruno reserva a rota inteira, que usa os dois trechos.

```

24-26 if err := b.CancelarReserva("bruno", reserva); err != nil {
    t.Fatalf("o cancelamento deveria funcionar: %v", err)
}

```

E cancela.

```

28-30 for i, n := range b.acharCarona(id).Ocupados {
    if n != 0 {
        t.Errorf("trecho %d deveria voltar a 0, esta em %d", i, n)
    }
}

```

Os **dois** trechos voltaram a zero.

```

34-36 if _, err := b.Reservar("carla", trecho(id, 0, 2)); err != nil {
    t.Errorf("carla deveria conseguir o assento devolvido: %v", err)
}

```

E o lugar serve para outra pessoa.

pbl/dados/cancelamento_test.go · linhas 39–79

```

39 func TestRegrasDeCancelarReserva(t *testing.T) {
40     original := TempoDeReserva
41     TempoDeReserva = time.Millisecond
42     defer func() { TempoDeReserva = original }()
43
44     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 3)
45
46     paga, _ := b.Reservar("bruno", trecho(id, 0, 1))
47     if err := b.Pagar("bruno", paga); err != nil {
48         t.Fatalf("o pagamento deveria funcionar: %v", err)
49     }
50
51     vencida, _ := b.Reservar("bruno", trecho(id, 0, 1))
52     time.Sleep(5 * time.Millisecond)
53     b.ExpirarVencidas()
54
55     if err := b.CancelarReserva("carla", paga); err == nil {
56         t.Error("carla nao pode cancelar reserva do bruno")
57     }
58     if err := b.CancelarReserva("bruno", 999); err == nil {
59         t.Error("reserva inexistente deveria dar erro")
60     }
}

```

```

61 if err := b.CancelarReserva("bruno", vencida); err == nil {
62     t.Error("reserva expirada nao pode ser cancelada")
63 }
64 if err := b.CancelarReserva("bruno", paga); err != nil {
65     t.Errorf("reserva paga pode ser cancelada: %v", err)
66 }
67 if err := b.CancelarReserva("bruno", paga); err == nil {
68     t.Error("cancelar duas vezes deveria dar erro")
69 }
70 if err := b.Pagar("bruno", paga); err == nil {
71     t.Error("reserva cancelada nao pode ser paga")
72 }
73
74 // A expirada ja tinha devolvido o assento dela. Se o cancelamento
75 // devolvesse de novo, o contador ficaria NEGATIVO.
76 if n := b.acharCarona(id).Ocupados[0]; n != 0 {
77     t.Errorf("esperava 0 assentos ocupados, deu %d", n)
78 }
79 }

```

46-49

```

paga, _ := b.Reservar("bruno", trecho(id, 0, 1))
if err := b.Pagar("bruno", paga); err != nil {
    t.Fatalf("o pagamento deveria funcionar: %v", err)
}

```

Uma reserva paga.

51-53

```

vencida, _ := b.Reservar("bruno", trecho(id, 0, 1))
time.Sleep(5 * time.Millisecond)
b.ExpirarVencidas()

```

E uma que expira de verdade, depois da espera.

55-71

```

if err := b.CancelarReserva("carla", paga); err == nil {
    t.Error("carla nao pode cancelar reserva do bruno")
}
if err := b.CancelarReserva("bruno", 999); err == nil {
    t.Error("reserva inexistente deveria dar erro")
}
if err := b.CancelarReserva("bruno", vencida); err == nil {
    t.Error("reserva expirada nao pode ser cancelada")
}
if err := b.CancelarReserva("bruno", paga); err != nil {
    t.Errorf("reserva paga pode ser cancelada: %v", err)
}
if err := b.CancelarReserva("bruno", paga); err == nil {
    t.Error("cancelar duas vezes deveria dar erro")
}
if err := b.Pagar("bruno", paga); err == nil {
    t.Error("reserva cancelada nao pode ser paga")
}

```

Seis situações em sequência: outra pessoa, número inexistente, reserva expirada (recusados), reserva paga (pode), cancelar de novo (recusado) e pagar a cancelada (recusado).

76-78

```

if n := b.acharCarona(id).Ocupados[0]; n != 0 {
    t.Errorf("esperava 0 assentos ocupados, deu %d", n)
}

```

A prova contra **devolução dupla**: a expirada já devolveu o lugar dela; se o cancelamento aceitasse a expirada, o contador ficaria negativo.

```

85 // TestCancelarCaronaCancelaConexaoInteira e o caso que exige cuidado: a
86 // reserva do bruno usa duas caronas. Quando uma delas e cancelada, a reserva
87 // inteira cai, e o assento dele na OUTRA carona tambem precisa voltar.
88 func TestCancelarCaronaCancelaConexaoInteira(t *testing.T) {
89     b, primeira := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
90
91     segunda, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 2, 40)
92     if err != nil {
93         t.Fatalf("nao consegui cadastrar: %v", err)
94     }
95
96     conexao, err := b.Reservar("bruno", []protocolo.Item{
97         {CaronaID: primeira, De: 0, Ate: 1},
98         {CaronaID: segunda, De: 0, Ate: 1},
99     })
100     if err != nil {
101         t.Fatalf("a reserva com conexao deveria funcionar: %v", err)
102     }
103
104     direta, err := b.Reservar("carla", trecho(primeira, 0, 1))
105     if err != nil {
106         t.Fatalf("a reserva direta deveria funcionar: %v", err)
107     }
108
109     afetadas, err := b.CancelarCarona("davi", segunda)
110     if err != nil {
111         t.Fatalf("o cancelamento deveria funcionar: %v", err)
112     }
113     if afetadas != 1 {
114         t.Errorf("esperava 1 reserva afetada, deu %d", afetadas)
115     }
116
117     if r := b.acharReserva(conexao); r.Estado != "cancelada" {
118         t.Errorf("a reserva com conexao deveria estar cancelada, esta %q", r.Estado)
119     }
120     if r := b.acharReserva(direta); r.Estado != "pendente" {
121         t.Errorf("a reserva da carla nao usa a carona cancelada, deveria seguir pendente, esta %q",
122 r.Estado)
123     }
124
125     // Na primeira carona so pode sobrar o assento da carla.
126     if n := b.acharCarona(primeira).Ocupados[0]; n != 1 {
127         t.Errorf("a primeira carona deveria ter 1 ocupado (so a carla), tem %d", n)
128     }
129
130     // Carona cancelada some da busca e recusa reserva nova.
131     if n := len(b.Buscar("Salvador", "Ilheus", "2026-09-14")); n != 0 {
132         t.Errorf("carona cancelada nao deveria aparecer na busca, apareceu %d vez(es)", n)
133     }
134     if _, err := b.Reservar("carla", trecho(segunda, 0, 1)); err == nil {
135         t.Error("nao deveria ser possivel reservar carona cancelada")
136     }
137 }

```

96-98 conexao, err := b.Reservar("bruno", []protocolo.Item{
 {CaronaID: primeira, De: 0, Ate: 1},
 {CaronaID: segunda, De: 0, Ate: 1},

O bruno faz uma reserva com conexão, usando as duas caronas.

```
104 direta, err := b.Reservar("carla", trecho(primeira, 0, 1))
```

A carla faz uma reserva só na primeira. `:=` funciona porque `direta` é nova.

```
109 afetadas, err := b.CancelarCarona("davi", segunda)
```

O davi cancela a **segunda** carona.

```
117-119 if r := b.acharReserva(conexao); r.Estado != "cancelada" {
        t.Errorf("a reserva com conexao deveria estar cancelada, esta %q", r.Estado)
    }
```

A reserva com conexão caiu inteira.

```
120-122 if r := b.acharReserva(direta); r.Estado != "pendente" {
        t.Errorf("a reserva da carla nao usa a carona cancelada, deveria seguir pendente, esta %q",
        r.Estado)
    }
```

A reserva da carla, que não usa a carona cancelada, continua pendente.

```
125-127 if n := b.acharCarona(primeira).Ocupados[0]; n != 1 {
        t.Errorf("a primeira carona deveria ter 1 ocupado (so a carla), tem %d", n)
    }
```

A cascata: na **primeira** carona, só sobrou o lugar da carla. O do bruno voltou.

```
130-134 if n := len(b.Buscar("Salvador", "Ilheus", "2026-09-14")); n != 0 {
        t.Errorf("carona cancelada nao deveria aparecer na busca, apareceu %d vez(es)", n)
    }
    if _, err := b.Reservar("carla", trecho(segunda, 0, 1)); err == nil {
        t.Error("nao deveria ser possivel reservar carona cancelada")
    }
```

A carona cancelada some da busca e recusa reservas novas.

pbl/dados/cancelamento_test.go · linhas 138–153

```
138 func TestRegrasDeCancelarCarona(t *testing.T) {
139     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
140
141     if _, err := b.CancelarCarona("davi", id); err == nil {
142         t.Error("davi nao e dono da carona e nao pode cancelar")
143     }
144     if _, err := b.CancelarCarona("ana", 999); err == nil {
145         t.Error("carona inexistente deveria dar erro")
146     }
147     if _, err := b.CancelarCarona("ana", id); err != nil {
148         t.Errorf("a dona deveria conseguir cancelar: %v", err)
149     }
150     if _, err := b.CancelarCarona("ana", id); err == nil {
151         t.Error("cancelar duas vezes deveria dar erro")
152     }
153 }
```

```

141-151  if _, err := b.CancelarCarona("davi", id); err == nil {
        t.Error("davi nao e dono da carona e nao pode cancelar")
    }
    if _, err := b.CancelarCarona("ana", 999); err == nil {
        t.Error("carona inexistente deveria dar erro")
    }
    if _, err := b.CancelarCarona("ana", id); err != nil {
        t.Errorf("a dona deveria conseguir cancelar: %v", err)
    }
    if _, err := b.CancelarCarona("ana", id); err == nil {
        t.Error("cancelar duas vezes deveria dar erro")
    }

```

Outro motorista, número inexistente, a dona (pode) e a dona de novo (recusado).

pbl/dados/cancelamento_test.go · linhas 159–194

```

159 func TestPassageirosPorTrecho(t *testing.T) {
160     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 2)
161
162     if _, err := b.Reservar("bruno", trecho(id, 0, 2)); err != nil { // os dois trechos
163         t.Fatalf("deveria funcionar: %v", err)
164     }
165
166     carla, err := b.Reservar("carla", trecho(id, 1, 2)) // so o segundo
167     if err != nil {
168         t.Fatalf("deveria funcionar: %v", err)
169     }
170     b.Pagar("carla", carla)
171
172     linhas, err := b.PassageirosDaCarona("ana", id)
173     if err != nil {
174         t.Fatalf("a dona deveria ver os passageiros: %v", err)
175     }
176
177     // linhas[0] e o cabecalho; depois vem uma linha por trecho.
178     if len(linhas) != 3 {
179         t.Fatalf("esperava cabecalho + 2 trechos, deu %d linhas: %v", len(linhas), linhas)
180     }
181
182     primeiro, segundo := linhas[1], linhas[2]
183
184     if !strings.Contains(primeiro, "bruno") || strings.Contains(primeiro, "carla") {
185         t.Errorf("Feira->Salvador deveria ter so o bruno: %q", primeiro)
186     }
187     if !strings.Contains(segundo, "bruno") || !strings.Contains(segundo, "carla (paga)") {
188         t.Errorf("Salvador->Ilheus deveria ter bruno e carla (paga): %q", segundo)
189     }
190
191     if _, err := b.PassageirosDaCarona("davi", id); err == nil {
192         t.Error("so a dona da carona pode ver os passageiros")
193     }
194 }

```

```

162-164  if _, err := b.Reservar("bruno", trecho(id, 0, 2)); err != nil { // os dois trechos
        t.Fatalf("deveria funcionar: %v", err)
    }

```

O bruno ocupa os dois trechos.

```
166-169 carla, err := b.Reservar("carla", trecho(id, 1, 2)) // so o segundo
      if err != nil {
          t.Fatalf("deveria funcionar: %v", err)
      }
```

A carla ocupa só o segundo.

```
170 b.Pagar("carla", carla)
```

E paga, para o estado aparecer diferente na lista.

```
172-175 linhas, err := b.PassageirosDaCarona("ana", id)
      if err != nil {
          t.Fatalf("a dona deveria ver os passageiros: %v", err)
      }
```

A ana, dona, pede a lista.

```
178-180 if len(linhas) != 3 {
      t.Fatalf("esperava cabecalho + 2 trechos, deu %d linhas: %v", len(linhas), linhas)
  }
```

Cabeçalho mais uma linha por trecho.

```
182 primeiro, segundo := linhas[1], linhas[2]
```

Atribuição de dois valores numa linha, como no Python.

```
184-186 if !strings.Contains(primeiro, "bruno") || strings.Contains(primeiro, "carla") {
      t.Errorf("Feira->Salvador deveria ter so o bruno: %q", primeiro)
  }
```

No primeiro trecho, só o bruno.

```
187-189 if !strings.Contains(segundo, "bruno") || !strings.Contains(segundo, "carla (paga)") {
      t.Errorf("Salvador->Ilheus deveria ter bruno e carla (paga): %q", segundo)
  }
```

No segundo, o bruno e a carla com o estado `(paga)`.

```
191-193 if _, err := b.PassageirosDaCarona("davi", id); err == nil {
      t.Error("so a dona da carona pode ver os passageiros")
  }
```

Outro motorista não pode ver.

pbl/dados/cancelamento_test.go · linhas 200–235

```
200 // TestReservaECancelamentoConcorrentes: 1000 goroutines reservam e cancelam ao
201 // mesmo tempo. Todas escrevem DUAS vezes (ocupar e liberar), entao se o mutex
202 // faltar em qualquer das duas funcoes, incrementos e decrementos se perdem e
203 // o contador final nao volta a zero.
204 func TestReservaECancelamentoConcorrentes(t *testing.T) {
205     const pares = 1000
206
207     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, pares)
208
209     var wg sync.WaitGroup
210     largada := make(chan struct{})
211
```

```

212     for i := 0; i < pares; i++ {
213         wg.Add(1)
214         go func() {
215             defer wg.Done()
216             <-largada
217
218             reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
219             if err != nil {
220                 t.Errorf("a reserva deveria funcionar: %v", err)
221                 return
222             }
223             if err := b.CancelarReserva("bruno", reserva); err != nil {
224                 t.Errorf("o cancelamento deveria funcionar: %v", err)
225             }
226         }()
227     }
228
229     close(largada)
230     wg.Wait()
231
232     if n := b.acharCarona(id).Ocupados[0]; n != 0 {
233         t.Errorf("tudo foi cancelado, mas sobraram %d assentos ocupados", n)
234     }
235 }

```

205 const pares = 1000

1000 goroutines, e a carona tem 1000 lugares.

218-222 reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
 if err != nil {
 t.Errorf("a reserva deveria funcionar: %v", err)
 return
 }

Cada uma reserva. `return` dentro da goroutine encerra só aquela goroutine, não o teste.

223-225 if err := b.CancelarReserva("bruno", reserva); err != nil {
 t.Errorf("o cancelamento deveria funcionar: %v", err)
 }

E cancela a própria reserva.

232-234 if n := b.acharCarona(id).Ocupados[0]; n != 0 {
 t.Errorf("tudo foi cancelado, mas sobraram %d assentos ocupados", n)
 }

Tudo foi cancelado, então o contador tem que voltar a zero. Sem o mutex no `CancelarReserva`: sobraram 11, 6 e 13.

pbl/dados/cancelamento_test.go · linhas 237-278

```

237 // TestCancelarCaronaDuranteReservas: o motorista cancela enquanto 500
238 // passageiros tentam reservar. Nao importa quem chega primeiro; o que nao pode
239 // acontecer e sobrar reserva ativa numa carona cancelada.
240 func TestCancelarCaronaDuranteReservas(t *testing.T) {
241     const pedidos = 500
242
243     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, pedidos)
244
245     var wg sync.WaitGroup
246     largada := make(chan struct{})

```

```

247
248     for i := 0; i < pedidos; i++ {
249         wg.Add(1)
250         go func() {
251             defer wg.Done()
252             <-largada
253             b.Reservar("bruno", trecho(id, 0, 1)) // pode falhar se o cancelamento vier antes
254         }()
255     }
256
257     wg.Add(1)
258     go func() {
259         defer wg.Done()
260         <-largada
261         if _, err := b.CancelarCarona("ana", id); err != nil {
262             t.Errorf("o cancelamento deveria funcionar: %v", err)
263         }
264     }()
265
266     close(largada)
267     wg.Wait()
268
269     for _, r := range b.Reservas {
270         if r.ativa() {
271             t.Errorf("reserva %d continua %s numa carona cancelada", r.ID, r.Estado)
272         }
273     }
274
275     if n := b.acharCarona(id).Ocupados[0]; n != 0 {
276         t.Errorf("carona cancelada deveria terminar sem assentos ocupados, tem %d", n)
277     }
278 }

```

241 const pedidos = 500

500 reservas e um cancelamento ao mesmo tempo.

253 b.Reservar("bruno", trecho(id, 0, 1)) // pode falhar se o cancelamento vier antes

O erro é ignorado: dependendo da ordem, a carona já pode estar cancelada.

261-263 if _, err := b.CancelarCarona("ana", id); err != nil {
 t.Errorf("o cancelamento deveria funcionar: %v", err)
 }

Uma goroutine a mais, que cancela, esperando a mesma largada.

269-271 for _, r := range b.Reservas {
 if r.ativa() {
 t.Errorf("reserva %d continua %s numa carona cancelada", r.ID, r.Estado)

A regra que não pode ser quebrada, qualquer que seja a ordem: nenhuma reserva ativa numa carona cancelada.

275-277 if n := b.acharCarona(id).Ocupados[0]; n != 0 {
 t.Errorf("carona cancelada deveria terminar sem assentos ocupados, tem %d", n)
 }

E nenhum lugar ocupado. Este teste também não foi sabotado para confirmar que detecta a falta do mutex.

53. servidor/main_test.go

Os dois testes do servidor usam conexões TCP reais contra a função `atender` de verdade, numa porta escolhida pelo sistema.

pbl/servidor/main_test.go · linhas 14–38

```
14 // servidorDeTeste sobe um ouvinte numa porta livre e atende UMA conexao.
15 // O canal devolvido e fechado quando o atender() daquela conexao termina.
16 func servidorDeTeste(t *testing.T) (endereco string, terminou chan struct{}) {
17     t.Helper()
18
19     // Porta 0: o sistema operacional escolhe uma porta livre sozinho.
20     ouvinte, err := net.Listen("tcp", "127.0.0.1:0")
21     if err != nil {
22         t.Fatalf("nao consegui abrir porta: %v", err)
23     }
24     t.Cleanup(func() { ouvinte.Close() })
25
26     terminou = make(chan struct{})
27
28     go func() {
29         conexao, err := ouvinte.Accept()
30         if err != nil {
31             return
32         }
33         atender(conexao, dados.NovoBanco())
34         close(terminou)
35     }()
36
37     return ouvinte.Addr().String(), terminou
38 }
```

16 `func servidorDeTeste(t *testing.T) (endereco string, terminou chan struct{}) {`

Retornos nomeados: `endereco` e `terminou` já são variáveis declaradas dentro da função, com valor zero, e o `return` no fim as devolve.

20 `ouvinte, err := net.Listen("tcp", "127.0.0.1:0")`

Porta `0` pede ao sistema operacional uma porta livre qualquer. Assim o teste nunca briga com um servidor de verdade rodando na 8080.

24 `t.Cleanup(func() { ouvinte.Close() })`

Fecha a porta quando o teste terminar, como uma fixture com `yield` no pytest.

26 `terminou = make(chan struct{})`

`=` e não `:=`: `terminou` já existe, é o retorno nomeado.

```
28-34 go func() {
    conexao, err := ouvinte.Accept()
    if err != nil {
        return
    }
    atender(conexao, dados.NovoBanco())
    close(terminou)
```

Aceita **uma** conexão, roda o `atender` real com um banco novo e, quando ele termina, fecha o canal. É assim que o teste descobre se a goroutine do cliente acabou.

```
37 return ouvinte.Addr().String(), terminou
```

`Addr()` devolve o endereço com a porta que o sistema escolheu.

pbl/servidor/main_test.go · linhas 40–66

```
40 // TestQuedaAbruptaEncerraAtendimento garante que o servidor aguenta um
41 // cliente que cai sem se despedir: processo morto, cabo puxado, wifi caiu.
42 //
43 // Nesses casos a conexao termina com RST, e a leitura devolve "connection
44 // reset" -- que NAO e io.EOF. Uma versao anterior do atender tratava qualquer
45 // erro diferente de EOF como JSON malformado e voltava a ler: a goroutine
46 // girava para sempre, consumindo 100% de um nucleo.
47 func TestQuedaAbruptaEncerraAtendimento(t *testing.T) {
48     endereco, terminou := servidorDeTeste(t)
49
50     cliente, err := net.Dial("tcp", endereco)
51     if err != nil {
52         t.Fatalf("nao consegui conectar: %v", err)
53     }
54
55     // SetLinger(0) faz o Close mandar RST em vez do encerramento educado.
56     cliente.(*net.TCPConn).SetLinger(0)
57     time.Sleep(100 * time.Millisecond) // garante que o atender ja esta lendo
58     cliente.Close()
59
60     select {
61     case <-terminou:
62         // certo: a goroutine percebeu a queda e encerrou
63     case <-time.After(2 * time.Second):
64         t.Fatalf("o atendimento nao terminou depois da queda do cliente: goroutine presa em laço")
65     }
66 }
```

```
48 endereco, terminou := servidorDeTeste(t)
```

Sobe o servidor de teste.

```
50-53 cliente, err := net.Dial("tcp", endereco)
    if err != nil {
        t.Fatalf("nao consegui conectar: %v", err)
    }
```

Conecta.

```
56 cliente.(*net.TCPConn).SetLinger(0)
```

Asserção de tipo. `net.Dial` devolve a interface `net.Conn`; `.(*net.TCPConn)` diz "sei que, por baixo, isto é uma conexão TCP", e dá acesso a métodos que só ela tem. Se o tipo real fosse outro, o programa cairia com `panic`.

`SetLinger(0)` faz o `Close` encerrar com RST, como um processo que morre ou uma rede que cai.

```
57 time.Sleep(100 * time.Millisecond) // garante que o atender ja esta lendo
```

Dá tempo de o `atender` começar a ler.

```
58 cliente.Close()
```

A queda.

```
60-64 select {
case <-terminou:
    // certo: a goroutine percebeu a queda e encerrou
case <-time.After(2 * time.Second):
    t.Fatalf("o atendimento nao terminou depois da queda do cliente: goroutine presa em laço")
```

`select` espera o primeiro de dois canais: `terminou` fechado (certo) ou `time.After` depois de 2 segundos (errado). Antes da correção, o teste falhava aqui, em 2,11 s; depois, passa em cerca de 0,1 s.

pbl/servidor/main_test.go · linhas 68–101

```
68 // TestJSONInvalidoNaoDerrubaConexao e o outro lado da mesma moeda: uma linha
69 // malformada NAO pode encerrar a conexao. O servidor responde o erro e
70 // continua atendendo aquele cliente.
71 func TestJSONInvalidoNaoDerrubaConexao(t *testing.T) {
72     endereco, _ := servidorDeTeste(t)
73
74     cliente, err := net.Dial("tcp", endereco)
75     if err != nil {
76         t.Fatalf("nao consegui conectar: %v", err)
77     }
78     defer cliente.Close()
79
80     leitor := bufio.NewReader(cliente)
81
82     fmt.Fprint(cliente, "isso nao e json\n")
83
84     r, err := protocolo.LerResposta(leitor)
85     if err != nil {
86         t.Fatalf("o servidor deveria responder o erro, mas: %v", err)
87     }
88     if r.OK {
89         t.Error("JSON invalido deveria voltar com ok=false")
90     }
91
92     fmt.Fprint(cliente, `{"acao":"ping"}'+"\n")
93
94     r, err = protocolo.LerResposta(leitor)
95     if err != nil {
96         t.Fatalf("a conexao deveria continuar viva depois do JSON invalido: %v", err)
97     }
98     if !r.OK || r.Mensagem != "pong" {
99         t.Errorf("esperava pong, veio %v", r)
100     }
101 }
```

```
72 endereco, _ := servidorDeTeste(t)
```

Aqui o canal de término não interessa.

```
78 defer cliente.Close()
```

Fecha no fim.

```
80 leitor := bufio.NewReader(cliente)
```

Para ler as respostas.

```
82 fmt.Fprint(cliente, "isso nao e json\n")
```

Escreve texto cru na conexão, sem passar pelo protocolo.

```
84-89 r, err := protocolo.LerResposta(leitor)
      if err != nil {
          t.Fatalf("o servidor deveria responder o erro, mas: %v", err)
      }
      if r.OK {
          t.Error("JSON invalido deveria voltar com ok=false")
      }

```

O servidor responde, e a resposta é de erro.

```
92 fmt.Fprint(cliente, `{"acao":"ping"}\n`)
```

Crases evitam escapar as aspas do JSON; `+` junta a quebra de linha.

```
94-99 r, err = protocolo.LerResposta(leitor)
      if err != nil {
          t.Fatalf("a conexao deveria continuar viva depois do JSON invalido: %v", err)
      }
      if !r.OK || r.Mensagem != "pong" {
          t.Errorf("esperava pong, veio %+v", r)
      }

```

A mesma conexão continua funcionando. `%+v` imprime a struct com o nome de cada campo, útil para depurar.

Parte VI — Rodar, demonstrar e empacotar

Como subir o sistema, o roteiro que mostra cada regra funcionando na apresentação, e os dois arquivos de Docker explicados linha a linha.

54. Rodar na máquina

```
cd pbl
go run ./servidor      # terminal 1: deixe visível, cada ação aparece aqui
go run ./cliente       # terminais 2, 3...: um por pessoa
```

Contas prontas, todas com senha `123`: **ana** e **davi** (motoristas), **bruno** e **carla** (passageiros). A opção **2) criar conta** cria outras.

Prazos curtos para demonstrar

Com o prazo real de 10 minutos, ninguém vê a expiração ao vivo. No PowerShell:

```
$env:VAIJUNTO_RESERVA="30s"; $env:VAIJUNTO_VARREDURA="5s"; go run ./servidor
```

No Git Bash ou Linux:

```
VAIJUNTO_RESERVA=30s VAIJUNTO_VARREDURA=5s go run ./servidor
```

DOIS CUIDADOS

No PowerShell, `$env:` vale para aquele terminal até ele ser fechado. E os dados sobrevivem ao servidor ser desligado, no `vaijunto.json`: para começar a demonstração do zero, com as caronas numeradas a partir de 1, apague o arquivo antes de subir o servidor.

55. Roteiro de demonstração

Com o servidor em prazos curtos e três clientes abertos:

#	Quem	Faz	Mostra
1	ana	Cadastra <code>Feira, Salvador, Ilheus</code> , data <code>2026-09-20</code> , 2 assentos, R\$30	<code>carona 1 cadastrada</code>
2	bruno	Busca Feira → Ilhéus e reserva	Reserva nos dois trechos
3	carla	Busca Salvador → Ilhéus, reserva e paga	Pagamento
4	ana	Opção 3, carona 1	Passageiros por trecho: <code>[1/2]</code> e <code>[2/2]</code>
5	bruno	Não paga; espera ~35 s	Servidor: <code>expirei 1 reserva(s)</code> ; o lugar volta
6	davi	Cadastra <code>Ilheus, Porto Seguro</code> na mesma data	—
7	bruno	Busca Feira → Porto Seguro	Opção com conexão: dois itens, R\$110
8	davi	Opção 4: cancela a carona dele	Cascata: reservas desfeitas, lugar volta também na carona da ana

9	—	Fecha um cliente pelo X da janela	Servidor: <code>caiu</code> ou <code>desconectou</code> ; os outros seguem
10	—	<code>go run ./carga -clientes 1000 -assentos 50</code>	Concorrência real: <code>PASSOU</code>

O teste de carga

Com o servidor rodando, em outro terminal:

```
go run ./carga
go run ./carga -clientes 1000 -assentos 50
go run ./carga -servidor 192.168.0.10:8080    # servidor em outra máquina
```

O final esperado:

```
verificacao:
[ok]    nenhum lugar vendido duas vezes
[ok]    todos os lugares disponiveis foram vendidos
[ok]    cada reserva aceita tem numero unico
[ok]    nenhum itinerario pela metade

PASSOU
```

Cada rodada cria usuários com nomes únicos e cancela as próprias caronas no fim. Os usuários criados ficam no arquivo de dados.

56. Dockerfile

O Dockerfile gera uma imagem com o servidor e o cliente. Ele usa **dois estágios**: o primeiro tem o compilador Go inteiro e compila; o segundo recomeça de uma imagem mínima e recebe só os executáveis prontos. Como Go gera executáveis que não dependem de nada instalado, a imagem final fica com poucos megabytes.

NUNCA EXECUTADO

A máquina de desenvolvimento não tem Docker. Os dois arquivos abaixo foram escritos com cuidado, mas **nunca rodaram**. Ver Parte VII.

pbl/Dockerfile · linhas 1–38

```
1  # Build em dois estagios.
2  #
3  # O primeiro estagio tem o compilador Go inteiro (~300 MB) e produz os dois
4  # binarios. O segundo copia so os binarios prontos para uma imagem minima.
5  # E por isso que a imagem final fica em poucos megabytes: Go gera binario
6  # estatico, entao nao precisa de runtime instalado.
7
8  FROM golang:1.27-alpine AS build
9
10 WORKDIR /src
11
12 # go.mod primeiro, sozinho: assim o Docker so refaz esta camada quando as
13 # dependencias mudam, e nao a cada alteracao de codigo.
14 COPY go.mod ./
```

```

15 RUN go mod download
16
17 COPY . .
18
19 # CGO_ENABLED=0 forca binario 100% estatico, sem depender de bibliotecas C
20 # do sistema. E o que permite rodar na imagem alpine crua.
21 RUN CGO_ENABLED=0 go build -o /bin/servidor ./servidor && \
22     CGO_ENABLED=0 go build -o /bin/cliente ./cliente
23
24 # -----
25
26 FROM alpine:3.20
27
28 COPY --from=build /bin/servidor /bin/servidor
29 COPY --from=build /bin/cliente /bin/cliente
30
31 # Onde o JSON fica gravado. O docker-compose monta um volume aqui para os
32 # dados sobreviverem ao container ser destruido.
33 RUN mkdir -p /dados
34 ENV VAIJUNTO_DADOS=/dados/vaijunto.json
35
36 EXPOSE 8080
37
38 CMD ["/bin/servidor"]

```

8 FROM golang:1.27-alpine AS build

Primeiro estágio. Parte da imagem oficial do Go 1.27 sobre Alpine Linux, uma distribuição mínima. `AS build` dá nome ao estágio, para o segundo poder copiar coisas dele. A versão precisa bater com o `go 1.27` do `go.mod`; a existência desta tag não foi conferida.

10 WORKDIR /src

Pasta de trabalho dentro da imagem. É criada se não existir, e os comandos seguintes rodam nela.

14-15 COPY go.mod ./ RUN go mod download

Copia só o `go.mod` e baixa as dependências. Cada instrução do Dockerfile vira uma camada guardada em cache; enquanto o `go.mod` não mudar, esta etapa é reaproveitada nos próximos builds. O projeto não tem dependências, então hoje não faz diferença, mas é o padrão.

17 COPY . .

Copia o resto do projeto, a partir da pasta onde o build é chamado (`pb1`), para `/src`.

21-22 RUN CGO_ENABLED=0 go build -o /bin/servidor ./servidor && \ CGO_ENABLED=0 go build -o /bin/cliente ./cliente

Compila os dois executáveis. `CGO_ENABLED=0` desliga a ligação com bibliotecas C do sistema e gera um binário totalmente estático, que roda numa imagem sem nada instalado. `&&` encadeia os comandos, e a barra no fim da linha continua o comando na linha seguinte. O programa `carga` não é compilado aqui (Parte VII).

26 FROM alpine:3.20

Segundo estágio: recomeça de uma imagem Alpine limpa. Nada do primeiro estágio vem junto, a não ser o que for copiado explicitamente.

```
28-29 COPY --from=build /bin/servidor /bin/servidor
COPY --from=build /bin/cliente /bin/cliente
```

Copia só os dois executáveis do estágio `build`. O compilador, com centenas de megabytes, fica para trás.

```
33-34 RUN mkdir -p /dados
ENV VAIJUNTO_DADOS=/dados/vaijunto.json
```

Cria a pasta dos dados e define `VAIJUNTO_DADOS`, que o servidor lê em `arquivoDeDados()`. `ENV` vale para todo container criado desta imagem.

```
36 EXPOSE 8080
```

Documenta que o container usa a porta 8080. Sozinho não abre nada: quem publica a porta é o `-p` do `docker run` ou o `ports` do `compose`.

```
38 CMD ["/bin/servidor"]
```

Comando padrão ao criar um container: sobe o servidor. O formato de lista executa o programa direto, sem passar por um shell. O `compose` troca este comando para rodar o cliente.

57. docker-compose.yml

O `compose` descreve os dois serviços e sobe tudo com um comando. O comentário do topo do arquivo lista os comandos de uso.

pbl/docker-compose.yml · linhas 1-36

```
1 # Sobe o servidor e deixa o cliente pronto para conectar.
2 #
3 # docker compose up -d --build    sobe o servidor
4 # docker compose run --rm cliente abre um cliente (pode repetir em varios
5 #                                terminais, para demonstrar concorrência)
6 # docker compose down            derruba tudo
7
8 services:
9   servidor:
10     build: .
11     ports:
12       - "8080:8080"
13     environment:
14       # Prazos curtos para a demonstração. Em uso normal, apague as duas
15       # linhas abaixo e o prazo volta a ser os 10 minutos do enunciado.
16       VAIJUNTO_RESERVA: 30s
17       VAIJUNTO_VARREDURA: 5s
18     volumes:
19       - dados:/dados
20
21   cliente:
22     build: .
23     command: /bin/cliente
24     environment:
25       # "servidor" e o nome do serviço acima: o Docker resolve para o IP do
26       # container. E por isso que o servidor precisa ouvir em ":8080" e não
27       # em "localhost:8080" -- senão ele recusaria conexão de fora.
28       VAIJUNTO_SERVIDOR: servidor:8080
29     depends_on:
30       - servidor
```

```
31 # Sem estes dois o menu nao consegue ler o teclado.
32 stdin_open: true
33 tty: true
34
35 volumes:
36 dados:
```

8 services:

A lista dos serviços, cada um um container.

9 servidor:

O nome do serviço. Também vira o nome dele na rede interna do compose: os outros containers o encontram pelo nome `servidor`.

10 build: .

Constrói a imagem a partir do Dockerfile desta pasta.

11-12 ports:
- "8080:8080"

Publica a porta no formato `máquina:container`. Conexões na porta 8080 da máquina chegam ao servidor, e é isso que permite um cliente de outra máquina conectar. As aspas evitam que o YAML leia o texto como outro tipo de valor.

13-17 environment:
Prazos curtos para a demonstracao. Em uso normal, apague as duas
linhas abaixo e o prazo volta a ser os 10 minutos do enunciado.
VAIJUNTO_RESERVA: 30s
VAIJUNTO_VARREDURA: 5s

Variáveis de ambiente do container. Os prazos curtos são para demonstração; o comentário lembra de apagá-los para o uso normal.

18-19 volumes:
- dados:/dados

Monta o volume `dados` em `/dados`, onde o JSON é gravado. O volume vive fora do container: derrubar e subir de novo não perde os dados.

21 cliente:

O serviço do cliente.

22-23 build: .
command: /bin/cliente

A mesma imagem; `command` troca o `CMD` do Dockerfile para rodar o cliente em vez do servidor.

24-28 environment:
"servidor" e o nome do servico acima: o Docker resolve para o IP do
container. E por isso que o servidor precisa ouvir em ":8080" e nao
em "localhost:8080" -- senao ele recusaria conexao de fora.
VAIJUNTO_SERVIDOR: servidor:8080

O cliente conecta no host `servidor`, que a rede do compose resolve para o container do servidor. É por isso que o servidor escuta em `:8080` e não em `localhost:8080`.

29-30

```
depends_on:
  - servidor
```

Sobe o servidor antes do cliente. Só garante a ordem de início, não que o servidor já esteja aceitando conexões.

32-33

```
stdin_open: true
tty: true
```

Mantém a entrada aberta e cria um terminal; sem isso, o menu não recebe o que é digitado. Correspondem ao `-i` e ao `-t` do `docker run`.

35-36

```
volumes:
  dados:
```

Declara o volume nomeado usado pelo servidor.

Os comandos

```
docker compose up -d --build      # constrói e sobe o servidor em segundo plano
docker compose run --rm cliente   # abre um cliente; repita em outros terminais
docker compose logs -f servidor   # acompanha o log do servidor
docker compose down               # derruba tudo (o volume de dados continua)
```

Em duas máquinas

O compose junta servidor e cliente numa mesma máquina. Para o cliente rodar em **outra** máquina do laboratório, o caminho é usar a imagem direto, apontando para o IP da máquina do servidor:

```
# máquina do servidor
docker compose up -d --build

# máquina do cliente (dentro da pasta pbl)
docker build -t vaijunto .
docker run -it --rm -e VAIJUNTO_SERVIDOR=IP_DO_SERVIDOR:8080 vaijunto /bin/cliente
```

O firewall da máquina do servidor precisa permitir conexões na porta 8080. Nada disto foi testado ainda.

Parte VII — O que ainda falta

As pendências para a entrega, e as limitações conhecidas do código, com o efeito de cada uma e como corrigir. Situação em 14/09/2026.

58. Prazos

Data	Marco
quinta, 17/09	Entrega
domingo, 20/09	Relatório no formato SBC, até 8 páginas
terça, 22/09	Apresentação

59. Pendências para a entrega

#	Pendência	Situação	Como resolver
1	Testar o Docker	Os dois arquivos existem e nunca rodaram. Pontos de risco: a tag <code>golang:1.27-alpine</code> , o <code>go mod download</code> sem dependências e a leitura do teclado no <code>compose run</code> .	Instalar o Docker Desktop, rodar os comandos da Parte VI e corrigir o que falhar.
2	Rodar em duas máquinas	Nunca testado.	Seguir "Em duas máquinas", na Parte VI, no laboratório.
3	Detector de corrida	<code>go test -race</code> nunca rodou: o Go instalado é de 32 bits.	Instalar o Go para Windows de 64 bits, ou rodar num container: <code>docker run --rm -v "\${PWD}:/src" -w /src golang:1.27 go test -race ./...</code> dentro de <code>pbl</code> . A imagem sem <code>-alpine</code> é necessária, porque o detector precisa de um compilador C.
4	Especificação do protocolo	Não existe um <code>protocolo.md</code> . O conteúdo está pronto na Parte II deste guia.	Copiar a seção do protocolo, com a tabela de ações e as conversas reais.
5	Relatório SBC	Não começado. Previsto para domingo.	A tabela "Mapa das regras" (Parte I), as seções de concorrência e resiliência (Parte II) e os números da carga servem de base.
6	Plano antigo na master	O <code>Documentação/Plano/README.md</code> descreve decisões que mudaram: prefixo de tamanho (hoje é <code>\n</code>), mutex por carona (hoje é um mutex global) e persistência só em memória (hoje é JSON em arquivo).	Atualizar o plano, ou explicar a mudança e o motivo no relatório.
7	Carga dentro do Docker	A imagem só tem servidor e cliente.	Acrescentar <code>go build -o /bin/carga ./carga</code> e o <code>COPY</code> correspondente no Dockerfile.
8	Commit das mudanças de 14/09	Cancelamentos, correção da queda, programa de carga, testes novos e README ficaram prontos para commit na <code>feat/revisao</code> .	Conferir se já foram commitados e abrir o PR.

60. Limitações conhecidas do código

Nada abaixo impede o sistema de funcionar ou quebra um requisito central. São pontos que um avaliador atento pode perguntar, em ordem aproximada de importância.

#	Limitação	Efeito	Como corrigir
1	Pagamento só olha o estado	Entre o vencimento do prazo e a próxima varredura (até 30 s), uma reserva vencida ainda pode ser paga.	No <code>Pagar</code> , recusar quando <code>time.Now().After(r.Expira)</code> , devolvendo os assentos na hora.
2	Buscar e reservar não conferem o tipo	Um motorista pode reservar mandando o pedido direto pelo protocolo.	No servidor, recusar <code>buscar</code> , <code>reservar</code> , <code>pagar</code> e <code>cancelar_reserva</code> quando o tipo não for passageiro.
3	Arquivo corrompido é sobrescrito	Se o JSON estiver ilegível ao subir, o servidor começa do zero e, na primeira alteração, apaga os dados antigos.	Gravar num arquivo temporário e renomear (a troca é atômica no sistema de arquivos); ao ler um arquivo ruim, guardá-lo como <code>.bak</code> antes de seguir.
4	Falha ao gravar só é impressa	A operação vale na memória, mas não no disco; um reinício perde a mudança.	Fazer <code>persistir</code> devolver erro e desfazer a operação, ou encerrar o servidor.
5	Senhas em texto puro	Quem lê o arquivo lê as senhas.	Guardar um hash com sal. <code>bcrypt</code> é o ideal, mas exige uma biblioteca externa (golang.org/x/crypto).
6	Data não é validada	Qualquer texto vira data; datas escritas de jeitos diferentes nunca se encontram na busca.	Conferir com <code>time.Parse("2006-01-02", data)</code> no cadastro e na busca.
7	Preço sem validação	Preço zero ou negativo é aceito.	Recusar <code>preco < 0</code> no <code>CadastrarCarona</code> .
8	Cidade repetida na rota	<code>indice</code> devolve a primeira ocorrência; uma rota que passa duas vezes pela mesma cidade não é buscada corretamente.	Recusar cidades repetidas no cadastro.
9	Busca limitada	No máximo uma conexão; e, sem horários, não confere se a segunda carona sai depois de a primeira chegar.	Acrescentar horários às caronas e compará-los na conexão.
10	Latência sob carga	Com 1000 clientes, p95 de 474 ms: tudo passa por um único mutex, e cada reserva aceita regrava o arquivo inteiro dentro dele.	<code>sync.RWMutex</code> para as leituras; gravação do arquivo fora da seção crítica, a partir de uma cópia do estado.
11	Dados só crescem	Reservas encerradas nunca saem da lista; cada rodada da carga deixa centenas de usuários. Buscas e gravações ficam mais lentas com o tempo.	Arquivar reservas encerradas; usar um arquivo de dados separado nas rodadas de carga.
12	Rajada de conexões novas	Centenas de conexões abertas no mesmo milissegundo são recusadas pelo sistema operacional. Os clientes não tentam de novo.	Tentar reconectar com uma pequena espera no <code>net.Dial</code> .

13	Sessões	A mesma conta pode estar logada em várias conexões ao mesmo tempo; o cliente não oferece trocar de conta (a ação <code>sair</code> existe no protocolo).	Aceitável para o projeto; se necessário, registrar logins ativos no servidor.
14	Contador compartilhado	Caronas e reservas dividem a numeração; o número sozinho não diz o que é.	Dois contadores, ou prefixos na exibição.
15	Testes de concorrência não confirmados	<code>TestExpiracaoConcorrente</code> e <code>TestCancelarCaronaDuranteReservas</code> nunca foram sabotados para ver se detectam a falta do mutex.	Repetir o experimento de tirar o <code>Lock</code> de <code>ExpirarVencidas</code> e de <code>CancelarCarona</code> .
16	Comentário desatualizado	O comentário de <code>TestReservaConcorrente</code> fala em 50 goroutines; o teste usa 200.	Ajustar o texto.

61. Sugestão de ordem

Quando	O quê
seg, 14/09	Commit e PR das mudanças do dia (#8). Estudar a Parte IV.
ter, 15/09	Docker (#1) e detector de corrida pelo container (#3). Limitações 1, 2 e 16, que são rápidas.
qua, 16/09	Duas máquinas no laboratório (#2), <code>protocolo.md</code> (#4), carga no Docker (#7), plano antigo (#6).
qui, 17/09	Entrega.
sáb e dom, 19–20/09	Relatório (#5).
seg, 21/09	Ensaio da apresentação com o roteiro da Parte VI.

Apêndice — Código completo

*Todos os arquivos do projeto, na íntegra, com os números de linha usados ao longo do guia.
Para consulta: as explicações estão nas partes IV, V e VI.*

62. go.mod

pbl/go.mod · linhas 1–3

```
1 module vaijunto
2
3 go 1.27
```

63. protocolo/protocolo.go

pbl/protocolo/protocolo.go · linhas 1–139

```
1 // Package protocolo define as mensagens que cliente e servidor trocam.
2 //
3 // A regra do protocolo e simples: UMA LINHA DE TEXTO = UMA MENSAGEM, e o
4 // conteudo da linha e um JSON. E isso que resolve o problema de o TCP nao ter
5 // fronteira de mensagem: quem le sabe que a mensagem acabou quando acha o \n.
6 package protocolo
7
8 import (
9     "bufio"
10    "encoding/json"
11    "errors"
12    "fmt"
13    "io"
14 )
15
16 // ErrMensagemInvalida marca o erro de "a linha chegou inteira, mas nao e um
17 // JSON valido". O problema e da MENSAGEM, e a conexao continua boa.
18 //
19 // Existe para quem le poder separar esse caso de um erro da CONEXAO (cliente
20 // fechou, caiu, rede sumiu) com errors.Is. Confundir os dois ja causou um bug:
21 // o servidor tratava queda abrupta como JSON ruim e ficava lendo para sempre.
22 var ErrMensagemInvalida = errors.New("mensagem invalida")
23
24 // Item e um pedaco de viagem: "na carona X, da cidade De ate a cidade Ate".
25 // De e Ate sao INDICES dentro da Rota da carona, nao nomes de cidade.
26 type Item struct {
27     CaronaID int `json:"carona_id"`
28     De       int  `json:"de"`
29     Ate      int  `json:"ate"`
30 }
31
32 // Opcao e uma viagem possivel que o servidor oferece ao passageiro.
33 //
34 // O servidor ja monta o Resumo pronto para exibir. Assim o cliente nao precisa
35 // conhecer a estrutura interna das caronas: ele so imprime o texto e devolve
36 // os Itens da opcao escolhida.
37 type Opcao struct {
38     Itens []Item `json:"itens"`
39     Resumo string `json:"resumo"`
40     Preco int    `json:"preco"`
41 }
42
43 // Pedido e o que o cliente manda para o servidor.
44 // Cada acao usa so os campos que interessam a ela; o resto vai vazio.
45 type Pedido struct {
```

```

46 Acao    string `json:"acao"`
47 Usuario string `json:"usuario,omitempty"`
48 Senha   string `json:"senha,omitempty"`
49 Tipo    string `json:"tipo,omitempty"` // registrar: motorista ou passageiro
50
51 // cadastrar
52 Rota     []string `json:"rota,omitempty"`
53 Data     string   `json:"data,omitempty"`
54 Assentos int       `json:"assentos,omitempty"`
55 Preco    int       `json:"preco,omitempty"`
56
57 // buscar
58 Origem   string `json:"origem,omitempty"`
59 Destino  string `json:"destino,omitempty"`
60
61 // reservar
62 Itens []Item `json:"itens,omitempty"`
63
64 // passageiros, cancelar_carona
65 CaronaID int `json:"carona_id,omitempty"`
66
67 // pagar, cancelar_reserva
68 ReservaID int `json:"reserva_id,omitempty"`
69 }
70
71 // Resposta e o que o servidor devolve para o cliente.
72 type Resposta struct {
73     OK      bool   `json:"ok"`
74     Erro    string `json:"erro,omitempty"`
75     Mensagem string `json:"mensagem,omitempty"`
76
77     Tipo      string `json:"tipo,omitempty"` // login
78     CaronaID  int    `json:"carona_id,omitempty"` // cadastrar
79     Opcoes   []Opcao `json:"opcoes,omitempty"` // buscar
80     Linhas   []string `json:"linhas,omitempty"` // listagens
81     ReservaID int    `json:"reserva_id,omitempty"` // reservar
82 }
83
84 // LerPedido le uma linha e transforma o JSON em Pedido.
85 //
86 // Dois tipos de erro podem voltar, e quem chama PRECISA distinguir:
87 // - erro de leitura (EOF, conexao resetada): a conexao acabou;
88 // - ErrMensagemInvalida: a linha veio, mas o JSON esta errado.
89 func LerPedido(leitor *bufio.Reader) (Pedido, error) {
90     linha, err := leitor.ReadString('\n')
91     if err != nil {
92         return Pedido{}, err
93     }
94
95     var p Pedido
96     if err := json.Unmarshal([]byte(linha), &p); err != nil {
97         return Pedido{}, fmt.Errorf("%w: %v", ErrMensagemInvalida, err)
98     }
99     return p, nil
100 }
101
102 // LerResposta le uma linha e transforma o JSON em Resposta.
103 // Os erros seguem a mesma regra do LerPedido.
104 func LerResposta(leitor *bufio.Reader) (Resposta, error) {
105     linha, err := leitor.ReadString('\n')
106     if err != nil {
107         return Resposta{}, err
108     }
109
110     var r Resposta
111     if err := json.Unmarshal([]byte(linha), &r); err != nil {
112         return Resposta{}, fmt.Errorf("%w: %v", ErrMensagemInvalida, err)

```

```

113     }
114     return r, nil
115 }
116
117 // EnviarPedido escreve o Pedido como uma linha de JSON.
118 func EnviarPedido(destino io.Writer, p Pedido) error {
119     return enviar(destino, p)
120 }
121
122 // EnviarResposta escreve a Resposta como uma linha de JSON.
123 func EnviarResposta(destino io.Writer, r Resposta) error {
124     return enviar(destino, r)
125 }
126
127 // enviar converte qualquer valor em JSON e escreve com o \n no fim.
128 // E minúscula, então só existe dentro deste pacote.
129 func enviar(destino io.Writer, valor any) error {
130     bytes, err := json.Marshal(valor)
131     if err != nil {
132         return err
133     }
134
135     bytes = append(bytes, '\n')
136
137     _, err = destino.Write(bytes)
138     return err
139 }

```

64. dados/dados.go

pbl/dados/dados.go · linhas 1–729

```

1 // Package dados guarda o estado do sistema e as regras que mexem nele.
2 //
3 // REGRA DE OURO DESTE PACOTE: toda função pública (maiúscula) abre com
4 // mu.Lock() e defer mu.Unlock(). Funções internas (minúsculas) NUNCA travam,
5 // porque são chamadas de dentro das públicas -- se elas travassem também, o
6 // programa congelaria (deadlock).
7 package dados
8
9 import (
10     "encoding/json"
11     "errors"
12     "fmt"
13     "os"
14     "strings"
15     "sync"
16     "time"
17
18     "vaijunto/protocolo"
19 )
20
21 // TempoDeReserva é quanto tempo a reserva fica de pé sem ser paga.
22 // É variável (não constante) para os testes poderem encurtar isso.
23 var TempoDeReserva = 10 * time.Minute
24
25 // Usuario é quem usa o sistema. Tipo é "motorista" ou "passageiro".
26 type Usuario struct {
27     Login string `json:"login"`
28     Senha string `json:"senha"`
29     Tipo  string `json:"tipo"`
30 }

```

```

31
32 // Carona e uma viagem oferecida por um motorista.
33 //
34 // Rota e a sequencia de cidades: ["Feira", "Salvador", "Ilheus"].
35 // Os TRECHOS sao os intervalos entre elas: Feira->Salvador e Salvador->Ilheus.
36 // Uma rota com N cidades tem N-1 trechos.
37 //
38 // Ocupados tem um contador por trecho. E o coracao da modelagem: a
39 // disponibilidade POR TRECHO sai naturalmente, sem estrutura extra.
40 type Carona struct {
41     ID          int          `json:"id"`
42     Motorista    string       `json:"motorista"`
43     Rota         []string     `json:"rota"`
44     Data         string       `json:"data"`
45     Assentos     int          `json:"assentos"` // total de assentos, igual em todo trecho
46     Preco        int          `json:"preco"`    // preco POR TRECHO
47     Ocupados     []int        `json:"ocupados"` // len = len(Rota)-1
48     Cancelada    bool         `json:"cancelada,omitempty"`
49 }
50
51 // Reserva e um pedido de assentos, que pode cobrir varias caronas.
52 type Reserva struct {
53     ID          int          `json:"id"`
54     Passageiro   string       `json:"passageiro"`
55     Itens        []protocolo.Item `json:"itens"`
56     Estado       string       `json:"estado"` // "pendente" | "paga" | "expirada" | "cancelada"
57     Motivo       string       `json:"motivo,omitempty"`
58     Expira       time.Time    `json:"expira"`
59 }
60
61 // Banco guarda TODO o estado do sistema em memoria.
62 //
63 // O mutex protege tudo que esta declarado abaixo dele. Como varias goroutines
64 // (uma por cliente conectado) mexem neste mesmo Banco, sem o mutex duas
65 // compras simultaneas poderiam ler o mesmo assento livre e vende-lo duas vezes.
66 //
67 // "mu" e "arquivo" comecam com minuscula, entao o encoding/json os ignora --
68 // e o que queremos: nem a fechadura nem o caminho do arquivo fazem parte dos
69 // dados salvos.
70 type Banco struct {
71     mu          sync.Mutex
72     arquivo     string
73
74     Usuarios    map[string]Usuario `json:"usuarios"`
75     Caronas     []Carona           `json:"caronas"`
76     Reservas    []Reserva          `json:"reservas"`
77     ProxID      int                `json:"prox_id"`
78 }
79
80 // NovoBanco cria o banco ja com usuarios de teste, sem persistencia.
81 //
82 // Devolve PONTEIRO (*Banco), nunca valor. Um sync.Mutex nao pode ser copiado:
83 // cada copia teria a sua propria fechadura, e proteger uma copia nao protegeria
84 // as outras. O proprio "go vet" reclama se voce tentar copiar.
85 func NovoBanco() *Banco {
86     return &Banco{
87         Usuarios: map[string]Usuario{
88             "ana": {Login: "ana", Senha: "123", Tipo: "motorista"},
89             "davi": {Login: "davi", Senha: "123", Tipo: "motorista"},
90             "bruno": {Login: "bruno", Senha: "123", Tipo: "passageiro"},
91             "carla": {Login: "carla", Senha: "123", Tipo: "passageiro"},
92         },
93         ProxID: 1,
94     }
95 }
96
97 // Carregar le o banco de um arquivo JSON. Se o arquivo nao existir (primeira

```

```

98 // execucao), começa do zero. A partir daqui toda alteracao e gravada sozinha.
99 //
100 // O enunciado proibe SGBD mas permite JSON: e exatamente isto.
101 func Carregar(caminho string) *Banco {
102     b := NovoBanco()
103     b.arquivo = caminho
104
105     conteudo, err := os.ReadFile(caminho)
106     if err != nil {
107         fmt.Printf("sem dados anteriores em %, começando do zero\n", caminho)
108         return b
109     }
110
111     if err := json.Unmarshal(conteudo, b); err != nil {
112         fmt.Printf("arquivo %s ilegível (%v), começando do zero\n", caminho, err)
113         return NovoBancoEm(caminho)
114     }
115
116     fmt.Printf("dados carregados de %s: %d carona(s), %d reserva(s)\n",
117         caminho, len(b.Caronas), len(b.Reservas))
118     return b
119 }
120
121 // NovoBancoEm cria um banco vazio que grava no caminho indicado.
122 func NovoBancoEm(caminho string) *Banco {
123     b := NovoBanco()
124     b.arquivo = caminho
125     return b
126 }
127
128 // -----
129 // Funcoes publicas: TODAS travam o mutex
130 // -----
131
132 // Autenticar confere login e senha. O segundo retorno diz se deu certo.
133 func (b *Banco) Autenticar(login, senha string) (Usuario, bool) {
134     b.mu.Lock()
135     defer b.mu.Unlock()
136
137     u, existe := b.Usuarios[login]
138     if !existe {
139         return Usuario{}, false
140     }
141
142     if u.Senha != senha {
143         return Usuario{}, false
144     }
145
146     return u, true
147 }
148
149 // CadastrarUsuario cria uma conta nova.
150 //
151 // Repare que a checagem "já existe" e a escrita no mapa acontecem DENTRO do
152 // mesmo Lock. Se a checagem ficasse fora, duas pessoas registrando o mesmo
153 // login ao mesmo tempo passariam as duas pela verificacao antes de qualquer
154 // uma escrever -- e a segunda sobrescreveria a primeira. E o mesmo raciocinio
155 // das duas fases do Reservar.
156 func (b *Banco) CadastrarUsuario(login, senha, tipo string) error {
157     login = strings.TrimSpace(login)
158
159     if login == "" {
160         return errors.New("informe o usuario")
161     }
162     if senha == "" {
163         return errors.New("informe a senha")
164     }

```

```

165     if tipo != "motorista" && tipo != "passageiro" {
166         return errors.New(`o tipo precisa ser "motorista" ou "passageiro"`)
167     }
168
169     b.mu.Lock()
170     defer b.mu.Unlock()
171
172     if _, existe := b.Usuarios[login]; existe {
173         return fmt.Errorf("o usuario %q ja existe", login)
174     }
175
176     b.Usuarios[login] = Usuario{Login: login, Senha: senha, Tipo: tipo}
177     b.persistir()
178
179     return nil
180 }
181
182 // CadastrarCarona registra uma carona nova e devolve o ID dela.
183 func (b *Banco) CadastrarCarona(motorista string, rota []string, data string, assentos, preco int)
(int, error) {
184     // A validacao vem ANTES do Lock: nao faz sentido segurar a fechadura
185     // (e travar todos os outros clientes) para conferir argumento.
186     if len(rota) < 2 {
187         return 0, errors.New("a rota precisa de pelo menos 2 cidades")
188     }
189     if assentos < 1 {
190         return 0, errors.New("a carona precisa de pelo menos 1 assento")
191     }
192     if data == "" {
193         return 0, errors.New("informe a data")
194     }
195
196     b.mu.Lock()
197     defer b.mu.Unlock()
198
199     c := Carona{
200         ID:        b.ProxID,
201         Motorista: motorista,
202         Rota:       rota,
203         Data:       data,
204         Assentos:   assentos,
205         Preco:      preco,
206         Ocupados:   make([]int, len(rota)-1), // um contador por trecho, zerado
207     }
208
209     b.ProxID++
210     b.Caronas = append(b.Caronas, c)
211     b.persistir()
212
213     return c.ID, nil
214 }
215
216 // Buscar devolve as opcoes de viagem de origem ate destino naquela data.
217 // Primeiro as caronas diretas, depois as que exigem UMA baldeacao.
218 func (b *Banco) Buscar(origem, destino, data string) []protocolo.Opcao {
219     b.mu.Lock()
220     defer b.mu.Unlock()
221
222     opcoes := b.buscarDiretas(origem, destino, data)
223     return append(opcoes, b.buscarComConexao(origem, destino, data)...)
224 }
225
226 // Reservar segura assentos em uma ou mais caronas, de forma ATOMICA.
227 //
228 // Esta e a funcao mais importante do projeto. O enunciado exige que, se o
229 // passageiro pegar um trecho, ele nao fique sem o outro. A garantia vem de
230 // duas fases dentro de UM UNICO Lock:

```

```

231 //
232 //     FASE 1 confere TODOS os itens sem alterar nada.
233 //     FASE 2 so roda se a fase 1 passou inteira.
234 //
235 // Como nada solta o mutex entre as duas, nenhuma outra goroutine consegue se
236 // intrometer no meio e roubar um assento que acabamos de conferir.
237 func (b *Banco) Reservar(passageiro string, itens []protocolo.Item) (int, error) {
238     if len(itens) == 0 {
239         return 0, errors.New("nenhum trecho escolhido")
240     }
241
242     b.mu.Lock()
243     defer b.mu.Unlock()
244
245     // ---- FASE 1: verificacao. NAO altera nada. ----
246     for _, item := range itens {
247         c := b.acharCarona(item.CaronaID)
248         if c == nil {
249             return 0, fmt.Errorf("carona %d nao existe", item.CaronaID)
250         }
251         if c.Cancelada {
252             return 0, fmt.Errorf("carona %d foi cancelada", item.CaronaID)
253         }
254         if item.De < 0 || item.Ate > c.trechos() || item.De >= item.Ate {
255             return 0, fmt.Errorf("trecho invalido na carona %d", item.CaronaID)
256         }
257         if c.livres(item.De, item.Ate) < 1 {
258             return 0, fmt.Errorf("sem assento livre na carona %d", item.CaronaID)
259         }
260     }
261
262     // ---- FASE 2: aplicacao. So chega aqui se TUDO passou. ----
263     for _, item := range itens {
264         b.acharCarona(item.CaronaID).ocupar(item.De, item.Ate)
265     }
266
267     r := Reserva{
268         ID:         b.ProxID,
269         Passageiro:  passageiro,
270         Itens:       itens,
271         Estado:      "pendente",
272         Expira:      time.Now().Add(TempoDeReserva),
273     }
274
275     b.ProxID++
276     b.Reservas = append(b.Reservas, r)
277     b.persistir()
278
279     return r.ID, nil
280 }
281
282 // Pagar confirma uma reserva pendente. Reserva paga nunca mais expira.
283 func (b *Banco) Pagar(passageiro string, reservaID int) error {
284     b.mu.Lock()
285     defer b.mu.Unlock()
286
287     r := b.acharReserva(reservaID)
288     if r == nil {
289         return fmt.Errorf("reserva %d nao existe", reservaID)
290     }
291     if r.Passageiro != passageiro {
292         return errors.New("esta reserva nao e sua")
293     }
294
295     switch r.Estado {
296     case "paga":
297         return errors.New("esta reserva ja foi paga")

```

```

298     case "expirada":
299         return errors.New("esta reserva expirou; os assentos voltaram para a fila")
300     case "cancelada":
301         return fmt.Errorf("esta reserva foi cancelada (%s)", r.Motivo)
302     }
303
304     r.Estado = "paga"
305     b.persistir()
306
307     return nil
308 }
309
310 // CancelarReserva desiste de uma reserva pendente ou paga e devolve os
311 // assentos de TODOS os trechos dela.
312 //
313 // E o espelho do Reservar: se a reserva cobre duas caronas, as duas recebem o
314 // assento de volta juntas, dentro do mesmo Lock.
315 func (b *Banco) CancelarReserva(passageiro string, reservaID int) error {
316     b.mu.Lock()
317     defer b.mu.Unlock()
318
319     r := b.acharReserva(reservaID)
320     if r == nil {
321         return fmt.Errorf("reserva %d nao existe", reservaID)
322     }
323     if r.Passageiro != passageiro {
324         return errors.New("esta reserva nao e sua")
325     }
326     if !r.ativa() {
327         return fmt.Errorf("esta reserva ja esta %s", r.Estado)
328     }
329
330     b.cancelar(r, "cancelada pelo passageiro")
331     b.persistir()
332
333     return nil
334 }
335
336 // CancelarCarona tira uma carona do ar e cancela todas as reservas ativas que
337 // passam por ela. Devolve quantas reservas foram afetadas.
338 //
339 // Atencao ao caso da conexao: uma reserva Feira->Ilheus pode usar esta carona
340 // num trecho e OUTRA carona no trecho seguinte. Liberar so a parte desta
341 // carona deixaria o passageiro com meia viagem -- exatamente o que a
342 // atomicidade proibe. Por isso a reserva inteira e cancelada, e os assentos
343 // voltam em TODAS as caronas dela.
344 func (b *Banco) CancelarCarona(motorista string, caronaID int) (int, error) {
345     b.mu.Lock()
346     defer b.mu.Unlock()
347
348     c := b.acharCarona(caronaID)
349     if c == nil {
350         return 0, fmt.Errorf("carona %d nao existe", caronaID)
351     }
352     if c.Motorista != motorista {
353         return 0, errors.New("esta carona nao e sua")
354     }
355     if c.Cancelada {
356         return 0, errors.New("esta carona ja foi cancelada")
357     }
358
359     c.Cancelada = true
360
361     motivo := fmt.Sprintf("carona %d cancelada pelo motorista", caronaID)
362     afetadas := 0
363
364     for i := range b.Reservas {

```

```

365     r := &b.Reservas[i]
366     if r.ativa() && r.usa(caronaID) {
367         b.cancelar(r, motivo)
368         afetadas++
369     }
370 }
371
372 b.persistir()
373
374 return afetadas, nil
375 }
376
377 // ExpirarVencidas devolve os assentos das reservas pendentes que passaram do
378 // prazo. Roda periodicamente numa goroutine do servidor. Devolve quantas
379 // reservas foram expiradas.
380 func (b *Banco) ExpirarVencidas() int {
381     b.mu.Lock()
382     defer b.mu.Unlock()
383
384     agora := time.Now()
385     expiradas := 0
386
387     for i := range b.Reservas {
388         // Ponteiro para o elemento REAL do slice. Com "for _, r := range"
389         // mexeríamos numa copia e o estado nunca mudaria.
390         r := &b.Reservas[i]
391
392         if r.Estado != "pendente" || agora.Before(r.Expira) {
393             continue
394         }
395
396         b.devolverAssentos(r)
397         r.Estado = "expirada"
398         expiradas++
399     }
400
401     if expiradas > 0 {
402         b.persistir()
403     }
404
405     return expiradas
406 }
407
408 // MinhasCaronas lista as caronas de um motorista, prontas para exibir.
409 func (b *Banco) MinhasCaronas(motorista string) []string {
410     b.mu.Lock()
411     defer b.mu.Unlock()
412
413     var linhas []string
414
415     for _, c := range b.Caronas {
416         if c.Motorista != motorista {
417             continue
418         }
419
420         situacao := ""
421         if c.Cancelada {
422             situacao = " | CANCELADA"
423         }
424
425         linhas = append(linhas, fmt.Sprintf("carona %d | %s | %s | R$%d/trecho | ocupados por trecho: %v
de %d%s",
426             c.ID, c.Data, strings.Join(c.Rota, " -> "), c.Preco, c.Ocupados, c.Assentos, situacao))
427     }
428
429     return linhas
430 }

```

```

431
432 // PassageirosDaCarona mostra, trecho a trecho, quem esta em cada assento.
433 // So o motorista dono da carona pode ver.
434 func (b *Banco) PassageirosDaCarona(motorista string, caronaID int) ([]string, error) {
435     b.mu.Lock()
436     defer b.mu.Unlock()
437
438     c := b.acharCarona(caronaID)
439     if c == nil {
440         return nil, fmt.Errorf("carona %d nao existe", caronaID)
441     }
442     if c.Motorista != motorista {
443         return nil, errors.New("esta carona nao e sua")
444     }
445
446     situacao := ""
447     if c.Cancelada {
448         situacao = " | CANCELADA"
449     }
450
451     linhas := []string{fmt.Sprintf("carona %d | %s | %s%s",
452         c.ID, c.Data, strings.Join(c.Rota, " -> "), situacao)}
453
454     // Para cada trecho, procura as reservas ativas cujo caminho passa por ele.
455     for t := 0; t < c.trechos(); t++ {
456         var nomes []string
457
458         for _, r := range b.Reservas {
459             if !r.ativa() {
460                 continue
461             }
462             for _, item := range r.Itens {
463                 if item.CaronaID == caronaID && item.De <= t && t < item.Ate {
464                     nomes = append(nomes, fmt.Sprintf("%s (%s)", r.Passageiro, r.Estado))
465                 }
466             }
467         }
468
469         lista := "ninguem"
470         if len(nomes) > 0 {
471             lista = strings.Join(nomes, ", ")
472         }
473
474         linhas = append(linhas, fmt.Sprintf(" %s -> %s [%d/%d]: %s",
475             c.Rota[t], c.Rota[t+1], c.Ocupados[t], c.Assentos, lista))
476     }
477
478     return linhas, nil
479 }
480
481 // MinhasReservas lista as reservas de um passageiro, prontas para exibir.
482 func (b *Banco) MinhasReservas(passageiro string) []string {
483     b.mu.Lock()
484     defer b.mu.Unlock()
485
486     var linhas []string
487
488     for _, r := range b.Reservas {
489         if r.Passageiro != passageiro {
490             continue
491         }
492
493         detalhe := ""
494         switch {
495             case r.Estado == "pendente":
496                 detalhe = fmt.Sprintf(" | expira em %s", time.Until(r.Expira).Round(time.Second))
497             case r.Motivo != "":

```

```

498     detalhe = " | " + r.Motivo
499 }
500
501     linhas = append(linhas, fmt.Sprintf("reserva %d | %s | %d trecho(s)%s",
502     r.ID, r.Estado, len(r.Itens), detalhe))
503 }
504
505     return linhas
506 }
507
508 // -----
509 // Funcoes internas: NENHUMA trava. Sao chamadas de dentro das publicas,
510 // que ja estao segurando o mutex.
511 // -----
512
513 // buscarDiretas acha caronas que sozinhas levam de origem ate destino.
514 func (b *Banco) buscarDiretas(origem, destino, data string) []protocolo.Opcao {
515     var opcoes []protocolo.Opcao
516
517     for _, c := range b.Caronas {
518         if c.Cancelada || c.Data != data {
519             continue
520         }
521
522         de := c.indice(origem)
523         ate := c.indice(destino)
524
525         // As duas cidades precisam estar na rota, e a origem precisa vir
526         // ANTES do destino: a carona so anda em um sentido.
527         if de == -1 || ate == -1 || de >= ate {
528             continue
529         }
530
531         livres := c.livres(de, ate)
532         if livres < 1 {
533             continue
534         }
535
536         opcoes = append(opcoes, protocolo.Opcao{
537             Itens: []protocolo.Item{{CaronaID: c.ID, De: de, Ate: ate}},
538             Preco: c.Preco * (ate - de),
539             Resumo: fmt.Sprintf("direto: carona %d (%s) %s | %d livre(s)",
540                 c.ID, c.Motorista, strings.Join(c.Rota[de:ate+1], " -> "), livres),
541         })
542     }
543
544     return opcoes
545 }
546
547 // buscarComConexao acha pares de caronas que, juntas, levam de origem ate
548 // destino trocando de veiculo numa cidade do meio.
549 //
550 // E aqui que a atomicidade do Reservar ganha sentido: uma opcao destas tem
551 // DOIS itens, e nao adianta conseguir o primeiro e perder o segundo.
552 func (b *Banco) buscarComConexao(origem, destino, data string) []protocolo.Opcao {
553     var opcoes []protocolo.Opcao
554
555     for _, primeira := range b.Caronas {
556         if primeira.Cancelada || primeira.Data != data {
557             continue
558         }
559
560         de := primeira.indice(origem)
561         if de == -1 {
562             continue
563         }
564

```

```

565 // Testa cada cidade depois da origem como ponto de baldeacao.
566 for meio := de + 1; meio < len(primeira.Rota); meio++ {
567     baldeacao := primeira.Rota[meio]
568
569     // Se a primeira carona ja chega no destino, isso e viagem direta:
570     // a outra funcao ja cuidou disso.
571     if baldeacao == destino {
572         continue
573     }
574     if primeira.livres(de, meio) < 1 {
575         continue
576     }
577
578     for _, segunda := range b.Caronas {
579         if segunda.ID == primeira.ID || segunda.Cancelada || segunda.Data != data {
580             continue
581         }
582
583         de2 := segunda.indice(baldeacao)
584         ate2 := segunda.indice(destino)
585         if de2 == -1 || ate2 == -1 || de2 >= ate2 {
586             continue
587         }
588         if segunda.livres(de2, ate2) < 1 {
589             continue
590         }
591
592         opcoes = append(opcoes, protocolo.Opcao{
593             Itens: []protocolo.Item{
594                 {CaronaID: primeira.ID, De: de, Ate: meio},
595                 {CaronaID: segunda.ID, De: de2, Ate: ate2},
596             },
597             Preco: primeira.Preco*(meio-de) + segunda.Preco*(ate2-de2),
598             Resumo: fmt.Sprintf("conexao em %s: carona %d (%s) + carona %d (%s)",
599                 baldeacao, primeira.ID, primeira.Motorista, segunda.ID, segunda.Motorista),
600         })
601     }
602 }
603 }
604
605 return opcoes
606 }
607
608 // cancelar desfaz uma reserva ativa: devolve os assentos e registra o motivo.
609 func (b *Banco) cancelar(r *Reserva, motivo string) {
610     b.devolverAssentos(r)
611     r.Estado = "cancelada"
612     r.Motivo = motivo
613 }
614
615 // devolverAssentos libera um assento em cada trecho de cada item da reserva.
616 // Usada pela expiracao e pelos dois cancelamentos.
617 func (b *Banco) devolverAssentos(r *Reserva) {
618     for _, item := range r.Itens {
619         if c := b.acharCarona(item.CaronaID); c != nil {
620             c.liberar(item.De, item.Ate)
621         }
622     }
623 }
624
625 // persistir grava o banco no arquivo. Se nao houver arquivo configurado
626 // (como nos testes), nao faz nada.
627 //
628 // Nao trava: quem chama ja esta com o mutex na mao. Gravar aqui dentro
629 // garante que o arquivo nunca pega o estado pela metade.
630 func (b *Banco) persistir() {
631     if b.arquivo == "" {

```

```

632     return
633 }
634
635 conteudo, err := json.MarshalIndent(b, "", " ")
636 if err != nil {
637     fmt.Println("erro ao converter os dados:", err)
638     return
639 }
640
641 if err := os.WriteFile(b.arquivo, conteudo, 0o644); err != nil {
642     fmt.Println("erro ao gravar", b.arquivo, ":", err)
643 }
644 }
645
646 // acharCarona devolve um PONTEIRO para a carona dentro do slice.
647 //
648 // Repare no "for i := range" em vez de "for _, c := range": o range por valor
649 // entrega uma COPIA de cada carona, e alterar a copia nao mudaria nada. Para
650 // modificar o elemento de verdade e preciso pegar o endereco dele.
651 func (b *Banco) acharCarona(id int) *Carona {
652     for i := range b.Caronas {
653         if b.Caronas[i].ID == id {
654             return &b.Caronas[i]
655         }
656     }
657     return nil
658 }
659
660 func (b *Banco) acharReserva(id int) *Reserva {
661     for i := range b.Reservas {
662         if b.Reservas[i].ID == id {
663             return &b.Reservas[i]
664         }
665     }
666     return nil
667 }
668
669 // ativa diz se a reserva ainda segura assentos.
670 func (r Reserva) ativa() bool {
671     return r.Estado == "pendente" || r.Estado == "paga"
672 }
673
674 // usa diz se algum item da reserva e da carona indicada.
675 func (r Reserva) usa(caronaID int) bool {
676     for _, item := range r.Itens {
677         if item.CaronaID == caronaID {
678             return true
679         }
680     }
681     return false
682 }
683
684 // trechos diz quantos trechos a carona tem: uma cidade a menos que a rota.
685 func (c Carona) trechos() int {
686     return len(c.Rota) - 1
687 }
688
689 // indice devolve a posicao da cidade na rota, ou -1 se ela nao estiver la.
690 func (c Carona) indice(cidade string) int {
691     for i, nome := range c.Rota {
692         if nome == cidade {
693             return i
694         }
695     }
696     return -1
697 }
698

```

```

699 // livres diz quantos assentos estao livres em TODO o caminho de "de" ate "ate".
700 //
701 // Viajar da cidade "de" ate a cidade "ate" usa os trechos de, de+1, ..., ate-1.
702 // O passageiro precisa do mesmo assento no caminho inteiro, entao o que vale e
703 // o trecho MAIS CHEIO: e ele que limita.
704 func (c Carona) livres(de, ate int) int {
705     menor := c.Assentos
706
707     for t := de; t < ate; t++ {
708         livre := c.Assentos - c.Ocupados[t]
709         if livre < menor {
710             menor = livre
711         }
712     }
713
714     return menor
715 }
716
717 // ocupar marca um assento em cada trecho do caminho.
718 func (c *Carona) ocupar(de, ate int) {
719     for t := de; t < ate; t++ {
720         c.Ocupados[t]++
721     }
722 }
723
724 // liberar devolve um assento em cada trecho do caminho.
725 func (c *Carona) liberar(de, ate int) {
726     for t := de; t < ate; t++ {
727         c.Ocupados[t]--
728     }
729 }

```

65. servidor/main.go

pbl/servidor/main.go · linhas 1–296

```

1  package main
2
3  import (
4      "bufio"
5      "errors"
6      "fmt"
7      "io"
8      "net"
9      "os"
10     "time"
11
12     "vaijunto/dados"
13     "vaijunto/protocolo"
14 )
15
16 // intervaloPadrao e de quanto em quanto tempo o servidor procura reservas
17 // vencidas. Pode ser encurtado por variavel de ambiente na hora de demonstrar.
18 const intervaloPadrao = 30 * time.Second
19
20 // sessao guarda quem esta logado NAQUELA conexao.
21 //
22 // Repare que ela NAO precisa de mutex: cada goroutine de atender() cria a sua
23 // propria sessao como variavel local, entao ninguem compartilha nada. So o
24 // Banco, que e o mesmo para todos, precisa de protecao.
25 type sessao struct {
26     usuario dados.Usuario

```

```

27 }
28
29 func main() {
30     varredura := configurarTempos()
31     banco := dados.Carregar(arquivoDeDados())
32
33     fmt.Printf("reserva expira em %s, varrendo a cada %s\n",
34         dados.TempoDeReserva, varredura)
35
36     // Goroutine de fundo: fica devolvendo assentos de reservas nao pagas.
37     go expirarPeriodicamente(banco, varredura)
38
39     ouvinte, err := net.Listen("tcp", ":8080")
40     if err != nil {
41         fmt.Println("erro ao abrir a porta:", err)
42         return
43     }
44     defer ouvinte.Close()
45
46     fmt.Println("servidor ouvindo em :8080")
47
48     for {
49         conexao, err := ouvinte.Accept()
50         if err != nil {
51             fmt.Println("erro ao aceitar conexao:", err)
52             continue
53         }
54
55         fmt.Println("cliente conectado:", conexao.RemoteAddr())
56         go atender(conexao, banco)
57     }
58 }
59
60 // expirarPeriodicamente roda para sempre, numa goroutine propria.
61 //
62 // Ela tambem mexe no Banco, entao disputa o mesmo mutex que as goroutines dos
63 // clientes -- e e justamente por isso que nao da problema.
64 func expirarPeriodicamente(banco *dados.Banco, intervalo time.Duration) {
65     relógio := time.NewTicker(intervalo)
66     defer relógio.Stop()
67
68     for range relógio.C {
69         if n := banco.ExpirarVencidas(); n > 0 {
70             fmt.Printf("expirei %d reserva(s) nao paga(s)\n", n)
71         }
72     }
73 }
74
75 // arquivoDeDados diz onde o banco JSON fica gravado.
76 // No Docker apontamos para um volume, para os dados sobreviverem ao container.
77 func arquivoDeDados() string {
78     if v := os.Getenv("VAIJUNTO_DADOS"); v != "" {
79         return v
80     }
81     return "vaijunto.json"
82 }
83
84 // configurarTempos le as variaveis de ambiente que encurtam os prazos.
85 // Para demonstrar a expiracao sem esperar 10 minutos:
86 //
87 //     VAIJUNTO_RESERVA=20s VAIJUNTO_VARREDURA=5s go run ./servidor
88 func configurarTempos() time.Duration {
89     if v := os.Getenv("VAIJUNTO_RESERVA"); v != "" {
90         if d, err := time.ParseDuration(v); err == nil {
91             dados.TempoDeReserva = d
92         }
93     }

```

```

94     intervalo := intervaloPadrao
95     if v := os.Getenv("VAIJUNTO_VARREDURA"); v != "" {
96         if d, err := time.ParseDuration(v); err == nil {
97             intervalo = d
98         }
99     }
100 }
101
102 return intervalo
103 }
104
105 func atender(conexao net.Conn, banco *dados.Banco) {
106     defer conexao.Close()
107
108     leitor := bufio.NewReader(conexao)
109
110     // A sessao nasce e morre junto com esta conexao.
111     var s sessao
112
113     for {
114         pedido, err := protocolo.LerPedido(leitor)
115
116         // A linha chegou inteira, mas nao e JSON valido: o problema e da
117         // mensagem, nao da conexao. Avisa o cliente e continua atendendo.
118         if errors.Is(err, protocolo.ErrMensagemInvalida) {
119             protocolo.EnviaResposta(conexao, protocolo.Resposta{OK: false, Erro: err.Error()})
120             continue
121         }
122
123         // Qualquer OUTRO erro e da conexao: o cliente fechou (EOF), caiu de
124         // forma abrupta (connection reset) ou a rede sumiu. Em todos os casos
125         // nao ha mais ninguem do outro lado, entao a goroutine precisa acabar.
126         //
127         // Uma versao anterior so encerrava no EOF e tratava o resto como JSON
128         // ruim: numa queda abrupta, voltava a ler, recebia o mesmo erro na hora
129         // e girava para sempre a 100% de um nucleo.
130         if err != nil {
131             if errors.Is(err, io.EOF) {
132                 fmt.Println("cliente desconectou:", conexao.RemoteAddr())
133             } else {
134                 fmt.Printf("conexao com %v caiu: %v\n", conexao.RemoteAddr(), err)
135             }
136             return
137         }
138
139         fmt.Printf("recebido de %v: acao=%q\n", conexao.RemoteAddr(), pedido.Acao)
140
141         resposta := executar(pedido, &s, banco)
142
143         if err := protocolo.EnviaResposta(conexao, resposta); err != nil {
144             fmt.Println("erro ao responder:", err)
145             return
146         }
147     }
148 }
149
150 // executar decide o que fazer com cada acao. Este switch e o roteador do
151 // servidor: e o indice do sistema inteiro.
152 //
153 // A sessao vem como PONTEIRO (*sessao) porque o case "login" precisa
154 // MODIFICAR quem esta logado. Se viesse por valor, alteraria uma copia.
155 func executar(p protocolo.Pedido, s *sessao, banco *dados.Banco) protocolo.Resposta {
156     // Barreira de autenticacao: vale para toda acao nova que criarmos daqui
157     // em diante, sem precisar repetir a checagem em cada case.
158     if exigeLogin(p.Acao) && s.usuario.Login == "" {
159         return erro("faca login primeiro")
160     }

```

```

161
162 switch p.Acao {
163 case "ping":
164     return protocolo.Resposta{OK: true, Mensagem: "pong"}
165
166 case "registrar":
167     if err := banco.CadastrarUsuario(p.Usuario, p.Senha, p.Tipo); err != nil {
168         return erro(err.Error())
169     }
170     return protocolo.Resposta{
171         OK: true,
172         Mensagem: fmt.Sprintf("conta criada para %s (%s)", p.Usuario, p.Tipo),
173     }
174
175 case "login":
176     u, ok := banco.Autenticar(p.Usuario, p.Senha)
177     if !ok {
178         return erro("usuario ou senha invalidos")
179     }
180
181     s.usuario = u
182     return protocolo.Resposta{
183         OK: true,
184         Tipo: u.Tipo,
185         Mensagem: fmt.Sprintf("bem-vindo, %s", u.Login),
186     }
187
188 case "sair":
189     s.usuario = dados.Usuario{}
190     return protocolo.Resposta{OK: true, Mensagem: "sessao encerrada"}
191
192 // ---- acoes do motorista ----
193
194 case "cadastrar":
195     if s.usuario.Tipo != "motorista" {
196         return erro("so motorista cadastra carona")
197     }
198
199     id, err := banco.CadastrarCarona(s.usuario.Login, p.Rota, p.Data, p.Assentos, p.Preco)
200     if err != nil {
201         return erro(err.Error())
202     }
203     return protocolo.Resposta{
204         OK: true,
205         CaronaID: id,
206         Mensagem: fmt.Sprintf("carona %d cadastrada", id),
207     }
208
209 case "minhas_caronas":
210     return protocolo.Resposta{OK: true, Linhas: banco.MinhasCaronas(s.usuario.Login)}
211
212 case "passageiros":
213     if s.usuario.Tipo != "motorista" {
214         return erro("so motorista ve os passageiros de uma carona")
215     }
216
217     linhas, err := banco.PassageirosDaCarona(s.usuario.Login, p.CaronaID)
218     if err != nil {
219         return erro(err.Error())
220     }
221     return protocolo.Resposta{OK: true, Linhas: linhas}
222
223 case "cancelar_carona":
224     if s.usuario.Tipo != "motorista" {
225         return erro("so motorista cancela carona")
226     }
227

```

```

228     afetadas, err := banco.CancelarCarona(s.usuario.Login, p.CaronaID)
229     if err != nil {
230         return erro(err.Error())
231     }
232     return protocolo.Resposta{
233         OK: true,
234         Mensagem: fmt.Sprintf("carona %d cancelada; %d reserva(s) desfeita(s) e assentos
devolvidos",
235             p.CaronaID, afetadas),
236     }
237
238     // ---- acoes do passageiro ----
239
240     case "buscar":
241         opcoes := banco.Buscar(p.Origem, p.Destino, p.Data)
242         return protocolo.Resposta{
243             OK: true,
244             Opcoes: opcoes,
245             Mensagem: fmt.Sprintf("%d opcao(oes) encontrada(s)", len(opcoes)),
246         }
247
248     case "reservar":
249         id, err := banco.Reservar(s.usuario.Login, p.Itens)
250         if err != nil {
251             return erro(err.Error())
252         }
253         return protocolo.Resposta{
254             OK: true,
255             ReservaID: id,
256             Mensagem: fmt.Sprintf("reserva %d criada; pague em ate %s ou os assentos voltam",
257                 id, dados.TempoDeReserva),
258         }
259
260     case "pagar":
261         if err := banco.Pagar(s.usuario.Login, p.ReservaID); err != nil {
262             return erro(err.Error())
263         }
264         return protocolo.Resposta{
265             OK: true,
266             Mensagem: fmt.Sprintf("reserva %d paga; assentos confirmados", p.ReservaID),
267         }
268
269     case "cancelar_reserva":
270         if err := banco.CancelarReserva(s.usuario.Login, p.ReservaID); err != nil {
271             return erro(err.Error())
272         }
273         return protocolo.Resposta{
274             OK: true,
275             Mensagem: fmt.Sprintf("reserva %d cancelada; assentos devolvidos", p.ReservaID),
276         }
277
278     case "minhas_reservas":
279         return protocolo.Resposta{OK: true, Linhas: banco.MinhasReservas(s.usuario.Login)}
280
281     default:
282         return erro(fmt.Sprintf("acao desconhecida: %q", p.Acao))
283     }
284 }
285
286 // erro monta uma Resposta de falha. Existe so para encurtar os cases.
287 func erro(mensagem string) protocolo.Resposta {
288     return protocolo.Resposta{OK: false, Erro: mensagem}
289 }
290
291 // exigeLogin diz quais acoes precisam de usuario autenticado.
292 // So "ping", "login" e "registrar" ficam de fora -- quem esta criando conta
293 // ainda nao tem como estar logado. Todo o resto exige.

```

```
294 func exigeLogin(acao string) bool {
295     return acao != "ping" && acao != "login" && acao != "registrar"
296 }
```

66. cliente/main.go

pbl/cliente/main.go · linhas 1–453

```
1 package main
2
3 import (
4     "bufio"
5     "fmt"
6     "net"
7     "os"
8     "strconv"
9     "strings"
10
11     "vaijunto/protocolo"
12 )
13
14 // cliente junta as tres coisas que toda tela precisa: por onde falar com o
15 // servidor, por onde ouvir a resposta, e por onde ler o teclado.
16 // Sem isso, cada funcao de menu teria que receber os tres como argumento.
17 type cliente struct {
18     conexao net.Conn
19     servidor *bufio.Reader
20     teclado  *bufio.Reader
21 }
22
23 func main() {
24     conexao, err := net.Dial("tcp", endereco())
25     if err != nil {
26         fmt.Println("nao consegui conectar:", err)
27         return
28     }
29     defer conexao.Close()
30
31     c := &cliente{
32         conexao: conexao,
33         servidor: bufio.NewReader(conexao),
34         teclado:  bufio.NewReader(os.Stdin),
35     }
36
37     tipo, ok := c.entrada()
38     if !ok {
39         return
40     }
41
42     // O tipo do usuario decide qual menu aparece.
43     if tipo == "motorista" {
44         c.menuMotorista()
45     } else {
46         c.menuPassageiro()
47     }
48 }
49
50 // endereco permite apontar para outro host, o que sera necessario no Docker:
51 //
52 //     VAIJUNTO_SERVIDOR=servidor:8080 go run ./cliente
53 func endereco() string {
54     if v := os.Getenv("VAIJUNTO_SERVIDOR"); v != "" {
```

```

55     return v
56 }
57 return "localhost:8080"
58 }
59
60 // -----
61 // Login
62 // -----
63
64 // entrada e a primeira tela: entrar com uma conta existente ou criar uma nova.
65 // Devolve o tipo do usuario, que decide qual menu aparece depois.
66 func (c *cliente) entrada() (string, bool) {
67     fmt.Println("=== Vaijunto ===")
68
69     for {
70         fmt.Println("\n1) entrar    2) criar conta    0) sair")
71
72         switch c.opcao() {
73             case "1":
74                 if tipo, ok := c.entrar(); ok {
75                     return tipo, true
76                 }
77             case "2":
78                 if tipo, ok := c.criarConta(); ok {
79                     return tipo, true
80                 }
81             case "0", "":
82                 return "", false
83             default:
84                 fmt.Println(" opcao invalida")
85             }
86         }
87     }
88
89     // entrar pede usuario e senha de uma conta que ja existe.
90     func (c *cliente) entrar() (string, bool) {
91         usuario, ok := c.ler("usuario: ")
92         if !ok {
93             return "", false
94         }
95
96         senha, ok := c.ler("senha: ")
97         if !ok {
98             return "", false
99         }
100
101         return c.autenticar(usuario, senha)
102     }
103
104     // criarConta registra um usuario novo e ja entra com ele.
105     func (c *cliente) criarConta() (string, bool) {
106         usuario, ok := c.ler("novo usuario: ")
107         if !ok {
108             return "", false
109         }
110
111         senha, ok := c.ler("senha: ")
112         if !ok {
113             return "", false
114         }
115
116         fmt.Println(" 1) motorista (oferece caronas)")
117         fmt.Println(" 2) passageiro (procura caronas)")
118
119         escolha, ok := c.ler("tipo: ")
120         if !ok {
121             return "", false

```

```

122 }
123
124 var tipo string
125 switch escolha {
126 case "1":
127     tipo = "motorista"
128 case "2":
129     tipo = "passageiro"
130 default:
131     fmt.Println(" escolha 1 ou 2")
132     return "", false
133 }
134
135 r, err := c.pedir(protocolo.Pedido{
136     Acao: "registrar",
137     Usuario: usuario,
138     Senha: senha,
139     Tipo: tipo,
140 })
141 if err != nil {
142     fmt.Println("conexao perdida:", err)
143     return "", false
144 }
145
146 if !r.OK {
147     fmt.Println(" erro:", r.Erro)
148     return "", false
149 }
150
151 fmt.Println(" ", r.Mensagem)
152
153 // Conta criada: entra direto, sem obrigar a digitar tudo de novo.
154 return c.autenticar(usuario, senha)
155 }
156
157 // autenticar manda o login e devolve o tipo do usuario.
158 func (c *cliente) autenticar(usuario, senha string) (string, bool) {
159     r, err := c.pedir(protocolo.Pedido{Acao: "login", Usuario: usuario, Senha: senha})
160     if err != nil {
161         fmt.Println("conexao perdida:", err)
162         return "", false
163     }
164
165     if !r.OK {
166         fmt.Println(" erro:", r.Erro)
167         return "", false
168     }
169
170     fmt.Println(" ", r.Mensagem)
171     return r.Tipo, true
172 }
173
174 // -----
175 // Menu do motorista
176 // -----
177
178 func (c *cliente) menuMotorista() {
179     for {
180         fmt.Println("\n1) cadastrar carona 2) minhas caronas 3) passageiros 4) cancelar carona
0) sair")
181
182         switch c.opcao() {
183         case "1":
184             c.cadastrarCarona()
185         case "2":
186             c.listar(protocolo.Pedido{Acao: "minhas_caronas"}, "voce ainda nao tem caronas")
187         case "3":

```

```

188         c.passageiros()
189     case "4":
190         c.cancelarCarona()
191     case "0", "":
192         return
193     default:
194         fmt.Println(" opcao invalida")
195     }
196 }
197 }
198
199 func (c *cliente) cadastrarCarona() {
200     texto, ok := c.ler("rota (cidades separadas por virgula): ")
201     if !ok {
202         return
203     }
204
205     // "Feira, Salvador, Ilheus" vira ["Feira" "Salvador" "Ilheus"].
206     var rota []string
207     for _, cidade := range strings.Split(texto, ",") {
208         cidade = strings.TrimSpace(cidade)
209         if cidade != "" {
210             rota = append(rota, cidade)
211         }
212     }
213
214     data, ok := c.ler("data (2026-09-14): ")
215     if !ok {
216         return
217     }
218
219     assentos, ok := c.lerNumero("assentos: ")
220     if !ok {
221         return
222     }
223
224     preco, ok := c.lerNumero("preco por trecho: ")
225     if !ok {
226         return
227     }
228
229     c.mostrar(c.pedir(protocolo.Pedido{
230         Acao:      "cadastrar",
231         Rota:       rota,
232         Data:       data,
233         Assentos:   assentos,
234         Preco:      preco,
235     })))
236 }
237
238 // passageiros mostra quem esta em cada trecho de uma carona.
239 func (c *cliente) passageiros() {
240     id, ok := c.lerNumero("numero da carona: ")
241     if !ok {
242         return
243     }
244
245     c.listar(protocolo.Pedido{Acao: "passageiros", CaronaID: id}, "carona sem dados")
246 }
247
248 // cancelarCarona pede confirmacao antes, porque desfaz as reservas de todos
249 // os passageiros daquela carona.
250 func (c *cliente) cancelarCarona() {
251     id, ok := c.lerNumero("numero da carona: ")
252     if !ok {
253         return
254     }

```

```

255
256     resposta, ok := c.ler(fmt.Sprintf("cancelar a carona %d? as reservas dos passageiros serao desfeitas
(s/n): ", id))
257     if !ok || strings.ToLower(resposta) != "s" {
258         fmt.Println(" nada foi cancelado")
259         return
260     }
261
262     c.mostrar(c.pedir(protocolo.Pedido{Acao: "cancelar_carona", CaronaID: id}))
263 }
264
265 // -----
266 // Menu do passageiro
267 // -----
268
269 func (c *cliente) menuPassageiro() {
270     for {
271         fmt.Println("\n1) buscar e reservar    2) minhas reservas    3) pagar    4) cancelar reserva    0)
sair")
272
273         switch c.opcao() {
274             case "1":
275                 c.buscarEReservar()
276             case "2":
277                 c.listar(protocolo.Pedido{Acao: "minhas_reservas"}, "voce ainda nao tem reservas")
278             case "3":
279                 c.pagar()
280             case "4":
281                 c.cancelarReserva()
282             case "0", "":
283                 return
284             default:
285                 fmt.Println(" opcao invalida")
286         }
287     }
288 }
289
290 func (c *cliente) buscarEReservar() {
291     origem, ok := c.ler("origem: ")
292     if !ok {
293         return
294     }
295
296     destino, ok := c.ler("destino: ")
297     if !ok {
298         return
299     }
300
301     data, ok := c.ler("data (2026-09-14): ")
302     if !ok {
303         return
304     }
305
306     r, err := c.pedir(protocolo.Pedido{
307         Acao:    "buscar",
308         Origem:  origem,
309         Destino: destino,
310         Data:    data,
311     })
312     if err != nil {
313         fmt.Println("conexao perdida:", err)
314         return
315     }
316
317     if !r.OK {
318         fmt.Println(" erro:", r.Erro)
319         return

```

```

320 }
321
322 if len(r.Opcoes) == 0 {
323     fmt.Println(" nenhuma carona encontrada")
324     return
325 }
326
327 // O servidor ja mandou o Resumo pronto: aqui so numeramos e imprimimos.
328 for i, o := range r.Opcoes {
329     fmt.Printf(" %d) %s | R$%d\n", i+1, o.Resumo, o.Preco)
330 }
331
332 escolha, ok := c.lerNumero("reservar qual? (0 cancela): ")
333 if !ok || escolha == 0 {
334     return
335 }
336
337 if escolha < 1 || escolha > len(r.Opcoes) {
338     fmt.Println(" opcao invalida")
339     return
340 }
341
342 c.mostrar(c.pedir(protocolo.Pedido{
343     Acao: "reservar",
344     Itens: r.Opcoes[escolha-1].Itens,
345 })))
346 }
347
348 func (c *cliente) pagar() {
349     id, ok := c.lerNumero("numero da reserva: ")
350     if !ok {
351         return
352     }
353
354     c.mostrar(c.pedir(protocolo.Pedido{Acao: "pagar", ReservaID: id}))
355 }
356
357 func (c *cliente) cancelarReserva() {
358     id, ok := c.lerNumero("numero da reserva: ")
359     if !ok {
360         return
361     }
362
363     c.mostrar(c.pedir(protocolo.Pedido{Acao: "cancelar_reserva", ReservaID: id}))
364 }
365
366 // -----
367 // Peças reaproveitadas
368 // -----
369
370 // pedir envia um Pedido e espera a Resposta. Toda conversa com o servidor
371 // passa por aqui.
372 func (c *cliente) pedir(p protocolo.Pedido) (protocolo.Resposta, error) {
373     if err := protocolo.EnviaPedido(c.conexao, p); err != nil {
374         return protocolo.Resposta{}, err
375     }
376     return protocolo.LerResposta(c.servidor)
377 }
378
379 // listar faz um pedido que devolve Linhas e imprime uma por uma.
380 func (c *cliente) listar(p protocolo.Pedido, seVazio string) {
381     r, err := c.pedir(p)
382     if err != nil {
383         fmt.Println("conexao perdida:", err)
384         return
385     }
386

```

```

387     if !r.OK {
388         fmt.Println(" erro:", r.Erro)
389         return
390     }
391
392     if len(r.Linhas) == 0 {
393         fmt.Println(" ", seVazio)
394         return
395     }
396
397     for _, linha := range r.Linhas {
398         fmt.Println(" " + linha)
399     }
400 }
401
402 // mostrar imprime o resultado de um pedir(). Recebe os DOIS retornos de uma
403 // vez, o que deixa as chamadas curtas: c.mostrar(c.pedir(...)).
404 func (c *cliente) mostrar(r protocolo.Resposta, err error) {
405     if err != nil {
406         fmt.Println("conexao perdida:", err)
407         return
408     }
409
410     if r.OK {
411         fmt.Println(" ", r.Mensagem)
412     } else {
413         fmt.Println(" erro:", r.Erro)
414     }
415 }
416
417 func (c *cliente) opcao() string {
418     escolha, ok := c.ler("> ")
419     if !ok {
420         return ""
421     }
422     return escolha
423 }
424
425 // ler mostra o rotulo e le uma linha do teclado. O segundo retorno e false
426 // quando a entrada acabou (ctrl+Z no Windows, ctrl+D no Linux).
427 func (c *cliente) ler(rotulo string) (string, bool) {
428     fmt.Print(rotulo)
429
430     texto, err := c.teclado.ReadString('\n')
431     if err != nil {
432         return "", false
433     }
434
435     // TrimSpace tira o \r\n do Enter. Sem isso "ping" viraria "ping\r\n" e
436     // nunca casaria com o case do servidor.
437     return strings.TrimSpace(texto), true
438 }
439
440 func (c *cliente) lerNumero(rotulo string) (int, bool) {
441     texto, ok := c.ler(rotulo)
442     if !ok {
443         return 0, false
444     }
445
446     n, err := strconv.Atoi(texto)
447     if err != nil {
448         fmt.Println(" precisa ser um numero")
449         return 0, false
450     }
451 }

```

```
452     return n, true
453 }
```

67. carga/main.go

pbl/carga/main.go · linhas 1–354

```
1  // Programa de teste de carga: N clientes REAIS, cada um com a sua conexao TCP,
2  // disputando os mesmos assentos ao mesmo tempo contra o servidor de verdade.
3  //
4  // Os testes do pacote dados chamam as funcoes direto. Aqui tudo passa pela
5  // rede: socket, protocolo JSON, goroutine por conexao no servidor e o mutex
6  // do Banco. E o teste que mostra o sistema inteiro aguentando concorrência.
7  //
8  // Com o servidor ja rodando:
9  //
10 //     go run ./carga
11 //     go run ./carga -clientes 500 -assentos 25
12 //     go run ./carga -servidor 192.168.0.10:8080
13 package main
14
15 import (
16     "bufio"
17     "flag"
18     "fmt"
19     "net"
20     "os"
21     "sort"
22     "strconv"
23     "sync"
24     "time"
25
26     "vaijunto/protocolo"
27 )
28
29 func main() {
30     padrao := "localhost:8080"
31     if v := os.Getenv("VAIJUNTO_SERVIDOR"); v != "" {
32         padrao = v
33     }
34
35     endereco := flag.String("servidor", padrao, "endereço do servidor")
36     clientes := flag.Int("clientes", 200, "quantos clientes disputam ao mesmo tempo")
37     assentos := flag.Int("assentos", 10, "lugares da carona mais apertada do itinerario")
38     flag.Parse()
39
40     if err := rodar(*endereco, *clientes, *assentos); err != nil {
41         fmt.Println("\nFALHOU:", err)
42         os.Exit(1)
43     }
44
45     fmt.Println("\nPASSOU")
46 }
47
48 // -----
49 // 0 cenário
50 // -----
51
52 func rodar(endereco string, clientes, assentos int) error {
53     // Sufixo unico: a data e os logins desta rodada nao colidem com nada que
54     // ja exista no servidor, entao o teste pode rodar varias vezes seguidas.
55     tag := strconv.FormatInt(time.Now().UnixNano(), 36)
```

```

56 data := "carga-" + tag
57
58 fmt.Printf("servidor %s | %d clientes | %d lugares disputados\n\n", endereco, clientes, assentos)
59
60 // ---- 1. O motorista monta o cenario ----
61
62 motorista, err := conectar(endereco)
63 if err != nil {
64     return fmt.Errorf("o motorista nao conectou: %w", err)
65 }
66 defer motorista.Close()
67
68 if err := entrar(motorista, "mot_"+tag, "motorista"); err != nil {
69     return err
70 }
71
72 // A primeira carona tem o DOBRO de lugares da segunda. Se a atomicidade
73 // falhar -- uma reserva ocupar a primeira sem ocupar a segunda -- sobra
74 // lugar ocupado na primeira sem par, e a verificacao do passo 4 enxerga.
75 a, err := motorista.exigir(protocolo.Pedido{
76     Acao: "cadastrar", Rota: []string{"Feira", "Salvador"},
77     Data: data, Assentos: 2 * assentos, Preco: 30,
78 })
79 if err != nil {
80     return err
81 }
82
83 b, err := motorista.exigir(protocolo.Pedido{
84     Acao: "cadastrar", Rota: []string{"Salvador", "Ilheus"},
85     Data: data, Assentos: assentos, Preco: 40,
86 })
87 if err != nil {
88     return err
89 }
90
91 // O itinerario disputado usa as DUAS caronas: tudo ou nada.
92 itinerario := []protocolo.Item{
93     {CaronaID: a.CaronaID, De: 0, Ate: 1},
94     {CaronaID: b.CaronaID, De: 0, Ate: 1},
95 }
96
97 fmt.Printf("itinerario: carona %d Feira->Salvador (%d lugares) + carona %d Salvador->Ilheus (%d
lugares)\n",
98     a.CaronaID, 2*assentos, b.CaronaID, assentos)
99
100 // ---- 2. Os clientes conectam, fazem login e disparam juntos ----
101
102 resultados := make(chan resultado, clientes)
103 largada := make(chan struct{})
104
105 // A preparacao (conectar + login) roda com no maximo 50 clientes por vez.
106 //
107 // O que este teste mede e a DISPUTA pela reserva, e ela continua com todos
108 // juntos, depois da largada. Ja abrir centenas de conexoes no mesmo
109 // milissegundo so estoura a fila de conexoes pendentes do sistema
110 // operacional, que tem tamanho limitado e recusa o excedente antes de o
111 // servidor sequer ver a conexao.
112 preparando := make(chan struct{}, 50)
113
114 var prontos, terminados sync.WaitGroup
115
116 for i := 0; i < clientes; i++ {
117     prontos.Add(1)
118     terminados.Add(1)
119
120     go func(n int) {
121         defer terminados.Done()

```

```

122
123 // Preparacao: cada cliente tem a SUA conexao e o SEU usuario.
124 preparando <- struct{}{} // pega uma das 50 vagas
125 c, err := conectar(endereco)
126 if err == nil {
127     defer c.Close()
128     err = entrar(c, fmt.Sprintf("pas_%s_%d", tag, n), "passageiro")
129 }
130 <-preparando // devolve a vaga
131
132 prontos.Done()
133 <-largada // espera todos estarem prontos
134
135 if err != nil {
136     resultados <- resultado{erro: err}
137     return
138 }
139
140 inicio := time.Now()
141 r, err := c.pedir(protocolo.Pedido{Acao: "reservar", Itens: itinerario})
142 latencia := time.Since(inicio)
143
144 switch {
145 case err != nil:
146     resultados <- resultado{erro: err}
147 case !r.OK:
148     resultados <- resultado{recusado: true, latencia: latencia}
149 default:
150     resultados <- resultado{reservaID: r.ReservaID, latencia: latencia}
151 }
152 }(i)
153 }
154
155 prontos.Wait()
156 fmt.Printf("%d clientes conectados e logados; disparando todos juntos...\n\n", clientes)
157
158 inicio := time.Now()
159 close(largada)
160 terminados.Wait()
161 duracao := time.Since(inicio)
162 close(resultados)
163
164 // ---- 3. Contagem ----
165
166 var latencias []time.Duration
167 ids := map[int]bool{}
168 aceitas, recusadas, falhas := 0, 0, 0
169 var primeiraFalha error
170
171 for r := range resultados {
172     switch {
173     case r.erro != nil:
174         falhas++
175         if primeiraFalha == nil {
176             primeiraFalha = r.erro
177         }
178         continue
179     case r.recusado:
180         recusadas++
181     default:
182         aceitas++
183         ids[r.reservaID] = true
184     }
185     latencias = append(latencias, r.latencia)
186 }
187
188 fmt.Printf("reservas aceitas: %d\n", aceitas)

```

```

189     fmt.Printf("reservas recusadas: %d (sem lugar: e o esperado)\n", recusadas)
190     fmt.Printf("falhas de conexao: %d\n", falhas)
191     if primeiraFalha != nil {
192         fmt.Println(" primeira falha:", primeiraFalha)
193     }
194     fmt.Printf("tempo da disputa:  %s\n", duracao.Round(time.Millisecond))
195     imprimirLatencias(latencias)
196
197     // ---- 4. Verificacao ----
198
199     fmt.Println("\nverificacao:")
200     v := &verificacao{}
201
202     v.conferir("nenhum lugar vendido duas vezes",
203         aceitas <= assentos,
204         fmt.Sprintf("%d reservas aceitas para %d lugares", aceitas, assentos))
205
206     esperadas := min(assentos, clientes-falhas)
207     v.conferir("todos os lugares disponiveis foram vendidos",
208         aceitas == esperadas,
209         fmt.Sprintf("esperava %d, vieram %d", esperadas, aceitas))
210
211     v.conferir("cada reserva aceita tem numero unico",
212         len(ids) == aceitas,
213         fmt.Sprintf("%d reservas mas so %d numeros diferentes", aceitas, len(ids)))
214
215     // Atomicidade: uma sonda conta, pelo proprio protocolo, quantos lugares
216     // sobraram em cada carona. Se toda reserva aceita ocupou as DUAS, sobra
217     // exatamente (lugares - aceitas) em cada uma.
218     sonda, err := conectar(endereco)
219     if err != nil {
220         return fmt.Errorf("a sonda nao conectou: %w", err)
221     }
222     defer sonda.Close()
223
224     if err := entrar(sonda, "sonda_"+tag, "passageiro"); err != nil {
225         return err
226     }
227
228     sobraB, errB := contarVagas(sonda, b.CaronaID, assentos)
229     sobraA, errA := contarVagas(sonda, a.CaronaID, 2*assentos)
230
231     v.conferir("nenhum itinerario pela metade",
232         errA == nil && errB == nil && sobraA == 2*assentos-aceitas && sobraB == assentos-aceitas,
233         fmt.Sprintf("sobraram %d na primeira (esperado %d) e %d na segunda (esperado %d)",
234             sobraA, 2*assentos-aceitas, sobraB, assentos-aceitas))
235
236     // Limpeza: cancelar as caronas desfaz tudo que esta rodada criou.
237     for _, id := range []int{a.CaronaID, b.CaronaID} {
238         motorista.pedir(protocolo.Pedido{Acao: "cancelar_carona", CaronaID: id})
239     }
240
241     if v.falhou {
242         return fmt.Errorf("alguma verificacao nao passou")
243     }
244     return nil
245 }
246
247 // contarVagas descobre quantos lugares sobraram numa carona reservando um por
248 // um ate o servidor recusar. Usa so o protocolo, sem olhar dentro do servidor.
249 func contarVagas(c *conexao, caronaID, limite int) (int, error) {
250     item := []protocolo.Item{{CaronaID: caronaID, De: 0, Ate: 1}}
251
252     for vagas := 0; vagas <= limite; vagas++ {
253         r, err := c.pedir(protocolo.Pedido{Acao: "reservar", Itens: item})
254         if err != nil {
255             return vagas, err

```

```

256     }
257     if !r.OK {
258         return vagas, nil
259     }
260 }
261
262 // Mais vagas do que a carona tem: o estado do servidor esta corrompido.
263 return limite + 1, nil
264 }
265
266 // -----
267 // Pecas de apoio
268 // -----
269
270 // conexao e um cliente minimo: o socket e o leitor com buffer dele.
271 type conexao struct {
272     net.Conn
273     leitor *bufio.Reader
274 }
275
276 func conectar(endereco string) (*conexao, error) {
277     c, err := net.Dial("tcp", endereco)
278     if err != nil {
279         return nil, err
280     }
281     return &conexao{Conn: c, leitor: bufio.NewReader(c)}, nil
282 }
283
284 func (c *conexao) pedir(p protocolo.Pedido) (protocolo.Resposta, error) {
285     if err := protocolo.EnviaPedido(c, p); err != nil {
286         return protocolo.Resposta{}, err
287     }
288     return protocolo.LerResposta(c.leitor)
289 }
290
291 // exigir e o pedir das etapas de preparacao: ali uma recusa do servidor
292 // tambem e erro, porque o teste nao consegue seguir sem aquilo.
293 func (c *conexao) exigir(p protocolo.Pedido) (protocolo.Resposta, error) {
294     r, err := c.pedir(p)
295     if err != nil {
296         return r, err
297     }
298     if !r.OK {
299         return r, fmt.Errorf("%s recusado: %s", p.Acao, r.Erro)
300     }
301     return r, nil
302 }
303
304 // entrar cria um usuario novo e faz login com ele na mesma conexao.
305 func entrar(c *conexao, login, tipo string) error {
306     if _, err := c.exigir(protocolo.Pedido{Acao: "registrar", Usuario: login, Senha: "carga", Tipo:
307 tipo}); err != nil {
308         return err
309     }
310     _, err := c.exigir(protocolo.Pedido{Acao: "login", Usuario: login, Senha: "carga"})
311     return err
312 }
313
314 type resultado struct {
315     reservaID int
316     recusado bool
317     erro error
318     latencia time.Duration
319 }
320
321 type verificacao struct {
322     falhou bool

```

```

322 }
323
324 func (v *verificacao) conferir(nome string, ok bool, detalhe string) {
325     if ok {
326         fmt.Printf(" [ok]      %s\n", nome)
327         return
328     }
329     fmt.Printf(" [FALHOU] %s: %s\n", nome, detalhe)
330     v.falhou = true
331 }
332
333 func imprimirLatencias(l []time.Duration) {
334     if len(l) == 0 {
335         return
336     }
337
338     sort.Slice(l, func(i, j int) bool { return l[i] < l[j] })
339
340     var soma time.Duration
341     for _, d := range l {
342         soma += d
343     }
344
345     percentil := func(p float64) time.Duration {
346         return l[int(p*float64(len(l)-1))]
347     }
348
349     arred := func(d time.Duration) time.Duration { return d.Round(100 * time.Microsecond) }
350
351     fmt.Printf("latencia da reserva: min %s | media %s | p50 %s | p95 %s | max %s\n",
352         arred(l[0]), arred(soma/time.Duration(len(l))), arred(percentil(0.50)),
353         arred(percentil(0.95)), arred(l[len(l)-1]))
354 }

```

68. dados/dados_test.go

pbl/dados/dados_test.go · linhas 1–561

```

1  package dados
2
3  import (
4      "fmt"
5      "sync"
6      "testing"
7      "time"
8
9      "vaijunto/protocolo"
10 )
11
12 // trecho monta o item "carona X, da cidade De ate a cidade Ate".
13 func trecho(caronaID, de, ate int) []protocolo.Item {
14     return []protocolo.Item{{CaronaID: caronaID, De: de, Ate: ate}}
15 }
16
17 // bancoComCarona devolve um banco novo com uma carona ja cadastrada.
18 func bancoComCarona(t *testing.T, rota []string, assentos int) (*Banco, int) {
19     t.Helper()
20
21     b := NovoBanco()
22     id, err := b.CadastrarCarona("ana", rota, "2026-09-14", assentos, 30)
23     if err != nil {
24         t.Fatalf("nao consegui cadastrar a carona: %v", err)

```

```

25 }
26
27 return b, id
28 }
29
30 // -----
31 // O TESTE PRINCIPAL: concorrencia
32 // -----
33
34 // TestReservaConcorrente e o teste que prova o tratamento de concorrencia.
35 //
36 // Uma carona com UM assento e 50 goroutines tentando reservar ao mesmo tempo.
37 // So uma pode vencer. Sem o mutex em Reservar, varias leriam "1 livre" antes
38 // de qualquer uma escrever, e o sistema venderia o mesmo assento varias vezes.
39 //
40 // Rode com o detector de corrida:
41 //
42 // go test -race ./...
43 //
44 // Para ver o teste falhar de proposito, comente o b.mu.Lock() de Reservar.
45 func TestReservaConcorrente(t *testing.T) {
46     const tentativas = 200
47
48     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
49
50     // Canal com espaco para todos: cada goroutine que conseguir reservar
51     // joga um aviso aqui. Usar canal evita precisar de outro mutex so para
52     // contar os sucessos.
53     sucessos := make(chan int, tentativas)
54
55     // WaitGroup e o contador que espera todas as goroutines terminarem.
56     var wg sync.WaitGroup
57
58     // largada trava todas as goroutines no mesmo ponto. Sem isso, a primeira
59     // a nascer ja teria reservado antes de a ultima existir, e nao haveria
60     // disputa nenhuma para testar.
61     largada := make(chan struct{})
62
63     for i := 0; i < tentativas; i++ {
64         wg.Add(1)
65
66         go func() {
67             defer wg.Done()
68
69             <-largada // espera aqui ate o canal ser fechado
70
71             if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err == nil {
72                 sucessos <- 1
73             }
74         }()
75     }
76
77     close(largada) // solta as 50 de uma vez
78     wg.Wait()     // espera todas terminarem
79     close(sucessos)
80
81     if n := len(sucessos); n != 1 {
82         t.Errorf("esperava exatamente 1 reserva bem sucedida, deu %d", n)
83     }
84
85     c := b.acharCarona(id)
86     if c.Ocupados[0] != 1 {
87         t.Errorf("esperava o trecho com 1 assento ocupado, deu %d", c.Ocupados[0])
88     }
89
90     if n := len(b.Reservas); n != 1 {
91         t.Errorf("esperava 1 reserva registrada, deu %d", n)

```

```

92 }
93 }
94
95 // TestContagemDeAssentos e o teste que pega a falta do mutex de forma
96 // CONFIÁVEL.
97 //
98 // O teste acima (1 assento, muitas goroutines) e a garantia de negocio, mas
99 // sozinho ele nao detecta bem o bug: quem largar primeiro ja ocupou o assento
100 // antes de as outras nascerem, e todas as demais falham "corretamente".
101 //
102 // Aqui e diferente: sao N assentos para N goroutines, entao TODAS entram na
103 // secao critica e TODAS escrevem. Sem o mutex, dois "Ocupados[t]++"
104 // simultaneos viram um so (o classico "lost update") e a contagem final fica
105 // MENOR que N.
106 func TestContagemDeAssentos(t *testing.T) {
107     const pedidos = 2000
108
109     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, pedidos)
110
111     var wg sync.WaitGroup
112     largada := make(chan struct{})
113
114     for i := 0; i < pedidos; i++ {
115         wg.Add(1)
116         go func() {
117             defer wg.Done()
118             <-largada
119             if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
120                 t.Errorf("reserva deveria funcionar, sobra assento: %v", err)
121             }
122         }()
123     }
124
125     close(largada)
126     wg.Wait()
127
128     if n := b.acharCarona(id).Ocupados[0]; n != pedidos {
129         t.Errorf("perdi incrementos: Ocupados = %d, esperado %d", n, pedidos)
130     }
131
132     if n := len(b.Reservas); n != pedidos {
133         t.Errorf("perdi reservas: %d registradas, esperado %d", n, pedidos)
134     }
135 }
136
137 // TestExpiracaoConcorrente roda a expiracao ao mesmo tempo que as reservas.
138 // Sao duas goroutines mexendo no MESMO contador de assentos por caminhos
139 // diferentes -- exatamente o cenario do servidor de verdade.
140 func TestExpiracaoConcorrente(t *testing.T) {
141     original := TempoDeReserva
142     TempoDeReserva = time.Millisecond
143     defer func() { TempoDeReserva = original }()
144
145     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 5)
146
147     var wg sync.WaitGroup
148
149     for i := 0; i < 30; i++ {
150         wg.Add(1)
151         go func() {
152             defer wg.Done()
153             b.Reservar("bruno", trecho(id, 0, 1))
154         }()
155     }
156
157     for i := 0; i < 30; i++ {
158         wg.Add(1)

```

```

159     go func() {
160         defer wg.Done()
161         b.ExpirarVencidas()
162     }()
163 }
164
165 wg.Wait()
166
167 // O numero final depende do sorteio das goroutines, mas o estado tem que
168 // ser COERENTE: nunca negativo, nunca acima do total de assentos.
169 c := b.acharCarona(id)
170 if c.Ocupados[0] < 0 || c.Ocupados[0] > c.Assentos {
171     t.Errorf("contador de assentos incoerente: %d (total %d)", c.Ocupados[0], c.Assentos)
172 }
173 }
174
175 // -----
176 // Atomicidade
177 // -----
178
179 // TestReservaAtomica confere o "pegou um trecho nao fica sem o outro".
180 // Se o segundo item falhar, o primeiro NAO pode ter sido aplicado.
181 func TestReservaAtomica(t *testing.T) {
182     b, livre := bancoComCarona(t, []string{"Ilheus", "Porto Seguro"}, 2)
183
184     cheia, err := b.CadastrarCarona("davi", []string{"Feira", "Salvador"}, "2026-09-14", 1, 30)
185     if err != nil {
186         t.Fatalf("nao consegui cadastrar: %v", err)
187     }
188
189     // Enche a segunda carona.
190     if _, err := b.Reservar("carla", trecho(cheia, 0, 1)); err != nil {
191         t.Fatalf("a primeira reserva deveria funcionar: %v", err)
192     }
193
194     // Agora pede as duas juntas. A carona livre vem PRIMEIRO de proposito:
195     // se a implementacao aplicasse item a item, ela ja teria sido ocupada
196     // quando a segunda falhasse.
197     _, err = b.Reservar("bruno", []protocolo.Item{
198         {CaronaID: livre, De: 0, Ate: 1},
199         {CaronaID: cheia, De: 0, Ate: 1},
200     })
201     if err == nil {
202         t.Fatal("a reserva deveria falhar: a segunda carona esta cheia")
203     }
204
205     if ocupados := b.acharCarona(livre).Ocupados[0]; ocupados != 0 {
206         t.Errorf("a carona livre foi alterada apesar da falha: ocupados = %d", ocupados)
207     }
208 }
209
210 // TestReservaMultiplaFunciona garante que o caso feliz de duas caronas
211 // tambem funciona -- senao o teste acima passaria com um Reservar que
212 // simplesmente recusa tudo.
213 func TestReservaMultiplaFunciona(t *testing.T) {
214     b, primeira := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
215
216     segunda, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 1, 40)
217     if err != nil {
218         t.Fatalf("nao consegui cadastrar: %v", err)
219     }
220
221     if _, err := b.Reservar("bruno", []protocolo.Item{
222         {CaronaID: primeira, De: 0, Ate: 1},
223         {CaronaID: segunda, De: 0, Ate: 1},
224     }); err != nil {
225         t.Fatalf("a reserva das duas deveria funcionar: %v", err)

```

```

226 }
227
228 if n := b.acharCarona(primeira).Ocupados[0]; n != 1 {
229     t.Errorf("primeira carona: esperava 1 ocupado, deu %d", n)
230 }
231 if n := b.acharCarona(segunda).Ocupados[0]; n != 1 {
232     t.Errorf("segunda carona: esperava 1 ocupado, deu %d", n)
233 }
234 }
235
236 // -----
237 // Disponibilidade por trecho
238 // -----
239
240 // TestDisponibilidadePorTrecho e o comportamento que diferencia o sistema:
241 // ocupar um trecho NAO ocupa a carona inteira.
242 func TestDisponibilidadePorTrecho(t *testing.T) {
243     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 1)
244
245     // Reserva so o primeiro trecho.
246     if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
247         t.Fatalf("deveria funcionar: %v", err)
248     }
249
250     casos := []struct {
251         nome          string
252         origem, destino string
253         querOpcoes     int
254     }{
255         {"o trecho seguinte continua livre", "Salvador", "Ilheus", 1},
256         {"o trecho reservado ficou cheio", "Feira", "Salvador", 0},
257         {"a viagem inteira precisa dos dois", "Feira", "Ilheus", 0},
258     }
259
260     for _, caso := range casos {
261         opcoes := b.Buscar(caso.origem, caso.destino, "2026-09-14")
262         if len(opcoes) != caso.querOpcoes {
263             t.Errorf("%s: %s->%s deu %d opcao(oes), queria %d",
264                 caso.nome, caso.origem, caso.destino, len(opcoes), caso.querOpcoes)
265         }
266     }
267 }
268
269 // TestBuscaSentidoUnico: a carona so anda para frente.
270 func TestBuscaSentidoUnico(t *testing.T) {
271     b, _ := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 2)
272
273     if n := len(b.Buscar("Ilheus", "Feira", "2026-09-14")); n != 0 {
274         t.Errorf("sentido contrario deveria dar 0 opcoes, deu %d", n)
275     }
276     if n := len(b.Buscar("Salvador", "Salvador", "2026-09-14")); n != 0 {
277         t.Errorf("origem igual ao destino deveria dar 0 opcoes, deu %d", n)
278     }
279     if n := len(b.Buscar("Feira", "Recife", "2026-09-14")); n != 0 {
280         t.Errorf("cidade fora da rota deveria dar 0 opcoes, deu %d", n)
281     }
282     if n := len(b.Buscar("Feira", "Ilheus", "2026-12-25")); n != 0 {
283         t.Errorf("data diferente deveria dar 0 opcoes, deu %d", n)
284     }
285 }
286
287 // TestBuscaComConexao: duas caronas que juntas levam o passageiro ao destino.
288 func TestBuscaComConexao(t *testing.T) {
289     b, _ := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
290
291     if _, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 2, 40); err !=
nil {

```

```

292     t.Fatalf("nao consegui cadastrar: %v", err)
293 }
294
295 opcoes := b.Buscar("Feira", "Ilheus", "2026-09-14")
296 if len(opcoes) != 1 {
297     t.Fatalf("esperava 1 opcao com conexao, deu %d", len(opcoes))
298 }
299
300 if n := len(opcoes[0].Itens); n != 2 {
301     t.Errorf("a opcao deveria ter 2 itens, tem %d", n)
302 }
303
304 // 30 do primeiro trecho + 40 do segundo.
305 if opcoes[0].Preco != 70 {
306     t.Errorf("preco esperado 70, deu %d", opcoes[0].Preco)
307 }
308 }
309
310 // -----
311 // Expiracao e pagamento
312 // -----
313
314 // TestExpiracaoDevolveAssento: quem nao paga perde o assento.
315 func TestExpiracaoDevolveAssento(t *testing.T) {
316     original := TempoDeReserva
317     TempoDeReserva = time.Millisecond
318     defer func() { TempoDeReserva = original }()
319
320     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
321
322     if _, err := b.Reservar("bruno", trecho(id, 0, 1)); err != nil {
323         t.Fatalf("deveria funcionar: %v", err)
324     }
325
326     // Antes de expirar, ninguem mais consegue.
327     if _, err := b.Reservar("carla", trecho(id, 0, 1)); err == nil {
328         t.Fatal("carla nao deveria conseguir: o assento esta ocupado")
329     }
330
331     time.Sleep(5 * time.Millisecond)
332
333     if n := b.ExpirarVencidas(); n != 1 {
334         t.Fatalf("esperava 1 reserva expirada, deu %d", n)
335     }
336
337     // Agora sim.
338     if _, err := b.Reservar("carla", trecho(id, 0, 1)); err != nil {
339         t.Errorf("carla deveria conseguir depois da expiracao: %v", err)
340     }
341 }
342
343 // TestReservaPagaNaoExpira: pagou, o assento e seu para sempre.
344 func TestReservaPagaNaoExpira(t *testing.T) {
345     original := TempoDeReserva
346     TempoDeReserva = time.Millisecond
347     defer func() { TempoDeReserva = original }()
348
349     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 1)
350
351     reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
352     if err != nil {
353         t.Fatalf("deveria funcionar: %v", err)
354     }
355
356     if err := b.Pagar("bruno", reserva); err != nil {
357         t.Fatalf("o pagamento deveria funcionar: %v", err)
358     }

```

```

359
360     time.Sleep(5 * time.Millisecond)
361
362     if n := b.ExpirarVencidas(); n != 0 {
363         t.Errorf("reserva paga nao pode expirar, mas expirou %d", n)
364     }
365
366     if _, err := b.Reservar("carla", trecho(id, 0, 1)); err == nil {
367         t.Error("o assento deveria continuar ocupado")
368     }
369 }
370
371 // TestRegrasDePagamento cobre os erros possiveis ao pagar.
372 func TestRegrasDePagamento(t *testing.T) {
373     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
374
375     reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
376     if err != nil {
377         t.Fatalf("deveria funcionar: %v", err)
378     }
379
380     if err := b.Pagar("carla", reserva); err == nil {
381         t.Error("carla nao deveria pagar reserva do bruno")
382     }
383     if err := b.Pagar("bruno", 999); err == nil {
384         t.Error("reserva inexistente deveria dar erro")
385     }
386     if err := b.Pagar("bruno", reserva); err != nil {
387         t.Errorf("o dono deveria conseguir pagar: %v", err)
388     }
389     if err := b.Pagar("bruno", reserva); err == nil {
390         t.Error("pagar duas vezes deveria dar erro")
391     }
392 }
393
394 // -----
395 // Login e cadastro
396 // -----
397
398 func TestAutenticar(t *testing.T) {
399     b := NovoBanco()
400
401     casos := []struct {
402         nome, login, senha string
403         quer                bool
404     }{
405         {"usuario e senha certos", "ana", "123", true},
406         {"senha errada", "ana", "outra", false},
407         {"usuario inexistente", "ninguem", "123", false},
408         {"senha vazia", "ana", "", false},
409     }
410
411     for _, caso := range casos {
412         if _, ok := b.Autenticar(caso.login, caso.senha); ok != caso.quer {
413             t.Errorf("%s: Autenticar(%q, %q) deu %v, queria %v",
414                 caso.nome, caso.login, caso.senha, ok, caso.quer)
415         }
416     }
417 }
418
419 func TestCadastrarUsuario(t *testing.T) {
420     b := NovoBanco()
421
422     if err := b.CadastrarUsuario("henrick", "senha1", "motorista"); err != nil {
423         t.Fatalf("deveria funcionar: %v", err)
424     }
425 }

```

```

426 u, ok := b.Autenticar("henrick", "senha1")
427 if !ok {
428     t.Fatal("deveria conseguir entrar com a conta recém-criada")
429 }
430 if u.Tipo != "motorista" {
431     t.Errorf("tipo esperado motorista, deu %q", u.Tipo)
432 }
433
434 casos := []struct {
435     nome, login, senha, tipo string
436 }{
437     {"login repetido", "henrick", "outra", "passageiro"},
438     {"login que ja vem no banco", "ana", "x", "passageiro"},
439     {"login vazio", "", "senha", "passageiro"},
440     {"login so com espacos", " ", "senha", "passageiro"},
441     {"senha vazia", "novo", "", "passageiro"},
442     {"tipo invalido", "novo", "senha", "piloto"},
443     {"tipo vazio", "novo", "senha", ""},
444 }
445
446 for _, caso := range casos {
447     if err := b.CadastrarUsuario(caso.login, caso.senha, caso.tipo); err == nil {
448         t.Errorf("%s: deveria dar erro, mas passou", caso.nome)
449     }
450 }
451
452 // O cadastro repetido nao pode ter trocado a senha do dono original.
453 if _, ok := b.Autenticar("henrick", "senha1"); !ok {
454     t.Error("a tentativa repetida sobrescreveu o usuario original")
455 }
456 }
457
458 // TestCadastroConcorrente: 500 goroutines tentando criar O MESMO login.
459 // So uma pode vencer. Se a checagem "ja existe" estivesse fora do Lock,
460 // varias passariam pela verificacao antes de qualquer uma escrever.
461 func TestCadastroConcorrente(t *testing.T) {
462     const tentativas = 500
463
464     b := NovoBanco()
465
466     sucessos := make(chan int, tentativas)
467     largada := make(chan struct{})
468     var wg sync.WaitGroup
469
470     for i := 0; i < tentativas; i++ {
471         wg.Add(1)
472         go func() {
473             defer wg.Done()
474             <-largada
475             if err := b.CadastrarUsuario("disputado", "senha", "passageiro"); err == nil {
476                 sucessos <- 1
477             }
478         }()
479     }
480
481     close(largada)
482     wg.Wait()
483     close(sucessos)
484
485     if n := len(sucessos); n != 1 {
486         t.Errorf("esperava exatamente 1 cadastro bem sucedido, deu %d", n)
487     }
488 }
489
490 // TestCadastrosSimultaneos e o par do TestContagemDeAssentos, agora para o
491 // mapa de usuarios: 300 logins DIFERENTES criados ao mesmo tempo.
492 //

```

```

493 // Como cada goroutine escreve mesmo (nenhuma e recusada), todas mexem no mapa
494 // em paralelo. Sem o Lock, o proprio runtime do Go derruba o programa com
495 // "fatal error: concurrent map writes" -- mapa nao suporta escrita simultanea.
496 func TestCadastrosSimultaneos(t *testing.T) {
497     const quantos = 300
498
499     b := NovoBanco()
500     iniciais := len(b.Usuarios)
501
502     var wg sync.WaitGroup
503     largada := make(chan struct{})
504
505     for i := 0; i < quantos; i++ {
506         wg.Add(1)
507         go func(n int) {
508             defer wg.Done()
509             <-largada
510             login := fmt.Sprintf("usuario%d", n)
511             if err := b.CadastrarUsuario(login, "senha", "passageiro"); err != nil {
512                 t.Errorf("cadastro de %s falhou: %v", login, err)
513             }
514         }(i)
515     }
516
517     close(largada)
518     wg.Wait()
519
520     if n := len(b.Usuarios); n != iniciais+quantos {
521         t.Errorf("esperava %d usuarios, deu %d", iniciais+quantos, n)
522     }
523 }
524
525 func TestCadastroInvalido(t *testing.T) {
526     b := NovoBanco()
527
528     casos := []struct {
529         nome      string
530         rota      []string
531         data      string
532         assentos  int
533     }{
534         {"rota com uma cidade so", []string{"Feira"}, "2026-09-14", 2},
535         {"rota vazia", nil, "2026-09-14", 2},
536         {"sem assentos", []string{"Feira", "Salvador"}, "2026-09-14", 0},
537         {"sem data", []string{"Feira", "Salvador"}, "", 2},
538     }
539
540     for _, caso := range casos {
541         if _, err := b.CadastrarCarona("ana", caso.rota, caso.data, caso.assentos, 30); err == nil {
542             t.Errorf("%s: deveria dar erro, mas passou", caso.nome)
543         }
544     }
545 }
546
547 // TestOcupadosNasceZero: uma rota de N cidades tem N-1 contadores.
548 func TestOcupadosNasceZero(t *testing.T) {
549     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus", "Porto Seguro"}, 3)
550
551     c := b.acharCarona(id)
552     if len(c.Ocupados) != 3 {
553         t.Fatalf("rota de 4 cidades deveria ter 3 trechos, tem %d", len(c.Ocupados))
554     }
555
556     for i, n := range c.Ocupados {
557         if n != 0 {
558             t.Errorf("trecho %d deveria nascer zerado, esta em %d", i, n)
559         }
560     }
561 }

```

```
560 }
561 }
```

69. dados/cancelamento_test.go

pbl/dados/cancelamento_test.go · linhas 1–278

```
1 package dados
2
3 import (
4     "strings"
5     "sync"
6     "testing"
7     "time"
8
9     "vaijunto/protocolo"
10 )
11
12 // -----
13 // Cancelar reserva
14 // -----
15
16 func TestCancelarReservaDevolveAssento(t *testing.T) {
17     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 1)
18
19     reserva, err := b.Reservar("bruno", trecho(id, 0, 2))
20     if err != nil {
21         t.Fatalf("deveria funcionar: %v", err)
22     }
23
24     if err := b.CancelarReserva("bruno", reserva); err != nil {
25         t.Fatalf("o cancelamento deveria funcionar: %v", err)
26     }
27
28     for i, n := range b.acharCarona(id).Ocupados {
29         if n != 0 {
30             t.Errorf("trecho %d deveria voltar a 0, esta em %d", i, n)
31         }
32     }
33
34     if _, err := b.Reservar("carla", trecho(id, 0, 2)); err != nil {
35         t.Errorf("carla deveria conseguir o assento devolvido: %v", err)
36     }
37 }
38
39 func TestRegrasDeCancelarReserva(t *testing.T) {
40     original := TempoDeReserva
41     TempoDeReserva = time.Millisecond
42     defer func() { TempoDeReserva = original }()
43
44     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 3)
45
46     paga, _ := b.Reservar("bruno", trecho(id, 0, 1))
47     if err := b.Pagar("bruno", paga); err != nil {
48         t.Fatalf("o pagamento deveria funcionar: %v", err)
49     }
50
51     vencida, _ := b.Reservar("bruno", trecho(id, 0, 1))
52     time.Sleep(5 * time.Millisecond)
53     b.ExpirarVencidas()
54
55     if err := b.CancelarReserva("carla", paga); err == nil {
```

```

56     t.Error("carla nao pode cancelar reserva do bruno")
57 }
58 if err := b.CancelarReserva("bruno", 999); err == nil {
59     t.Error("reserva inexistente deveria dar erro")
60 }
61 if err := b.CancelarReserva("bruno", vencida); err == nil {
62     t.Error("reserva expirada nao pode ser cancelada")
63 }
64 if err := b.CancelarReserva("bruno", paga); err != nil {
65     t.Errorf("reserva paga pode ser cancelada: %v", err)
66 }
67 if err := b.CancelarReserva("bruno", paga); err == nil {
68     t.Error("cancelar duas vezes deveria dar erro")
69 }
70 if err := b.Pagar("bruno", paga); err == nil {
71     t.Error("reserva cancelada nao pode ser paga")
72 }
73
74 // A expirada ja tinha devolvido o assento dela. Se o cancelamento
75 // devolvesse de novo, o contador ficaria NEGATIVO.
76 if n := b.acharCarona(id).Ocupados[0]; n != 0 {
77     t.Errorf("esperava 0 assentos ocupados, deu %d", n)
78 }
79 }
80
81 // -----
82 // Cancelar carona
83 // -----
84
85 // TestCancelarCaronaCancelaConexaoInteira e o caso que exige cuidado: a
86 // reserva do bruno usa duas caronas. Quando uma delas e cancelada, a reserva
87 // inteira cai, e o assento dele na OUTRA carona tambem precisa voltar.
88 func TestCancelarCaronaCancelaConexaoInteira(t *testing.T) {
89     b, primeira := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
90
91     segunda, err := b.CadastrarCarona("davi", []string{"Salvador", "Ilheus"}, "2026-09-14", 2, 40)
92     if err != nil {
93         t.Fatalf("nao consegui cadastrar: %v", err)
94     }
95
96     conexao, err := b.Reservar("bruno", []protocolo.Item{
97         {CaronaID: primeira, De: 0, Ate: 1},
98         {CaronaID: segunda, De: 0, Ate: 1},
99     })
100     if err != nil {
101         t.Fatalf("a reserva com conexao deveria funcionar: %v", err)
102     }
103
104     direta, err := b.Reservar("carla", trecho(primeira, 0, 1))
105     if err != nil {
106         t.Fatalf("a reserva direta deveria funcionar: %v", err)
107     }
108
109     afetadas, err := b.CancelarCarona("davi", segunda)
110     if err != nil {
111         t.Fatalf("o cancelamento deveria funcionar: %v", err)
112     }
113     if afetadas != 1 {
114         t.Errorf("esperava 1 reserva afetada, deu %d", afetadas)
115     }
116
117     if r := b.acharReserva(conexao); r.Estado != "cancelada" {
118         t.Errorf("a reserva com conexao deveria estar cancelada, esta %q", r.Estado)
119     }
120     if r := b.acharReserva(direta); r.Estado != "pendente" {
121         t.Errorf("a reserva da carla nao usa a carona cancelada, deveria seguir pendente, esta %q",
122 r.Estado)

```

```

122 }
123
124 // Na primeira carona so pode sobrar o assento da carla.
125 if n := b.acharCarona(primeira).Ocupados[0]; n != 1 {
126     t.Errorf("a primeira carona deveria ter 1 ocupado (so a carla), tem %d", n)
127 }
128
129 // Carona cancelada some da busca e recusa reserva nova.
130 if n := len(b.Buscar("Salvador", "Ilheus", "2026-09-14")); n != 0 {
131     t.Errorf("carona cancelada nao deveria aparecer na busca, apareceu %d vez(es)", n)
132 }
133 if _, err := b.Reservar("carla", trecho(segunda, 0, 1)); err == nil {
134     t.Error("nao deveria ser possivel reservar carona cancelada")
135 }
136 }
137
138 func TestRegrasDeCancelarCarona(t *testing.T) {
139     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, 2)
140
141     if _, err := b.CancelarCarona("davi", id); err == nil {
142         t.Error("davi nao e dono da carona e nao pode cancelar")
143     }
144     if _, err := b.CancelarCarona("ana", 999); err == nil {
145         t.Error("carona inexistente deveria dar erro")
146     }
147     if _, err := b.CancelarCarona("ana", id); err != nil {
148         t.Errorf("a dona deveria conseguir cancelar: %v", err)
149     }
150     if _, err := b.CancelarCarona("ana", id); err == nil {
151         t.Error("cancelar duas vezes deveria dar erro")
152     }
153 }
154
155 // -----
156 // Passageiros por trecho
157 // -----
158
159 func TestPassageirosPorTrecho(t *testing.T) {
160     b, id := bancoComCarona(t, []string{"Feira", "Salvador", "Ilheus"}, 2)
161
162     if _, err := b.Reservar("bruno", trecho(id, 0, 2)); err != nil { // os dois trechos
163         t.Fatalf("deveria funcionar: %v", err)
164     }
165
166     carla, err := b.Reservar("carla", trecho(id, 1, 2)) // so o segundo
167     if err != nil {
168         t.Fatalf("deveria funcionar: %v", err)
169     }
170     b.Pagar("carla", carla)
171
172     linhas, err := b.PassageirosDaCarona("ana", id)
173     if err != nil {
174         t.Fatalf("a dona deveria ver os passageiros: %v", err)
175     }
176
177     // linhas[0] e o cabecalho; depois vem uma linha por trecho.
178     if len(linhas) != 3 {
179         t.Fatalf("esperava cabecalho + 2 trechos, deu %d linhas: %v", len(linhas), linhas)
180     }
181
182     primeiro, segundo := linhas[1], linhas[2]
183
184     if !strings.Contains(primeiro, "bruno") || strings.Contains(primeiro, "carla") {
185         t.Errorf("Feira->Salvador deveria ter so o bruno: %q", primeiro)
186     }
187     if !strings.Contains(segundo, "bruno") || !strings.Contains(segundo, "carla (paga)") {
188         t.Errorf("Salvador->Ilheus deveria ter bruno e carla (paga): %q", segundo)
189     }
190 }

```

```

189 }
190
191 if _, err := b.PassageirosDaCarona("davi", id); err == nil {
192     t.Error("so a dona da carona pode ver os passageiros")
193 }
194 }
195
196 // -----
197 // Concorrencia nos cancelamentos
198 // -----
199
200 // TestReservaECancelamentoConcorrentes: 1000 goroutines reservam e cancelam ao
201 // mesmo tempo. Todas escrevem DUAS vezes (ocupar e liberar), entao se o mutex
202 // faltar em qualquer das duas funcoes, incrementos e decrementos se perdem e
203 // o contador final nao volta a zero.
204 func TestReservaECancelamentoConcorrentes(t *testing.T) {
205     const pares = 1000
206
207     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, pares)
208
209     var wg sync.WaitGroup
210     largada := make(chan struct{})
211
212     for i := 0; i < pares; i++ {
213         wg.Add(1)
214         go func() {
215             defer wg.Done()
216             <-largada
217
218             reserva, err := b.Reservar("bruno", trecho(id, 0, 1))
219             if err != nil {
220                 t.Errorf("a reserva deveria funcionar: %v", err)
221                 return
222             }
223             if err := b.CancelarReserva("bruno", reserva); err != nil {
224                 t.Errorf("o cancelamento deveria funcionar: %v", err)
225             }
226         }()
227     }
228
229     close(largada)
230     wg.Wait()
231
232     if n := b.acharCarona(id).Ocupados[0]; n != 0 {
233         t.Errorf("tudo foi cancelado, mas sobraram %d assentos ocupados", n)
234     }
235 }
236
237 // TestCancelarCaronaDuranteReservas: o motorista cancela enquanto 500
238 // passageiros tentam reservar. Nao importa quem chega primeiro; o que nao pode
239 // acontecer e sobrar reserva ativa numa carona cancelada.
240 func TestCancelarCaronaDuranteReservas(t *testing.T) {
241     const pedidos = 500
242
243     b, id := bancoComCarona(t, []string{"Feira", "Salvador"}, pedidos)
244
245     var wg sync.WaitGroup
246     largada := make(chan struct{})
247
248     for i := 0; i < pedidos; i++ {
249         wg.Add(1)
250         go func() {
251             defer wg.Done()
252             <-largada
253             b.Reservar("bruno", trecho(id, 0, 1)) // pode falhar se o cancelamento vier antes
254         }()
255     }

```

```

256
257     wg.Add(1)
258     go func() {
259         defer wg.Done()
260         <-largada
261         if _, err := b.CancelarCarona("ana", id); err != nil {
262             t.Errorf("o cancelamento deveria funcionar: %v", err)
263         }
264     }()
265
266     close(largada)
267     wg.Wait()
268
269     for _, r := range b.Reservas {
270         if r.ativa() {
271             t.Errorf("reserva %d continua %s numa carona cancelada", r.ID, r.Estado)
272         }
273     }
274
275     if n := b.acharCarona(id).Ocupados[0]; n != 0 {
276         t.Errorf("carona cancelada deveria terminar sem assentos ocupados, tem %d", n)
277     }
278 }

```

70. servidor/main_test.go

pbl/servidor/main_test.go · linhas 1–101

```

1  package main
2
3  import (
4      "bufio"
5      "fmt"
6      "net"
7      "testing"
8      "time"
9
10     "vaijunto/dados"
11     "vaijunto/protocolo"
12 )
13
14 // servidorDeTeste sobe um ouvinte numa porta livre e atende UMA conexao.
15 // O canal devolvido e fechado quando o atender() daquela conexao termina.
16 func servidorDeTeste(t *testing.T) (endereco string, terminou chan struct{}) {
17     t.Helper()
18
19     // Porta 0: o sistema operacional escolhe uma porta livre sozinho.
20     ouvinte, err := net.Listen("tcp", "127.0.0.1:0")
21     if err != nil {
22         t.Fatalf("nao consegui abrir porta: %v", err)
23     }
24     t.Cleanup(func() { ouvinte.Close() })
25
26     terminou = make(chan struct{})
27
28     go func() {
29         conexao, err := ouvinte.Accept()
30         if err != nil {
31             return
32         }
33         atender(conexao, dados.NovoBanco())
34         close(terminou)

```

```

35 }()
36
37 return ouvinte.Addr().String(), terminou
38 }
39
40 // TestQuedaAbruptaEncerraAtendimento garante que o servidor aguenta um
41 // cliente que cai sem se despedir: processo morto, cabo puxado, wifi caiu.
42 //
43 // Nesses casos a conexao termina com RST, e a leitura devolve "connection
44 // reset" -- que NAO e io.EOF. Uma versao anterior do atender tratava qualquer
45 // erro diferente de EOF como JSON malformado e voltava a ler: a goroutine
46 // girava para sempre, consumindo 100% de um nucleo.
47 func TestQuedaAbruptaEncerraAtendimento(t *testing.T) {
48     endereco, terminou := servidorDeTeste(t)
49
50     cliente, err := net.Dial("tcp", endereco)
51     if err != nil {
52         t.Fatalf("nao consegui conectar: %v", err)
53     }
54
55     // SetLinger(0) faz o Close mandar RST em vez do encerramento educado.
56     cliente.(*net.TCPConn).SetLinger(0)
57     time.Sleep(100 * time.Millisecond) // garante que o atender ja esta lendo
58     cliente.Close()
59
60     select {
61     case <-terminou:
62         // certo: a goroutine percebeu a queda e encerrou
63     case <-time.After(2 * time.Second):
64         t.Fatal("o atendimento nao terminou depois da queda do cliente: goroutine presa em laço")
65     }
66 }
67
68 // TestJSONInvalidoNaoDerrubaConexao e o outro lado da mesma moeda: uma linha
69 // malformada NAO pode encerrar a conexao. O servidor responde o erro e
70 // continua atendendo aquele cliente.
71 func TestJSONInvalidoNaoDerrubaConexao(t *testing.T) {
72     endereco, _ := servidorDeTeste(t)
73
74     cliente, err := net.Dial("tcp", endereco)
75     if err != nil {
76         t.Fatalf("nao consegui conectar: %v", err)
77     }
78     defer cliente.Close()
79
80     leitor := bufio.NewReader(cliente)
81
82     fmt.Fprint(cliente, "isso nao e json\n")
83
84     r, err := protocolo.LerResposta(leitor)
85     if err != nil {
86         t.Fatalf("o servidor deveria responder o erro, mas: %v", err)
87     }
88     if r.OK {
89         t.Error("JSON invalido deveria voltar com ok=false")
90     }
91
92     fmt.Fprint(cliente, `{"acao":"ping"}\n`)
93
94     r, err = protocolo.LerResposta(leitor)
95     if err != nil {
96         t.Fatalf("a conexao deveria continuar viva depois do JSON invalido: %v", err)
97     }
98     if !r.OK || r.Mensagem != "pong" {
99         t.Errorf("esperava pong, veio %+v", r)

```

```
100 }
101 }
```

71. Dockerfile

pbl/Dockerfile · linhas 1–38

```
1  # Build em dois estagios.
2  #
3  # O primeiro estagio tem o compilador Go inteiro (~300 MB) e produz os dois
4  # binarios. O segundo copia so os binarios prontos para uma imagem minima.
5  # E por isso que a imagem final fica em poucos megabytes: Go gera binario
6  # estatico, entao nao precisa de runtime instalado.
7
8  FROM golang:1.27-alpine AS build
9
10 WORKDIR /src
11
12 # go.mod primeiro, sozinho: assim o Docker so refaz esta camada quando as
13 # dependencias mudam, e nao a cada alteracao de codigo.
14 COPY go.mod ./
15 RUN go mod download
16
17 COPY . .
18
19 # CGO_ENABLED=0 forca binario 100% estatico, sem depender de bibliotecas C
20 # do sistema. E o que permite rodar na imagem alpine crua.
21 RUN CGO_ENABLED=0 go build -o /bin/servidor ./servidor && \
22     CGO_ENABLED=0 go build -o /bin/cliente ./cliente
23
24 # -----
25
26 FROM alpine:3.20
27
28 COPY --from=build /bin/servidor /bin/servidor
29 COPY --from=build /bin/cliente /bin/cliente
30
31 # Onde o JSON fica gravado. O docker-compose monta um volume aqui para os
32 # dados sobreviverem ao container ser destruido.
33 RUN mkdir -p /dados
34 ENV VAIJUNTO_DADOS=/dados/vaijunto.json
35
36 EXPOSE 8080
37
38 CMD ["/bin/servidor"]
```

72. docker-compose.yml

pbl/docker-compose.yml · linhas 1–36

```
1  # Sobe o servidor e deixa o cliente pronto para conectar.
2  #
3  #   docker compose up -d --build      sobe o servidor
4  #   docker compose run --rm cliente  abre um cliente (pode repetir em varios
5  #                                   terminais, para demonstrar concorrência)
6  #   docker compose down              derruba tudo
7
8  services:
```

```
9  servidor:
10    build: .
11    ports:
12      - "8080:8080"
13    environment:
14      # Prazos curtos para a demonstracao. Em uso normal, apague as duas
15      # linhas abaixo e o prazo volta a ser os 10 minutos do enunciado.
16      VAIJUNTO_RESERVA: 30s
17      VAIJUNTO_VARREDURA: 5s
18    volumes:
19      - dados:/dados
20
21  cliente:
22    build: .
23    command: /bin/cliente
24    environment:
25      # "servidor" e o nome do servico acima: o Docker resolve para o IP do
26      # container. E por isso que o servidor precisa ouvir em ":8080" e nao
27      # em "localhost:8080" -- senao ele recusaria conexao de fora.
28      VAIJUNTO_SERVIDOR: servidor:8080
29    depends_on:
30      - servidor
31      # Sem estes dois o menu nao consegue ler o teclado.
32    stdin_open: true
33    tty: true
34
35  volumes:
36    dados:
```